



CSUPREM

A 2/3 Dimensional Semiconductor Process Simulator

© 2004-2014 Crosslight Software Inc.

© 1993 Stanford University

All rights reserved

CSUPREM User's Manual V3.0

Updated: 2014.4

© 2004-2014 Crosslight Software Inc.

© 1993 Stanford University

All rights reserved

Trademark Notice

LASTIP, PICS3D, APSYS, PROCOM, CSUPREM, SimuCenter, SimuLastip, SimuPics3d, SimuApsys, LayerBuilder, Layer3d, GeoEditor, CrosslightView, CrosslightEdit are trademarks of Crosslight Software Inc.

Contents

About This Document	17
CHAPTER 1.....	18
Introduction	18
1.1 What is Csuprem.....	18
1.2 Capabilities.....	19
1.2.1 Ion implantation	19
1.2.2 Epitaxial layer doping.....	19
1.2.3 Etching	19
1.2.4 Deposition.....	20
1.2.5 Diffusion.....	20
1.2.6 Oxidation.....	20
CHAPTER 2.....	22
Getting started.....	22
2.1 Installation	22
2.1.1 IBM-PC/Windows.....	22
2.1.2 LINUX.....	23
2.2 Useful files.....	23
2.3 Running Csuprem.....	24
2.4 Getting started on Windows.....	25
2.4.1 Walking through an example.....	25
2.4.2 Series.....	26
CHAPTER 3.....	27
Useful Tool for 3D Simulation	27
3.1 Introduction	27
3.2 Concept of 3D segment, plane, taper and bending.....	27
3.3 Converting existing 2D input files into 3D	29
3.3.1 How to restart a stalled 3D simulation	29
3.3.2 Converting existing 2D process files	29
3.3.3 Converting existing 2D device files	30
3.4 Definition of structure in z-direction	31
3.5 Reading GDSII data for 3D simulation	31

CHAPTER 4.....	32
Physical Models.....	32
4.1 Introduction	32
4.2 Deposition models	32
4.2.1 Introduction	32
4.3 Analytical ion implantation models	33
4.3.1 Introduction	33
4.3.2 1D Gauss and Pearson IV models	33
4.3.3 Central moments	34
4.3.4 Gauss model.....	35
4.3.5 Pearson model	35
4.3.6 Dual Pearson model.....	36
4.3.7 Analytical implant table	37
4.3.8 2D implant models.....	38
4.4 Direct import of SIMS and Monte Carlo profile.....	39
4.4.1 Method	39
4.4.2 SIMS file format	39
4.5 Etching models.....	40
4.5.1 Introduction	40
4.5.2 Geometrical etching	41
4.5.3 Original etching algorithms.....	41
4.5.4 New etching methods.....	43
4.6 Stress Analysis in Csuprem	44
4.6.1 2D stress equations.....	44
4.6.2 Finite Element Review	47
4.6.3 Finite Element Discretization.....	48
4.6.4 Application of finite element theory to solve displacement equations.....	53
4.6.5 Dirichlet bounary condition	55
4.6.6 Neumann bounary condition.....	55
4.6.7 3D Stress equations in Csuprem	55
4.7 Intrinsic stress in SiGe	57
4.7.1 Si/SiGe interface	57
4.7.2 Stress model for Si/SiGe MOSFET	59

4.8	Oxidation models	60
4.8.1	Introduction	60
4.8.2	2D Analytical models	61
4.8.3	Numerical models	63
4.8.4	Vertical model	64
4.8.5	Compressible model with no stress	64
4.8.6	Non-linear viscous model	65
4.9	Diffusion and segregation models	67
4.9.1	Introduction	67
4.9.2	Pair-diffusion mechanisms	68
4.9.3	Intrinsic and extrinsic dopant diffusion	70
4.9.4	Fermi diffusion model	72
4.9.5	Two-dimensional diffusion model	74
4.9.6	Fully coupled diffusion model	78
4.9.7	Steady state diffusion model	78
4.9.8	Segregation models	78
4.9.9	Clustering and activation models	79
CHAPTER 5		81
One and Two Dimensional Examples		81
Introduction		81
5.1	Example 1: Boron anneal-1D	81
5.2	Example 2: Boron OED -1D	86
5.3	Example 3: OED time	89
5.4	Example 4: Boron OED - 2D	92
5.5	Example 5: LDD cross section	97
5.6	Example 6: One dimensional oxide growth	105
5.7	Example 7: Fully Recessed Oxide Growth	108
5.8	Example 8: Shear stress	113
5.9	Example 9: Stress Dependent Oxidation	117
5.10	Example 18: Diffusion in Presence of Dislocation Loops	122
5.11	Example 19: Interstitial Cluster Dissolution	126
5.12	Example 20: Nondilute Diffusion Using Full.Cpl	130
5.13	Example 21: Stanford BiCMOS 2D Field Oxidation Simulation	134

5.14 SiGe induced uniaxial stress in silicon	137
CHAPTER 6.....	140
Three Dimensional Examples.....	140
6.1 Introduction	140
6.1.1 Reference structure with segregation on x-y plane	141
6.1.2 Structure with segregation in z-direction	143
6.2 Example: Reference Structure for Full 3D MOSFET Simualtion.....	146
6.2.1 Description.....	146
6.2.2 Numerical Results and discussion.....	148
6.3 Example: STI Structure for Full 3D MOSFET Simualtion	150
6.3.1 Description.....	150
6.3.2 Numerical Results and discussion.....	152
6.4 Example: Independent Gate FinFET.....	154
6.4.1 Introduction	154
6.4.2 Description.....	154
6.4.3 Numerical Results	158
6.5 Example: Dependent Double Gate FinFET.....	161
6.5.1 Description.....	161
6.5.2 Numerical Results	165
CHAPTER 7.....	168
CSuprem Statements	168
7.1 3d.implant.method	168
7.1.1 Synopsis	168
7.1.2 Description.....	168
7.1.3 Examples	169
7.2 3d.method	169
7.2.1 Synopsis	169
7.2.2 Description.....	169
7.2.3 Examples	170
7.3 3d_mesh	170
7.3.1 Synopsis	170
7.3.2 Description.....	170
7.3.3 Examples.....	171

7.4	activation.model	171
7.4.1	Synopsis	171
7.4.2	Description	171
7.4.3	Examples	171
7.5	al.impurity	172
7.5.1	Synopsis	172
7.5.2	Description	172
7.5.3	Examples	174
7.5.4	References	175
7.6	antimony	175
7.6.1	Synopsis	175
7.6.2	Description	176
7.6.3	Examples	178
7.6.4	References	178
7.7	arsenic	178
7.7.1	Synopsis	179
7.7.2	Description	179
7.7.3	Examples	181
7.7.4	References	181
7.8	beryllium	182
7.8.1	Synopsis	182
7.8.2	Description	182
7.8.3	Examples	184
7.8.4	References	185
7.9	boron	185
7.9.1	Synopsis	185
7.9.2	Description	185
7.9.3	Examples	188
7.9.4	References	188
7.10	Boundary	188
7.10.1	Synopsis	189
7.10.2	Description	189
7.10.3	Examples	189

7.10.4	Bugs.....	190
7.11	carbon	190
7.11.1	Synopsis	190
7.11.2	Description.....	190
7.11.3	Examples	192
7.11.4	References	192
7.12	cesium.....	193
7.12.1	Synopsis	193
7.12.2	Description.....	193
7.12.3	Examples	194
7.13	change_material	194
7.13.1	Synopsis	195
7.13.2	Description.....	195
7.13.3	Examples	196
7.14	connect_region.....	196
7.14.1	Synopsis	196
7.14.2	Description.....	196
7.14.3	Examples	197
7.15	contour.....	197
7.15.1	Synopsis	197
7.15.2	Description.....	197
7.15.3	Examples	198
7.15.4	Bugs.....	198
7.16	define	198
7.16.1	Synopsis	198
7.16.2	Description.....	198
7.16.3	Examples	199
7.17	deposit	199
7.17.1	Synopsis	199
7.17.2	Description.....	199
7.17.3	Examples	201
7.18	diffuse	201
7.18.1	Synopsis	201

7.18.2	Description.....	201
7.18.3	Examples	204
7.18.4	Bugs.....	204
7.19	dislocation.....	205
7.19.1	Synopsis	205
7.19.2	Description.....	205
7.19.3	Examples	206
7.20	double_mesh	206
7.20.1	Synopsis	206
7.20.2	Description.....	206
7.20.3	Example.....	207
7.21	eliminate	207
7.21.1	Synopsis	207
7.21.2	Description.....	207
7.21.3	Examples	208
7.22	etch	208
7.22.1	Synopsis	208
7.22.2	Description.....	208
7.22.3	Examples	210
7.23	export.....	211
7.23.1	Synopsis	211
7.23.2	Description.....	211
7.23.3	Examples	212
7.24	extend	212
7.24.1	Synopsis	212
7.24.2	Description.....	212
7.24.3	Examples	213
7.25	fill_hole	213
7.25.1	Synopsis	213
7.25.2	Description.....	213
7.25.3	Examples	214
7.26	flip_y	214
7.26.1	Synopsis	214

7.26.2	Description	215
7.26.3	Examples	215
7.27	generic	215
7.27.1	Synopsis	215
7.27.2	Description	215
7.27.3	Examples	218
7.28	germanium	218
7.28.1	Synopsis	218
7.28.2	Description	219
7.28.3	Examples	221
7.28.4	References	221
7.29	gold	221
7.29.1	Synopsis	221
7.29.2	Description	222
7.29.3	Examples	223
7.30	iigeneric	223
7.31	iiigeneric	223
7.32	implant	224
7.32.1	Synopsis	224
7.32.2	Description	224
7.32.3	Examples	225
7.33	include	226
7.33.1	Synopsis	226
7.33.2	Description	226
7.33.3	Examples	226
7.34	indium	226
7.34.1	Synopsis	226
7.34.2	Description	227
7.34.3	Examples	229
7.34.4	References	230
7.35	initialize	230
7.35.1	Synopsis	230
7.35.2	Description	230

7.35.3	Examples	231
7.36	interface_label	231
7.36.1	Synopsis	231
7.36.2	Description.....	232
7.36.3	Examples	232
7.37	interstitial.....	232
7.37.1	Synopsis	232
7.37.2	Description.....	233
7.37.3	Examples	237
7.37.4	References	237
7.38	isilicon	238
7.38.1	Synopsis	238
7.38.2	Description.....	238
7.38.3	Examples	240
7.38.4	References	241
7.39	ivgeneric.....	241
7.40	line	241
7.40.1	Synopsis	241
7.40.2	Description.....	241
7.40.3	Examples	242
7.41	local_heating.....	243
7.41.1	Synopsis	243
7.41.2	Description.....	243
7.41.3	Examples	244
7.42	loose_mesh.....	244
7.42.1	Synopsis	244
7.42.2	Description.....	244
7.42.3	Examples	244
7.43	magnesium.....	245
7.43.1	Synopsis	245
7.43.2	Description.....	245
7.43.3	Examples	247
7.43.4	References	247

7.44	mask.....	248
7.44.1	Synopsis	248
7.44.2	Description.....	248
7.44.3	Examples	249
7.45	material.....	249
7.45.1	Synopsis	249
7.45.2	Description.....	250
7.45.3	Examples	251
7.46	material_define.....	251
7.46.1	Synopsis	251
7.46.2	Description.....	252
7.46.3	Examples	253
7.47	memselectivetch.....	253
7.47.1	Synopsis	253
7.47.2	Description.....	253
7.47.3	Examples	253
7.48	merge.....	253
7.48.1	Synopsis	253
7.48.2	Description.....	254
7.48.3	Examples	254
7.49	method.....	255
7.49.1	Synopsis	255
7.49.2	Description.....	255
7.49.3	Examples	260
7.50	mode.....	260
7.50.1	Synopsis	260
7.50.2	Description.....	260
7.50.3	Examples	261
7.51	ni.impurity.....	261
7.51.1	Synopsis	261
7.51.2	Description.....	261
7.51.3	Examples	263
7.51.4	References	264

7.52	option.....	264
7.52.1	Synopsis	264
7.52.2	Description.....	264
7.52.3	Examples	266
7.53	oxide.....	266
7.53.1	Synopsis	266
7.53.2	Description.....	267
7.53.3	Examples	270
7.54	phosphorus	270
7.54.1	Synopsis	270
7.54.2	Description.....	270
7.54.3	Examples	273
7.54.4	References	273
7.55	plot.1d.....	273
7.55.1	Synopsis	273
7.55.2	Description.....	274
7.55.3	Examples	275
7.56	plot.2d.....	276
7.56.1	Synopsis	276
7.56.2	Description.....	276
7.56.3	Examples	278
7.57	print.1d	278
7.57.1	Synopsis	278
7.57.2	Description.....	278
7.57.3	Examples	279
7.58	profile.....	280
7.58.1	Synopsis	280
7.58.2	Description.....	280
7.58.3	Examples	280
7.59	quit.....	281
7.59.1	Synopsis	281
7.59.2	Description.....	281
7.59.3	Examples	281

7.60	rectangle_deposit_mesh	281
7.60.1	Synopsis	281
7.60.2	Description	281
7.60.3	Examples	282
7.61	rectangle_deposit_method	282
7.61.1	Synopsis	282
7.61.2	Description	282
7.61.3	Examples	283
7.62	region	283
7.62.1	Synopsis	283
7.62.2	Description	283
7.62.3	Examples	284
7.63	regrid	284
7.63.1	Synopsis	285
7.63.2	Description	285
7.63.3	Examples	287
7.64	restart	287
7.64.1	Synopsis	287
7.64.2	Description	287
7.64.3	Examples	287
7.65	select	287
7.65.1	Synopsis	287
7.65.2	Description	288
7.65.3	Examples	290
7.66	selenium	290
7.66.1	Synopsis	290
7.66.2	Description	290
7.66.3	Examples	292
7.66.4	References	293
7.67	set_etch_geometry	293
7.67.1	Synopsis	293
7.67.2	Description	293
7.67.3	Examples	294

7.68	smooth_interface	294
7.68.1	Synopsis	294
7.68.2	Description.....	294
7.68.3	Examples	295
7.69	stress.....	295
7.69.1	Synopsis	295
7.69.2	Description.....	295
7.69.3	Examples	296
7.70	structure.....	296
7.70.1	Synopsis	296
7.70.2	Description.....	296
7.70.3	Examples	299
7.71	tin	299
7.71.1	Synopsis	299
7.71.2	Description.....	299
7.71.3	Examples	301
7.71.4	References	302
7.72	trap.....	302
7.72.1	Synopsis	302
7.72.2	Description.....	302
7.72.3	Examples	303
7.72.4	References	303
7.73	truncate.....	303
7.73.1	Synopsis	303
7.73.2	Description.....	304
7.73.3	Examples	304
7.74	uniform_doping	304
7.74.1	Synopsis	304
7.74.2	Description.....	304
7.74.3	Examples	305
7.75	vacancy	305
7.75.1	Synopsis	305
7.75.2	Description.....	306

7.75.3	Examples	309
7.75.4	References	310
7.76	vgeneric.....	311
7.77	zinc	311
7.77.1	Synopsis	311
7.77.2	Description.....	311
7.77.3	Examples	313
Appendix A		314
Impurity and Material Indices		314
References		319

About This Document

The CSUPREM User's Manual consists of seven chapters. The first chapter is a general introduction of the CSUPREM program.

Chapter.2 helps the user to install and run the software on different platforms.

Chapter.3 introduces new commands to set up and generate a 3D simulation. The auxiliary program geo3d is described along with geometric concepts of segments, planes, tapers and bendings.

Chapter.4 describes advanced physical and numerical models used in CSUPREM.

Chapter.5 contains 2D example supported by the software. The first 21 examples were based on Stanford's Suprem4.gs and used as quality control example sets. More 2D examples with new models and new structure are also described.

Chapter.6 contains 3D examples and numerical results.

Chapter.7 lists all the commands supported by CSuprem which are fully compatible with Suprem4.gs from Stanford.

CHAPTER 1

Introduction

The fabrication of the modern integrated circuits (ICs) involves many process steps which may need months to be realized. The explosive development costs and high complexity of these fabrication steps require advanced numerical simulation tools. A process simulation software is indispensable not only in development and optimization of IC fabrication processes but it can also predict accurately the doping profiles needed in device simulators. The shrinking of the device dimensions in today's highly packed integrated circuits demands a process simulator that includes the multidimensional effects and more accurate physical models. Our goals and priorities are to provide the semiconductor industries with highly accurate and reliable numerical simulation software in 1, 2 and 3 dimensions.

1.1 What is Csuprem

Csuprem is Crosslight's advanced version of the 2D process simulator Suprem-IV.gs [2] originally developed by Stanford university. It simulates the processing steps involved in sub-micron Silicon and Gallium Arsenide fabrication technology. It is based on well established, calibrated, and advanced physical models. It can be used as valuable Technology Computer-Aided Design (TCAD) tool to optimize the processing time, the design, and to control the cost of today's

highly complicated integrated circuits.

1.2 Capabilities

Csuprem simulates and provides cross-sections of various structures based on advanced physical models for ion implantation, diffusion, oxidation, and annealing. Its capability also includes the basic models to simulate etching and deposition steps. Although, it has the capability to interface with other simulators that accurately simulate etching and deposition of the thin films on the semiconductor surface. Csuprem structure is designed to handle moving boundaries which occur during oxidation, deposition, or etching of device structure. It can model the stress gradient produced by overlying films. The diffusion, annealing, implantation, and oxidation models are point defect based and are believed to be the most advanced physical models used in any process simulation software.

1.2.1 Ion implantation

1-Available impurities: antimony, arsenic, boron, bf2, cesium, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic.

2-Physical models: Two different types of analytical models are available: **Gauss** distribution, and **Pearson IV** distribution. Experimental SIMS profile or Monte-Carlo data may be imported directly into the device.

3-Damage: The damage due to the implant can be calculated. The data is from Hobler and Selberherr [30] and exist only for antimony, arsenic, boron and phosphorus.

1.2.2 Epitaxial layer doping

Epitaxial layer may be modeled as a material layer with uniform grown-in dopants. Gaussian function may be used to obtain more accurate doping profile.

1.2.3 Etching

1-Materials: silicon, oxide, oxynitride, nitride, polysilicon, photoresist, aluminium, GaAs, SiGe, generic-material.

2-Region shapes: Etching capability allows the user to specify the different etch regions in quick and easy manners. The user can also supply an input file containing the coordinates associated with the etched surface. This file could be generated by an auxiliary program called Etch_geometry which includes physically based etch models.

3-Auxiliary etch programs: CSuprem comes with two auxiliary etch related programs. The etch_geometry program is capable of producing etch shapes based on isotropic and directional etching rates in reactive ion etching (RIE). The program etch_image is a graphic program that converts experimental (SEM) image into etch profile data. Profile data from both programs may be imported into the etch command to produce accurate etching shapes.

1.2.4 Deposition

1-Materials: silicon, oxide, oxynitride, nitride, polysilicon, photoresist, aluminium, gallium-Arsenide, generic-materials.

2-Dopant types: antimony, arsenic, boron, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic.

3-Input file: dopant profile may be imported into device during the deposit step.

1.2.5 Diffusion

1- Point-defect based diffusion

2- Transient enhanced diffusion for damage and clustering

3- Oxidation enhanced and retarded diffusion

4- Interface segregation models

5- Dynamic and static clustering model for As

6- Diffusion within poly-silicon, oxide, and oxide nitride layers.

7- Capability of different diffusion coefficients for grown-in and as-implanted impurities.

8- Capability for rapid thermal annealing.

1.2.6 Oxidation

There are five oxide models from simple to more advanced including stress effects on oxide growth rate and volume expansions. All these models are based on the Deal-Grove theory.

1- Error-function fit to bird's beak shape model.

2- Parameterized error-function model.

3- A model where oxidant diffuses and the oxide boundaries move vertically at the rate determined by the local oxidant concentration.

4- A compressible viscous flow model

5- An incompressible viscous flow model.

6- Dry and wet oxidation capability

CHAPTER 2

Getting started

2.1 Installation

2.1.1 IBM-PC/Windows

The minimum system requirements are IBM-compatible PC with *500MB* of memory, *500MB* of hard disk space and *500MB* of swap space. Later and more stable versions of OS such as Windows 2000, XP or Vista are recommended.

The installshield will guide you through the installation process. Version specific instructions may come with installation CD or with downloads. The program may be launched from the Start-Program's menu.

The simulation package consists of several executables, a number of auxiliary files and examples under the installation directory which the users may choose from the installshield. The graphic user interface (GUI) programs are in a separate GUI directory under the installation directory. The general program control GUI is the SimuCsuprem which may be launched from the Start menu.

Windows is the original platform of code development and all GUI components are available and supported on Windows.

2.1.2 LINUX

The core simulation software may also be ported to LINUX without the Windows GUI. Since the type of LINUX may vary from time to time or from version to version, please check with Crosslight on the supported LINUX platforms.

Some GUI components have been ported from Windows to LINUX. At the time of this writing, the output GUI CrosslightView has been ported to LINUX using the WINE emulator so that simulation results of CSUPREM may be viewed directly on LINUX.

The Csuprem package is usually contained in a single file named "csuprem.tar" (or the compressed version: csuprem.tar.gz, csuprem.tgz, or csuprem.tar.z) for the convenience of data transfer. Expansion of this file results in installation directory named csuprem/.

Usually, a shell script, such as Csuprem.sh or Csuprem.csh are supplied with the installation. Please set the path in the script and copy it to the example project directory. Then, change directory to the example project directory and type the following to run the program

```
chmod u+x Csuprem.sh
./Csuprem.sh example.in
```

will enable running the program Please follow the procedures contained in the README files under the installation directory on how to run the simulator on LINUX.

To install the output GUI CrosslightView, you need to install emulator WINE with assistance from Crosslight (some patches of WINE are needed). Then, launch the CrosslightView program with command:

```
wine crosslightview.exe
```

2.2 Useful files

It is useful to know some basics of files involved in process modeling using CSUPREM. For simplicity, we assume the Windows operating system (LINUX version should be similar) and installation on

```
c:\crosslig\Csuprem\
```

- The main simulation program is csuprem.exe and resides at

```
c:\crosslig\Csuprem\bin\
```

Running a simulation means using it on an input file containing process commands.

- Control of the simulation is through a main input file (also named input decks) containing commands such as deposit, etch and diffuse. The file extension of the input files is .in. Examples of input files can be found under

c:\crosslig\Csuprem\examples\

- A complete table of Csuprem commands and associated parameters is included in a file named suprem.key.

c:\crosslig\Csuprem\

This file contains the actual commands used by the current version of Csuprem. The program reads this file every time it runs. It is sometimes convenient to look up this file to see if any new commands not yet included in the GUI are available. **IMPORT:** please do not revise this file or Csuprem may not run.

- The default model of Csuprem is contained in a file named Modelrc located at

c:\crosslig\Csuprem\

It is a collection of Csuprem commands to describe the physical models of various processes. For example, the default oxidation rate model parameters and dopant solubilities are defined in this file. Every time Csuprem runs, it checks the model parameters in the Modelrc. It is advised that revision of this Modelrc file be reserved for Csuprem experts.

- The default model parameter for implantation is contained in a file named sup4gs.imp, located at

c:\crosslig\Csuprem\

Again, it is advised that revision of this file be reserved for Csuprem experts.

- For 3D-simulation, the mesh planes positions in the z-direction are defined in a file named zmesh.zst. It must be located in the same directory as the main input file.

2.3 Running Csuprem

Applying the executable of Csuprem program on this input file activates the process simulation. This means simply change directory to the current directory of the input file and type:

[fullpath] csuprem.exe inputfile

on MS-DOS prompt for Windows or Shell prompt for LINUX. Here [fullpath] is the full path of the csuprem executable (for example: *C:\crosslig\cusprem*).

If your installation comes with a full GUI, the above may be launched from GUI instead of from a MS-DOS prompt/LINUX shell.

An input file contains a collection of statements (or commands) recognized by Csuprem program. The statements have been invented by the programmers of Csuprem to allow the user to interact with the simulator. There are 2 basic input file types with different file extensions: .in, .str. These are used to solve the equations, plot the solution using a public domain graphic program GNUPLOT, and the Crosslight visualization program CrosslightView.

For Windows users, a Graphic User Interface (GUI) is used to help running the program so that most actions can be controlled using point and click. If the GUI is bothering you or not functioning due to installation problems, you can always go back to the basics and use the MS-DOS prompt. However, you have to type the full path of the executable program correctly (or use a batch file to avoid repeated typing). It is always recommended to use MS-DOS prompt as a debug tool if there is something wrong with the simulation program, since it will always give you the real time display of what is going on.

On Linux, it is recommended that you use the shell scripts (such as Csuprem.sh, etc.) that comes with the package to run the input files. These are used to set the environmental variables and set the paths before running the program.

2.4 Getting started on Windows

2.4.1 Walking through an example

The fastest way of learning a new software package is to follow some simple examples. Let us go through an example with you. We will simulate a simple 1D Boron diffusion process in silicon structure which is included in this package. The example input file boron.in should have been installed in the directory

c:\crosslig\Csuprem\examples\Stanford_Original\exam1

Please follow these steps:

- Launch SimuCsuprem. This may be done by clicking "start—> programs—> Csuprem—> SimuCsuprem".
- Choose "File—>Open Project" and browse to the above example directory and open

boron.in file. On the left side window under the Input tab, you should see boron.in display as an input file.

- Right click on boron.in and select **Simulate** or use the "run" bottom located in the **menu bar** and the simulation should run.
- Click on **Output** tab on the left window and you should see a number of output files including boron.str. Right click on boron.str and select Open with CrosslightView. The results of boron profile similar to Figure 2.1
- That's it !

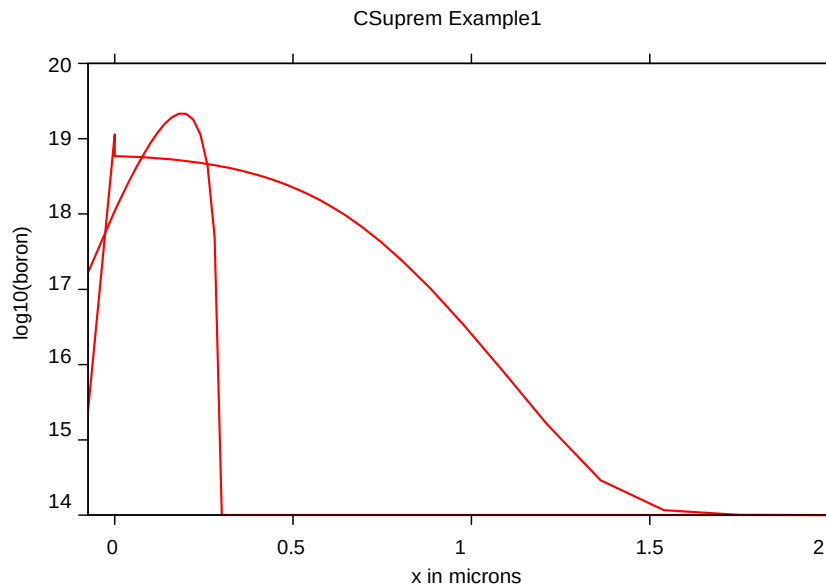


Figure 2. 1: Implanted and annealed Boron profile

2.4.2 Series

In SimuCsuprem, there is a special tab for series. Series is useful when multiple values of a process parameter needs to be tested. Instead of changing the parameter every time, the SimuCsuprem will allow user to select multiple numbers using start value, end value and step. Please refer to the instruction movie for detail usage.

CHAPTER 3

Useful Tool for 3D Simulation

3.1 Introduction

The user needs to set up three types of input files to run a 3D simulation. The main file (or driver file, *.in file) where almost all the processing commands are set. The xy-geometry files holding the geometrical data for the face of each segment. Finally, another geometry file holding the geometrical data for the mesh in z-direction. In the following sections, we guide the user to set up and combine these files to run a 3D simulation.

3.2 Concept of 3D segment, plane, taper and bending

3D simulation in Crosslight's simulator depends on the stacking up 2D planes. This technique is convenient and flexible since one may start from existing 2D mesh which has been well established in previous simulations. Also the mesh in each plane can be controlled individually according to physical variable on that plane so as to minimize the CPU time. It is powerful since one can bend the plane in any way, rotate it as in cylindrical coordinate system and combine it with rectangle coordinates.

The fundamental issue is that once the planes are established, how would the simulator treats

the interaction or coupling between the complicated mesh between different planes. This is the task of the geo3d.exe program which takes zmesh.zst as its input file. The geo3d.exe program figures out how mesh points on different planes should be coupled with each other and pass that information to CSUPREM simulator.

The concept of **plane** takes its usual meaning to mean a single mesh grid located on a certain constant z-coordinate. A plane is always associated with a reference z-coordinate.

The concept of **segment** or **z-segment** is used to present a collection of planes having the same (x,y) grid points. The material on each plane within a segment must also be identical, even though solution variables may vary. In another word, segment is a collection of structurally identical planes. Please note that the z-coordinates of the planes within the segment may vary in any ways, as well as the mesh points on the planes may be shifted in z-direction (see also the definition of bending below), as long as the (x,y) coordinates are identical within a segment. Internally, the code uses the same numerical arrays to store the x-y mesh data within a segment.

Bending is used to mean the shift of z-coordinate of mesh points in a specific way (described by functions or tables). For example, if a plane is to take the shape of part of a sphere, it is represented by a function describing the shift of z-coordinate around the reference z-coordinate, as in the following expression:

$$z_{bending} = z_0 + \sqrt{R^2 - (x - x_0)^2 - (y - y_0)^2} - z_{ref} \quad 3.0-1$$

where (x_0, y_0, z_0) is the center of the sphere, R the radius, z_{ref} is the reference position of the plane. In the limit of zero bending, the z-coordinate reduces to a constant z_{ref} .

Taper is used to mean how the mesh would connect from one segment to the next when the dimensions of the segments are different. When mesh points couple between segment perpendicularly, we call it having no taper. If the points couple with an angle, we define it as coupling with taper. By default, no taper connect is used by the simulation program.

Let us illustrate this using a simple example. Suppose segments A and B have identical height but the x-dimension is different. Segment A has $x=(0\ 5)$ and B has $x=(0\ 6)$. By default, segment A and B only interact with each other perpendicularly (meaning no taper), and for segment B, only mesh points from 0 to 5 are coupled to segment A. When we use taper, the points on segment A at $x=5$ would connect to segment B at $x=6$ with an angle.

In practical 3D simulation in CSUPREM, oxidation step is usually involved. Since oxidation depends on dopant, stress and oxidant flow in z-direction, the mesh on all planes may be different once the mesh points start moving. So it is usually impossible to organize the planes into segments to save internal memory. Therefore, we normally define one plane per each segment and freely use plane and segment interchangeably in CSUPREM documents.

3.3 Converting existing 2D input files into 3D

Most devices have strong 2D nature and many basic properties can be simulated using a 2D mesh already. An easy approach for 3D simulation is to start from a successful 2D simulation and simply convert the input files into 3D. Here is a brief guideline on how to convert both process and device input files from 2D to 3D.

3.3.1 How to restart a stalled 3D simulation

Restarting a stalled quasi3d or 3D simulation is similar to that for 2D. The only difference is that the **restart** command should be placed right after the **3d_mesh** command to get the z-plane information. Then, the program would have all the 3D mesh information and would be ready to jump to the position of the .str to continue a stalled simulation.

3.3.2 Converting existing 2D process files

Assuming you already ran through a 2D simulation. Here are the procedures of converting it into a simple 3-plane 3D simulation file. Please note that it is recommended that you initially attempt to reproduce the same results in 2D by using uniform planes. Then, you will be ready to change the conditions of some of the planes to make it real 3D.

1) use the following lines at the beginning:

```
mode three.dim 3d_mesh nsegm=3 infile=mymesh
```

here mymesh1.in, mymesh2.in and mymesh3.in contain commands to define mesh lines, regions and exposed interfaces, as used in 2D simulation.

2) find an existing template of zmesh.zst from any of the 3D examples. Edit it to define the z-position of the 3 mesh planes. Place it in the same directory as the main input deck. This will be used by CSUPREM when running in 3D.

3) Make 2 copies of each etch command and append segm=1,2,3 to all the etch commands, respectively. For example:

```
etch oxide all
```

will be converted to

```
etch oxide all segm=1
etch oxide all segm=2
```

`etch oxide all segm=3`

The above 3 steps would convert a 2D input deck into a working 3D input file. After it runs through the simulation, you may start to vary some planes by using different etching commands or by using a different initial mesh/region for different planes. Please note that deposition and implantation are the same for all planes. Only etching can vary from plane to plane (as a result of mask/photolithography).

When running a 3D simulation, instead of using "mode three.dim", it is recommended that you use "mode quasi3d" first. This would let you run through all the planes quickly. Sometime, poor initial mesh may cause a crash and it is better to find out while you are still doing quasi3d. Full 3D generally takes a longer simulation time.

3.3.3 Converting existing 2D device files

Assuming you already ran through a 2D device simulation using a .sol file. Here are the steps to convert it into a working 3D input file.

1) at the beginning of the .sol file, add

`3d_solution_method 3d_flow=yes`

2) define the position of the z-planes by copying all the z_structure commands from zmesh.zst in the project folder of 3D process simulation to the .sol file, placing them to following the line of "3d_solution_method" above.

3) keep the single load_mesh command same as before.

4) enclose all statements of suprem_property, load_macro, (also material property commands such as affinity and impact_chynoweth), suprem_contact and contact statements inside a pair of

```
begin_zmater zseg_num=1,2,3...
...
end_zmater
```

Here zseg_num is really the plane number since we normally use one plane per segment in CSUPREM+APSYS in case oxidation makes all planes different. You need to make one set of "begin_zmater end_zmater" for each plane since there may be variation from plane to plane. For example, some planes may have contacts while others may not. You may wish to use the loop feature of APSYS to avoid some writing if you have many planes. Please note there is a difference between "metal material" and "contact." "Contact" here means pure geometric boundary condition while a metal material means a real material with properties such as

resistivity and work function.

5) Keep all the equilibrium, newton_par and scan commands same as for 2D.

3.4 Definition of structure in z-direction

The definition of device structure in z-direction is through a file zmesh.zst residing at the same folder as the main input file. This file is used by the geo3d.exe program which communicates with the main CSUPREM simulator to provide plane to plane coupling information. Since zmesh.zst and geo3d.exe are part of the APSYS device simulator, the user is referred to the APSYS documents for more details. We only give a brief summary of the common usage of the commands in zmesh.zst.

The command **z_structure** in zmesh.zst is really the only one a user needs to pay close attention to. It is used to define the reference position of the planes with a segment. As we stated earlier, it is common to use one plane per segment in CSUPREM simulation and this means defining the positions manually for all the planes (unless you use the GDS2MASK utility).

Taper connections are defined using one or more of the parameters containing the word **taper** within the **z_structure** command. To define bending of the planes, you need to use the command **bend_xy_plane** once or several times for one or more segments/planes.

If some of the planes rotates around a center, you need to define them as part of a cylindrical system using the **cylindrical** flag within the same **z_structure** commands.

3.5 Reading GDSII data for 3D simulation

It makes sense to use the GDS2MASK utility for 3D simulation because the utility automatically cuts up the layout into multiple planes and produces the zmesh.zst file for 3D simulation. Since 3D simulations cost considerable amount of CPU time, it is highly recommended that a similar 2D simulation has been verified before an attempt to use GDSII data in a 3D simulation. In many cases, the bottleneck is not how good the cuts are made in the GDSII files, but the construction of a reasonable 2D mesh for the planes used in the simulation. Ideally, we recommend at least some of the planes cut from GDSII pass a 2D simulation test before submitting for a 3D simulation job. This ensures the mesh on the planes would not cause any numerical problem on their own. If a 2D simulation fails, the 3D simulation would certainly fail. In such a case, it is easier to fix the 2D problem first.

The user is recommended to consult the previous chapter for more details on using the GDS2MASK utility.

CHAPTER 4

Physical Models

4.1 Introduction

In this chapter, we present the equations of the different models used in Csuprem.

4.2 Deposition models

4.2.1 Introduction

The deposition of thin film is used for fabrication of various materials like poly-crystalline silicon, nitride, silicon dioxide, and silicides. The most used techniques are chemical vapor deposition (**CVD**) and physical vapor deposition (**PVD**) [27]. The atomic layer deposition (**ALD**) is new to the semiconductor technology. It has been used for the fabrication of simple metal oxides such as Al_2O_3 for trench capacitors and metal nitrides for copper barriers.

The simulation of the physical deposition will provide information about the shape of the films as well as about the quality of the deposited films. However, Csuprem provides only a purely geometrical deposition. The deposition simulation is activated using the command **deposit** and its parameters as **thickness**, material name to be deposited. Csuprem allows the user to set the grid spacing in the deposited region on the deposit command using the parameters **divisions**

and **space**. The divisions parameter controls the number of vertical grid spacings (i.e number of sub-layers) in the deposited region in the case of planar deposition. Its default is 1. The space parameter represents the average spacing between points along the outside edge of the deposit. Csuprem allows the user to also deposit a doped layer like ploy-silicon using the **deposit** command and its parameter **conc**. For a complete description of this command see the Reference Users's Manual [2].

4.3 Analytical ion implantation models

4.3.1 Introduction

The most widely used doping technique in modern integrated circuits technology is ion implantation. Ionized dopants atoms are accelerated by an electrostatic field, strike and penetrate into the target material. The dose can be controlled by controlling the ion current. The profile depth is controlled by the implantation energy. Since dose and depth can be adjusted electrically ion implantation is highly controllable and reproducible [27], [32]. The implanted ions cause implantation damage by displacing the wafer atoms (defects generations) or by creating clusters and precipitates. A subsequent thermal annealing is necessary to heal as much as possible of implantation damages. A rapid thermal anneal (RTA) at very high temperature allows to repair the damage with a minimal dopant diffusion. Ion implantation into a single crystal causes an other problem due to the anisotropy of the target. Along the major crystal axis the implanted impurities collide with fewer target atoms and can thus penetrate deeper into the crystal. This phenomenon referred to as *channelling* [4] can be significantly reduced by slightly tilting the ion beam against the preferred orientation [27], [31], [33]. In this section, we will present the mathematical expression of the 1D and 2D analytical models used in Csuprem.

4.3.2 1D Gauss and Pearson IV models

We first consider one dimensional distributions. To describe ion implantation profiles not all the analytical distributions are suitable [4]. $F(x)$ is termed univariate cumulative distribution function if:

- a) $F(x)$ is non-decreasing:

$$F(x_1) \leq F(x_2), x_1 < x_2 \quad 4.1$$

- b) $F(x)$ is everywhere continuous from the right:

$$F(x) = \lim_{h \rightarrow 0+} F(x + h) \quad 4.2$$

c) $F(x)$ fulfills:

$$F(-\infty) = 0, F(+\infty) = 1 \quad 4.3$$

Only distribution functions with these properties can be used for implantation distributions. Furthermore, we only allow continuous functions so that we can write $F(x)$ as

$$F(x) = \int_{-\infty}^x f(t)dt \quad 4.4$$

Using the relation $F(+\infty) = 1$ and equation 4.4 the final distribution of implanted dopants is given by

$$C(x) = N_d f(x) \quad 4.5$$

where N_d is the implantation dose per unit area. $f(x)$ is the probability density (or frequency function). Most of analytical ion implantation models are based on the conventional central moments and their characteristic parameters. Gauss and Pearson *IV* models provided by Csuprem are based on the central moments [4],[27].

4.3.3 Central moments

The i -th non central moments of a frequency function $f(x)$ are given by:

$$m_i = \int_{-\infty}^{+\infty} x^i f(x) dx. \quad 4.6$$

By introducing the mean value of the distribution, we get the central moments:

$$\mu_i = \int_{-\infty}^{+\infty} (x - R_p)^i f(x) dx. \quad 4.7$$

The characteristic parameters: Projected range R_p , standard deviation σ_p , skewness γ , and the kurtosis or excess β are given by:

$$R_p = m_1 \quad 4.8$$

$$\sigma_p = \sqrt{\mu_2} \quad 4.9$$

$$\gamma = \frac{\mu_3}{\sigma_p^3} \quad 4.10$$

$$\beta = \frac{\mu_4}{\sigma_p^4} \quad 4.11$$

4.3.4 Gauss model

The most popular Gaussian distribution is based on the central moments. It uses only the two first characteristic parameters R_p and σ_p and it is given by:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma_p} \exp\left(-\frac{(x-R_p)^2}{2\sigma_p^2}\right) \quad 4.12$$

This distribution is only a first order accurate. The lack of accuracy in the tail of the density function in case of nonsymmetric ion implantation profiles constrain the usage of other frequency functions as Pearson *IV* density [35].

4.3.5 Pearson model

An approach followed by Hoflker [35] for fitting a frequency function to experimental data is to use a Pearson type IV distribution function. The whole family of Pearson distributions [36] is based on the solutions of the Pearson differential equation:

$$\frac{df}{dy} = \frac{y-a}{b_0+b_1y+b_2y^2}, y = x - R_p \quad 4.13$$

The Pearson coefficients a , b_0 , b_1 , and b_2 can be determined by the first four characteristic parameters as follows:

$$a = b_1 \quad 4.14$$

$$b_0 = -\frac{\sigma_p^2(4\beta-3\gamma^2)}{10\beta-12\gamma^2-18} \quad 4.15$$

$$b_1 = -\frac{\gamma\sigma_p(\beta+3)}{10\beta-12\gamma^2-18} \quad 4.16$$

$$b_2 = -\frac{(2\beta-3\gamma^2-6)}{10\beta-12\gamma^2-18} \quad 4.17$$

The solutions of equation 4.13 depend on the roots of the equation

$$b_0 + b_1y + b_2y^2 = 0$$

which eventually depend on the values of γ , and β . Height different solutions: Gaussian, Pearson type *I*, Type *II*,...and type *VII* can be obtained.

If $\gamma = 0$, and $\beta = 3$ we obtain the Gaussian distribution which is the limited case for all types of Pearson distributions.

If $\gamma \neq 0$ and $\beta < 3 + 1.5\gamma^2$, we get the Pearson type *II*.

The case when the equation $b_0 + b_1y + b_2y^2 = 0$ does not have real roots corresponds to Pearson type *IV*. This case arises when:

$$0 < \gamma < 32 \quad 4.18$$

$$\beta > \frac{39\gamma^2 + 48 + 6(\gamma^2 + 4)^{3/2}}{32 - \gamma^2} \quad 4.19$$

For the description of implantation profiles only Pearson type *IV* and type *VII* distribution can be generally applied. These frequency functions have a single maximum at $y = a$ and decay monotonously to zero on both sides. The type *VII* distribution is not skewed which limits its application. The general solution of the Pearson differential equation 4.13 under the conditions 4.18 which characterizes the Pearson *IV* is given by:

$$f(x) = K(-(b_0 + b_1(x - R_p) + b_2(x - R_p)^2))^{\frac{1}{2b_2}} \exp\left(-\frac{\frac{b_1}{b_2} + 2a}{\sqrt{4b_2b_0 - b_1^2}} \operatorname{atan}\left(\frac{2b_2(x - R_p) + b_1}{\sqrt{4b_2b_0 - b_1^2}}\right)\right) \quad 4.20$$

where the normalization constant K is given by

$$\int_{-\infty}^{\infty} f(x) dx = 1 \quad 4.21$$

If no values for the kurtosis are available, the following universal expression is used:

$$\beta = 2.8 + 2.4\gamma^2$$

4.3.6 Dual Pearson model

The Pearson type *IV* involves the first four moments and it is then more accurate to predict a doping profile in many applications. However, it is not really suitable to predict the typical channelling tail observed when channelling phenomenon are presents. In this case, the Dual Pearson *IV* is used:

$$f(x) = N_d[(1 - \zeta)f_1(x, R_p, \sigma_p, \gamma_p, \beta_2) + \zeta f_2(x, R'_p, \sigma'_p, \gamma'_p, \beta'_2)] \quad 4.22$$

Where f_1 is the Pearson IV for the surface region and f_2 is the Pearson IV for the bulk region. R_p , σ_p , γ_p and β_2 are the **projected range**, **standard deviation**, **skewness** and the **kurtosis**, respectively, for the first Pearson distribution. Similarly, those with ' are for the second Pearson function distribution describing the channeling tail. For more details please see [37], [38].

4.3.7 Analytical implant table

The implant table is placed within CSuprem installation folder and named sup4gs.imp. This file can be edited by user. For single Pearson model, there are 8 columns of data and 13 columns for dual Pearson:

1. material index
2. impurity index
3. implant energy
4. projected range
5. standard deviation
6. skewness
7. kurtosis
8. lateral standard deviation
9. projected range of the 2nd Pearson function
10. standard deviation of the 2nd Pearson function
11. skewness of the 2nd Pearson function
12. kurtosis of the 2nd Pearson function
13. percentage of the 2nd Pearson function

An example of implant table for boron is shown in Figure 4.1 which display moment parameters for both single and dual Pearson distributions.

#	#	Material: Silicon		[3]									
#	#	Element : boron		[5]									
#	#	energy	rp	drp	gamma	beta2	lrp	rp_p	drp_p	gamma_p	beta2_2	zita	
3	5	0.25	-0.0008	0.002	0.00001	4.00	0.002	0.005	0.005	6.3	688	0.25	
3	5	0.5	0.00311	0.0029	0.22632	7.000	0.0029	0.012	0.01	6.0	659	0.03421	
3	5	1.0	0.0000	0.009444	0.00001	4.00	0.009444	-0.947e-3	0.005	6.3	688	0.25	
3	5	2.0	0.00111	0.007778	-0.72222	5.000	0.0077777	0.015	0.005	0.63	68.8	0.001	
3	5	5.0	0.03778	0.021111	1.13333	5.000	0.0211110	0.025	0.005	0.63	68.8	0.001	
3	5	10.	0.0351	0.0182	-0.078	2.9195	0.0250						
3	5	20.	0.0685	0.0296	-0.313	3.0633	0.0424						
3	5	30.	0.1011	0.0384	-0.480	3.2746	0.0569						
3	5	40.	0.1326	0.0455	-0.611	3.5045	0.0694						
3	5	50.	0.1629	0.0515	-0.718	3.7393	0.0803						
3	5	60.	0.1921	0.0566	-0.811	3.9744	0.0899						
3	5	70.	0.2202	0.0610	-0.892	4.2078	0.0986						
3	5	80.	0.2473	0.0649	-0.964	4.4390	0.1064						
3	5	90.	0.2736	0.0683	-1.030	4.6675	0.1135						
3	5	100.	0.2990	0.0714	-1.091	4.8935	0.12						
3	5	110.	0.3237	0.0742	-1.147	5.1169	0.1260						

Figure 4. 1: A part of the implant table containing low energy data for boron in silicon. The first two numbers are material and impurity indices and the rest are Pearson function moments for single and dual Pearson function(s).

To change the implant table it is necessary to know the internal ion and material indices. Table A in Appendix A provides a complete list of ion and impurity and material indices recognized by the CSuprem program. Please note the use of "ion" and "impurity". The former refers to the ion implantation profile model while the latter to the diffusion model. For example, BF2 is implanted as ion BF2 but diffused as B, as indicated in the table.

To create a new implant table for a new material, please use material numbers 9, 10, 11, 12, and 13 for imaterial, iimaterial, iiimaterial, ivmaterial, and vmaterial, respectively. By default, these new materials assume the default parameters of of poly silicon and all models for poly silicon (such as diffusion model) may be used for them.

New impurities not implemented in CSuprem should use generic, iigeneric, iiigeneric, ivgeneric and vgeneric. The impurity indices are also listed in Appendix A.

The program will automatically switch between single and dual Pearson models depending on the number of columns available. For a given ion energy E , logarithmic interpolation is made between available data of two nearest energies E_a and E_b as follows:

$$\ln(f(x, E)) = \ln(C_N) + g_a \ln[f_a(x, E_a)] + g_b \ln[f_b(x, E_b)] \quad 4. 23$$

$$g_a = (E_b - E)/(E_b - E_a)$$

$$g_b = (E - E_a)/(E_b - E_a)$$

where C_N is a normalization constant to ensure the dopant profile is normalized to the total implant dose.

4.3.8 2D implant models

The 2D ion implantation profile is based on the convolution of the 1D lateral (y-coordinate) and 1D longitudinal (x-coordinate) profiles and can be formulated in general form as:

$$C(x, y) = N_d C_1(x, t(y)) * C_2(y) \quad 4. 24$$

where $*$ is the convolution product and $t(y)$ represents the thickness of the mask or coating when the implantation is done through a mask (as pad oxide,...etc.). If no mask is present then $t(k) = 0$. C_1 is the depth dependent longitudinal concentration. C_1 could be chosen to be a Gaussian or Pearson *IV*. The user can set the models of C_1 using the **implant** command and its parameters **gauss** or **pearson**. C_2 is the lateral doping concentration. In the actual version of Csuprem, C_2 is chosen to be a Gaussian distribution and it is given by:

$$C_2(y) = \frac{1}{2} \left(\operatorname{erfc} \left(\frac{y-l}{\sqrt{2}\sigma_{py}} \right) - \operatorname{erfc} \left(\frac{y+l}{\sqrt{2}\sigma_{py}} \right) \right), \forall y \in [-l, l] \quad 4.25$$

where $2l$ is the lateral length of the material wafer. $\operatorname{erfc}(y)$ is the complementary error function defined as:

$$\operatorname{erfc}(y) = \frac{2}{\sqrt{\pi}} \int_y^{\infty} \exp(-t^2) dt \quad 4.26$$

σ_{py} is the lateral standard deviation. In this current version of Csuprem, the lateral standard deviation is independent of the longitudinal direction (x-coordinate). And it is by default equal to the standard deviation σ_p presented previously.

The ion implantation causes a lot of damage to the wafer material. The user can include the model of damage-induced injection of point defects using the **implant** command and its parameters **damage** and **max.damage** described in details in the previous chapter.

4.4 Direct import of SIMS and Monte Carlo profile

4.4.1 Method

CSuprem offers a new feature to allow the importation of experimental or Monte Carlo generated doping profile for one or more materials. In the case of device with multiple materials the program automatically takes care of dose matching between different materials, whether they are from SIMS or from analytical models.

One may think of the use of SIMS as simply replacing the Pearson functions in the longitudinal direction.

4.4.2 SIMS file format

The most simple format is for a series of SIMS or Monte Carlo (MC) profile with all the incident angle at zero. If the first row is: "#log10", it means the doping concentration result is obtained from log10 operation. First column is the coordinate, second column is the doping concentration.

#log10	
0.135126146809E+00	0.216488754272E+02
0.148602582945E+01	0.212062296623E+02
0.229965627924E+01	0.208093444935E+02
0.391829213638E+01	0.203819863695E+02
0.580538292156E+01	0.199393815353E+02

0.742509690900E+01	0.194662218847E+02
0.904265463584E+01	0.190846652874E+02
0.106609311162E+02	0.186725743390E+02
0.128161174527E+02	0.182299899702E+02
0.149702256588E+02	0.178332071281E+02
0.163153753131E+02	0.176348361724E+02
0.181956379397E+02	0.174823076741E+02
0.198081643918E+02	0.173144915348E+02
0.216877082648E+02	0.171924973877E+02
0.246407124732E+02	0.170247835752E+02
0.289377882057E+02	0.167045003337E+02

4.5 Etching models

4.5.1 Introduction

The resist patterns generated by lithography is transferred into the layer underneath using etching processes [27]. Etching process can be performed using chemical attack, physical damage, or a combination of both.

Chemical or wet etching:

Chemical or wet etches use liquid agents that are applied to the surface of the substrate. The wet etching can be highly selective and generally does not damage the wafer. It has a high reliability and is used for total or partial removal of whole regions. But, it is not suitable for the decreasing structure sizes [27] and it is not successful for anisotropic etches.

Physical or dry etching:

Physical or dry etching is on the opposite a highly anisotropic process and then able to transfer small sizes. Dry etching is based on reactive ion-etch process (RIE), plasma etch processes (PE), and reactive ion-beam etching (RIBE) in some applications [32].

An accurate simulator should include the main effects present during an etching process. These effects can be:

- Chemical enhanced sputtering
- damage induced etching
- sidewall passivation
- burrowing effect [27], [43]

Three-dimensional etching and deposition simulators are motivated by the 3D-effects as *proximity effect* [39],[40], [41], [32].

4.5.2 Geometrical etching

In the actual version of Csuprem, the etching process simulation is only geometrical. The injection of defects and the diffusion of impurities due to the physical etching are not included. In the same way, the shape of the etched region is not simulated.

The user should specify the shape of the etched region using the **etch** command and its parameters as was described in the previous chapter.

4.5.3 Original etching algorithms

CSuprem implements three different algorithms for the etching function. The default of **etch.type** is 3.

The original design in Suprem4.gs assumes a relatively simple region geometry. When region geometry becomes more complicated, the original etch function may have trouble handling the complex mesh system. This usually happens in dry etching of SiO_2 of complicated geometry. We use two examples of try etch to illustrate typical situations that the original etching method may fail.

Figure 4.2 shows a mesh layer (SiO_2) deposited on a structure with three stairs. A dry etch operation intends to take away the area above the red line. The end result will be two isolated regions left over by the dry etch as indicated by the arrows.

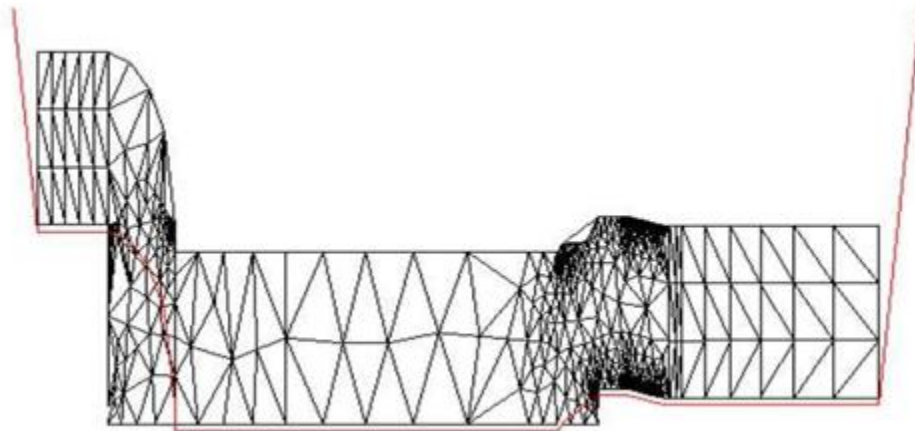


Figure 4. 2: A case zigzag shaped layer being dry etched, resulting in two isolated regions. The red line is the etch line.

Figure 4.3 shows another example that the original method may fail. A layer of SiO_2 is deposited on a structure is large step. A dry etch intends to take away the area above the red line and will leave behind two isolated regions.

Therefore a common mesh problem (causing simulator to crash) comes from the original etch

function which has difficulty in handing material region of zigzag shapes that will leave behind multiple isolated regions.

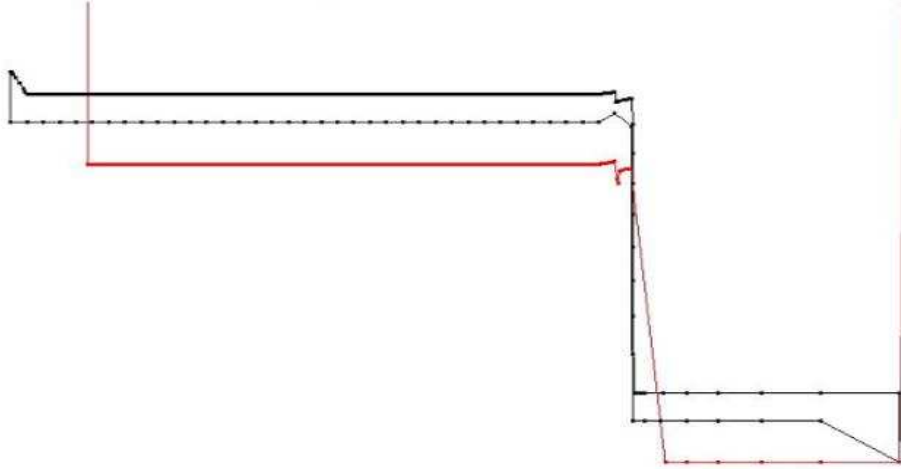


Figure 4. 3: A layered material with a large step being dry etched, resulting in two isolated regions. The red line is the etch line. Such a case will cause the original Suprem4.gs to fail.

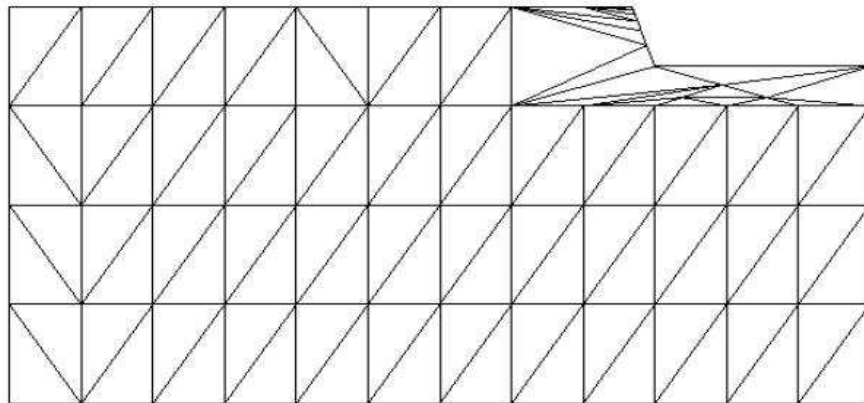


Figure 4. 4: Schematic showing the type 1 etch where the affected area is being re-meshed, causing large change in mesh structures.

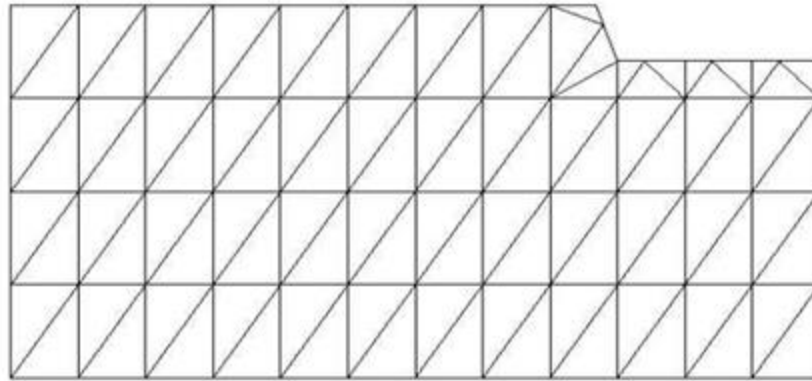


Figure 4. 5: Schematic showing the new type 2 etch where the affected area uses the original mesh lines wherever possible.

4.5.4 New etching methods

Two new type of etch functions have been implemented by Crosslight. For `etch.type=1`, we borrow the basic idea of finding the difference between the original region and the region to be etched away. Then, the region left behind will be re-meshed. For this new design, we ensure the problem of multiple region is resolved before re-meshing.

Both `etch.type=0` and `etch.type=1` take the similar steps of firstly finding the new region after etch, then all original mesh were discarded and new mesh is created to fit the new boundary after etch. One disadvantage is that the mesh after etch may be vastly different than the one before and thus makes it difficult to control the mesh density distribution. Please see Figure 4.4.

To achieve better mesh control a new etch type, `type=2`, has been created. It is based on the following design. Consider all the triangles within a region to be etched. If a whole triangle is within the etch region, it will be discarded. If a whole triangle is outside of the etch region, it will be left intact. If the etch line intercepts with a triangle, find the new triangle after intercept. In case the intercepted polygon becomes polygon than than triangle, further division is performed to generate two or more new triangles. If the points of the new triangles are close to the etch line, move them to the etch line to avoid tiny triangles. Please see Figures 4.4 and 4.5 for an explanation of the two new etch function designs.

Using similar method as in `type2`, `etch.type type3` makes sure the triangles outside etch region stay the same. The method is as following: If the entire triangle is within the etch region, it should be eliminated. If the entire triangle is outside the etch region, it will be kept. All the triangles that cross over the etch region need to be recorded and rearranged. First built different polygons from the meshes on the etch region and outside etch region and then reshape these polygons to triangles. These triangles will then be added into the region. The difference between `type3` and `type2` is how to deal with the triangles cross over the etch regions. Use option command to switch between etch types, e.g. `option etch.type=3`.

4.6 Stress Analysis in Csuprem

4.6.1 2D stress equations

The elastic stress model is based on the Newton's second law of motion governing deformation:

$$\rho a_i = \frac{\partial \sigma_{ij}}{\partial x_j} + b_i \quad \text{in } \Omega \quad 4.27$$

where ρ is the density of the body, a_i is the acceleration, σ_{ij} is the local stress tensor, and b_i is the body force (e.g. gravitational or electromagnetic forces). If we assume negligible body force and acceleration, we get:

$$\frac{\partial \sigma_{ij}}{\partial x_j} = 0 \quad \text{in } \Omega \quad 4.28$$

$$[\sigma_{ij}] = \sigma = \begin{pmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{xy} \end{pmatrix}$$

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} = 0 \quad \text{in } \Omega \quad 4.29$$

$$\frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} = 0 \quad \text{in } \Omega \quad 4.30$$

We notice here that we have 3 unknowns and 2 equations. To solve these equations we use:

1) Hookean's elastic law that expresses the relation between stress σ and strain ε :

$$\sigma = D(\varepsilon - \varepsilon_0) + \sigma_0 \quad 4.31$$

where

$$\varepsilon = \begin{pmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \varepsilon_{xy} \end{pmatrix}$$

ε_0 is the initial strain. σ_0 is the initial stress. The stiffness tensor D for isotropic elastic material is given by:

$$D = \begin{pmatrix} c_{11} & c_{12} & 0 \\ c_{12} & c_{11} & 0 \\ 0 & 0 & c_{44} \end{pmatrix}$$

where

$$c_{11} = \frac{E(1-\nu)}{(1+\nu)(1-2\nu)}$$

$$c_{12} = \frac{E\nu}{(1+\nu)(1-2\nu)}$$

$$c_{44} = \frac{E}{1+\nu}$$

In Csuprem, c_{44} is given by:

$$c_{44} = \frac{E}{2(1+\nu)}$$

and E is the Young's Modulus, and ν is the Poisson's ratio. These constants are stored in the array `matco[]` and we have:

$$matco[0] = c_{11}, matco[1] = c_{12}, matco[2] = c_{44}$$

, and the C-function calculating these constants is : *plane - strain*($E, \nu, matco$)

Remark1: In Csuprem code ε_0 holds the thermal stress due to a change in temperature. The thermal stress model is included using the parameter $lcte = a * T + b$ in the command *material* as:

$$material \quad lcte = 2 * T + 3.0 \quad intrin.sig = 1.4e^{10} \quad nitride$$

$$eps[nitride][XX] = eps[nitride][YY] = \int_{temp1}^{temp2} (a * T + b) dT$$

and the temperature change is set using the command *stress* as:

$$stress \quad temp1 = 600 \quad temp2 = 1000$$

The initial stress σ_0 corresponds to the intrinsic stress given by *intrin.sig* paramter. In Csuprem code, we exactly have:

$$istress[nitride][XX] = FEnd[i] \rightarrow sig[0] = 1.4e^{10}$$

Remark2: The command *stress* is the command that turns on the stress calculation.

2) The relation between strain and displacement $V = \begin{pmatrix} V_x \\ V_y \end{pmatrix}$:

$$\varepsilon_{xx} = \frac{\partial V_x}{\partial x} \tag{4.32}$$

$$\varepsilon_{yy} = \frac{\partial V_y}{\partial y} \tag{4.33}$$

$$\varepsilon_{xy} = \frac{\partial v_y}{\partial x} + \frac{\partial v_x}{\partial y} \quad 4.34$$

By using Hooke's law, we get:

$$\sigma_{xx} = c_{11} * \frac{\partial v_x}{\partial x} + c_{12} * \frac{\partial v_y}{\partial y} + \sigma_{0,xx} \quad 4.35$$

$$\sigma_{yy} = c_{12} * \frac{\partial v_x}{\partial x} + c_{11} * \frac{\partial v_y}{\partial y} + \sigma_{0,yy} \quad 4.36$$

$$\sigma_{xy} = c_{44} * \frac{\partial v_x}{\partial y} + c_{44} * \frac{\partial v_y}{\partial x} + \sigma_{0,xy} \quad 4.37$$

Now if we take the derivatives, we get:

$$\frac{\partial \sigma_{xx}}{\partial x} = c_{11} * \frac{\partial^2 v_x}{\partial x^2} + c_{12} * \frac{\partial^2 v_y}{\partial x \partial y} + \frac{\partial \sigma_{0,xx}}{\partial x} \quad 4.38$$

$$\frac{\partial \sigma_{yy}}{\partial y} = c_{12} * \frac{\partial^2 v_x}{\partial y \partial x} + c_{11} * \frac{\partial^2 v_y}{\partial y^2} + \frac{\partial \sigma_{0,yy}}{\partial y} \quad 4.39$$

$$\frac{\partial \sigma_{xy}}{\partial x} = c_{44} * \frac{\partial^2 v_x}{\partial x \partial y} + c_{44} * \frac{\partial^2 v_y}{\partial x^2} + \frac{\partial \sigma_{0,xy}}{\partial x} \quad 4.40$$

$$\frac{\partial \sigma_{xy}}{\partial y} = c_{44} * \frac{\partial^2 v_x}{\partial y^2} + c_{44} * \frac{\partial^2 v_y}{\partial y \partial x} + \frac{\partial \sigma_{0,xy}}{\partial y} \quad 4.41$$

By applying the Newton's second law, we get:

$$c_{11} * \frac{\partial^2 v_x}{\partial x^2} + c_{44} * \frac{\partial^2 v_x}{\partial y^2} + c_{12} * \frac{\partial^2 v_y}{\partial x \partial y} + c_{44} * \frac{\partial^2 v_y}{\partial y \partial x} = -bx \quad 4.42$$

$$c_{12} * \frac{\partial^2 v_x}{\partial y \partial x} + c_{44} * \frac{\partial^2 v_x}{\partial x \partial y} + c_{44} * \frac{\partial^2 v_y}{\partial x^2} + c_{11} * \frac{\partial^2 v_y}{\partial y^2} = -by \quad 4.43$$

$$bx = \frac{\partial \sigma_{0,xx}}{\partial x} + \frac{\partial \sigma_{0,xy}}{\partial y} \quad 4.44$$

$$by = \frac{\partial \sigma_{0,yy}}{\partial y} + \frac{\partial \sigma_{0,xy}}{\partial x} \quad 4.45$$

The equations solved in Csuprem to get the stress distribution from displacement are exactly those above (4.42)-(4.45).

Remark1: The variables that are important for us are the derivatives of the displacement and not the displacement itself.

4.6.2 Finite Element Review

We can summarize the approximation methods to 5 methods:

- 1-Finite element method
- 2-Finite box method
- 3-Finite difference method
- 4-Spectral method
- 5-Wavelet method

The finite element method is just an approximation method based on polynomials.

The first advantage of this method is that the user has the freedom to choose the degree of the polynomials. This choice will depend on the order of the problem to solve and the accuracy needed by the user. Suppose we want to solve the following Poisson's equation with Dirichlet and Neumann boundary conditions:

Find $u(x, y) \in V(\Omega)$ such that:

$$\Delta u = f \quad \text{in } \Omega \quad 4.46$$

$$\nabla u \cdot n = 0 \quad \text{in } \partial\Omega_N \quad 4.47$$

$$u = 0 \quad \text{in } \partial\Omega_D \quad 4.48$$

where $V(\Omega)$ is a set of regular functions. For this problem it is chosen to be the space of distributions $H_0^1(\Omega)$. We have no need to elaborate on its definition. Let's forget it for now.

If we multiply the first equation in the above system by any **shape function** $v \in V(\Omega)$ and we integrate over Ω , then we get:

$$\int_{\Omega} \Delta u \cdot v \, dx \, dy = \int_{\Omega} f \cdot v \, dx \, dy \quad \forall v \in V(\Omega) \quad 4.49$$

By using an integration by part, the problem to solve is now:

Find $u \in V(\Omega)$ such that:

$$-\int_{\Omega} \nabla u \cdot \nabla v \, dx \, dy + \int_{\partial\Omega_N} \nabla u \cdot n v \, d\sigma + \int_{\partial\Omega_D} \nabla u \cdot n v \, d\sigma = \int_{\Omega} f \cdot v \, dx \, dy \quad \forall v \in V(\Omega) \quad 4.50$$

Since both Dirichlet and Neumann boundary conditions are homogeneous the integrals on the boundary are null. Then, we obtain:

Find $u \in V(\Omega)$ such that:

$$-\int_{\Omega} \nabla u \cdot \nabla v dx dy = \int_{\Omega} f \cdot v dx dy \quad \forall v \in V(\Omega) \quad 4.51$$

4.6.3 Finite Element Discretization

The basic ideas of finite element discretization are summarized in the following 2 points:

1) The domain Ω is discretized or decomposed into a finite set of elements. In 2D, these elements may be triangles T , rectangles, or squares. Assume we are using a decomposition based on triangles. Let Ω_h be the union of these triangles (i.e. $\Omega_h = \cup T$) where h is the smallest edge (or diameter) in this decomposition. Then Ω is approximated by Ω_h (i.e. $\Omega = \text{Limit}_{h \rightarrow 0} \Omega_h$).

2) The space $V(\Omega)$ is approximated by the discrete space $V_h(\Omega_h)$ (i.e. $V(\Omega) = \text{Limit}_{h \rightarrow 0} V_h(\Omega_h)$).

Let u_h be an approximation of u (i.e. $u = \text{Limit}_{h \rightarrow 0} u_h$) and v_h be an approximation of v (i.e. $v = \text{Limit}_{h \rightarrow 0} v_h$). Then, the discrete problem to solve is:

Find $u_h \in V_h(\Omega_h)$ such that:

$$-\int_{\Omega_h} \nabla u_h \cdot \nabla v_h dx dy = \int_{\Omega_h} f_h \cdot v_h dx dy \quad \forall v_h \in V_h(\Omega_h) \quad 4.52$$

To define our finite element approximation, we just need to define the discrete space $V_h(\Omega_h)$. In finite element, this discrete space is simply defined as a set of functions v_h which are:

- a) smooth on the whole space Ω_h
- b) polynomials of degree k on each element (triangle in our case)

The discrete space $V_h(\Omega_h)$ is a vectorial space of finite dimension n , where n is smaller or equal to the number of mesh points. Let $(\omega_0, \omega_1, \dots, \omega_n)$ be the basis functions of $V_h(\Omega_h)$. These basis functions are defined by the following:

$$\omega_i(x_i, y_i) = 1 \quad \forall i = 0, \dots, n-1$$

$$\omega_i(x_j, y_j) = 0 \quad \forall i = 0, \dots, n-1, j = 0, \dots, n-1, i \neq j$$

We should note that each basis function $\omega_i(x, y)$ is associated with a given node (x_i, y_i) and is a polynomial of degree k on each triangle T .

Remark2:

The additional work that we have to do in finite element method compared to finite box method is the calculation of the basis functions $(\omega_i)_{i=0, n-1}$ and their derivatives.

The unknown u_h is just a linear combination of the basis functions $(\omega_i(x, y))_{i=0, n-1}$:

$$u_h = \sum_{i=0}^{n-1} u_h^i \cdot \omega_i(x, y)$$

where u_h^i is given by:

$$u_h^i = u_h(x_i, y_i), i = 0, \dots, n-1$$

Finally, the constants $u_h^i, i = 0, \dots, n-1$ are the unknowns that we are looking for. If we choose the shape function v_h to be any basis function $\omega_j, j = 0, \dots, n-1$, then the discrete problem to solve can be written as a linear system:

Find the vector $u_h = (u_h^0, u_h^1, \dots, u_h^{n-1})$ such that:

$$-\sum_{i=0}^{n-1} u_h^i \int_{\Omega_h} \nabla \omega_i \cdot \nabla \omega_j dx dy = \int_{\Omega} f_h \cdot \omega_j dx dy \quad \forall j = 0, \dots, n-1 \quad 4.53$$

Remark3:

The index j represents the row index and i represents the column index of the associated matrix.

To write the above linear system in a matrix form, we simply set:

$$A = (A_{JI}), A_{JI} = \int_{\Omega_h} \nabla \omega_i \cdot \nabla \omega_j dx dy$$

$$B = (B_j), B_j = \int_{\Omega} f_h \cdot \omega_j dx dy$$

then we have to solve:

Find the vector u_h such that:

$$A * u_h = B \quad 4.54$$

Let's assume as in Csuprem that the degree of approximation k in this finite element discretization is 2. Then each basis function w_i on each triangle T is a polynomial of degree 2:

$$\omega_i(x, y) = a_0 + a_1 * x + a_2 * y + a_3 * x^2 + a_4 * xy + a_5 * y^2$$

To determinate this basis function, we need to calculate 6 constants $(a_0, a_1, a_2, a_3, a_4, a_5)$. To do this we define 6 points (or nodes) on each triangle. We use the 3 corner points (x_0, y_0) , (x_1, y_1) , and (x_2, y_2) of each triangle T defined by the mesh and we add 3 middle edge points (x_3, y_3) , (x_4, y_4) , and (x_5, y_5) .

We set:

$$a_{ji}^T = \int_T \nabla \omega_i \cdot \nabla \omega_j dx dy \quad \forall (i,j) = 0,5$$

a_{ji}^T is the Stiffness matrix. The global matrix A_{JI} is then given by

$$A_{JI} = \sum_T a_{ji}^T$$

The global right hand side B_J is given by:

$$B_J = \sum_T b_j^T, \quad b_j^T = \int_T f_h \cdot \omega_j dx dy \quad \forall j = 0,5$$

The work we need to do in any finite element method is to calculate the 2 integrals a_{ji}^T and b_j^T on a given triangle T .

People use 2 different ways to calculate these integrals:

- 1) Some people calculate these integrals directly on the current triangle T .
- 2) Some people use a linear transformation to map a current element T defined with global coordinate (x,y) into a standard element $\hat{T}$ defined in local coordinates (ξ,η) . The corner points of this standard triangle are $(0,0)$, $(1,0)$, and $(0,1)$. Then these integrals are calculated on the standard element $\hat{T}$. The basis functions and their global derivatives are then calculated in a very simple way on standard $\hat{T}$. In Csuprem, we use this transformation.

Let's express the linear transformation that mappes local coordinates (ξ,η) into the global coordinates (x,y) . In this mapping, x is a linear function of (ξ,η) and y is a linear function of (ξ,η) . By the rules of partial differential equations, the derivatives of the basis functions have the form:

$$\frac{\partial \omega_i}{\partial \xi} = \frac{\partial \omega_i}{\partial x} * \frac{\partial x}{\partial \xi} + \frac{\partial \omega_i}{\partial y} * \frac{\partial y}{\partial \xi} \quad 4.55$$

$$\frac{\partial \omega_i}{\partial \eta} = \frac{\partial \omega_i}{\partial x} * \frac{\partial x}{\partial \eta} + \frac{\partial \omega_i}{\partial y} * \frac{\partial y}{\partial \eta} \quad 4.56$$

In matrix notation, we have:

$$\begin{pmatrix} \frac{\partial \omega_i}{\partial \xi} \\ \frac{\partial \omega_i}{\partial \eta} \end{pmatrix} = \begin{pmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} \end{pmatrix} \cdot \begin{pmatrix} \frac{\partial \omega_i}{\partial x} \\ \frac{\partial \omega_i}{\partial y} \end{pmatrix} = J \cdot \begin{pmatrix} \frac{\partial \omega_i}{\partial x} \\ \frac{\partial \omega_i}{\partial y} \end{pmatrix}$$

The derivatives of the basis function with respect to global coordinates are now given by:

$$\begin{pmatrix} \frac{\partial \omega_i}{\partial x} \\ \frac{\partial \omega_i}{\partial y} \end{pmatrix} = J^{-1} \cdot \begin{pmatrix} \frac{\partial \omega_i}{\partial \xi} \\ \frac{\partial \omega_i}{\partial \eta} \end{pmatrix}$$

So, we need to calculate the inverse of J . In 2D, or even in 3D, this is a simple task. In Csuprem, this is calculated in the function `tri6_shape(s,t, cord,xsj,shp)` defined in the file `tri6.c`. This function calculates:

- a- The shape functions $\omega_i(\xi, \eta)_{i=0,5}$ on the standard triangle
- b- their derivatives with respect to ξ and η
- c- The Jaccobian matrix J , its determinant $\det(J)$ and its inverse J^{-1}

The stiffness matrices are calculated by the function:

$$\text{tri6_stiff}(\text{stiff}, \text{wrhs}, \text{xl}, \text{disp}, \text{coeff}, \text{mat})$$

defined in the same file `tri6.c`.

Now we will use this transformation as was exactly done in Csuprem to calculate the integrals involved in the stiffness matrix a_{ji}^T and in the right hand side b_j^T . Let's set:

$$\omega_i(\mathbf{x}(\xi, \eta), \mathbf{y}(\xi, \eta)) = \widehat{\omega}_i(\xi, \eta)$$

The basis functions $\widehat{\omega}_i(\xi, \eta)$ are defined on the standard triangle $\widehat{T}$.

Now, we have:

$$\int_T \begin{pmatrix} \frac{\partial \omega_i}{\partial x} \\ \frac{\partial \omega_i}{\partial y} \end{pmatrix} \cdot \begin{pmatrix} \frac{\partial \omega_j}{\partial x} \\ \frac{\partial \omega_j}{\partial y} \end{pmatrix} dx dy = \int_{\widehat{T}} J^{-1} \begin{pmatrix} \frac{\partial \widehat{\omega}_i}{\partial \xi} \\ \frac{\partial \widehat{\omega}_i}{\partial \eta} \end{pmatrix} \cdot J^{-1} \begin{pmatrix} \frac{\partial \widehat{\omega}_j}{\partial \xi} \\ \frac{\partial \widehat{\omega}_j}{\partial \eta} \end{pmatrix} \parallel \det(J) \parallel d\xi d\eta$$

The same transformation is used for the right hand side. In Csuprem, the above integral on $\widehat{T}$ is calculated using Gauss numerical integration with a constant weight ω_g ($\omega_g = 0.16666$) and 3 points defined on the middle of each edge of $\widehat{T}$ (i.e: (0.5,0), (0.5,0.5), (0,0.5)).

In Csuprem, we use a second order finite element approximation. This means we are using a quadratic interpolation in Csuprem. Then this numerical integration is not exact. In case of linear interpolation this Gauss numerical integration is exact.

We should note that :

$$\int_{\widehat{T}} = \int_0^1 \int_0^1$$

Now let's calculate explicitly the basis functions $\widehat{\omega}_i(\xi, \eta)$ and their derivatives as it was exactly

done in Csuprem. The basis functions $\widehat{\omega}_i(\xi, \eta)$ are defined in the same way as $\omega_i(x, y)$. Then, we have:

$$\widehat{\omega}_i(\xi, \eta) = a0 + a1 * \xi + a2 * \eta + a3 * \xi^2 + a4 * \xi\eta + a5 * \eta^2 \quad 4.57$$

$$\widehat{\omega}_i(\xi_i, \eta_i) = 1 \quad \text{and} \quad \widehat{\omega}_i(\xi_j, \eta_j) = 0 \quad \forall (i, j) = 0, 5 \quad i \neq j \quad 4.58$$

Let's put as in Csuprem in the C-function *tri6_shape*(....):

$$lam0 = 1 - \xi - \eta \quad 4.59$$

$$lam1 = \xi \quad 4.60$$

$$lam2 = \eta \quad 4.61$$

By solving the linear system of 6 equations and 6 unknowns given in the equation (4.58), we get:

$$\widehat{\omega}_0(\xi, \eta) = lam0 * (2 * lam0 - 1) \quad 4.62$$

$$\widehat{\omega}_1(\xi, \eta) = lam1 * (2 * lam1 - 1) \quad 4.63$$

$$\widehat{\omega}_2(\xi, \eta) = lam2 * (2 * lam2 - 1) \quad 4.64$$

$$\widehat{\omega}_3(\xi, \eta) = 4 * lam1 * lam2 \quad 4.65$$

$$\widehat{\omega}_4(\xi, \eta) = 4 * lam2 * lam0 \quad 4.66$$

$$\widehat{\omega}_5(\xi, \eta) = 4 * lam0 * lam1 \quad 4.67$$

The derivatives of the basis functions as in Csuprem are given exactly by the following:

$$\frac{\partial \widehat{\omega}_0(\xi, \eta)}{\partial \xi} = -1 * (4 * lam0 - 1) \quad 4.68$$

$$\frac{\partial \widehat{\omega}_0(\xi, \eta)}{\partial \eta} = -1 * (4 * lam0 - 1) \quad 4.69$$

$$\frac{\partial \widehat{\omega}_1(\xi, \eta)}{\partial \xi} = 4 * lam0 - 1 \quad 4.70$$

$$\frac{\partial \widehat{\omega}_1(\xi, \eta)}{\partial \eta} = 0 \quad 4.71$$

$$\frac{\partial \widehat{\omega}_2(\xi, \eta)}{\partial \xi} = 0 \quad 4.72$$

$$\frac{\partial \widehat{\omega}_2(\xi, \eta)}{\partial \eta} = 4 * lam2 - 1 \quad 4.73$$

$$\frac{\partial \widehat{\omega}_3(\xi, \eta)}{\partial \xi} = 4 * \eta \quad 4.74$$

$$\frac{\partial \widehat{\omega}_3(\xi, \eta)}{\partial \eta} = 4 * \xi \quad 4.75$$

$$\frac{\partial \widehat{\omega}_4(\xi, \eta)}{\partial \xi} = -4 * \eta \quad 4.76$$

$$\frac{\partial \widehat{\omega}_4(\xi, \eta)}{\partial \eta} = 4 * (1 - \xi - 2 * \eta) \quad 4.77$$

$$\frac{\partial \widehat{\omega}_5(\xi, \eta)}{\partial \xi} = 4 * (1 - 2 * \xi - \eta) \quad 4.78$$

$$\frac{\partial \widehat{\omega}_5(\xi, \eta)}{\partial \eta} = -4 * \xi \quad 4.79$$

4.6.4 Application of finite element theory to solve displacement equations

Now let's apply the above theory to the equations of the displacements given by (4.42)-(4.45) that we need to solve. First, we should note that we are dealing with a vectorial problem where the unknown is the vector $V = \begin{pmatrix} V_x \\ V_y \end{pmatrix}$. Then we have two degrees of freedom on each node and the local basis functions $W_{I=0,11}$ for this vectorial problem can be defined as: $W_I = \begin{pmatrix} \omega_{i=0,5}^1 \\ \omega_{i=0,5}^2 \end{pmatrix} \quad \forall I = 0,11$. If we multiply the equations (4.42) and (4.43) by a shape function $W_{JG}, JG = 0, \dots, 2 * nn$, integrate over Ω_h , and use an integration by part, then we get:

$$\begin{pmatrix} \sum_T a_{IJ}^T & \sum_T a_{I(J+1)}^T \\ \sum_T a_{(I+1)J}^T & a_{(I+1)(J+1)}^T \end{pmatrix} \cdot \begin{pmatrix} VX \\ VY \end{pmatrix} = \begin{pmatrix} \sum_T b_x^T \\ \sum_T b_y^T \end{pmatrix}, \quad \forall (I, J) = 0, 11$$

where

$$VX = \begin{pmatrix} V_x^0 \\ V_x^1 \\ \cdot \\ \cdot \\ V_x^{nn-1} \end{pmatrix}, \quad VY = \begin{pmatrix} V_y^0 \\ V_y^1 \\ \cdot \\ \cdot \\ V_y^{nn-1} \end{pmatrix}$$

We should remind that nn is the total number of finite element points. The total number of finite element points is the total number of the mesh points plus the extra points that we add on the middle of each edge in the case of second order interpolation as in Csuprem. The points and the nodes are similar in the approximation used in Csuprem.

The stiffness matrices a_{IJ}^T and $a_{(I+1)J}^T$ are associated with the component V_x and $a_{I(J+1)}^T$ and $a_{(I+1)(J+1)}^T$ are associated with V_y . These stiffness matrices are given by:

$$a_{IJ}^T = \int_T \left(c_{11} * \frac{\partial \omega_i^1}{\partial x} * \frac{\partial \omega_j^1}{\partial x} + c_{44} * \frac{\partial \omega_i^1}{\partial y} * \frac{\partial \omega_j^1}{\partial y} \right) dx dy \quad 4.80$$

$$a_{I(J+1)}^T = \int_T \left(c_{12} * \frac{\partial \omega_i^2}{\partial y} * \frac{\partial \omega_j^2}{\partial x} + c_{44} * \frac{\partial \omega_i^2}{\partial x} * \frac{\partial \omega_j^2}{\partial y} \right) dx dy \quad 4.81$$

$$a_{(I+1)J}^T = \int_T \left(c_{12} * \frac{\partial \omega_i^1}{\partial x} * \frac{\partial \omega_j^1}{\partial y} + c_{44} * \frac{\partial \omega_i^1}{\partial y} * \frac{\partial \omega_j^1}{\partial x} \right) dx dy \quad 4.82$$

$$a_{(I+1)(J+1)}^T = \int_T \left(c_{44} * \frac{\partial \omega_i^2}{\partial x} * \frac{\partial \omega_j^2}{\partial x} + c_{11} * \frac{\partial \omega_i^2}{\partial y} * \frac{\partial \omega_j^2}{\partial y} \right) dx dy \quad 4.83$$

As in Csuprem, the right hand side b_x^T and b_y^T are given by:

$$b_x^T = - \int_T \left(\frac{\partial \sigma_{0,xx}}{\partial x} + \frac{\partial \sigma_{0,xy}}{\partial y} \right) \cdot \omega_j^1 dx dy \quad 4.84$$

$$b_y^T = - \int_T \left(\frac{\partial \sigma_{0,xy}}{\partial x} + \frac{\partial \sigma_{0,yy}}{\partial y} \right) \cdot \omega_j^2 dx dy \quad 4.85$$

In Csuprem, the stiffness matrices are explicitly calculated using the transformation J and the Gauss numerical integration with 3 points on the middle of each edge. In Csuprem, the stiffness matrix a_{IJ}^T and the right hand side b_x^T for example are explicitly calculated by:

$$a_{IJ}^T = \int_0^1 \int_0^1 \left(c_{11} * \frac{\partial \omega_i^1}{\partial x} * \frac{\partial \omega_j^1}{\partial x} + c_{44} * \frac{\partial \omega_i^1}{\partial y} * \frac{\partial \omega_j^1}{\partial y} \right) \parallel \det(J) \parallel d\xi d\eta \quad 4.86$$

$$= \sum_{k=0}^3 \left(c_{11} * \frac{\partial \omega_i^1}{\partial x} * \frac{\partial \omega_j^1}{\partial x} + c_{44} * \frac{\partial \omega_i^1}{\partial y} * \frac{\partial \omega_j^1}{\partial y} \right) (\xi_k, \eta_k) \parallel \det(J) \parallel \omega_g \quad 4.87$$

$$b_x^T = \int_0^1 \int_0^1 \left(\frac{\partial \sigma_{0,xx}}{\partial x} + \frac{\partial \sigma_{0,xy}}{\partial y} \right) \cdot \omega_j^1 \parallel \det(J) \parallel d\xi d\eta \quad 4.88$$

$$= \sum_{k=0}^3 \left(\left(\frac{\partial \sigma_{0,xx}}{\partial x} + \frac{\partial \sigma_{0,xy}}{\partial y} \right) \cdot \omega_j^1 \right) (\xi_k, \eta_k) \parallel \det(J) \parallel \omega_g \quad 4.89$$

The other stiffness matrices and the right hand side are calculated in the same way.

4.6.5 Dirichlet boundary condition

In Csuprem, we have two types of boundaries. The exposed boundary and the reflected boundary. The Dirichlet boundary conditions for displacement are set on the reflected boundary and are given by:

$V_x = 0$ on edge ej if V_x is perpendicular to ej and $V_y = 0$ on edge ej if V_y is perpendicular to ej .

4.6.6 Neumann boundary condition

In the stiffness matrices presented above for displacement, all the terms on the boundary $\partial\Omega_h$ due to the integration by part are missing. We then assume that an homogeneous Neumann boundary condition is used on the exposed and the reflected boundaries. The terms on the boundary due to the integration by part are the following:

$$\int_{\partial\Omega_h} c_{11} \frac{\partial V_x}{\partial x} \cdot n_x \cdot \omega_j^1 d\sigma, \int_{\partial\Omega_h} c_{44} \frac{\partial V_x}{\partial y} \cdot n_y \cdot \omega_j^1 d\sigma, \int_{\partial\Omega_h} c_{12} \frac{\partial V_y}{\partial y} \cdot n_x \cdot \omega_j^2 d\sigma, \int_{\partial\Omega_h} c_{44} \frac{\partial V_y}{\partial x} \cdot n_y \cdot \omega_j^2 d\sigma$$

$$\int_{\partial\Omega_h} c_{12} \frac{\partial V_x}{\partial x} \cdot n_y \cdot \omega_j^1 d\sigma, \int_{\partial\Omega_h} c_{44} \frac{\partial V_x}{\partial y} \cdot n_x \cdot \omega_j^1 d\sigma, \int_{\partial\Omega_h} c_{44} \frac{\partial V_y}{\partial x} \cdot n_x \cdot \omega_j^2 d\sigma, \int_{\partial\Omega_h} c_{11} \frac{\partial V_y}{\partial y} \cdot n_y \cdot \omega_j^2 d\sigma$$

4.6.7 3D Stress equations in Csuprem

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + \frac{\partial \sigma_{zx}}{\partial z} = 0 \quad 4.90$$

$$\frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + \frac{\partial \sigma_{yz}}{\partial z} = 0 \quad 4.91$$

$$\frac{\partial \sigma_{zx}}{\partial x} + \frac{\partial \sigma_{yz}}{\partial y} + \frac{\partial \sigma_{zz}}{\partial z} = 0 \quad 4.92$$

$$\sigma = [\sigma_{ij}] = \begin{pmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \\ \sigma_{yz} \\ \sigma_{zx} \end{pmatrix}$$

$$\sigma = D(\varepsilon - \varepsilon_0) + \sigma_0 \quad 4.93$$

where

$$\boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \varepsilon_{zz} \\ \varepsilon_{xy} \\ \varepsilon_{yz} \\ \varepsilon_{zx} \end{pmatrix}$$

$$\mathbf{D} = \begin{pmatrix} c_{11} & c_{12} & c_{12} & 0 & 0 & 0 \\ c_{12} & c_{11} & c_{12} & 0 & 0 & 0 \\ c_{12} & c_{12} & c_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & c_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & c_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & c_{44} \end{pmatrix}$$

$$\varepsilon_{xx} = \frac{\partial V_x}{\partial x} \quad 4.94$$

$$\varepsilon_{yy} = \frac{\partial V_y}{\partial y} \quad 4.95$$

$$\varepsilon_{zz} = \frac{\partial V_z}{\partial z} \quad 4.96$$

$$\varepsilon_{xy} = \frac{\partial V_y}{\partial x} + \frac{\partial V_x}{\partial y} \quad 4.97$$

$$\varepsilon_{yz} = \frac{\partial V_z}{\partial y} + \frac{\partial V_y}{\partial z} \quad 4.98$$

$$\varepsilon_{zx} = \frac{\partial V_z}{\partial x} + \frac{\partial V_x}{\partial z} \quad 4.99$$

$$c_{11} * \frac{\partial^2 V_x}{\partial x^2} + c_{44} * \frac{\partial^2 V_x}{\partial y^2} + c_{44} * \frac{\partial^2 V_x}{\partial z^2} + c_{12} * \frac{\partial^2 V_y}{\partial x \partial y} + c_{44} * \frac{\partial^2 V_y}{\partial y \partial x} + c_{12} * \frac{\partial^2 V_z}{\partial x \partial z} + c_{44} * \frac{\partial^2 V_z}{\partial z \partial x} = -a \quad 4.100$$

$$c_{44} * \frac{\partial^2 V_x}{\partial x \partial y} + c_{12} * \frac{\partial^2 V_x}{\partial y \partial x} + c_{44} * \frac{\partial^2 V_y}{\partial x^2} + c_{11} * \frac{\partial^2 V_y}{\partial y^2} + c_{44} * \frac{\partial^2 V_y}{\partial z^2} + c_{12} * \frac{\partial^2 V_z}{\partial y \partial z} + c_{44} * \frac{\partial^2 V_z}{\partial z \partial y} = -b \quad 4.101$$

$$c_{44} * \frac{\partial^2 V_x}{\partial x \partial z} + c_{12} * \frac{\partial^2 V_x}{\partial z \partial x} + c_{44} * \frac{\partial^2 V_y}{\partial y \partial z} + c_{12} * \frac{\partial^2 V_y}{\partial z \partial y} + c_{44} * \frac{\partial^2 V_z}{\partial x^2} + c_{44} * \frac{\partial^2 V_z}{\partial y^2} + c_{11} * \frac{\partial^2 V_z}{\partial z^2} = -c \quad 4.102$$

$$a = \frac{\partial \sigma_{0,xx}}{\partial x} + \frac{\partial \sigma_{0,xy}}{\partial y} + \frac{\partial \sigma_{0,zx}}{\partial z} \quad 4.103$$

$$b = \frac{\partial \sigma_{0,yy}}{\partial y} + \frac{\partial \sigma_{0,xy}}{\partial x} + \frac{\partial \sigma_{0,yz}}{\partial z} \quad 4.104$$

$$c = \frac{\partial \sigma_{0,zx}}{\partial x} + \frac{\partial \sigma_{0,yz}}{\partial y} + \frac{\partial \sigma_{0,zz}}{\partial z} \quad 4.105$$

$$\sigma_{0,xx} = (a_{Si} G_{Si} h_{Si} + a_{Ge} G_{Ge} h_{Ge} h_{Ge} / (G_{Si} h_{Si} + G_{Ge} h_{Ge})) \quad 4.106$$

$$\sigma_{0,yy,i} = a_i [1 - D^i(\sigma_{0,xx}/a_i - 1)]; D = D(c_{ij}, 001, 110, 111) \quad 4.107$$

4.7 Intrinsic stress in SiGe

4.7.1 Si/SiGe interface

Strain engineering is promising for advanced silicon technologies because it may enhance the mobilities of hole and electron. Using SiGe pockets to generate the uniaxial stress along the channel is widely used. In this case the intrinsic stress of SiGe is the component of residual stress due to variations in the deposition process and is dependent on factors such as: deposition rate, thickness and temperature. The intrinsic stress of SiGe film on Si due to lattice mismatch can be estimated as follows.

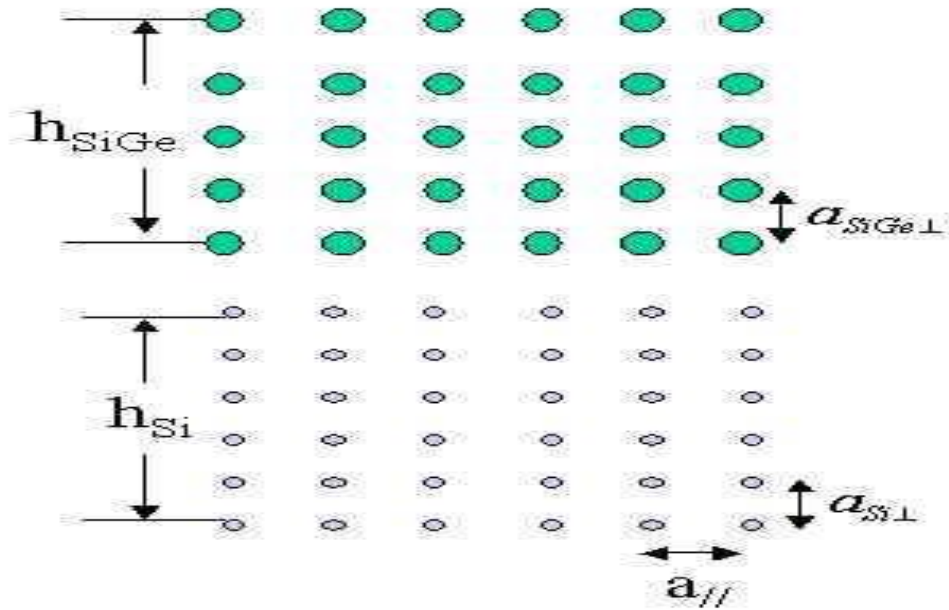


Figure 4. 6: Schematic of Si/SiGe interface.

We suppose $\text{Si}(1-x)\text{Ge}(x)$ thin film with thickness of h_{SiGe} is deposited on Si substrate with thickness of h_{Si} . Strains will be generated due to the mismatch of lattice constants. The strains

in the materials can be determined by minimizing the macroscopic elastic energy, under the constraint that the lattice constant in the plane, $a_{\parallel}$, remains the same throughout the structure. The lattice constants will be denoted by letter a , and the subscript $\parallel$ and $\perp$ are used to indicate lattice spacings parallel or perpendicular to the plane of the interface after strain. Please see Figure 4.6 which we use to refer to uniaxial and biaxial strain/stress and these terms are not to be confused with the uniaxial stress in the MOSFET conduction channel.

Following van de Walle[53], we get the strained lattice constants indicated in Figure 4.6:

$$a_{\parallel} = (a_{Si}G_{Si}h_{Si} + a_{SiGe}G_{SiGe}h_{SiGe})/(G_{Si}h_{Si} + G_{SiGe}h_{SiGe}) \quad 4.108$$

$$a_{i\perp} = a_i[1 - D^i(a_{\parallel}/a_i - 1)] \quad 4.109$$

where i denotes the materials, Si or SiGe, a_i denotes the unstrained lattice constants of the respective materials, and G_i is the shear modulus given by

$$G_i = 2(C_{11}^i + 2C_{12}^i)(1 - D^i/2) \quad 4.110$$

The constant D depends on the elastic constants C_{11} , C_{12} and C_{44} of the respective materials, and on the interface orientation

$$D_{001} = 2(C_{12}/C_{11}) \quad 4.111$$

$$D_{110} = (C_{11} + 3C_{12} - 2C_{44})/(C_{11} + C_{12} + 2C_{44}) \quad 4.112$$

$$D_{111} = 2(C_{11} + 3C_{12} - 2C_{44})/(C_{11} + C_{12} + 4C_{44}) \quad 4.113$$

The ratio of strained $a_{\parallel}$ and $a_{\perp}$ to the unstrained lattice constants determines the strain components parallel and perpendicular to the interface

$$\varepsilon_{i\parallel} = (a_{\parallel}/a_i - 1) \quad 4.114$$

$$\varepsilon_{i\perp} = (a_{\perp}/a_i - 1) \quad 4.115$$

At the horizontal interface between SiGe and Si, we assume the uniaxial stress (perpendicular to the interface in Figure 4.6) σ_{yy}^h is zero and the biaxial stress in SiGe is obtained by

$$\sigma_{xx}^h = (C_{11} + C_{12})\varepsilon_{\parallel} + C_{12}\varepsilon_{\perp} \quad 4.116$$

At the vertical interface shown in Figure 4.7, the biaxial stress σ_{yy}^v is calculated as above, and we assume the uniaxial stress σ_{xx}^v is not zero and can be obtained as

$$\sigma_{xx}^v = E \varepsilon_{\perp} \quad 4.117$$

where E is the Young's modulus.

Finally, the total intrinsic stresses within SiGe due to both horizontal and vertical interface lattice mismatch are

$$\sigma_{xx} = \sigma_{xx}^h + \sigma_{xx}^v \quad 4.118$$

$$\sigma_{yy} = \sigma_{yy}^v \quad 4.119$$

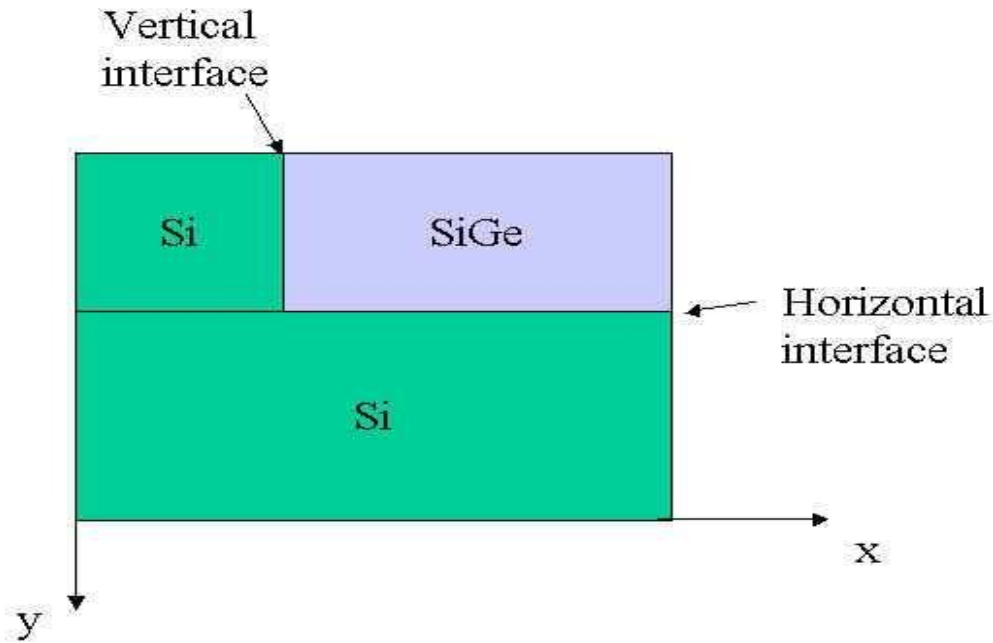


Figure 4. 7: Schematic of a simplified Si/SiGe MOSFET configuration. The SiGe part represents the drain SiGe pocket in the MOSFET.

4.7.2 Stress model for Si/SiGe MOSFET

Referring to a simple configuration in Figure 4.7, we consider the stress distribution in Si/SiGe MOSFET where the source and drain are formed by SiGe pockets which produce uniaxial stress in the Si channel.

According to van de Walle[53], strain can be produced in both Si and SiGe but since the Si part is more massive than the SiGe pockets, the strain within Si is neglected and we only consider

intrinsic strain/stress within SiGe as derived in the previous subsection.

Csuprem solves the stress equations assuming intrinsic stress within SiGe and obtains stress distribution in the whole device. It is worth noting that the uniaxial stress in the Si channel mostly comes from the biaxial strain of the horizontal Si/SiGe interface. Such biaxial strain/stress with respect to the horizontal interface translates into uniaxial stress along the conduction channel and help enhance mobility therein.

4.8 Oxidation models

4.8.1 Introduction

Oxidation is the process of converting silicon on the wafer surface into oxide. The chemical reaction already starts at room temperature, and the resulting very thin oxide film is called native oxide. The oxidation rate can be accelerated by exposing the wafer to oxygen or water vapor at high temperature. The growth of the thermally grown oxide depends on temperature, pressure, crystal orientation, doping level, the impurity contamination of the silicon substrate, and stress due to non-planar wafer structures [27],[2]. The grown oxide is commonly used as insulator between single devices, to act as gate oxide in MOS structures and to serve as mask against dopant implantation [27]. In Csuprem, there are two techniques to get the silicon dioxide (SiO_2) from pure silicon (Si)

- **dry oxidation:** the silicon wafer is settled to a pure oxygen atmosphere at a temperature above 900°C . At the surface of the wafer a chemical reaction



takes place. Because of a very low oxidation velocity ($< 100\text{nm}/h$), this method is often used to generate high quality thin oxide layers [27].

- **wet oxidation:** the silicon wafer is settled to a water vapor atmosphere and the reaction equation



This process has a rather high oxidation velocity ($> 20\text{nm}/min$) and is therefore often used to get thick oxide layers for insulation and passivation techniques.

From oxidation modelling point of view, Csuprem provides both analytical and numerical models. Numerical models treat the oxide as a viscous incompressible fluid [48], [49], [32], [44], [45]. This involves solving the simplified Navier-Stokes equations inside the growing oxide [48],

[49]. The interstitials injection phenomenon (known as oxidation enhanced phenomenon) from oxide to the bulk of silicon [50], [51], segregation of dopants (as segregation of boron to oxide) phenomenon, as well as stress effects are included in Csuprem numerical models.

The main problem encountered in the simulation of the oxidation is the movement of the oxide/silicon interface. This means, for each time step, we solve the equations of the numerical model and then we regenerate the mesh and we recalculate the interface boundaries. This requires an efficient mesh generator and efficient data managements for this huge and frequently data update [52]. From the numerical point of view, the *finite element discretization* is preferred for oxidation simulation. Since, the grid quality can be relaxed compared to the *Voronoi finite box technique*. The new technology involves oxidation of non-rectangular structures [54], [27], then a three-dimensional oxidation simulator is needed to model arbitrary structures and include the three-dimensional effects [32].

4.8.2 2D Analytical models

In Csuprem, the 2D-oxidation models are based on the Deal Grove model [3], [4]. The basic idea of the Deal Grove was the assumption of the steady state situation between three flux:

1- Oxidant's flux

The flux of the oxidant ($O_2, H_2O, \dots$), F_1 , from the bulk of gas to the gas/oxide interface is give by

$$F_1 \cdot n = h(C^* - C^0) \quad 4.122$$

where n is the normal to the interface gas/oxide oriented positively from gas to oxide. C^* is the concentration of the oxidant in the oxide which will be in the equilibrium with the partial pressure in the bulk of the gas. C^0 is the concentration of the oxidant at the oxide surface. h is the gas phase mass transfer coefficient.

2- Diffusion flux

The diffusion flux, F_2 , across the oxide is given by the Fick's law

$$F_2 \cdot n = -D \nabla C \cdot n \quad 4.123$$

where D is the diffusion coefficient of the oxidant in the oxide. C is the actual concentration of the oxidant in the moving oxide layer.

3- Reaction flux

The reaction flux, F_3 , corresponds to the oxidation reaction at the oxide/silicon interface. It is given by

$$F_3 \cdot n = -k_s C^i \quad 4.124$$

where n is the vector normal to the oxide/silicon interface. k_s represents the chemical surface reaction rate. C^i is the oxidant concentration at the interface oxide/silicon [4].

In the steady state condition, these fluxes are identical and can be expressed as:

$$F_1 = F_2 = F_3 = F \quad 4.125$$

The flux of oxidant reaching the oxide/silicon interface is described in 1D by the differential equation developed by Deal Grove:

$$\frac{N_1 dx_{ox}}{dt} = F \quad 4.126$$

where N_1 is the number of oxidant molecules incorporated into a unit volume of oxide. x_{ox} is the growing oxide thickness. $dx_{ox}dt$ represents the oxide growth rate. The solution to the 1D-oxide rate equation is

$$x_{ox}(t) = \sqrt{(A/2 + x_{ox}(0))^2 + Bt} - A/2 \quad 4.127$$

Some times, this equation is written as:

$$\frac{dx_{ox}}{dt} = \frac{B}{A + 2x_{ox}} \quad 4.128$$

with

$$A = 2D \left(\frac{1}{k_s} + \frac{1}{h} \right) \quad 4.129$$

$$B = 2D \frac{C^*}{N_1} \quad 4.130$$

The equation 4.127 is frequently written in slightly different way:

$$x_{ox}(t)^2 + Ax_{ox}(t) = B(t + \tau) \quad 4.131$$

with

$$\tau = \frac{x_{ox}(0)^2 + Ax_{ox}(0)}{B} \quad 4.132$$

B is referred to as the parabolic growth rate coefficient. Since for large t 4.131 approaches

$$x_{ox}(t)^2 + Ax_{ox}(t) = Bt; t \gg \frac{A^2}{4B} \quad 4.133$$

For small time, we observe that BA describes a linear growth rate :

$$x_{ox}(t)^2 = \frac{B}{A}(t + \tau); t \ll \frac{A^2}{4B} - \tau \quad 4.134$$

For thin film, Csuprem includes a correction term to the equation 4.135 as follows:

$$\frac{dx_{ox}}{dt} = \frac{B}{A+2x_{ox}} + C_c \quad 4.135$$

where C_c is given by

$$C_c = \text{thinox.0} e^{\left(\frac{\text{thinox.E}}{KT}\right)} e^{\left(-\frac{x_{ox}}{\text{thinox.l}}\right)} P^{\text{thinox.P}} \quad 4.136$$

where P is the partial pressure. The user can set the parameters thinox.0 , thinox.E , thinox.l using the **oxide** command.

The 2D-analytical model, in Csuprem, is based on the error-function that fits to the bird's beak shapes and is given by [4].

$$x(y, t) = b \cdot x_{ox}(t) \frac{1}{2} \text{erf}\left(\frac{\sqrt{2}(y-a)}{k_1 x_{ox}(t)}\right) \quad 4.137$$

where b is the amount of silicon consumed to produce one unit of oxide. x_{ox} is the oxide thickness as a function of time given by the one dimensional theory of Deal Grove which we have sketched above. a determines the location of the mask edge ($y > a$ is the free surface oxidizing surface); k_1 denotes the ratio of lateral to vertical oxidation and is considered to be a function of the mask thickness.

The analytical models are used for a simple planar structures.

4.8.3 Numerical models

From mathematical point of view, the numerical oxidation models can be described by a coupled system of partial differential equations:

- **diffusion equation:** describing the diffusion of the oxidant in the existing silicon dioxide.
- **chemical reaction equation:** describing the transformation of silicon to silicon-dioxide.
- **displacement equations:** describing the displacement and expansion of the oxide layer and

its neighboring materials. At high temperature, the oxide layer is modelled as an elastic [32], viscoelastic [46], [47], [32], [45] , or viscous compressible or incompressible flow [48], [49], [32], [44], [45]. Due to unknown motion of the interface between silicon and silicon dioxide this leads to a free boundary problem [32],[44],[45].

In Csuprem, the oxide layer is modelled as viscous flow. Three numerical models are provided by Csuprem.

4.8.4 Vertical model

This model does not account for the stress effects on oxidation growth. It can be used only for planar structures where the oxide growth is one dimensional. This model is based on diffusion equation of oxidants in the oxide (SiO_2), and uses the interface fluxes (F_1 , F_3) as boundary conditions. The velocity of the moving boundary SiO_2/Si is calculated at each time step from the oxidant concentration at the interface and surface reaction rate. The equations to be solved are given by:

$$\frac{\partial C}{\partial t} = \text{div}(F) \text{ in } SiO_2 \quad 4.138$$

$$F = -D\nabla C \text{ in } SiO_2 \quad 4.139$$

$$F \cdot n = F_1 \cdot n \text{ on } gas/SiO_2 \quad 4.140$$

$$F \cdot n = F_3 \cdot n \text{ on } SiO_2/Si \quad 4.141$$

The velocity of the growing oxide, V_{ox} , is given by:

$$V_{ox} \cdot n = \frac{k_s C}{N_1} \text{ on } SiO_2/Si \quad 4.142$$

At each time step, the thickness the oxide is given by

$$x_{ox}(t + dt) = V_{ox} \cdot n \cdot dt \quad 4.143$$

where dt is the actual time step.

At each time step of the oxidation, the silicon material is removed. The ratio of the silicon material consumed, at each time step, due to the volume expansion of the oxide can be set by the user using the **oxide** command and its parameter **alpha**.

4.8.5 Compressible model with no stress

In this model the oxide material is considered as incompressible viscous flow [32]. This model is more accurate than the vertical model. It can be used for non-planar structures where the

oxide flow is in tow dimension. But, it does not include the stress effects. In this model, the displacement equations are derived from the incompressible Navier-Stokes equations where the acceleration and gravitational terms are neglected. The equations of this model are given by

$$\frac{\partial^2 V_x}{\partial x^2} + \frac{\partial^2 V_y}{\partial y^2} = \frac{1}{\mu} \frac{\partial P}{\partial x} \quad \text{in } SiO_2 \quad 4.144$$

$$\frac{\partial^2 V_x}{\partial x^2} + \frac{\partial^2 V_y}{\partial y^2} = \frac{1}{\mu} \frac{\partial P}{\partial y} \quad \text{in } SiO_2 \quad 4.145$$

$$\frac{\partial V_x}{\partial x} + \frac{\partial V_y}{\partial y} = 0 \quad \text{in } SiO_2 \quad 4.146$$

In Csuprem, the incompressibility condition in the system 4.144 is replaced by an artificial compressibility condition to improve the conditioning of the discrete system. For this reason, this model is called compressible model. So, in Csuprem we solve

$$\frac{\partial V_x}{\partial x} + \frac{\partial V_y}{\partial y} = -\frac{1-2.Poiss.r}{\mu} P \quad \text{in } SiO_2 \quad 4.147$$

where $V = (V_x, V_y)$ is the velocity of the growing oxide in 2D. μ is the oxide viscosity given by:

$$\mu = \frac{Young.m}{2+2.Poiss.r} \quad 4.148$$

and P is the hydrostatic pressure.

These simplified Navier-Stokes equations should be solved self consistently with the oxidation growth equations of the vertical model described previously due to the close coupling.

The material's Young's modulus $Young.m$ and the material's Poisson's ratio $Poiss.r$ can be set by the user using **material** command.

To account for stress effects on the oxidation growth rate, Csuprem provides two models.

4.8.6 Non-linear viscous model

This model takes into account the effect of stress $\sigma = (\sigma_{xx}, \sigma_{yy}, \sigma_{xy})$ on the diffusion coefficient D of the oxidant, the oxide viscosity μ , and the interface reaction rate coefficient k_s . In this model, the diffusion coefficient is given by:

$$D = D(t + dt, \sigma) = D(t, \sigma) \cdot e^{V_d(\sigma_{xx} + \sigma_{yy})} \quad 4. 149$$

where t is the previous time, dt is the time step, and the parameter V_d is the activation volume of oxidant diffusion with respect to pressure.

The oxidant viscosity is given by:

$$\mu = \mu(t + dt, \sigma) = \mu(t, \sigma) \cdot \frac{ts \cdot \frac{V_c}{2K_b T}}{\sinh\left(ts \cdot \frac{V_c}{2K_b T}\right)} \quad 4. 150$$

where the parameter V_c is the activation volume of the viscosity and ts is the total shear stress given by:

$$ts = \frac{1}{2} \sqrt{(\sigma_{xx} - \sigma_{yy})^2 + 4\sigma_{xy}^2} \quad 4. 151$$

The reaction rate coefficient is given by:

$$k_s = k_s(t + dt, \sigma) = k_s(t, \sigma) \cdot e^{-\left(\frac{V_r \sigma_n + V_t \sigma_t}{K_b T}\right)} \quad 4. 152$$

where V_r is the activation volume of reaction rate with respect to normal stress σ_n given by:

$$\sigma_n = \sigma_{xx} \cdot n_x^2 + \sigma_{yy} \cdot n_y^2 + 2 \cdot \sigma_{xy} \cdot n_x \cdot n_y \quad 4. 153$$

the parameter V_t is the activation volume of the reaction rate with respect to tangential stress σ_t given by:

$$\sigma_t = \sigma_{xx} \cdot n_y^2 + \sigma_{yy} \cdot n_x^2 - 2 \cdot \sigma_{xy} \cdot n_x \cdot n_y \quad 4. 154$$

The parameters V_d , V_c , V_t , and V_d can be set by the user using the **oxide** command and its parameters Vc , Vr , Vd , and Vt .

At the initial time $t = 0$, the coefficients D , k_s , and μ can be given by:

$$D = D_0; \mu = \mu_0; k_s = k_{s0}$$

where D_0 , μ_0 , and k_{s0} satisfy an Arrhenius expression. The user can set the viscosity μ_0 by using the material command and its parameters $visc.0$, and $visc.E$. The parameter D_0 , and

k_{s0} can be set by the oxide command and its parameters *diff.0*, *diff.E* and *trn.0*, and *trn.E* respectively.

In this model, we solve the equations of vertical model, compressible model, and additional linear equations to calculate the stress tensor in the moving oxide given by [17sill]:

$$\sigma_{xx} - \sigma_{yy} = 2\mu_0 \left(\frac{\partial V_x}{\partial x} + \frac{\partial V_y}{\partial y} \right) \quad 4.155$$

$$\sigma_{xx} + \sigma_{yy} = \frac{2\mu_0}{1-2.Poiss.r} \left(\frac{\partial V_x}{\partial x} + \frac{\partial V_y}{\partial y} \right) \quad 4.156$$

$$\sigma_{xy} = \mu_0 \left(\frac{\partial V_x}{\partial x} + \frac{\partial V_y}{\partial y} \right) \quad 4.157$$

The viscosity μ_0 is given by the Arrhenius expression.

4.9 Diffusion and segregation models

4.9.1 Introduction

After ion implantation, deposition, or etching a subsequent thermal diffusion processing (also called annealing) is performed. This thermal diffusion processing is needed mainly for the following two raisons. The implanted impurity atoms partly reside on interstitial sites inside the crystal lattice and are, thus, electrically inactive. Secondly, the implanted ions displace substrate atoms (dislocations or line-defects) and then cause implantation damage. So, the thermal diffusion treatment helps to heal as much as possible of the implantation damages and to activate the impurities (i.e. impurity becomes dopant) by moving them on substitutional lattice sites.

For example, a Rapid Thermal Annealing (RTA) at a very high temperatures allows to repair the damage with minimum impurity movement.

The species involved in the diffusion are the impurities and the point-defects (interstitials and vacancies). In the following sections, we present the diffusion mechanism and the diffusion models supported in Csuprem. The diffusion of the charged impurities, interstitials and vacancies is due to their unequal concentrations and to the internal electric field.

The derivation of the dopants and defect diffusion equations is based on the continuum theory using Fick's diffusion laws and the atomistic theory [27]. The latest includes the interaction of dopants with the lattice atoms and defects.

There are four different diffusion models in Csuprem.

4.9.2 Pair-diffusion mechanisms

The new generation of diffusion models are based on the idea of pair-diffusion models. These models do not only represent the effect of charged impurities on each other but they also describe the reactions of impurities and the lattice point-defects.

In Csuprem, the diffusion models for the impurities are based the pair-diffusion mechanism. The impurity atoms can't diffuse on their own. But, they need the neighboring point-defects as diffusion vehicle. When the binding energy between an impurity and a neighboring defect is not weak the impurity and the the point-defect will move as a pair dopant-defect until they get separated by a recombination or other effects. It is common to denote any diffusing dopant involved in any diffusion mechanism as dopant-defect pairs. We label a dopant A paired with a vacancy V as AV pair, a dopant A paired with interstitial I as AI pair. So, the diffusivities of dopants represent the diffusivities of dopant-defect pairs.

The recombination between the paired dopants and other species (as unpaired defects) is not included in the equations in Csuprem. The chemical reaction rate equations of these type of recombination are: $AV + I \Leftrightarrow A$ in case of Frank-Turnbull recombination and $AI + V \Leftrightarrow A$ in case of dissolution process [27].

Csuprem includes three basic diffusion mechanisms for dopant diffusion models [1], [55], [56], [27].

- **Direct diffusion mechanism**

In direct diffusion, impurities which have small ionic radii can travel directly from one interstitial site to another. Especially, Group-I and Group-III elements are diffusing directly and are therefore fast diffusers.

- **Vacancy mechanism**

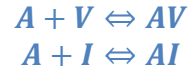
In this diffusion mechanism, a substitutional dopant exchange its position with an adjacent or neighboring vacancy. The vacancy should move at least to a third-neighbor site away from the dopant to prevent oscillations. For more details about this regime see [55].

- **Interstitials mechanism**

This mechanism is also called **kick-out** mechanism. In case of silicon, the dopant at a substitutional site is approached by a silicon interstitial and kicked out to reside at interstitial position while the original self-interstitial has disappeared by occupying the regular lattice site. This interstitial dopant can now move towards an adjacent lattice site to regenerate a self-silicon interstitial by the same kick-out mechanism.

The most common p-type dopant Boron diffuses basically via interstitials. The n-type dopant Phosphorus diffuses via interstitials at low doping and via a dual mechanism at high doping

concentrations with a strong vacancy diffusion tendency. Arsenic uses also interstitials and vacancies for its diffusion. But, its interstitial mechanism is limited due to the relatively large ionic radii [27]. Large impurities as Antimony only diffuse via vacancies. The rate equations for the basic atomistic diffusion mechanisms are:



In Csuprem, the diffusion models of point defects are based on two concepts: paired and unpaired diffusion. This means that a point defect can move freely (unpaired diffusion) or can move in pair with its binding dopant (paired diffusion). In Csuprem, the unpaired defects (free defects) are explicitly represented by separate equations. But, the paired defects diffusion is implicitly included in the paired dopant diffusion equations.

The diffusivity of paired and unpaired point defects could be very different. The diffusion of defects is also limited or stopped by the recombination effects. The recombination reaction between interstitials and vacancies is included in Csuprem models.

Table 4.1: Chemical reactions of pair-diffusion models with interstitials (I) and vacancies (V).

Silicon	Boron	Phosphorus	Arsenic	Antimony
$I + V \rightleftharpoons 0$	$B + I \rightleftharpoons BI$	$P + I \rightleftharpoons PI$	$As + I \rightleftharpoons AsI$	$Sb + V \rightleftharpoons SbV$
	$BI + V \rightleftharpoons B$	$P + V \rightleftharpoons PV$	$As + V \rightleftharpoons AsV$	$SbV + I \rightleftharpoons Sb$
		$PI + V \rightleftharpoons P$	$AsI + V \rightleftharpoons As$	
		$PV + I \rightleftharpoons P$	$AsV + I \rightleftharpoons As$	

There are four different diffusion models in Csuprem. The accuracy of each model depends mainly on how the point defects are modelled and simulated.

Fermi diffusion model

This model is the fastest in CPU time. In this simple model, the free point defects equations are not solved. Since the free point defects are supposed to be in thermodynamical equilibrium. In this model, only the equations representing the diffusion of dopant-defect pairs are solved. And their diffusivity coefficients are then supposed to be time independent. So, this model can not be used to simulate transient enhanced diffusion, oxidation enhanced or retarded diffusion in which the free point defects are not in thermodynamical equilibrium.

Two-dimensional diffusion model

In this model, the free point defects are not supposed to be in equilibrium all the time. But, they are evolved in time. So, the equations representing their diffusion are solved. On the other hand, the diffusivity coefficients of the dopant-defects pair diffusion equations are enhanced to include the transient effects of the free point defects. So, the diffusion of the dopants is strongly affected by the diffusion of the free point defects. But, in this model, the diffusion of free point defects is not affected by the diffusion of dopants. Mathematically speaking, the

diffusion equations of free point defects are decoupled from the diffusion equations of dopants. This is one way coupling. This model could be used to include the oxidation enhanced or retarded diffusion effects.

This model is more expensive in CPU time than the Fermi model.

Fully coupled diffusion model

In this model, the diffusion of the free point defects is affected by the diffusion of the dopants. This coupling adds the dopant-defect pair diffusion fluxes to the diffusion fluxes of the free point defects. This model is an extension of two dimensional model which is, in turn, an extension of the Fermi model.

The fully coupled model is physically more accurate and it was used in different process simulations to predict different physical effects observed in the processing as the emitter push effect. experiments. But, even this model still suffer from not taking into account explicitly paired and unpaired dopants point defects diffusion.

Steady state diffusion model

This model is a particular case of two dimensional model in which the point defects are assumed to be in the steady state.

4.9.3 Intrinsic and extrinsic dopant diffusion

Dopant diffusion theories are based mainly on the approaches: 1) the continuum theory which is based on Fick's diffusion laws and 2) the atomistic theory which includes the rate equations modelling the interactions of point-defects and the diffusing species [27], [55]. The flux of each dopant C_T due to the gradient of the concentrations C_T , at intrinsic doping condition ($C_A = n_i$) is modelled by the Fick's first:

$$J_T = -D \cdot \nabla C_T \quad 4.158$$

where J_T is the diffusion flux of the dopants, D the diffusion coefficient, and n_i is the intrinsic electron concentration.

The conservation of dopant atoms is expressed by the continuity equation known as Fick's second law

$$\frac{\partial C_T}{\partial t} = -\text{div}(J_T) \quad 4.159$$

In the case of low doping (intrinsic diffusion) the measured diffusion profiles agree with Fick's laws. But at high doping ($n_i = C_T$), the diffusion profiles deviate from those predicted by the Fick's laws [27]. At diffusion temperatures the dopants are usually ionized and the released electrons cause an electric field. This later includes a field enhancement term (drift term) in the normal diffusion flux.

$$J_T = -D_T \cdot \nabla C_T + v \cdot C_T \quad 4.160$$

The velocity of the charged particles can be described with

$$v = \mu E = -\mu \nabla \psi \quad 4.161$$

where E is the electric field and ψ is the electrostatic potential. The mobility μ is related to the diffusion coefficient by Einstein's relation

$$\mu = D_T \cdot \frac{q}{kT} \quad 4.162$$

where q denotes the elementary charge and k is the Boltzmann constant. By including 4.162 and 4.161 into 4.160, and taking more than one dopant into account, Fick's law is extended to

$$J_T = -D_T \cdot (\nabla C_T + z_T \frac{q}{kT} C_T \cdot \nabla \psi) \quad 4.163$$

where z_T denotes the charge state of the belonging dopant (+1 for singly charged acceptors and -1 for singly charged donors).

The electrostatic ψ is determined by the Poisson equation

$$\text{div}(\epsilon \cdot \nabla \psi) = q \cdot (n - p - C_{net}) \quad 4.164$$

where

$$C_{net} = -\sum_T z_T \cdot C_T \quad 4.165$$

represents the net concentration of all ionized dopants and n , p the concentration of electrons and holes, respectively.

By assuming the thermodynamic equilibrium, Boltzmann statistic, and the charge neutrality, the carrier concentrations n and p satisfy:

$$n \cdot p = n_i^2 \quad 4.166$$

$$n = n_i \cdot \exp(\psi \frac{q}{kT}) \quad 4.167$$

$$p = n_i \cdot \exp(-\psi \frac{q}{kT}) \quad 4.168$$

$$n - p - C_{net} = 0 \quad 4.169$$

the electrostatic potential can be given by

$$\psi = \frac{kT}{q} \cdot \text{arsinh}\left(\frac{C_{net}}{2n_i}\right) \quad 4.170$$

and the gradient of ψ

$$\nabla\psi = -\frac{kT}{q} \cdot \frac{1}{\sqrt{C_{net}^2 + 4n_i^2}} \cdot \sum_i z_i C_i \quad 4.171$$

The flux J_T for a dopant C_T now depends on the gradients of the concentration C_i of other diffusing dopants:

$$J_T = -D_T \cdot \left(1 + \frac{C_T}{\sqrt{C_{net}^2 + 4n_i^2}}\right) \cdot \nabla C_T - D_T \cdot \frac{z_T C_T}{\sqrt{C_{net}^2 + 4n_i^2}} \cdot \sum_{i \neq T} z_i \cdot \nabla C_i \quad 4.172$$

The equation 4.172 is reduced to

$$J_T = -D_T \cdot \left(1 + \frac{C_T}{\sqrt{C_T^2 + 4n_i^2}}\right) \cdot \nabla C_T \quad 4.173$$

in case of one dopant. We note that an effective diffusion coefficient with a field enhancement factor λ can be deduced

$$D_{eff} = -D_T \cdot \left(1 + \frac{C_T}{\sqrt{C_T^2 + 4n_i^2}}\right) = \lambda \cdot D_T \quad 4.174$$

We should note that the dopant diffusion equations derived above do not include the pairing diffusion. These can be seen as generic equations for free dopants diffusion.

4.9.4 Fermi diffusion model

The dopant diffusion equation solved by Csuprem is:

$$\frac{\partial C_T}{\partial t} = \text{div} \left[D_{AV} C_A \nabla \ln \left(C_A \frac{n}{n_i} \right) + D_{AI} C_A \nabla \ln \left(C_A \frac{n}{n_i} \right) \right] \quad 4.175$$

for n-type dopants. For p-type dopants, nn_i should be replaced by pn_i . Here, C_T is the total chemical concentration (which includes the active and non active species), and C_A is the total electrically active concentration. D_{AV} is the dopant-vacancy pair diffusion coefficient and D_{AI} is the paring diffusivity with interstitials. These paring diffusivities can be given by the following general expressions which include different charge states: neutral (x), singly negative (m), doubly negative (mm), singly positive (p), and doubly positive (pp):

$$D_{AI} = F_I \cdot [D_x + D_m \left(\frac{n}{n_i}\right) + D_{mm} \left(\frac{n}{n_i}\right)^2 + D_p \left(\frac{p}{n_i}\right) + D_{pp} \left(\frac{p}{n_i}\right)^2] \quad 4.176$$

$$D_{AV} = (1 - F_I) \cdot [D_x + D_m \left(\frac{n}{n_i}\right) + D_{mm} \left(\frac{n}{n_i}\right)^2 + D_p \left(\frac{p}{n_i}\right) + D_{pp} \left(\frac{p}{n_i}\right)^2] \quad 4.177$$

F_I is the fraction of interstitialcy or interstitial assisted diffusion and $(1 - F_I)$ is the fraction of vacancy assisted diffusion.

Example:

$$D_{AV} = (1 - F_I) \cdot [D_x + D_m \left(\frac{n}{n_i}\right)] \quad 4.178$$

$$D_{AI} = F_I \cdot [D_x + D_m \left(\frac{n}{n_i}\right)] \quad 4.179$$

for Arsenic (As) dopant.

The intrinsic diffusivities D_{aa} where $a \in \{x, m, mm, p, pp\}$ are expressed using arrhenius formula:

$$D_{aa} = D_{aa.0} \cdot \exp\left(\frac{-D_{aa.E}}{kT}\right) \quad 4.180$$

The user can specify the pre-factors $D_{aa.0}$ and the activation energies $D_{aa.E}$ for implanted and grown-in dopants.

Example:

arsenic silicon $D_{ix.0}=8.0$ $D_{ix.E} = 3.65$

Here, $D_{ix.0}$ and $D_{ix.E}$ are the pre-factor and the activation energy for the diffusivity with neutral defects. The default values of all the intrinsic diffusivities are given in the User's Reference Manual [2].

To include the electric field effects, the following expression was used

$$\frac{q}{kT} \cdot \nabla \psi = \ln \left(\frac{n}{n_i} \right) \quad 4.181$$

for n-type dopant and for p-type n is replaced by p .

It is important to know that all the dopant diffusion equations are coupled by the potential. So, in this model we solve a system of equation corresponding to the diffusing impurities. The free defect diffusion equations are not solved.

4.9.5 Two-dimensional diffusion model

This model includes the transient diffusion of the free point-defects. So, in this model we need to solve the dopant and the free point-defect diffusion equations. The diffusion coefficients of dopants now depend also on the concentration of point defects C_I , C_V , $C_I^{\ddot{a}}$, and $C_V^{\ddot{a}}$. C_I and C_V are the concentrations of interstitials and vacancies at the current time and $C_I^{\ddot{a}}$ and $C_V^{\ddot{a}}$ are their related concentrations at the thermodynamical equilibrium. The equations involved in this model are as follows.

$$\frac{\partial C_T}{\partial t} = \text{div}[J_{AV} + J_{AI}], \forall A = 1, ND \quad \text{in } \Omega \quad 4.182$$

$$\frac{\partial (C_I - C_{ET})}{\partial t} = \text{div}[-J_I - \sum_A J_{AI}] - R \quad \text{in } \Omega \quad 4.183$$

$$\frac{\partial C_{ET}}{\partial t} = -K_T \left[C_{ET} C_I - \frac{e^*}{1-e^*} C_I^* (C_{TT} - C_{ET}) \right], \quad \text{in } \Omega \quad 4.184$$

$$\frac{\partial C_V}{\partial t} = \text{div}[-J_V - \sum_A J_{AV}] - R, \quad \text{in } \Omega \quad 4.185$$

where

$$J_{AI} = D_{AI} C_A \frac{C_I}{C_I^{\ddot{a}}} \nabla \ln \left(C_A \frac{C_I}{C_I^{\ddot{a}}} \frac{n}{n_i} \right) \quad 4.186$$

$$J_{AV} = D_{AV} C_A \frac{C_V}{C_V^{\ddot{a}}} \nabla \ln \left(C_A \frac{C_V}{C_V^{\ddot{a}}} \frac{n}{n_i} \right) \quad 4.187$$

$$-J_I = D_I C_I^* \nabla \frac{C_I}{C_I^{\ddot{a}}} \quad 4.188$$

$$-J_V = D_V C_V^* \nabla \frac{C_V}{C_V^{\ddot{a}}} \quad 4.189$$

$$R = K_R(C_I C_V - C_I^* C_V^*) \quad 4.190$$

Ω represents the computation domain.

J_{AI} and J_{AV} refer to the flux of impurity A diffusing as pair with interstitials and vacancies respectively.

Note: In two-dimensional model, the fluxes J_{AI} and J_{AV} are ignored in the interstitial and vacancy Equations 4.183 and 4.185, respectively.

ND is the number of the diffusing dopants.

C_{ET} is the number of empty interstitial traps

J_I is the flux of free interstitials which represent the effects of an electric field on the charged portion of interstitials.

J_V is the flux of free vacancies.

D_I , and D_V are the diffusion coefficient of the free interstitials and free vacancies respectively.

R represents all sources of bulk recombinations of interstitials. The trap Equation 4.184 represents the evolution in time of the empty traps. It's expression is derived from the chemical reaction



This trap reaction was described by Griffin [57]. This reaction explains some of the wide variety of diffusion coefficients extracted from several different experimental conditions.

C_{TT} is the total trap concentration. The user can set C_{TT} using the command **trap** with its parameter **total**. For more complete discussion, see the User's Reference Manual [2].

K_T is the trap reaction coefficient

$e^* = C_{ET}^* C_{TT}$ is the equilibrium trap occupancy ratio. K_T and e^* are temperature dependent and their expression follow Arrhenius expression. The user can set these parameters using the **trap** command and its parameters *frac.0*, *frac.E*, *ktrap.0*, and *ktrap.E*.

Example:

trap silicon total= 2.e18 frac.0=0.5 frac.E=0.0

Instead of the reaction, the time derivative is used in the trap equation because it has better properties in the numerical simulation.

The trap model used here is involved on ongoing research, and may be out of date at any time. The trap concentration will depend on the thermal history of the wafer, starting material, stress, and temperature [2].

The equilibrium concentration of interstitials C_I^* in extrinsic conditions depends on the various charge states (neutral, negative, double negative, triple negative, positive, double positive, and triple positive) and is given by

$$C_I^* = C_{star} \frac{neu + neg(\frac{n}{n_i}) + dneg(\frac{n}{n_i})^2 + tneg(\frac{n}{n_i})^3 + pos(\frac{p}{n_i}) + dpos(\frac{p}{n_i})^2 + tpos(\frac{p}{n_i})^3}{neu + neg + dneg + tneg + pos + dpos + tpos} \quad 4.192$$

C_{star} is the total equilibrium concentration in intrinsic doping concentration. The parameters $neu, neg, dneg, tneg, pos, dpos, tpos$ take into account the defects of different charge states in intrinsic doping case.

The equilibrium concentration of vacancies C_V^* in extrinsic conditions is given by a similar expression. The expressions of all these parameters follow the arrhenius formula. The user can set these parameters using the command **interstitial** (or **vacancy**) with the parameters

Cstar.0, Cstar.E, neu.0, neu.E, neg.0, neg.E, dneg.0, dneg.E, tneg.0, tneg.E, pos.0, pos.E, dpos.0, dpos.E, tpos.0 and tpos.E.

Example:

```
inter silicon neu.0=1 neg.0=1 pos.0=0 dneg.0=0 inter silicon neu.E=0 neg.E=0 pos.E=0 dneg.E=0
```

Boundary conditions

The solvability of the above system of the coupled partial differential equations 4.182, 4.183, 4.184, and 4.185 depends on the type of the boundary (or interface) conditions. The boundary conditions are defined at the top surface (exposed surface), at the bottom surface (backside surface) and at the side surfaces (reflecting surfaces). They can also be defined on the interfaces between two materials if the diffusing specie is defined on one material only.

Boundary conditions for dopants

The boundary condition for the dopants can be of mixed type:

$$A \cdot C_T + B \cdot \nabla C_T \cdot n = S \quad \text{on} \quad \partial\Omega \quad 4.193$$

where $\partial\Omega$ represents the boundary of the computational domain, n is the normal to the boundary, S represents the source term when it exists.

Boundary conditions for interstitials

The interstitials satisfy a flux balance boundary condition as described by Hu [58].

$$J_I n + K_I (C_I - C_I^*) = g \quad \text{on} \quad \Gamma_{ex} \subset \partial\Omega \quad 4.194$$

where Γ_{ex} can represent the exposed boundary or the interface boundary in case of oxidation. K_I is the surface recombination rate. It depends on the time during oxidation. The term g is the generation, if any, at the surface. The generation term g exists in case of oxidation. This boundary condition has been used with success on several different surface types [2].

The surface recombination rate K_I in case of oxidation (moving boundary) is given by

$$K_I = K_{surf} (k_{rat} \left(\frac{v_{ox}}{B/A}\right)^{K_{pow}} + 1) \quad 4.195$$

where v_{ox} is the interface velocity, an B/A is the Deal Grove oxide growth coefficient (see oxide command). The full model applies only to oxide, since it is the only interface that can grow.

For the non-mobile interfaces (gas,oxynitride,nitride,poly) K_{surf} is the only important parameter (i.e. $K_I = K_{surf}$).

The user can set K_{surf} , K_{rat} , K_{pow} on the **interstitial** command using the following parameters

Ksurf.0, Ksurf.E, krat.0,Krat.E, Kpow.0,Kpow.E

Csuprem offers tow models (Growth injection model and time injection model) to calculate the generation rate g of interstitials or vacancies at the moving oxide boundary during oxidation. These two models can be selected using the **interstitial** or **vacancy** commands with the parameters **growth.inj** and **time.inj**. In the growth injection model the generation rate is given by

$$g = \theta V_{mole} v_{ox} \left(\frac{v_{ox}}{B/A}\right)^{G_{pow}} \quad 4.196$$

where θ is the fraction of silicon atoms consumed during growth that are re-injected as interstitials. V_{mole} is the lattice density of the consumed material. G_{pow} is a power parameter. Once again, this model only makes sense for oxide since it is currently the only material which can have a growth velocity.

In the time injection model the generation rate is given by

$$g = A(t + t_0)^{T_{pow}} \quad 4.197$$

where t is the time in the diffusion in seconds. This can be used to represent the injection of

interstitials from an oxynitride layer. The parameters A , t_0 , and T_{pow} follow an Arrhenius expression and can be set by the user using the following parameters

$A.0$, $A.E$, $t0.0$, $t0.E$, $Tpow.0$, $Tpow.E$.

For a complete discussion on how to set all the other parameters see the User's Reference Manual [2].

Boundary conditions for vacancies

The vacancies satisfy the same type of boundary (or interface) condition as interstitials.

4.9.6 Fully coupled diffusion model

The equations involved in this model are similar to those described in the tow dimensional model. The only difference is the fluxes from the dopants $\sum_A J_{AV}$ and $\sum_A J_{AI}$ are now calculated and included in the interstitial and vacancy equations. This also means that the interstitial and vacancy diffusion equations are coupled to the dopant diffusion equations. This model can be selected using the **method** command and its parameter **full.cpl**.

4.9.7 Steady state diffusion model

This model is similar to the two dimensional model but assumes that the point defects are in steady state. This model can be selected using the **method** command and its parameter **steady**.

4.9.8 Segregation models

Segregation models represent an interface condition for dopant across an interface between two regions. In general the impurity concentrations within the two regions is not in the equilibrium thus a diffusion flux over the interface results trying to balance the dislocation of dopants.

The segregation model is computed using the following model

$$J_{seg} \cdot n = T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 4.198$$

where $J_{seg} \cdot n$ is the normal flux of dopants flowing from material 1 to material 2 across the interface. C_1 and C_2 are the concentrations in material 1 and 2 respectively. T_r is the transport velocity which takes into account the rate of dopants crossing the interface. M_{12} is the segregation term taking into account the variation of the threshold solubility of dopants in the two neighboring materials. For each diffusing dopant A , we have

$$M_{12} = \frac{TS_1(A)}{TS_2(A)} \quad 4.199$$

where $TS_1(A)$ is the threshold solubility of the dopant A in material 1. $TS_2(A)$ is the threshold solubility of the dopant A in material 2. M_{12} and T_r are calculated using a standard Arrhenius relationship. Csuprem allows the user to set these parameters using **impurity name** as command with its parameters **Seg.0**, **Seg.E**, **Trn.0**, and **Trn.E**. The materials are ordered as: the first material in the command is material 1 and the second material is the material 2. The two materials should be separated by forward slash.

Example:

antimony silicon /oxide Seg.0=30. Seg.E=0 Trn.0= 1.66e-7

This command will change the segregation parameters at the silicon/oxide interface for the dopant antimony. The solubility of antimony in silicon is 30 times the oxide solubility.

4.9.9 Clustering and activation models

Due to high implantation doses in the new technology involving shallow junctions engineering, heavily doped silicon forms inactivated aggregates of particles known as *precipitates* and *clusters* [27] during a thermal treatment (diffusion or annealing). While clusters are restricted to specific configuration of atoms neglecting any crystal properties, precipitates can host thousands of atoms and form second phase regions. The classical interpretation of these high concentration effects is that at a certain temperature only a limited amount of these impurities is able to dissolve on substitutional sites in the crystal (active and mobile dopants). Impurities above the solubility limit (called also threshold) form precipitates or clusters. Precipitates are electrically inactive and immobile [27].

Some models assume that clusters are partially active. [27]. It is not clear yet, whether to favor clustering or precipitation models for certain impurity species. Baccus proposed a clustering model for Boron where the clusters are modelled to be inactive but mobile [59]. Most researchers, modelling high concentration arsenic profiles, prefer the clustering approach [27], [60], [61]. As dopant concentration above the solubility limit is common in today's technology, we need to account for the clustering phenomenon for the simulation of the dopant diffusion.

In Csuprem, we still use the classical models where clusters are assumed to be non-active and non-mobile. They can not diffuse. Likewise, the impurities above the solubility limit are considered non-active and non diffusing. Csuprem offers two electrical activation models for all impurities. The first activation model is the clustering model with vacancies for Arsenic. The second activation model is based on experimental data. The later is used for all impurities except Arsenic.

Activation and clustering model for Arsenic

It is supposed that the total dopant concentration C_T is divided into a mobile active C_A and immobile non-active clustered C_A^{cl} part [27]. The Arsenic active concentration is calculated using:

$$C_T = C_A + C_A^{cl} = C_A(1 + C_{tn}(\frac{n}{n_i})^2 \frac{C_V}{C_V^*}) \quad 4. 200$$

Physically, this model represents the clustering of Arsenic with doubly negative vacancies. The total clustering coefficient C_{tn} obeys a standard Arrhenius relationship. Cusprem allows the user to set this parameter using **arsenic** command and its parameters **Ctn.0**, **Ctn.E**.

Example:

arsenic silicon Ctn.0 =5.9e-24 Ctn.E=0.60

Activation model for other dopants

The activation model for other dopants than Arsenic is based on experimental data. The solid solubility limit C_T^{sl} as function of temperature is stored in experimental data table for every dopant. From the current temperature an extrapolation is used to calculate from the data table a fitted solubility limit C_T^{fsl} . Then, the active concentration is given by

$$C_A = C_T^{fsl} (1 + (1 - a) \cdot \ln \frac{(\frac{C_T}{C_T^{fsl}})^{-a}}{1 - a}) \quad \text{if } C_T > C_T^{fsl} \quad 4. 201$$

$$C_A = C_T^{fsl} \quad \text{if } C_T < C_T^{fsl} \quad 4. 202$$

Csuprem allows the user to add a single temperature and solid solubility limit C_T^{sl} point to the experimental data table using the impurity command and its parameters **ss.temp**, and **ss.conc**

Example:

magnesium gaas ss.temp=700 ss.conc=1.e+19

CHAPTER 5

One and Two Dimensional Examples

Introduction

This chapter is dedicated to describing examples supported by the CSuprem software. Examples 1 to 21 are based on 1 or 2 dimensional structures from Stanford University and are used as a reference for quality control purpose. Examples beyond the first twenty-one are created by Crosslight and are used to illustrate new features supported by the software.

We only document the silicon related examples here while those related to GaAs technology are to be included in a future version of this manual, depending on user demands.

5.1 Example 1: Boron anneal-1D

This example performs a simple anneal of a boron implant. The final structure is saved, and then various post processing is performed. The input file for the simulation is in the ``examples\Stanford_Original\exam1_boron" directory, in the file boron.in.

```

#some set stuff
option quiet
mode one.dim

#the vertical definition
line x loc = 0      spacing = 0.01 tag = top
line x loc = 0.50   spacing = 0.01
line x loc = 2.0    spacing = 0.01 tag=bottom

#the silicon wafer
region silicon xlo = top xhi = bottom

#set up the exposed surfaces
bound exposed  xlo = top  xhi = top

#calculate the mesh
init boron conc=1.0e14

#the pad oxide
deposit oxide thick=0.075

#the uniform boron implant
implant boron  dose=3e14  energy=70

#plot the initial profile
select z=log10(boron)
plot.1d  x.ma=2.0 y.mi=14.0 y.max=20.0

#the diffusion card
diffuse time=30 temp=1100

#save the data
structure out=boron.str

#plot the final profile
select z=log10(bor)
plot.1d x.ma=2.0 y.mi=14.0 y.max=20.0 cle=f axis=f

```

The first lines in the input deck set some basic CSUPREM options.

```

#some set stuff
option quiet

```

mode one.dim

The '#' character is the comment character for CSUPREM. The entire line after the '#' is ignored. The first command instructs the simulator to be quiet about what it is doing and it is the default option. The second command specifies that the simulation will be done in one dimension. It is therefore necessary to specify only vertical grid information.

The next section begins the definition of the mesh to be used for the simulation. The section

#the vertical definition

```
line x loc = 0      spacing = 0.01 tag = top
line x loc = 0.50   spacing = 0.01
line x loc = 2.0    spacing = 0.01 tag=bottom
```

describes the locations of the x lines in the mesh. CSUPREM, in one dimensional mode, defines x to be the direction into the wafer. In two-dimensional mode, x is across the wafer top and y is the direction into the wafer. This simulation is to be done on a one dimensional problem, therefore lines are only specified in the x direction. The first statement places a mesh line at the top of wafer, x coordinate 0.0 μm and tags the line as "top". The spacing is set to 0.02 μm . A line is placed at 0.5 μm which is the expected starting depth of the implanted profile. The spacing is set to 0.02 μm . Since the spacing between the first two mesh lines is the same, 0.02 μm , there will be mesh lines placed 0.02 μm apart between 0.0 and 0.50 μm . Finally a line is placed at 2.0 μm , which is greater than the depth of the profile after the anneal. The spacing here is set to 0.25 μm . In the case where spacings are different in neighboring lines, the spacing is graded between them. The distance between mesh lines near 0.5 μm will be 0.02 μm and the distance will get progressively closer to 0.25 μm as the mesh gets closer to 2.0 μm . The line statement fixes line locations in the mesh as well as the average spacing between lines.

The next two sections describe the device starting material and the surfaces which are exposed to gas.

#the silicon wafer

```
region silicon xlo = top xhi = bottom
```

#set up the exposed surfaces

```
bound exposed xlo = top xhi = top
```

The region statement is used to define the starting materials. In this case the wafer is silicon with no initial masking layers. The silicon area is defined to extend between the lines tagged top and bottom in the vertical direction. Looking back at the mesh line definition section, this corresponds to the entire area that had been defined. The initial simulation area is completely silicon.

The bound statement allows the definition of the front and backsides of the wafer. Any gases on the diffuse, deposit, and etch statements are applied to the surface marked exposed. It is important to define the top of the wafer for CSUPREM to correctly simulate these actions. Most mistakes are made by ignoring to define the exposed surface.

The next line informs CSUPREM that the mesh has been defined and should be computed.

```
#calculate the mesh
init boron conc=1.0e14
```

This statement computes the locations of lines given the spacings, triangulates the rectangular mesh, and computes geometry information. The initial doping is boron with a concentration of 10^{14} cm^{-3} . The next line adds a pad oxide to the wafer.

```
#the pad oxide
deposit oxide thick=0.075
```

The deposited oxide is specified to 0.075 μm thick. This represents the pad oxide that is grown before the uniform boron implant. Rather than simulate the growth, it is simpler to add a deposited oxide the correct thickness of the pad.

The next statement performs the implant of the boron.

```
#the uniform boron implant
implant boron dose=3e14 energy=70
```

The implant is modeled with a Pearson-IV distribution. The energy and dose are $3 * 10^{14} \text{ cm}^{-2}$ and 70 KeV respectively. This produces an abrupt boron profile with a junction near 0.3 μm . The next commands choose a plot variable and display it.

```
#plot the initial profile
select z=log10(boron)
plot.1d x.ma=2.0 y.mi=14.0 y.max=20.0
```

This statement will generate a file with extension .tmp. Applying the GNUPLOT (a public domain plotting software) on this .tmp file will produce the graphic display. The plot will look like Figure 5.1. The select statement selects the plot variable to be the log base ten of the boron concentration. The plot.1d statement does not need to specify the location of the cross section in one dimension. The plot should extend between the minimum value of y through 2.0 μm . Dimensions on the plot command always refer to the plot axis. In this case, the x axis is the depth dimension, and the y axis is the log of the boron concentration. The minimum and maximum on the y axis are set to 10^{14} and 10^{20} . The y axis values have to be specified in the units of the selected variable, log10 of concentration.

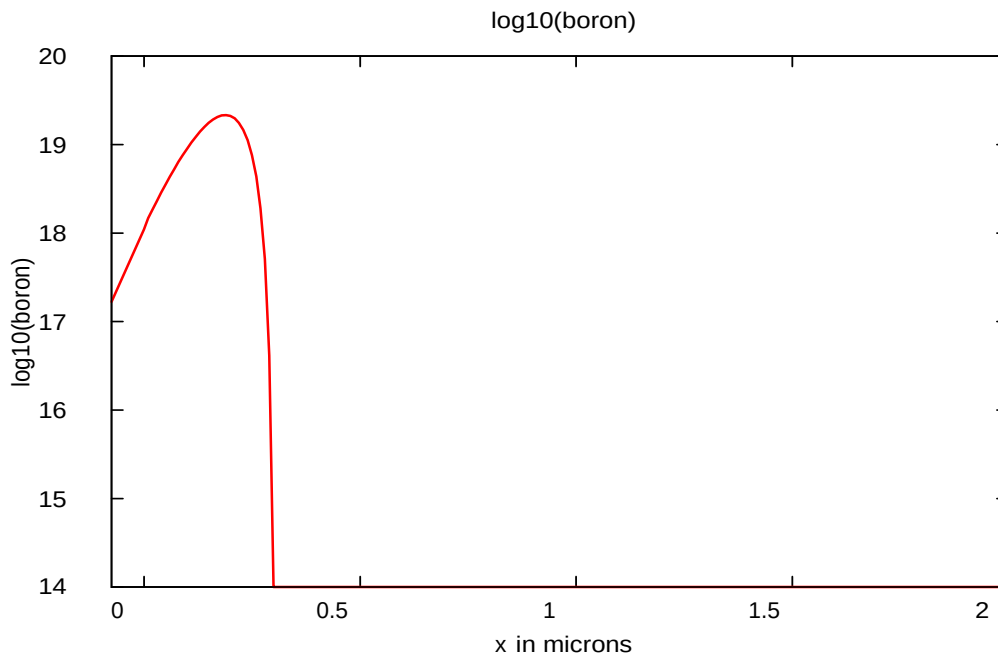


Figure 5. 1: Implanted boron profile.

The diffuse command simulates the 30 minute, 1100 C anneal.

```
#the diffusion card
diffuse time=30 temp=1100
```

The default ambient is inert, so no oxide will be grown. The next step is to save the data for further examination.

```
#save the data
structure out=boron.str
```

This saves the data in the file boron.str. This file can be read in using the init command or the structure command. The following script will plot the final boron concentration.

```
#plot the final profile
select z=log10(bor)
plot.1d x.ma=2.0 y.mi=14.0 y.max=20.0 cle=f axis=f
```

The first statement picks the log base ten of the boron concentration to plot. The plot will be placed on the earlier plot. The ``cle=f axi=f" instructs CSUPREM to not clear and not compute new axes. Figure 5.2 shows a deeper junction as expected, as well as segregation into the oxide. The spike at the surface indicates that the dopant concentration on oxide surface has increased by about 20 times due to annealing.

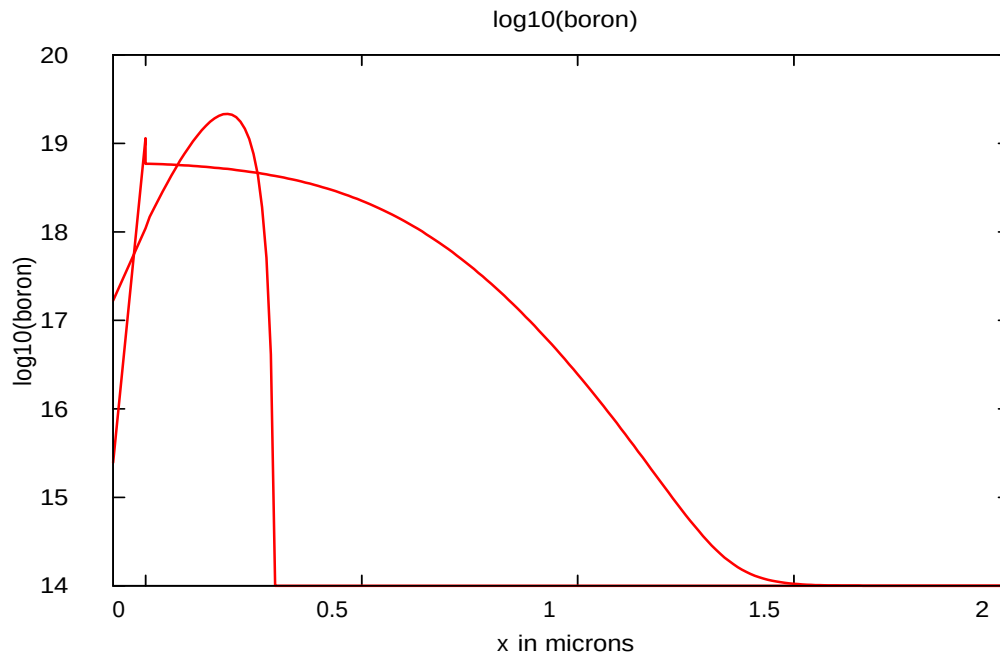


Figure 5. 2: Implanted boron profile. Comparison of annealed and implanted boron profiles.

5.2 Example 2: Boron OED -1D

This example performs a simple anneal of a boron implant under a dry oxygen ambient. The interstitials are injected by the growing oxide and will enhance the boron diffusivity. This example will produce a deeper boron junction than Example 1 because of the interstitial injection. The final structure is saved, and then various post processing is performed. The input file for the simulation is in the ``examples\Stanford_Original\exam_oed'' directory, in the file oed.in.

```
#some set stuff
option quiet
mode one.dim
```

```
#the vertical definition
line x loc = 0      spacing = 0.02  tag = top
line x loc = 1.50   spacing = 0.05
line x loc = 5.0    spacing = 0.5
line x loc = 400.0          tag=bottom
```

```
#the silicon wafer
```

```

region silicon xlo = top xhi = bottom

#set up the exposed surfaces
bound exposed xlo = top xhi = top

#calculate the mesh
init boron conc=1.0e14

#the pad oxide
deposit oxide thick=0.075

#the uniform boron implant
implant boron dose=3e14 energy=70

#plot the initial profile
select z=log10(boron)
plot.1d x.ma=2.0 y.mi=14.0 y.max=20.0 x.min=-0.1

#the diffusion card
method init=1.0e-3 two.d
diffuse time=30 temp=1100 dry

#save the data
structure out=oed.str

#plot the final profile
select z=log10(bor)
plot.1d cle=f axi=f

```

This example is very similar to Example 1. It starts by instructing Csuprem to be relatively quiet about the progress of computation. The x line section is considerably different. In this example, we wish to calculate the oxidation enhanced diffusion of a boron layer under a growing oxide. Therefore, the vertical specification needs to be different.

```

#the vertical definition
line x loc = 0      spacing = 0.02  tag = top
line x loc = 1.50   spacing = 0.05
line x loc = 5.0    spacing = 0.5
line x loc = 400.0          tag=bottom

```

This is similar to the first example. However, we expect the junction to be deeper, therefore the tight grid spacing is maintained out to 1.5 μm . The spacing is increased out to a depth of 5 μm . The backside is placed at the full thickness of the wafer since the defects can travel great distances into the silicon substrate. The next series of lines define the region, boundary, and

initialize the wafer. A starting oxide is deposited and the boron implant is performed. These steps are all identical to Example 1.

The next set of commands choose a plot variable and display it. This plot will be identical to Figure 5.3 of Example 1. The diffuse command contains the directive to simulate the 30 minute, 1100 C drive-in and anneal.

```
#the diffusion card
method init=1.0e-3 two.d
diffuse time=30 temp=1100 dry
```

The ambient is dry oxygen.

In comparison to Example 1, more time steps are needed. This is for two reasons. First, the defects move faster than the boron and they limit the size of the time step initially. At longer times, the defects begin to approach steady state, and the boron diffusion limits the time step. Since the boron diffusivity is enhanced by the injection of interstitials, the time steps have to be smaller than in Example 1 to resolve the faster changes in the boron.

The next step is to save the data for further examination.

```
#save the data
structure out=boron.str
```

This saves the data in the file boron.str. This file can be read in using the initialize command or the structure command.

The following script will plot the final boron concentration.

```
#plot the final profile
select z=log10(bor)
plot.1d cle=f axi=f
```

The first command picks the log base ten of the boron concentration to plot. The plot will be placed on the earlier plot.

The ``cle=f axi=f'' instructs Csuprem to not clear and not compute new axes.

The profile in Figure 5.3 shows a deeper junction as expected, as well as segregation into the oxide. The spike at the surface indicates that the surface oxide concentration is about 20 times larger than the silicon surface concentration.

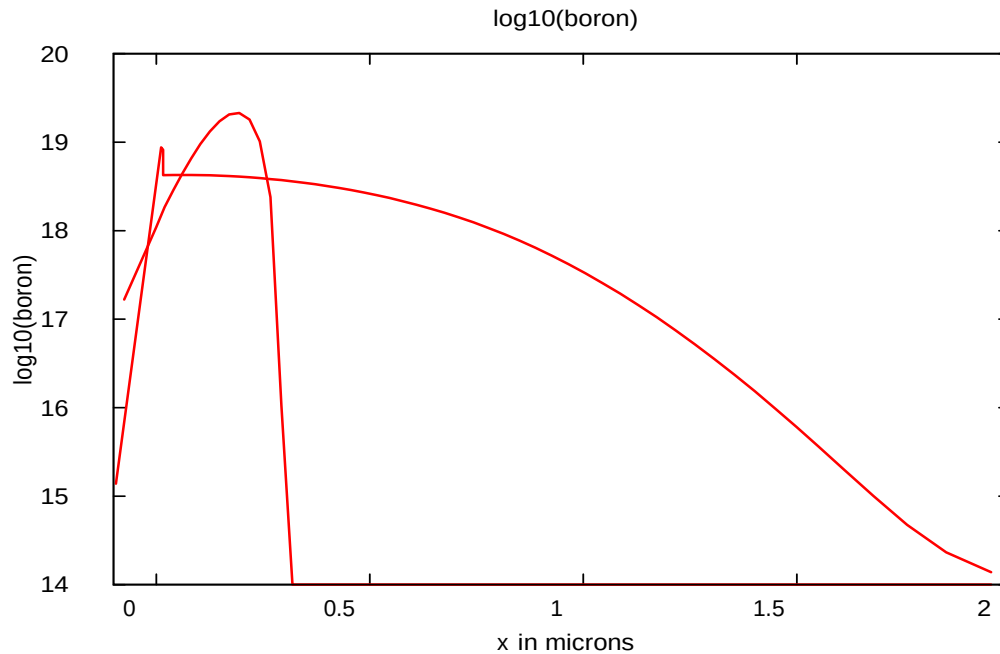


Figure 5. 3: Final structure comparing the boron before and after anneal.

5.3 Example 3: OED time

This example uses the movie command to look at the time dependence of the interstitial concentration resulting from oxide growth. This simulation will be of a silicon membrane 28 μm thick. This will allow the simulation of the effective diffusivity. The input file for the simulation is in the ``examples\Stanford_Original\exam3_oed_time" directory, in the file oed_time.in.

```
#some set stuff
option quiet
mode one.dim
```

```
#the vertical definition
line x loc = 0      spacing = 0.02 tag = top
line x loc = 1.50   spacing = 0.05
line x loc = 5.0    spacing = 0.5
line x loc = 28.0   tag=bottom
```

```
#the silicon wafer
region silicon xlo = top xhi = bottom
```

```

#set up the exposed surfaces
bound exposed   xlo = top   xhi = top
bound backsid   xlo = bottom xhi = bottom

#calculate the mesh
init

#the pad oxide
deposit oxide thick=0.02

#the diffusion card
method init=1.0e-3 two.d

select z=(inter/ci.star)
plot.1d y.min=1.0 y.max=3.5
diffuse time=240 temp=1100 dry
movie="select z=(inter/ci.star) && plot.1d axis=f clear=f"

#save the data
structure out=oed_time.str

```

A remark ``#continued" is used to indicate that an original long command line has been broken and continued onto the next line for the purpose of this document. In a simulation input file, all parameters belonging to the same command should be placed within a single command line. This example is very similar to Example 2. It starts by instructing Csuprem to be relatively quiet about the progress of computation. The x line section is somewhat different. In this example, we wish to calculate the oxidation enhanced diffusion resulting from a growing oxide. Therefore, the vertical specification needs to be different.

```

#the vertical definition
line x loc = 0      spacing = 0.02 tag = top
line x loc = 1.50   spacing = 0.05
line x loc = 5.0    spacing = 0.5
line x loc = 28.0   tag=bottom

```

This is similar to the last example. The backside is placed at the membrane thickness so that we can compute the interstitials at this interface.

The next series of commands define the region, boundary, and initialize the wafer. There are only two differences between this example and the previous one. First is the specification of a backside interface. This will allow the specification of boundary conditions for the defects. Second, no boron implant is performed. This will speed the simulation. A starting oxide is deposited next. The method command specifies the defects should be solved and sets a small

initial time step. The next set of commands initialize some variables that will be used in the diffuse statement.

The basic procedure is to set up a plotting coordinate system for the interstitials. Then, the movie parameter is used to export interstitial profile to be plotted on the same coordinate system, as in the following command lines.

```
select z=(inter/ci.star)
plot.1d y.min=1.0 y.max=3.5
diffuse time=240 temp=1100 dry
movie="select z=(inter/ci.star) && plot.1d axis=f clear=f"
```

The first command in the movie line is to select the scaled interstitial concentration, CI/CI^* as the plot variable. The parameter parameter force the solver to export data to be used by the plot.1d command. ``axis=f" causes all the plots to share the same axis as in the previous plot.1d.

Please note that the time step in a movie plot is determined by the solver according to the initial time step in method command, convergence condition in the diffuse command. To achieve control of time interval of concentration profile plotting, you may experiment with different settings of initial time interval and maximum time step. Figure 5.4 shows a series of curves moving deeper into the substrate indicating propagation of the interstitials as a result of oxidation on the top interface.

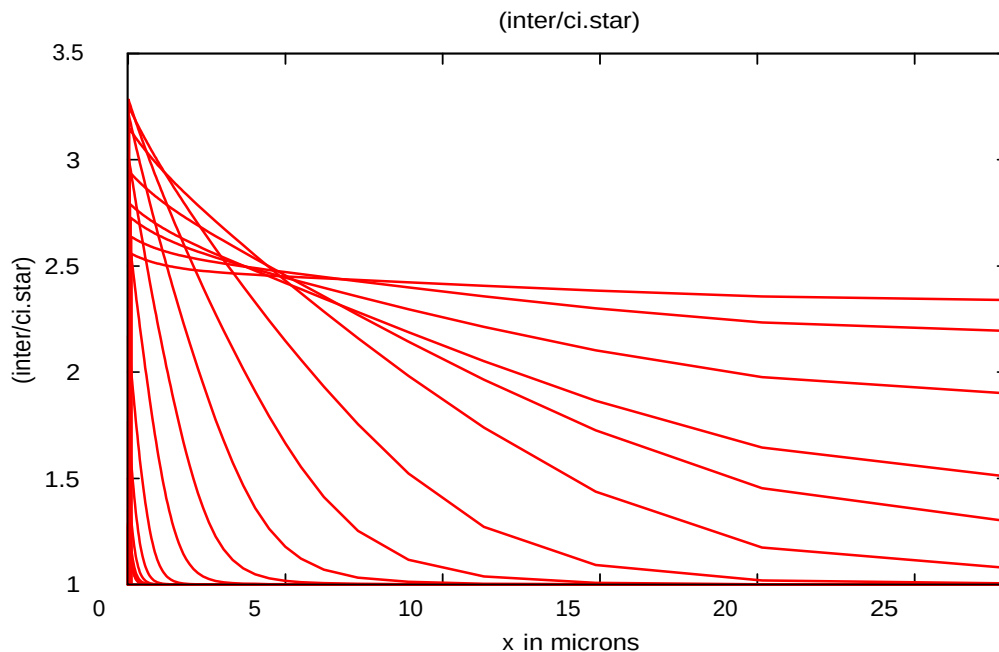


Figure 5. 4: Time dependence of scaled interstitial concentration.

5.4 Example 4: Boron OED - 2D

This example performs a simple anneal of a boron implant under a dry oxygen ambient. The interstitials are injected by the growing oxide and will enhance the boron diffusivity. This example will use a two dimensional structure to investigate the lateral extent of the oxidation enhanced diffusion behavior. In addition, the two dimensional growth of the oxide will be simulated. The final structure is saved, and then various post processing is performed. The input file for the simulation is in the "examples\Stanford_Original\exam4_oed_2d" directory, in the file oed_2d.in.

```
#some set stuff
option quiet
```

```
#the x dimension definition
line x loc = -2.0      tag = left
line x loc = 0.0       spacing=0.05
line x loc = 5.0       tag = right
```

```
#the vertical definition
line y loc = 0         spacing = 0.02      tag = top
line y loc = 1.5       spacing = 0.05
line y loc = 400              tag=bottom
```

```
#the silicon wafer
region silicon xlo = left xhi = right ylo = top yhi = bottom
```

```
#set up the exposed surfaces
bound exposed  xlo = left xhi = right ylo = top  yhi = top
```

```
#calculate the mesh
init boron conc=1.0e14
```

```
#the pad oxide
deposit oxide thick=0.02
```

```
#deposit the nitride mask
deposit nitride thick=0.05
```

```
etch nitride right p1.x=0.0 p2.x=0.0
```

```
#the boron implant
implant boron dose=3e14 energy=40
```

```
#the diffusion card
```

```
method two.d
diffuse time=30 temp=1100 dry
```

```
#save the data
structure out=oed_2d.str
```

```
select z=log10(boron)
plot.2d bound fill y.max=2.0
foreach v (11 to 19 step 1)
  contour val=v
end
```

```
#plot the final profile
select z=(inter/ci.star)
plot.2d bound fill y.max=10
foreach v (1.25 to 3.0 step 0.25)
  contour val=v
end
```

```
select z=(inter/ci.star)
plot.1d y.v=0.5
```

```
select z=(inter/ci.star)
plot.1d x.v=5.0 x.ma=75.0
```

This example is very similar to Example 2. It starts by instructing Csuprem to be relatively quiet about the progress of computation. The x line specification is different for the two dimensional device. For this example, an x line is placed at $-2 \mu m$ and another at $5 \mu m$ for the left and right of the device. The area of most interest is the middle area near where the mask edge will be placed. Consequently, a tighter spacing is specified at the mask edge.

The next series of lines define the y lines, region, boundary, and initialize the wafer. A starting oxide is deposited and the boron implant is performed. These steps are all similar to Example 2. Following the boron implant, a mask is deposited and etched.

```
#deposit the nitride mask
deposit nitride thick=0.05
etch nitride right p1.x=0.0 p2.x=0.0
```

The mask material is specified to be $0.05 \mu m$ of nitride. This will mask the oxide growth. The nitride is etched off to the right of a vertical line at x equal to $0 \mu m$. The oxide will grow on the right and inject interstitials there. The diffuse command contains the directive to simulate the 30 minute, 1100 C drive-in and anneal.

```
#the diffusion card
method two.d
diffuse time=30 temp=1100 dry
```

The ambient is dry oxygen. This diffusion step will produce output similar to Example 2. The next step is to save the data for further examination.

```
#save the data structure out=oed.str
```

This saves the data in the file oed.str. This file can be read in using the structure command. The final lines will plot the final boron concentration, in two dimensions

```
#plot the final profile
select z=(inter/ci.star)
plot.2d bound fill y.max=10
foreach v (1.25 to 3.0 step 0.25)
  contour val=v
end
```

The first statement picks the log base ten of the boron concentration to plot. The next statement instructs Csuprem to plot the two dimensional outline of the device. The bound parameter asks for the material boundaries to be plotted. The area plotted will extend only down to y equal to 2 μm . The contour statements are plotted as part of loop. The foreach command repeats 9 times and sets the variable v to the value specified in the loop. Figure 5.5 shows that the lateral extent of the OED is well over two microns. There is little lateral difference in the shape of the profile.

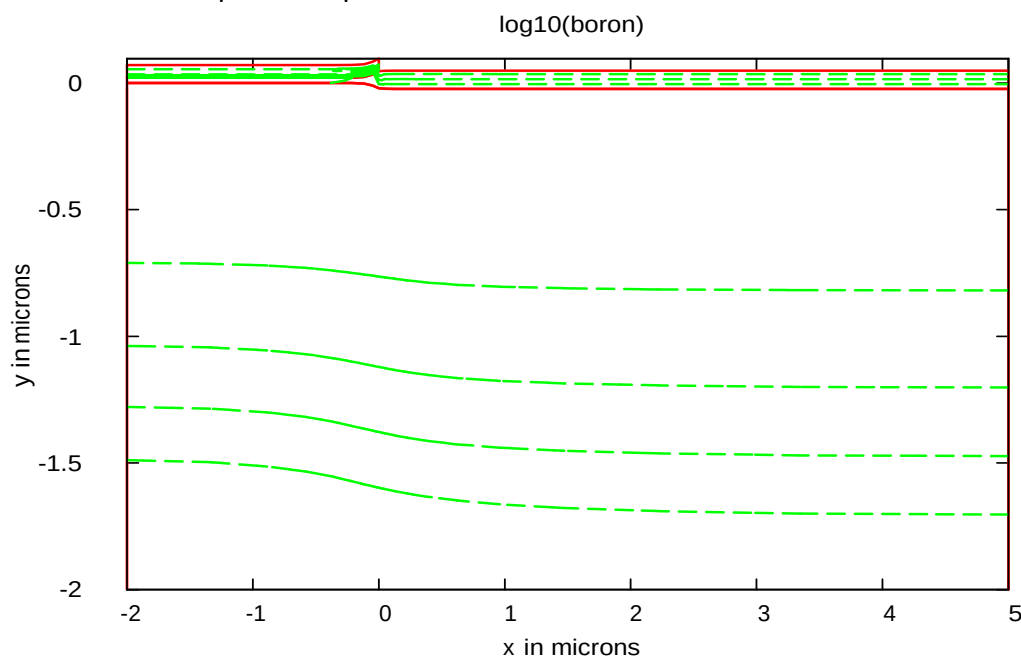


Figure 5. 5: Boron contours resulting from the implant and anneal.

The interstitial concentration can also be plotted. This will verify that there is little lateral decay in the defect profile.

```
#plot the final profile
select z=(inter/ci.star)
plot.2d bound fill y.max=10
foreach v (1.25 to 3.0 step 0.25)
  contour val=v
end
```

Figure 5.6 shows the result. There is more shape to the defects than the boron. However, the profile is fairly flat across the top of the wafer.

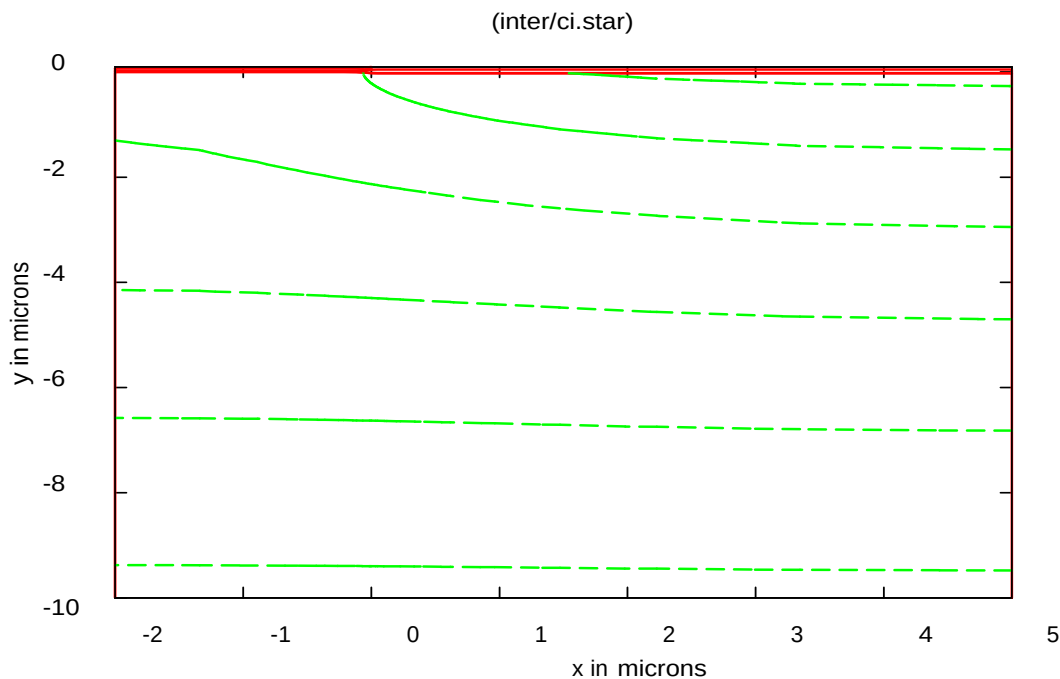


Figure 5. 6: Interstitial contours resulting from the injection.

The interstitial concentration can also be plotted in cross section and is shown in Figure 5.7

```
plot.1d y.v=0.5
```

This command will plot the interstitial concentration $0.5 \mu\text{m}$ below the surface.

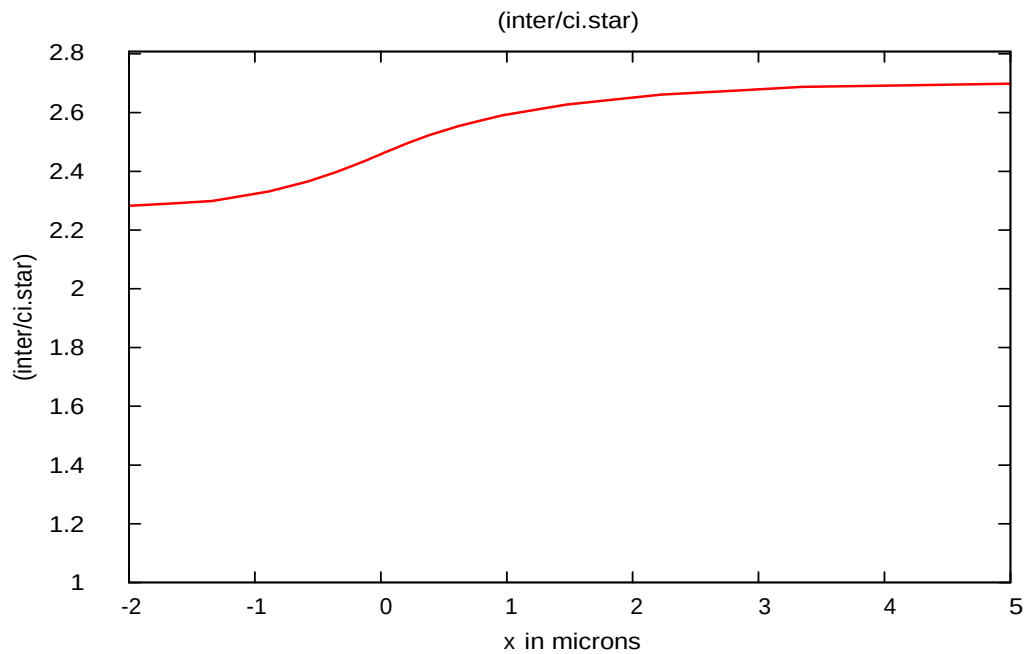


Figure 5. 7: Lateral cross section of the interstitial profile.

The value can also be plotted vertically, and is shown in Figure 5.8.

`plot.1d x.v=5.0 x.ma=75.0`

This shows the penetration into the bulk of the interstitials. The penetration is limited by both the bulk recombination of the interstitials with vacancies and the trapping mechanism.

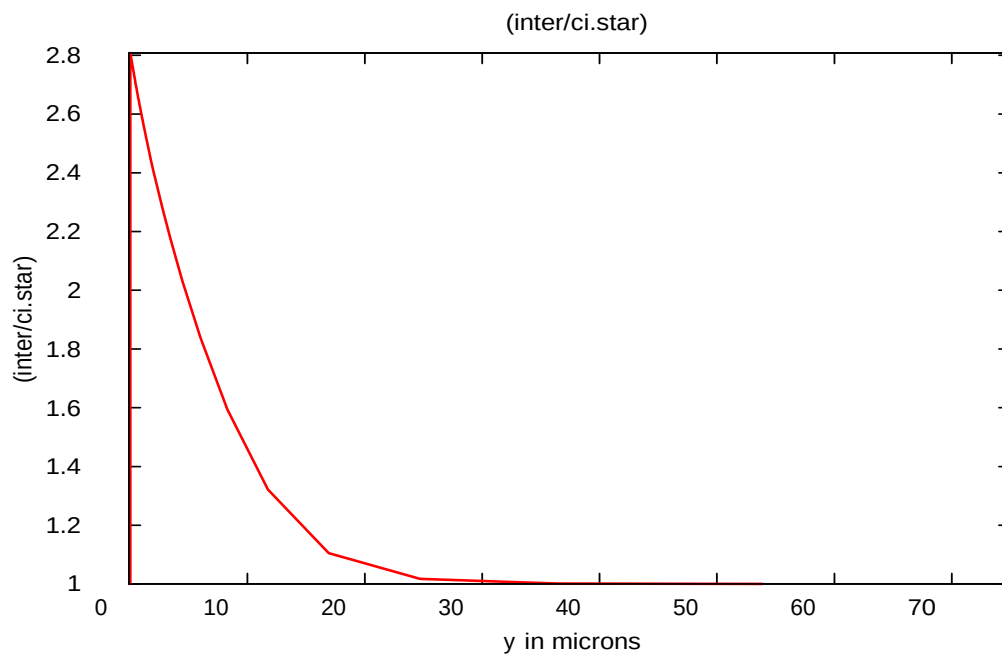


Figure 5. 8: Vertical cross section of the interstitial profile.

5.5 Example 5: LDD cross section

This example simulates the anneal of a lightly doped drain cross section. This example will produce a cross section appropriate to pass to Stanford's PISCES-II and Crosslight's APSYS for device simulation. This example is in the directory ``examples\Stanford_Original\exam5_1dd'', and the input file is listed as:

```
option quiet
option original=true etch.type=1
#
phos poly /gas Trn.0=0.0
bor poly /gas Trn.0=0.0
phos oxide /gas Trn.0=0.0
bor oxide /gas Trn.0=0.0
#change the phosphorus diffusivity like this
#phos silicon Dix.0=0.0 Dim.0=0.0 Dimm.0=0.0
#
line x loc=0.0 tag=lft spacing=0.25
line x loc=0.45          spacing=0.03
line x loc=0.75          spacing=0.03
line x loc=1.4           spacing=0.25
line x loc=1.5 tag=rht spacing=0.25

line y loc=0.0 tag=top spacing=0.01
line y loc=0.1                      spacing=0.01
line y loc=0.25          spacing=0.05
line y loc=3.0 tag=bot

region silicon xlo=lft xhi=rht ylo=top yhi=bot
bound exposed xlo=lft xhi=rht ylo=top yhi=top
bound backside xlo=lft xhi=rht ylo=bot yhi=bot
init boron conc=1.0e16

#deposit the gate oxide
deposit oxide thick=0.025

#channel implant
implant boron dose=1.0e12 energy=15.0

#deposit the gate poly
deposit poly thick=0.500 div=10 phos conc=1.0e19

#anneal the beast
diff time=10 temp=1000
```

```
#etch the poly away
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55
```

```
#anneal this step
diffuse time=30.0 temp=950
```

```
struct outf=poly.str
```

```
#do the phosphorus implant
implant phos dose=1.0e13 energy=50.0
```

```
#deposit the oxide spacer
deposit oxide thick=0.400 spac=0.05
```

```
#etch the spacer back
etch dry oxide thick=0.420
```

```
struct outf=imp2.str
#after etch anneal
method vert fermi grid.ox=0.0
diffuse time=30 temp=950 dry
```

```
#implant the arsenic
implant ars dose=5.0e15 energy=80.0
```

```
#deposit a cap oxide
deposit oxide thick=0.15 space=0.03
```

```
struct outf=imp4.str
```

```
#do the final anneal
diffuse time=20 temp=950
struct outf=ldd.str
```

```
select z=log10(phos+ars)-log10(bor)
plot.2d bound fill y.max=1.0
contour val=0.0
```

The command:

```
option original=true etch.type=1
```

is used to turn on original version of etch and deposit model as inherited from Stanford. The default etching and deposition model are implemented by Crosslight and they are etch type 3 and vertical mesh respectively.

The next section begins the definition of the mesh to be used for the simulation. The section

```
phos poly /gas Trn.0=0.0
bor poly /gas Trn.0=0.0
phos oxide /gas Trn.0=0.0
bor oxide /gas Trn.0=0.0
```

turns off the out diffusion of phosphorus and boron. In this simulation, there are anneals of bare poly. To keep the time step from being limited by the out-diffusion behavior, the gas transport is turned off by setting the transport rate pre-exponential constant to zero.

The next section describes the locations of the x lines in the mesh.

```
line x loc=0.0 tag=lft    spacing=0.25
line x loc=0.45          spacing=0.03
line x loc=0.75          spacing=0.03
line x loc=1.4           spacing=0.25
line x loc=1.5 tag=rht    spacing=0.25
```

Csuprem defines x to be the direction across the top of the wafer, and y to be the vertical dimension into the wafer. Similar to Example 4, the location of the mesh lines is chosen to minimize the spatial error and to represent the location of various etch steps.

The next section of input describes the location and spacings of the vertical mesh lines.

```
line y loc=0.0 tag=top    spacing=0.01
line y loc=0.1           spacing=0.01
line y loc=0.25          spacing=0.05
line y loc=3.0 tag=bot
```

The lines are chosen close together near the surface and increasing away from it. Tight spacing is maintained only near the top surface. Since the structure will be passed to APSYS, a depth of 3.0 μm is chosen so that the eventual substrate contact will be deep to not unduly influence the simulation. The next two sections describe the device starting material and the surfaces which are exposed to gas.

```
region silicon xlo=lft xhi=rht ylo=top yhi=bot
bound exposed xlo=lft xhi=rht ylo=top yhi=top
bound backside xlo=lft xhi=rht ylo=bot yhi=bot
```

The region statement is used to define the starting materials. In this case the wafer is silicon with no initial masking layers. The bound statement allows the definition of the front and backsides of the wafer. Any gasses on the diffuse, deposit, and etch commands are applied to the surface marked exposed. The backside will not influence the process simulation. However, the backside is the location of a contact when the specification of the electrodes is made for PISCES-II. For APSYS simulation, the backside definition is not needed and the command is kept for compatibility reasons.

The next line informs Csuprem that the mesh has been defined and should be computed.

```
init boron conc=1.0e16
#deposit the gate oxide
deposit oxide thick=0.025
```

The first line initializes the starting material. The deposited oxide is specified to 0.025 μm thick. This represents the gate oxide that is grown before the uniform boron implant.

The next statement performs the channel implant of the boron.

```
#channel implant
implant boron dose=1.0e12 energy=15.0
```

The implant is modeled with a Pearson-IV distribution. The energy and dose are $10^{12} cm^{-2}$ and 15 KeV respectively. This produces an abrupt boron profile with a shallow junction.

The next commands define the poly deposition and anneal. The poly deposition is done first, then the heat cycle that follows does the temperature cycle that occurs during actual poly deposition.

```
#deposit the gate poly
deposit poly thick=0.500 div=10 phos conc=1.0e19
#anneal the beast
diff time=10 temp=1000
#etch the poly away
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55
#anneal this step
diffuse time=30.0 temp=950
struct outf=poly.str
```

The first pair of statements deposit the poly and then provide a temperature step representing the deposition. This temperature step will cause diffusion of the threshold adjust implant. The poly is put down with ten grid layers and doped to concentration of $10^{19} cm^{-3}$ phosphorus. The next statements etch the gate material and perform a poly anneal. The structure is then saved in the file ``poly.str".

The phosphorus light implant and spacer definition comes next.

```
#deposit the oxide spacer
deposit oxide thick=0.400 spac=0.05
#etch the spacer back
etch dry oxide thick=0.420
struct outf=imp2.str
```

This implant is the one which will form the lightly doped drain. The next two steps of the full device are to deposit and etch the spacer oxide. The spacer is deposited with 10 divisions, and the curvature is resolved to $0.04 \mu m$. The oxide is etched back a thickness of $0.42 \mu m$. The dry parameter indicates that the oxide surface is to be dropped straight down. Finally, the structure at this stage is saved.

The oxide is regrown and the phosphorus annealed during the next step.

```
#after etch anneal
method vert fermi grid.ox=0.0
diffuse time=30 temp=950 dry
```

The method command specifies the oxide will grow vertically. No injection of interstitials will be simulated during this oxide growth. The diffuse command specifies that the anneal is to be done for 30 minutes at 950 C in dry O₂.

The next step implants the heavily doped portion of the drain, and deposits the cap oxide.

```
#implant the arsenic
implant ars dose=5.0e15 energy=80.0
#deposit a cap oxide
deposit oxide thick=0.15 space=0.03
struct outf=imp4.str
```

The arsenic is implanted through the new oxide growth and then a cap oxide is deposited. The structure is saved in the file ``imp4.str".

Finally, we want to do the final anneal.

```
#do the final anneal
diffuse time=20 temp=950
struct outf=ldd.str
```

This anneal will be carried out for 20 minutes at 950 C. The structure is saved in the file ldd.str. At this point, any of the structure files can be read back and examined to check the results. Commands and analysis similar to that found in Example 4 may be performed.

To just perform a simple check, the following commands should produce a plot.

```
plot.2d bound fill y.max=1.0  
select z=log10(phos+ars)-log10(bor)  
contour val=0.00
```

The first command plots the device outline. The second command selects the difference in the logs of the doping. This tends to produce much smoother contours for plotting purposes. The final command plots the junction location, as shown in Figure 5.9.

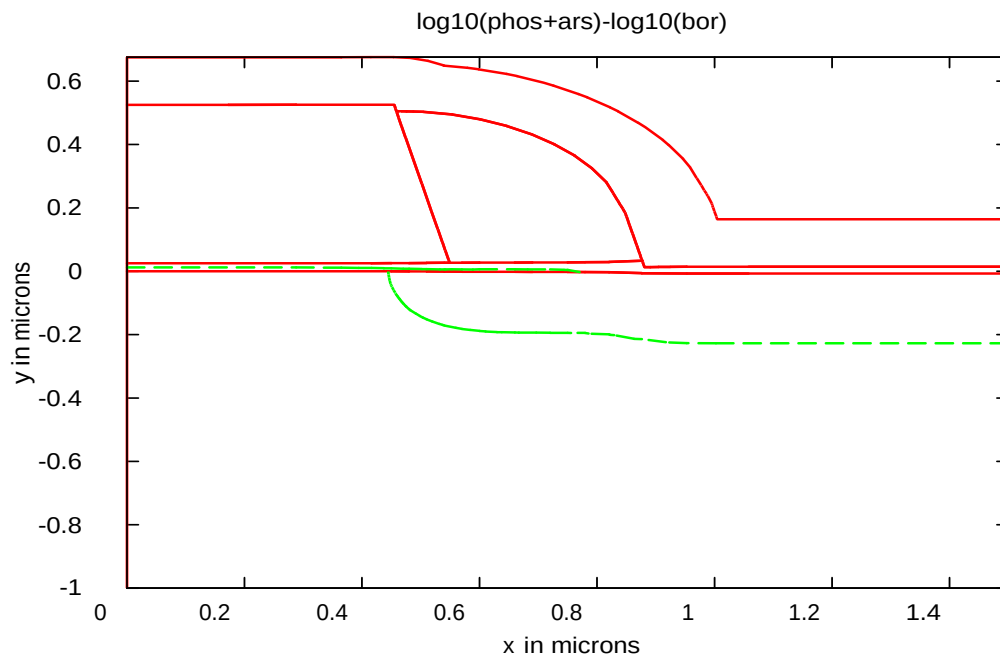


Figure 5. 9: Final LDD Device Structure with junction contour line.

Figure 5.10 is the boron 2D profile generated by CrosslightView from ldd.str.

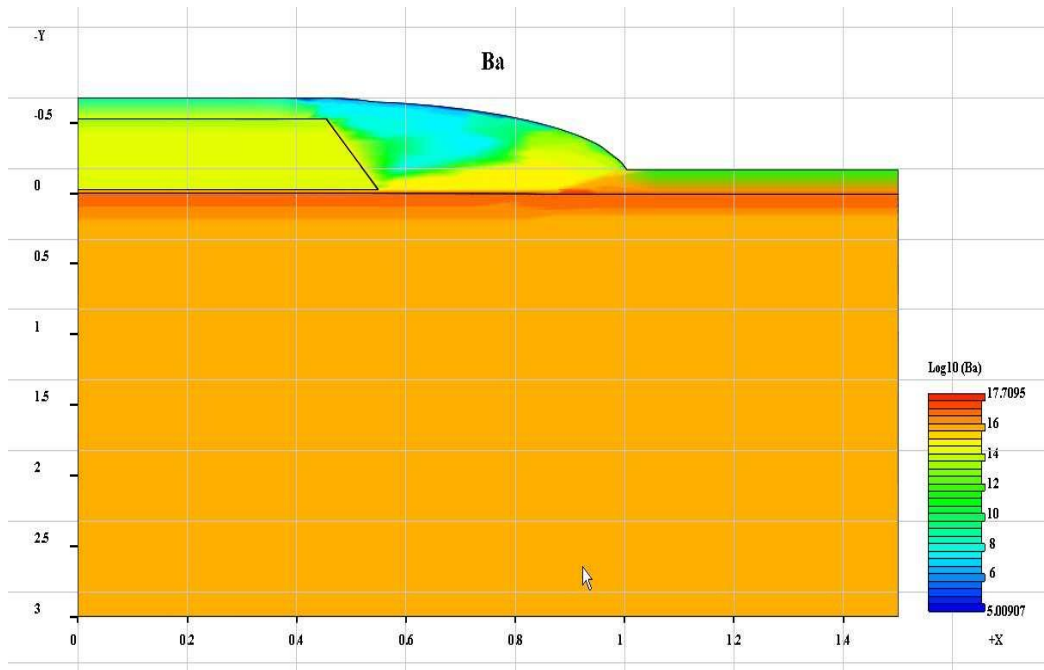


Figure 5.10: Boron 2D profile generated by CrosslightView.

Figure 5.11 is the phosphorus 2D profile generated by CrosslightView from ldd.str.

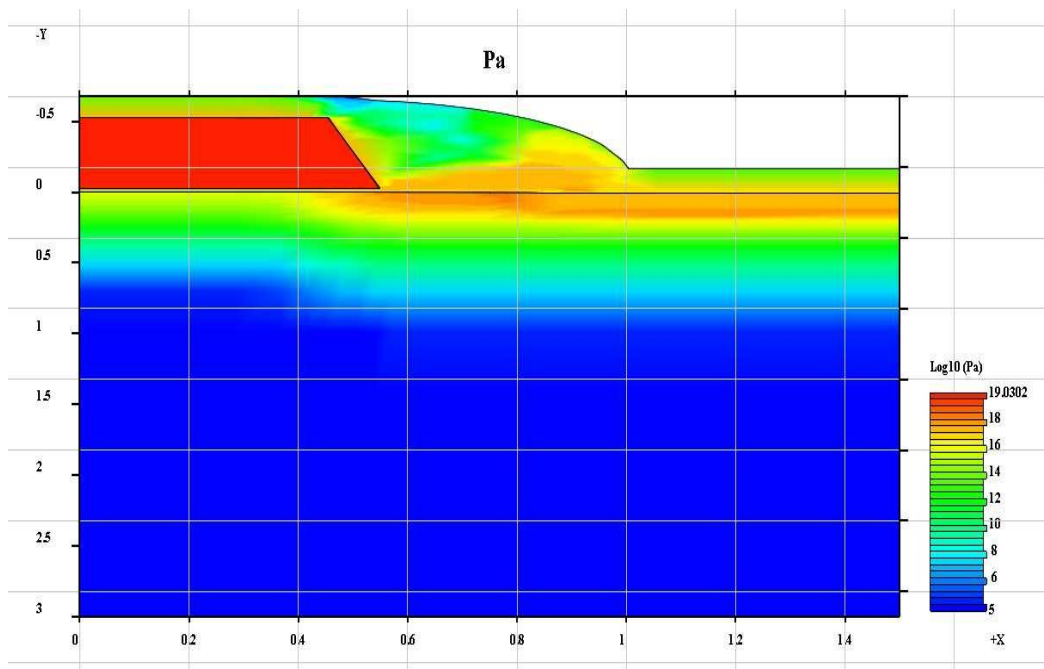


Figure 5.11: Phosphorus 2D profile generated by CrosslightView.

Figure 5.12 is the arsenic 2D profile generated by CrosslightView from Idd.str.

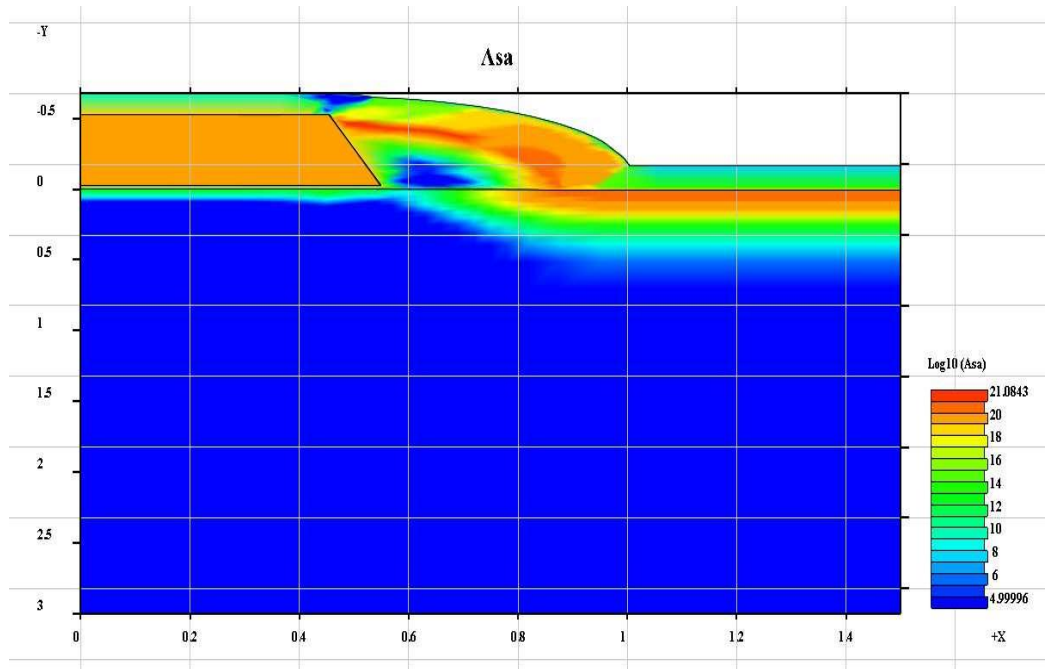


Figure 5.12: Arsenic 2D profile generated by CrosslightView.

Finally Figure 5.13 is a plot of $\log(\text{As}+\text{P})-\log(\text{B})$ (to determine junction depth) generated by CrosslightView from Idd.str.

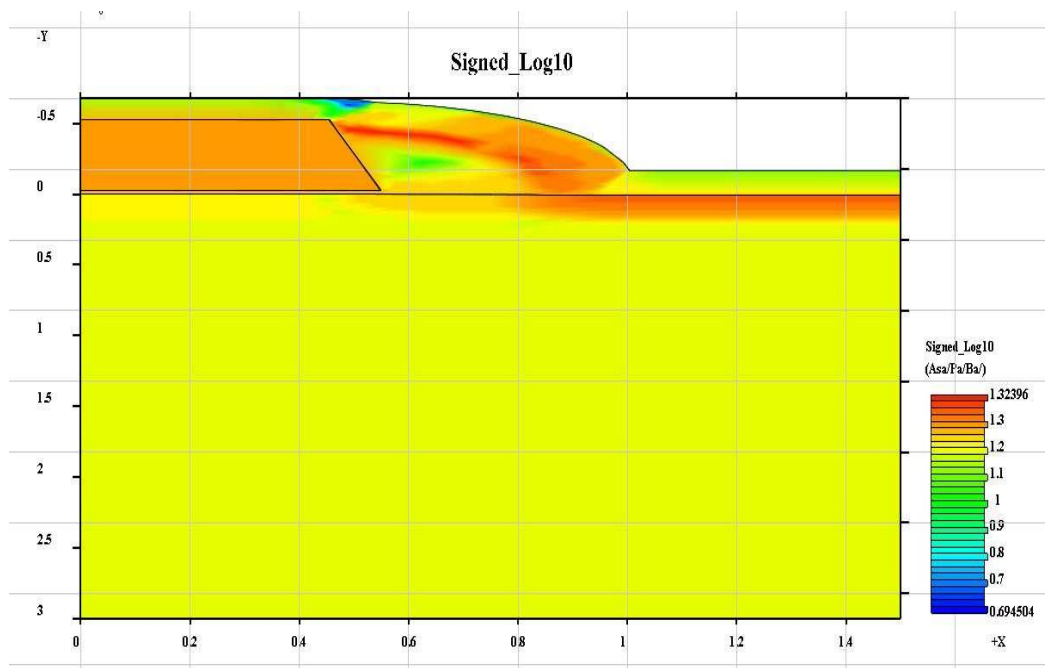


Figure 5.13: Junction 2D profile $[\log(\text{donor})-\log(\text{acceptor})]$ generated by CrosslightView.

5.6 Example 6: One dimensional oxide growth

This example compares the oxide thickness grown for different orientations. The input file for the simulation is in the ``examples\Stanford_Original\exam6" directory and listed as follows:

```
#---some set stuff
option quiet

foreach 0 (100 110 111)

#---the minimal mesh
line x loc = 0.0 tag = left
line x loc = 1.0 tag = right

line y loc = 0.0 tag = top
line y loc = 1.0 tag=bottom

region silicon xlo = left xhi = right ylo = top yhi = bottom

bound exposed  xlo = left xhi = right ylo = top  yhi = top

init ori=0

#---the oxidation step
meth  init.time=100  grid.ox=0.03
diffuse time=30 temp=900 wet

select z=y*1e8 label="Depth(A)"
print.1d x.v=0 form=8.0f layers

struct out=$0.str

end

The loop

foreach o (100 110 111) ... end
```

runs its body three times for three different orientations.

The first few lines of the loop body define the simplest mesh possible. It has exactly four lines (one rectangle), and a front side. The method command chooses the oxide grid spacing to be

0.03 μm , knowing that 30 minutes at 900 C will grow about 0.1 μm of oxide. The oxidation step chooses the vertical model by default. The vertical model solves the Deal-Grove diffusion equation for oxidant, and uses the calculated velocities to move the oxide vertically everywhere. For the purposes of thickness on a planar or near-planar structure, the accuracy is good enough. Grid is added automatically to the oxide as it grows, by default at increments of 0.1 μm . The time step is initially 0.1 second; subsequently it is automatically chosen to add no more than 14 of the grid increment per time step.

When the diffusion is done, the select statement fills the ``z vector" with the y (vertical) locations of the points, scaled to angstroms. The print.1d statement then prints the y displacements along the left side of the mesh.

The print.1d output is as follows:

```
---> ori=100
y coordinate in um    Depth(A)    Material
-0.01081667          -108        oxide
  0.02939687           294        oxide
  0.03488766           349        oxide
  0.03488766           349        silicon
```

```
--->ori=110
y coordinate in um    Depth(A)    Material
-0.02276796          -228        oxide
  0.01709697           171        oxide
  0.04614729           461        oxide
  0.04614729           461        silicon
```

```
--->" init ori=111"
y coordinate in um    Depth(A)    Material
-0.06455074          -646        oxide
-0.02986426          -299        oxide
  0.00977623            98        oxide
  0.05295895           530        oxide
  0.05295895           530        silicon
```

The results are saved in three different files 100.str, 110.str and 111.str and are plotted in Figures 5.14, 5.15, and 5.16 respectively. These results clearly show different oxide thickness for different silicon crystal orientations.

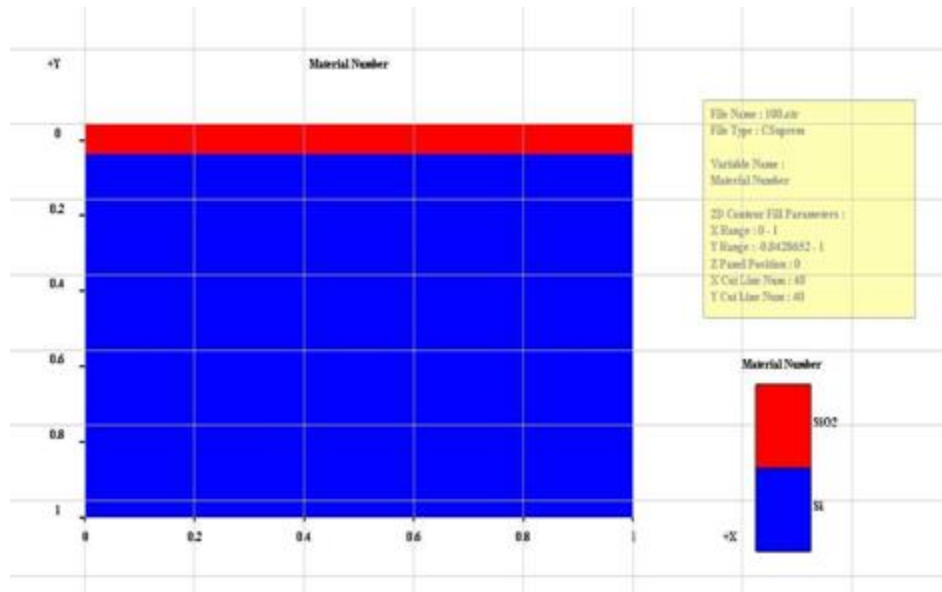


Figure 5.14: Simulated structure showing oxide growth in silicon 100 direction.

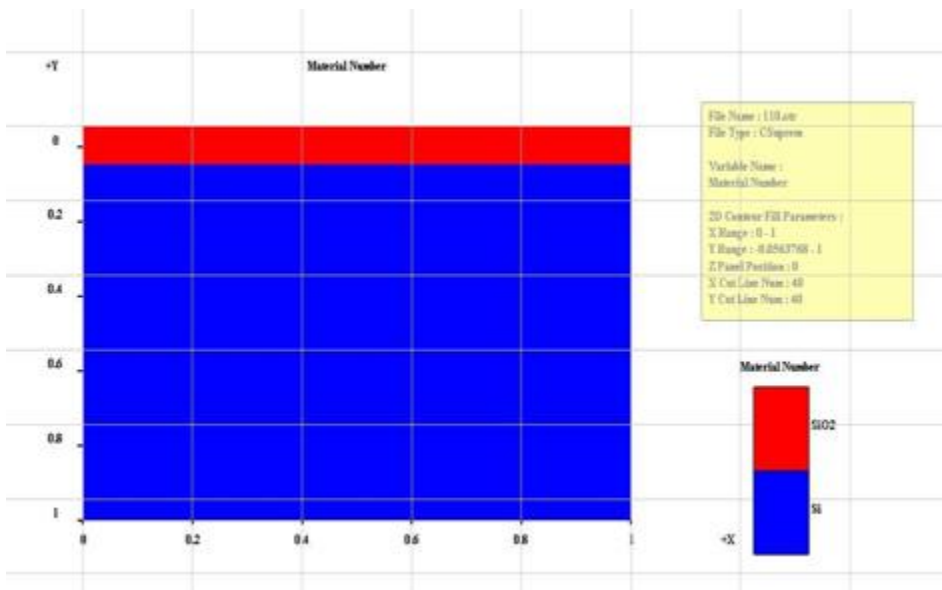


Figure 5.15: Simulated structure showing oxide growth in silicon 110 direction.

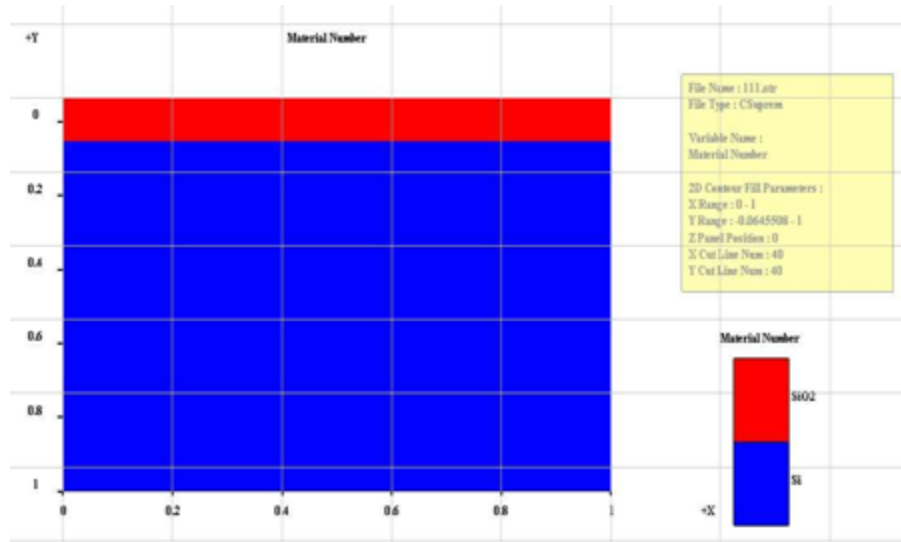


Figure 5. 16: Simulated structure showing oxide growth in silicon 111 direction.

5.7 Example 7: Fully Recessed Oxide Growth

This example shows the evolution of an oxide profile during semi-recessed oxidation. It uses the Deal-Grove model to calculate the oxide growth rate at the silicon/oxide interface, and treats the deformation of the oxide and nitride as viscous incompressible flow. The input file for the simulation is in the ``examples\Stanford_Original\exam7_beak'' directory. The input file is listed as follows.

```
# --Recessed LOCOS cross section: recess 0.3 um, grow 0.54um (0.23 + 0.31)--
option quiet
```

```
#-----Substrate mesh definition
```

```
line y loc=0   spac=0.05   tag=t
```

```
line y loc=0.6 spac=0.2
```

```
line y loc=1           tag=b
```

```
line x loc=-1         spac=0.2 tag=l
```

```
line x loc=-0.2       spac=0.05
```

```
line x loc=0          spac=0.05
```

```
line x loc=1          spac=0.2 tag=r
```

```
region silicon xlo=l xhi=r ylo=t yhi=b
```

```
bound expo xlo=l xhi=r ylo=t yhi=t
```

```
init or=100
```

```
#-----Anisotropic silicon etch
etch silicon left p1.x=-0.218 p1.y=0.3 p2.x=0 p2.y=0

#-----Pad oxide and nitride mask
deposit oxide thick=0.02
deposit nitride thick=0.1
etch nitride left p1.x=0
etch oxide left p1.x=0

plot.2d grid bound

#-----Field oxidation
meth compr

plot.2d bound y.mi=-0.5 line.b=2
diffuse tim=90 tem=1000 weto2 movie="plot.2 b cle=f axi=f"

#diffuse tim=90 temp=1000 weto2 movie="plot.2d bound y.mi=-0.5 line.b=2"

struct outfile=fc.str

plot.2d bound flow vleng=0.1
```

The first line asks for as little output as possible. The grid definition comes next. First the vertical lines are defined:

```
line y loc=0 spac=0.05 tag=t
line y loc=0.6 spac=0.2
line y loc=1 tag=b
```

Ninety minutes in 1000 C steam grows about $0.54 \mu m$ of oxide on $\langle 100 \rangle$ silicon. In the process, $0.24 \mu m$ of silicon is consumed. The second line is around the expected final depth of the oxide, and there is one more line at $1 \mu m$ to round out the grid.

```
line x loc=-1 spac=0.2 tag=l
line x loc=-0.2 spac=0.05
line x loc=0 spac=0.05
line x loc=1 spac=0.2 tag=r
```

The x lines run from -1 to 1 for symmetry, with the mask edge at 0. The extra refinement around $0.25 \mu m$ is to prepare for the silicon etch, which will be at an angle of 54 degrees and therefore has a lateral extent of $0.3 \mu m / \tan 54 \text{ degrees}$ approximately equals 0.218.

```
region silicon xlo=l xhi=r ylo=t yhi=b
bound expo xlo=l xhi=r ylo=t yhi=t
```

```
init or=100
```

The region statement identifies the entire area as silicon substrate. It refers to the tags defined on the x and y lines. These tags are used to label lines uniquely so that new lines can be added or subtracted easily without renumbering. The boundary statement identifies the top surface as being exposed. (Csuprem does not assume the top is exposed.) Layer depositions, oxidations and impurity predepositions only happen on "exposed" surfaces, so this statement must not be omitted. The initialize command causes the initial rectangular mesh to be generated. The substrate orientation is $\langle 100 \rangle$.

```
#-----Anisotropic silicon etch
etch silicon left p1.x=-0.218 p1.y=0.3 p2.x=0 p2.y=0
```

The silicon substrate is etched. The etch statement removes all silicon found lying to the left of the line joining the points (-0.218, 0.3) and (0,0).

```
#-----Pad oxide and nitride mask
deposit oxide thick=0.02
deposit nitride thick=0.1
etch nitride left p1.x=0
etch oxide left p1.x=0
```

```
plot.2d grid bound
```

The pad oxide is put down, then nitride is deposited on top and patterned. The patterning is presumed to remove the underlying pad oxide also. The plot command shows the structure before oxidation and is shown in Figure 5.17.

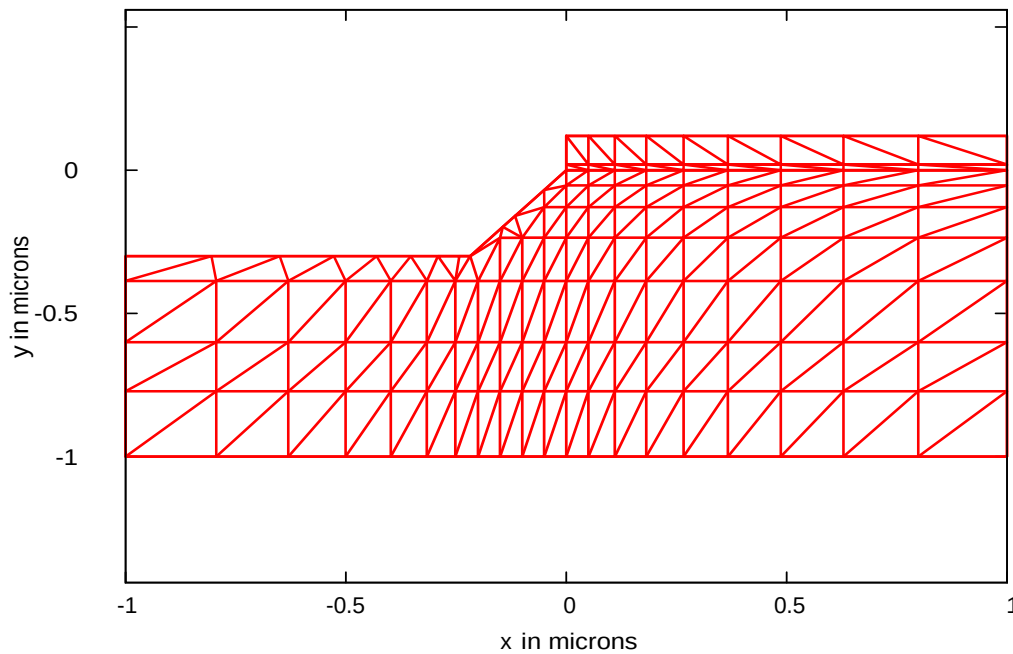


Figure 5.17: Initial grid structure before oxidation.

```
#-----Field oxidation
meth compr
```

The compressible flow model is chosen for oxidation. The incompressible model could also have been used, but for this example where the stress is not desired, the compressible model is faster and nearly as accurate.

```
plot.2d bound y.mi=-0.5 line.b=2
diffuse tim=90 tem=1000 weto2 movie="plot.2 b cle=f axi=f"
```

The diffuse statement is the point of the exercise. For 90 minutes, oxide grows at the silicon interface. The new oxide being formed pushes up the old oxide and nitride layers which cover it, causing the characteristic bird's head profile. The movie parameter lists any extra actions to take at each time step. In this case, the boundary is plotted without erasing or re-scaling the picture. A series of outlines is generated, one from each time step, illustrating the evolution of the oxide profile (Figure 5.18). The first plot.2d bound defined a plotting window large enough so that the subsequent plots without axes would fit inside it.

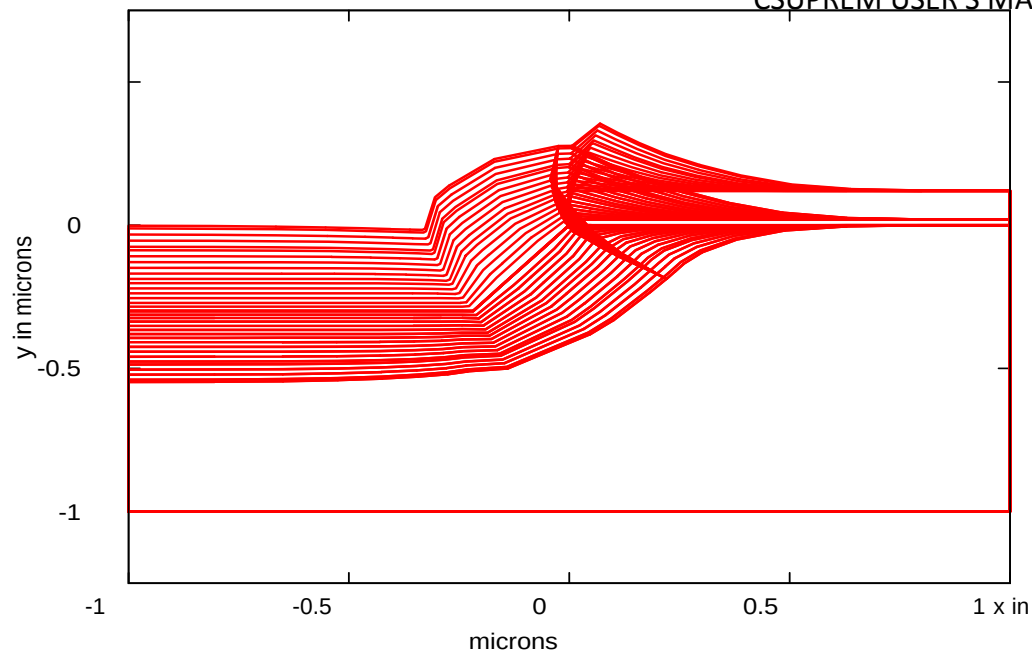


Figure 5. 18: Evolution of bird's beak profile.

`plot.2d bound flow vleng=0.1`

The flow pattern at the last time step can be plotted with the statement above. The parameter `vleng` is the length to draw the longest velocity vector. The other vectors are scaled proportionally. This results in Figure 5.19. The flow is normal at the interface but becomes more vertically oriented away from it.

The structure at the end of oxidation is plotted by CrosslightView is in Figure 5.20.

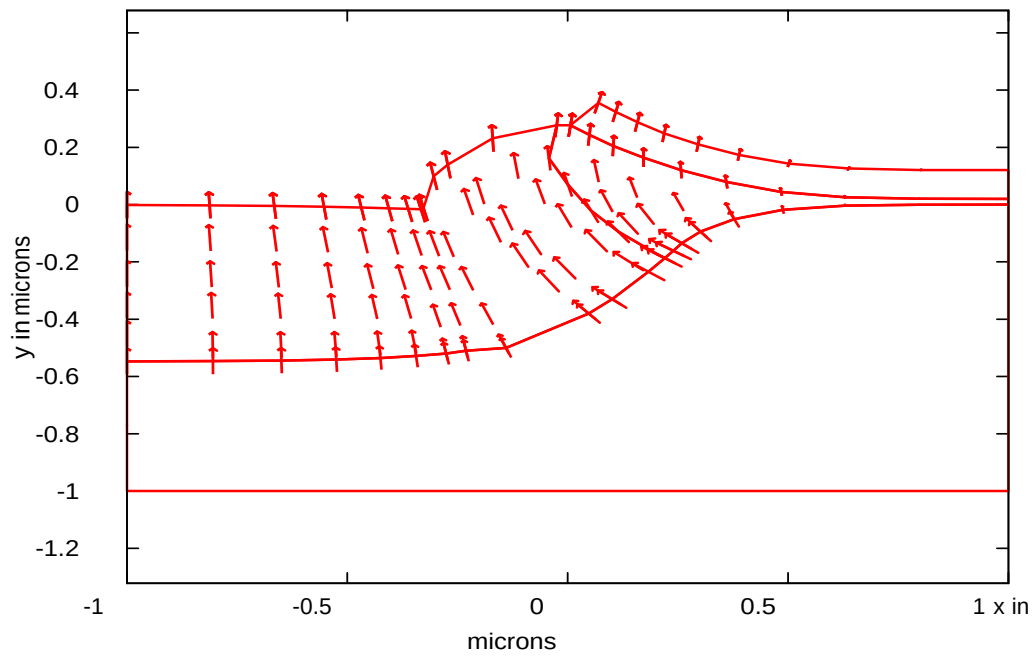


Figure 5.19: Flow vectors at the end of oxidation.

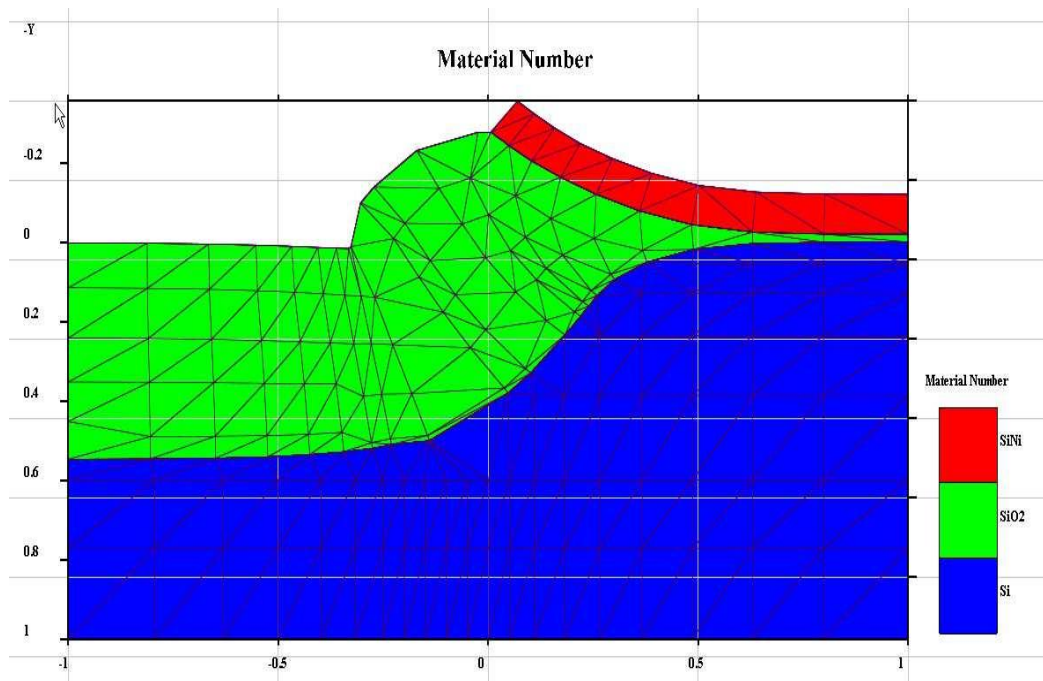


Figure 5.20: Bird's beak by CrosslightView at the end of oxidation.

5.8 Example 8: Shear stress

This example calculates the extent of shear stress in the silicon substrate, generated by a film edge. It is well known that when Si_3N_4 is deposited on silicon by chemical vapor deposition, a large intrinsic stress is present in the layer. If the film is continuous and sufficiently thin, this stress does not present a problem. The substrate is much thicker (1000 times) than the film, so the substrate stress is 1000 times less than the film stress. If the film is etched, however, there is a localized stress in the silicon close to the film edge which is of the same order of magnitude as the stress in the film. This large stress can induce dislocations directly, or indirectly through several mechanisms. It can also induce dislocations to glide from implanted regions into masked regions, causing junction leakage. Similar stress patterns are set up by thermal expansion mismatches between the substrate and overlying films. A simulation of the substrate stress induced by a nitride film is shown below. This example can be found in the "examples\Stanford_Original\exam8" directory under the name. A $\langle 111 \rangle$ substrate is considered with mask edges aligned along the $\langle 110 \rangle$ directions. The nitride film is 0.02 microns thick. The intrinsic stress in the nitride is assumed to be $1.4 \times 10^{10} \text{ dynes/cm}^2$, as reported in Irene [1]. The input deck for the simulation begins as follows.

```
option quiet
#
# nitride on silicon example
foreach SEP ( 10 5 4 2.5 )

    line x loc=(-$SEP) tag=l
    line x loc=-2.0 spac=0.3
    line x loc=0 spac=0.1 tag=m
    line x loc=2.0 spac=0.3
    line x loc=$SEP tag=r

    line y loc=0 spac=0.1 tag=si
    line y loc=2 spac=0.3
    line y loc=5 tag=b

    region silicon xlo=l xhi=r ylo=si yhi=b
    bound expos xlo=l xhi=r ylo=si yhi=si
    initial ori=111
```

Quite a large piece of substrate is analyzed, starting with a space 10 microns by 5 microns. The foreach loop chooses four different widths to examine the effect of different stripe separations. The structure being simulated has reflecting boundary conditions (no perpendicular displacement) on the left and right sides. Those boundary conditions correspond to a single instance of a repeating pattern of nitride stripes. When the stripes are widely separated, there is little interference between the stress field of one stripe and the next, and the results are found to be independent of the stripe separation. At smaller separations, the patterns begin to overlap. This example will show the stress pattern for widely separated stripes and the modifications that occur as the stripes are brought closer together.

The backside of the wafer also has a reflecting boundary condition. Although that approximation is not quite physical, the nitride film primarily exerts a horizontal force on the substrate, the assumption does not seriously affect the result. This can be verified by using different backside thicknesses; 2 microns or 100 microns gives identical results. The exposed surface is the only surface with free displacements. The spacing around the film edge is 0.1 microns. This could be reduced for more accuracy, at the cost of more cpu time.

```
deposit nitride thick=0.02 div=2
etch nitride left p1.x = 0
```

The nitride is deposited directly on silicon and patterned.

```
material intrin.sig = 1.4e10 nitride
```

The initial nitride stress is specified in *dynes/cm²*.

```
stress temp1=1000 temp2=1000
```

This calculates the stress distribution arising from the nitride initial stress. Stress arising from thermal expansion mismatch would be included by specifying a different temp2. Be warned that large thermal steps often bring into play many more complicated phenomena than the simple thermal expansion mismatch analyzed in Csuprem. For instance breakdown of film adhesion or structural change may occur, but are not taken into account in the program.

The principal slip system in silicon is in the $\langle 110 \rangle$ direction on 111 planes. This corresponds to the σ_{xy} shear force in the plane of the simulation. The value $3.0E7$ is considered by Hu[2] to be the critical shear stress for slip in silicon. Dislocations found in regions where the shear stress is larger than that value will move under the stress field of the nitride film. Therefore when analyzing stress in the substrate, a principal concern is the extent of the σ_{xy} equal to $3.0E7$ contour.

```
plot.2d bound x.mi=-2.0 x.ma=2.0 y.ma=4.0 cl=f
select z=Sxy
contour val=-3.0e7
end
```

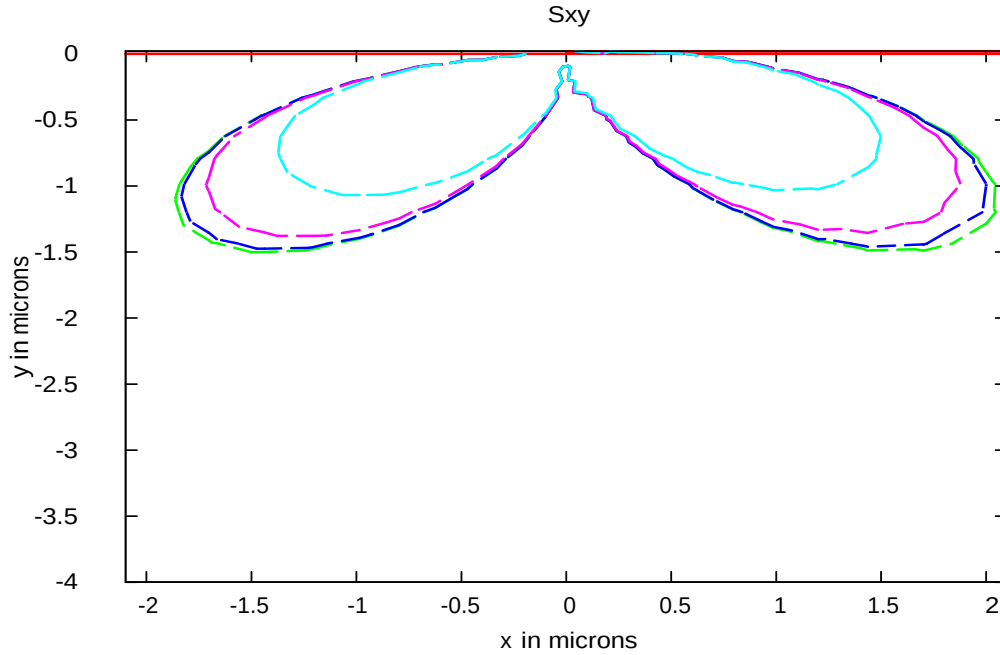


Figure 5. 21: Critical Shear Stress Contours.

The contours of σ_{xy} equal to $3.0E7$ in the silicon are shown in Figure 5.21. The contour is a double lobe because the shear stress is related to the polar components of stress through

$$\sigma_{xy} = (\sigma_{rr} - \sigma_{\theta\theta})\sin 2\theta + \sigma_{r\theta}\cos 2\theta \sin 2\theta \quad 5.1$$

The function $\sin 2\theta$ is at a maximum around 45 degrees from the vertical, and is zero at $\theta = 90$. Both σ_{rr} and $\cos 2\theta$ change sign moving from left to right, so that the sign of σ_{xy} is the same throughout. This means that a dislocation which is being driven under the mask by the stress field will continue to move in that direction after passing the mask edge. However as it passes through the center, the decrease in shear stress may leave it stranded under the mask edge. Figure 5.21 shows that the area of influence of the nitride film is many times greater than its thickness, about 2 microns horizontally, and 1.5 microns vertically. The largest lobe corresponds to the 10 microns stripe. The result for the 5 microns stripe lies so closely on top that it cannot be distinguished. Thus either can be a reasonable approximation for infinitely separated stripes. The 4 microns and 2.5 microns lobes are somewhat smaller. This shows that the shear stress fields from neighboring stripes tend to cancel each other in their overlap areas, reducing the influence of the nitride film somewhat.

References

- [1] E. A. Irene, J. Electronic Mat., 5(3), p. 287, (1976).
- [2] S. M. Hu, "Film-edge-induced Stress in Silicon Substrates," Appl. Phys. Lett, 32(1), p. 5, 1978

5.9 Example 9: Stress Dependent Oxidation

This example shows the use of the stress-dependent oxidation model. Experimental LOCOS profiles are generally of two distinct types [1]. When the nitride mask is more than two to three times thicker than the pad oxide, the oxide/silicon interface and the oxide gas surface is kinked. For thinner nitride masks, the shape can be approximately be described by an error-function. The kinks were not observed in the first generation of oxidation simulators, because they result from stress effects on the growth coefficients. Early oxidation programs did not take stress into account and found essentially identical oxide shapes irrespective of nitride thickness. This example shows how the stress-dependent model in CSUPREM can predict such second-order effects. The example is given in the input file in the "examples\Stanford_Original\exam9" directory.

```
# --LOCOS cross section with stress effects
opt chat
# --Substrate mesh definition--
line x loc=-1 spac=0.2 tag=l
line x loc=0   spac=0.05 tag=c
line x loc=1   spac=0.2 tag=r

line y loc=0 tag=t
line y loc=1 tag=b

region silicon xlo=l xhi=r ylo=t yhi=b
bound expo xlo=l xhi=r ylo=t yhi=t

init ori=100

# --Pad oxide and nitride mask--
deposit oxide thick=0.02 div=1
deposit nitride thick=0.15 div=1
etch nitride left p1.x=0

#-----Field oxidation
oxide  Vc=300  Vr=30  Vd=25 stress.dep=t

meth viscous oxide.rel=1e-2

plot.2 bound y.mi=-0.3 x.mi=-0.3 x.ma=0.3

diffuse tim=90 tem=1000 weto2 movie="plot.2 bound cl=f ax=f line.b=1"
```

```
stru outf=exp9.str
```

The grid structure and nitride/oxide sandwich is very similar to the fully-recessed oxide example. The substrate grid is as sparse as possible (two lines). The lateral grid is a little coarse to compensate for the increased computation time used in the nonlinear model. The new statements are as follows:

```
oxide Vc=300 Vr=30 Vd=25 stress.dep=t
```

The nonlinear stress-dependent model is turned on. The default is to turn it off since it is much more expensive to run than the linear model. The activation volume for plastic flow V_c , for the stress-dependent reaction rate V_r , and for the diffusivity V_d , are taken from the defaults in the model file. The values used were derived by fitting Kao's cylinder oxidation data [2].

```
meth viscous oxide.rel=1e-2
```

The viscous model is chosen, because only this model takes stress effects into account. (Only the viscous model calculates stress accurately enough to feed back into the coefficients). The relative error criterion is chosen as 1% in the velocities. The default of $1.0E-6$ is rather tight and requires more CPU time. The diffuse statement is as usual.

This input file was executed twice, once with 0.05 microns of nitride and once with 0.15 microns of nitride. The output contains the usual features, along with many lines as follows:

```
Newton      loop      0  cut          1  upd      0.9785  orhs      3.945  rhs      3.69E-15
Newton      loop     1*  cut          1  upd     8.94E-13  orhs     3.69E-15  rhs     1.00E-38
Continuation step    #0  to      lambda =          0.25  step          0.25
Newton      loop      0  cut          1  upd      0.1415  orhs     0.000966  rhs     0.000558
Newton      loop      2  cut          1  upd      0.2453  orhs     0.001736  rhs     0.005291
Newton      loop      2  cut          0.25  upd      0.2453  orhs     0.001736  rhs     0.001325
Newton      loop      4  cut          0.25  upd      0.1258  orhs     0.001258  rhs     0.001938
Newton      loop      4  cut          0.063  upd      0.1258  orhs     0.001258  rhs     0.001188
Newton      loop      6  cut          0.063  upd      0.09528  orhs     0.001177  rhs     0.001114
Newton      loop      8  cut          0.13  upd      0.06899  orhs     0.001107  rhs     0.001033
Newton      loop     10  cut          0.25  upd      0.03523  orhs     0.001025  rhs     0.000854
Newton      loop     12  cut          0.5  upd      0.02074  orhs     0.000862  rhs     0.000443
Newton      loop     14  cut          1  upd      0.01121  orhs     0.00047  rhs     0.000126
Newton      loop     15*  cut          1  upd      0.00101  orhs     0.000126  rhs     1.00E-38
Continuation step    #0  to      lambda =          0.5  step          0.25
Newton      loop      0  cut          1  upd      0.4536  orhs     0.03935  rhs     0.05515
Newton      loop      0  cut          0.25  upd      0.4536  orhs     0.03935  rhs     0.02751
.....
.....
Newton      loop     10  cut          0.031  upd      0.7223  orhs     0.01903  rhs     0.0197
```

```

Continued    too    far    backing off.
Continuation step # -1 to    lambda =    0.375 step    0.125
Newton      loop    0 cut          1 upd    0.4509 orhs    0.03796 rhs    0.04957
Newton      loop    0 cut          0.25 upd    0.4509 orhs    0.03796 rhs    0.02221
.....
.....
Newton      loop    20 cut          0.031 upd    0.2287 orhs    0.01222 rhs    0.01656
Continued    too    far    backing off.
Continuation step # -2 to    lambda =    0.3125 step    0.0625
Newton      loop    0 cut          1 upd    0.361 orhs    0.03488 rhs    0.03238
Newton      loop    2 cut          1 upd    0.5928 orhs    0.03143 rhs    0.06065
.....
.....
Newton      loop    30 cut          0.5 upd    0.01595 orhs    0.002296 rhs    0.00145
Newton      loop    32 cut          1 upd    0.009169 orhs    0.001448 rhs    1.00E-38
Continuation step # -1 to    lambda =    0.4375 step    0.125
Newton      loop    0 cut          1 upd    0.07792 orhs    0.0172 rhs    0.03778
Newton      loop    0 cut          0.25 upd    0.07792 orhs    0.0172 rhs    0.01468
.....
.....
Newton      loop    22 cut          1 upd    0.0036 orhs    0.001816 rhs    1.00E-38
Continuation step # 0 to    lambda =    0.5 step    0.0625
Newton      loop    0 cut          1 upd    0.01835 orhs    0.005257 rhs    0.001574
Newton      loop    1* cut          1 upd    0.004715 orhs    0.001574 rhs    1.00E-38
Continuation step # 0 to    lambda =    1 step    0.5
Newton      loop    0 cut          1 upd    0.1938 orhs    0.1795 rhs    0.06693
Newton      loop    2 cut          1 upd    0.05735 orhs    0.05586 rhs    0.01965
.....
.....
Newton      loop    9 cut          0.063 upd    0.01719 orhs    0.000602 rhs    0.000564
Newton      loop    11 cut          0.13 upd    0.00913 orhs    0.000563 rhs    1.00E-38

```

The nonlinear solver works by proceeding first from the linear solution. The linear solution takes exactly two Newton steps. The first has a large update step 0.9785, which reduces the error from 3.945 to $3.685e-015$. The second Newton step $8.9e-013$ then of course is trivial since the solution has been reached. The Newton loop counter is incremented by two whenever a new Jacobian is factorized, and by one when only the error is recalculated. The stress-dependent viscosity is turned on (continuation step 0 to $\lambda = 0.25$). The first step is 0.14 and decrease the error from 0.00097 to 0.00056. A second Newton step 0.25 is taken from there and is found to increase the error from 0.0017 to 0.0053. A quarter-step (cut 0.25) in that direction is tried without success. The solver continues to cut back step until the error is decreased from 0.001177 to 0.001114. Since this is successful, the cutback factor is increased from 0.063 to 0.13.

The eighth Newton step (#14) reduces the error by a large factor, so on the subsequent step the same Jacobian is recycled, indicated by the asterisk. The recycled Jacobian proves to be effective, since the error goes from 0.0001264 to $1e - 038$. Since the update of 0.00101 is less than the accuracy criterion. The first nonlinear problem has been solved.

The stress-dependent reaction rate is then turned on (continuation to 0.5). In this case, the Newton process fails. Successively smaller steps along the Newton direction are taken, but at each attempted new position the error is larger than the current location. The smallest step tried is 0.031; the next would be 0.031/4 which is less than 1% of the Newton update and the situation is considered hopeless. The program then backs off by trying an intermediate lambda of 0.375. This corresponds to a stress dependence which is only half as strong as the desired dependence. However it is without success either. The program then backs off by trying an even smaller lambda of 0.3125. After 16 Newton steps, this weaker problem is solved. The program increases lambda from 0.3125 to 0.4375 and the solution is used as an initial guess. After 11 Newton steps this problem is solved too. The solution is then used for the problem first tried, that with the full stress dependence (lambda = 0.5). The improved initial guess leads to a successful solution of the problem after 2 Newton steps. Finally, the stress-dependent diffusivity is turned on (lambda =1) and it took 6 Newton steps to get convergent. The total number of Newton loops is therefore about 56, compared to 1 for a linear problem. Thus the nonlinear problem is about 56 times as expensive to solve as a linear problem.

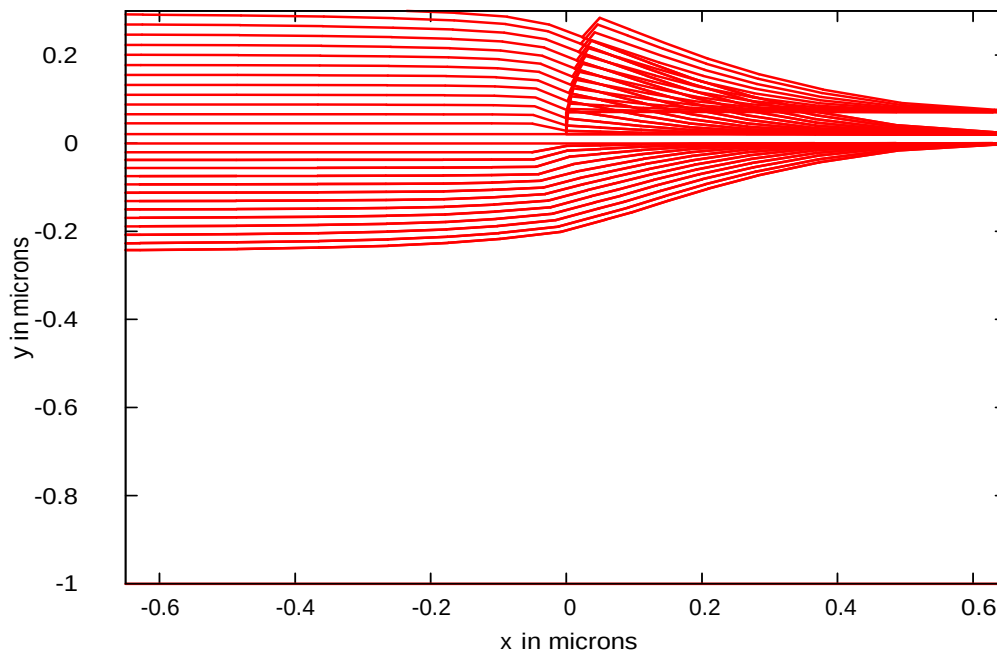


Figure 5. 22: Comparison of LOCOS Shapes with 500 A Nitride and 1500 A Nitride. This case has 500 A Nitride.

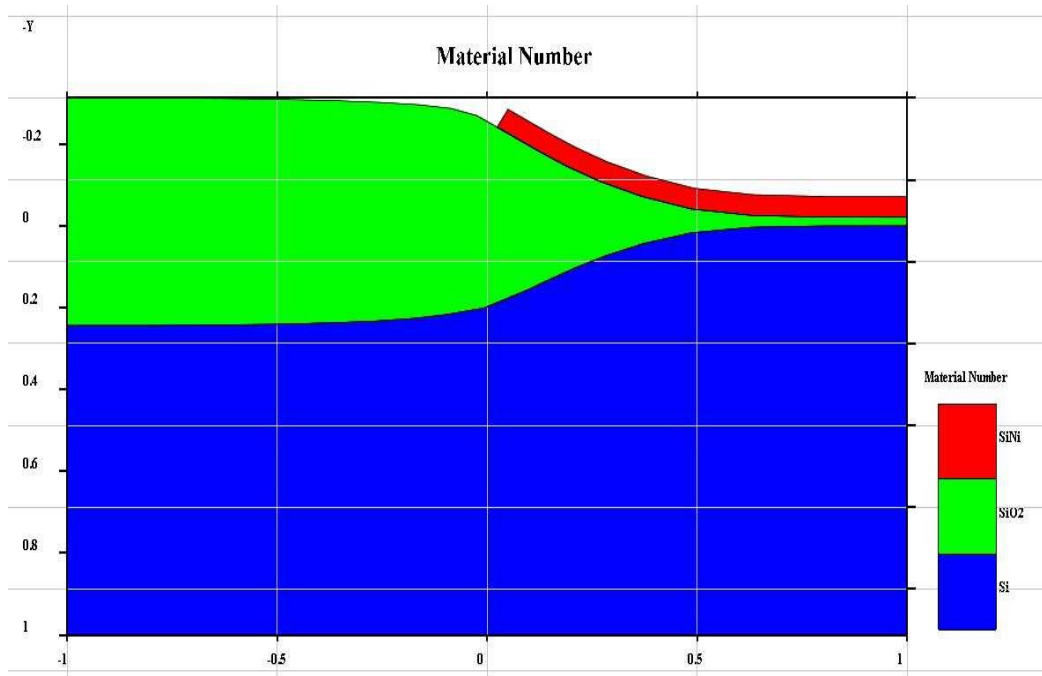


Figure 5.23: Bird's beak by Crosslightview. This case has 500 A Nitride.

The different oxide shapes are shown in Figure 5.22 and Figure 5.24. The different oxide shapes by CrosslightView are shown in Figure 5.23 and Figure 5.25. The thicker nitride clearly has the effect of reducing the bird's beak.

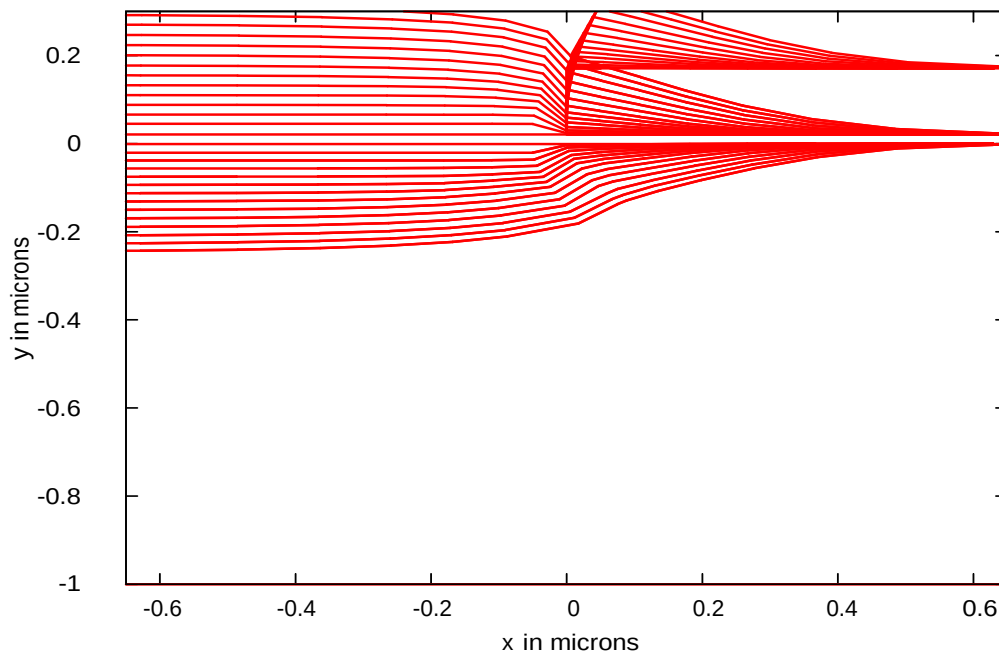


Figure 5.24: Comparison of LOCOS Shapes with 500 A Nitride and 1500 A Nitride. This case has 1500 A Nitride.

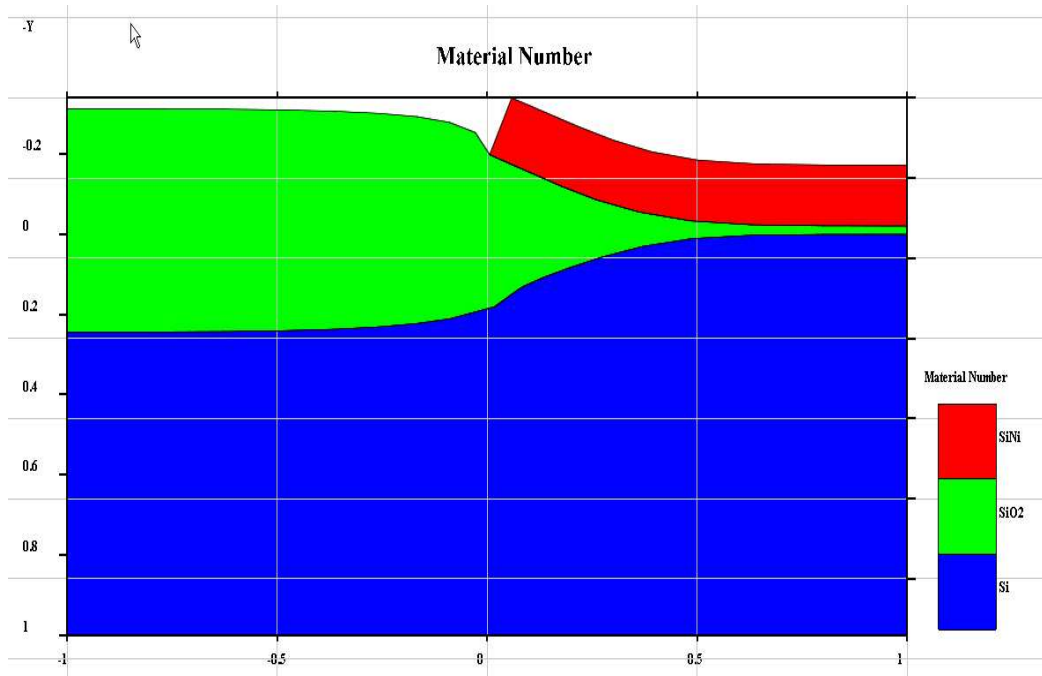


Figure 5. 25: Bird's beak by Crosslightview. This case has 1500 Å Nitride.

References

- [1] N. Guillemot, G. Pananakakis and P. Chenevier, IEEE Transactions on Electron Devices, ED-34, (1987).
- [2] D. B. Kao, J. P. McVittie, W. D. Nix and K. C. Saraswat, "Two-Dimensional Silicon Oxidation Experiments and Theory," IEDM Tech. Digest, 1985.

5.10 Example 18: Diffusion in Presence of Dislocation Loops

In this example, the one-dimensional model for dislocation loops is enabled. The ability of the dislocation loops to absorb excess injected interstitials during thermal oxidation is demonstrated. The input file for this simulation is in the "examples\Stanford_Original\exam18" directory in the file exam18.in.

```
option quiet
mode one.dim
```

```
# set boron fi and diffusion coefficients
boron silicon Fi=0.8
boron silicon Dix.0=0.037 Dix.E=3.46 Dip.0=0.72 Dip.E=3.46
```

```
# set up grid
line x loc=0.0 spacing=0.2 tag=top
line x loc=0.135 spacing=0.005
line x loc=0.145 spacing=0.005
line x loc=0.155 spacing=0.005
line x loc=0.165 spacing=0.005
line x loc=0.175 spacing=0.005
line x loc=0.185 spacing=0.005
line x loc=0.195 spacing=0.005
line x loc=0.205 spacing=0.005
line x loc=0.215 spacing=0.005
line x loc=1.0 spacing=0.01
line x loc=1.5 spacing=0.01
line x loc=2.0 spacing=0.01
line x loc=5.0 spacing=0.5
line x loc=200.0 spacing=10.0 tag=bottom

# initialize structure
region silicon xlo=top xhi=bottom
bound exposed xlo=top xhi=top
bound backside xlo=bottom xhi=bottom
init phosphorus conc = 1.00e+14 orient = 100

# initialize method and errors
method two.d
method inter rel.err=.001
method vacan rel.err=.001

# turn off traps
trap silicon total=0 enable=f
interst silicon ktrap.0=1.0e-30 ktrap.E=0.0

# only use neutral defects
interst silicon neu.0=1.0 neg.0=0.0 dneg.0=0.0 pos.0=0.0 dpos.0=0.0
interst silicon neu.E=0.0 neg.E=0.0 dneg.E=0.0 pos.E=0.0 dpos.E=0.0

# set vacancy parameters
vacancy silicon Cstar.0=8.0e17 Cstar.E=0.503
vacancy silicon D.0=3.02e7 D.E=4.652
vacancy silicon /oxide Ksurf.0=5.556e6 Ksurf.E=3.983

# set interstitial parameters
interst silicon D.0=3.031e9 D.E=3.66
```

```

interst silicon Cstar.0=1.49e11 Cstar.E=0.0
interst silicon /gas Ksurf.0=2.009e11 Ksurf.E=3.455
interst silicon /oxide Ksurf.0=2.009e11 Ksurf.E=3.455

# bulk recombination
interst silicon Kr.0=242.48 Kr.E=3.66
vacancy silicon Kr.0=242.48 Kr.E=3.66

# silicon /gas interface parameters
interstitial silicon /gas Krat.0= 0.          Krat.E= 0.
interstitial silicon /gas Kpow.0= 0.          Kpow.E= 0.

# silicon /oxide interface parameters
interstitial silicon /oxide Krat.0= 2400.      Krat.E= 0.
interstitial silicon /oxide Kpow.0= 0.5        Kpow.E= 0.
interstitial silicon /oxide Gpow.0= 0.0        Kpow.E= 0.

# vacancy interface parameters
vacancy silicon /gas Krat.0= 0.          Krat.E= 0.
vacancy silicon /gas Kpow.0= 0.          Kpow.E= 0.
vacancy silicon /oxide Krat.0= 0.          Krat.E= 0.
vacancy silicon /oxide Kpow.0= 0.          Kpow.E= 0.

# change injection parameter, theta, to fit normal OED
interst silicon /oxide theta.0=1.63*1.0e-2 theta.E=0.0

# read in epi boron profile
profile infile=initepi.dat boron

# initialize defects
method fermi
diffuse temp=900 time=0.001

# turn on dislocation loop model
dislocation rinit=235e-8 looploc=1750e-8 rho=4.5e9 fdrdt=1/1.75
dislocation loopgdt=0.001 gamma=4.375e13 omega=2.0e-23 burger=3.14e-8
dislocation mu=4.975e23 nu=0.27864583333 ro=3.14e-8

method two.d init=1e-6

# 900C, 2 hour wet oxidation
diffuse time=120 temp=900 weto2 press=0.9

# save structure file

```

```
struct out=example18.str
```

```
profile infile=initepi.dat ant
```

```
select z=log10(ant)
plot.1d x.mi=1 x.ma=2 y.mi=16
```

```
select z=log10(bor)
plot.1d x.mi=1 x.ma=2 y.mi=16
```

The following command reads in a doping file named initepi.dat, containing a boron one-dimensional doping profile.

```
profile infile=initepi.dat boron
```

The first dislocation card is used to set the initial radius of the dislocation loops, their location, their density, and their effectiveness at capturing interstitials. The second dislocation card defines the simulation accuracy, the internal energy of the stacking fault, the volume per silicon atom, and the magnitude of the Burger's vector associated with the dislocation loop. Finally, the last dislocation card sets the shear modulus, Poisson's ratio, and the core radius of the dislocation loop.

```
profile infile=initepi.dat ant
```

```
select z=log10(ant)
plot.1d x.mi=1 x.ma=2 y.mi=16
```

```
select z=log10(bor)
plot.1d x.mi=1 x.ma=2 y.mi=16
```

The above commands produce the boron profile plotting before and after the oxidation including the dislocation loops. They are used to compare to a simulation result of normal OED without the loops (exam18b.in), as in Figure 5.26.

The presence of the dislocation loops substantially reduces the OED of the boron. The final simulated loop radius after oxidation is 593Å.

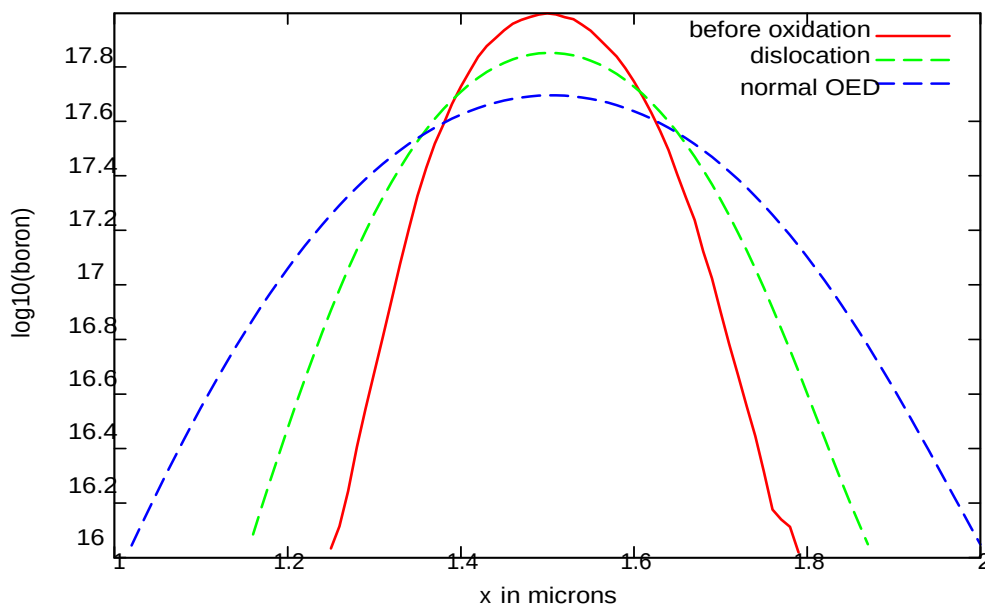


Figure 5. 26: Comparison of simulations of oxidation-enhanced boron diffusion with and without a layer of dislocation loops centered at **1750Å** from the surface.

5.11 Example 19: Interstitial Cluster Dissolution

In this example, the dislocation loop model is used in reverse in order to mimic interstitial cluster dissolution. The input file for this simulation is in the "examples\Stanford_Original\exam19" directory in the file example19.in.

```
set echo
mode one.dim

# set boron fi and diffusion coefficient
boron silicon Fi=0.8
boron silicon Dix.0=0.037 Dix.E=3.46 Dip.0=0.72 Dip.E=3.46

# set up grid
line x loc=0.0 spacing=0.005 tag=top
line x loc=0.135 spacing=0.005
line x loc=0.145 spacing=0.005
line x loc=0.155 spacing=0.0003
line x loc=0.165 spacing=0.0003
line x loc=0.175 spacing=0.0003
line x loc=0.185 spacing=0.0003
line x loc=0.195 spacing=0.0003
```

```
line x loc=0.205 spacing=0.005
line x loc=0.215 spacing=0.005
line x loc=0.3 spacing=0.01
line x loc=0.6 spacing=0.01
line x loc=1.0 spacing=0.01
line x loc=5.0 spacing=0.5
line x loc=200.0 spacing=10.0 tag=bottom

# initialize structure
region silicon xlo=top xhi=bottom
bound exposed xlo=top xhi=top
bound backside xlo=bottom xhi=bottom
init phosphorus conc = 1.00e+14 orient = 100

# set up method and errors
method two.d
method inter rel.err=.001
method vacan rel.err=.001

# turn off traps
trap silicon total=0 enable=f
interst silicon ktrap.0=1.0e-30 ktrap.E=0.0

# only use neutral defects
interst silicon neu.0=1.0 neg.0=0.00 dneg.0=0.0 pos.0=0.00 dpos.0=0.0
interst silicon neu.E=0.0 neg.E=0.00 dneg.E=0.0 pos.E=0.00 dpos.E=0.0

# vacancy parameters
vacancy silicon Cstar.0=4.77e18 Cstar.E=0.713
vacancy silicon D.0=6.34e3 D.E=3.29
vacancy silicon /oxide Ksurf.0=1174.0 Ksurf.E=2.62

# interstitial parameters
interst silicon D.0=1.03e6 D.E=3.22
interst silicon /gas Ksurf.0=74.6 Ksurf.E=1.53
interst silicon /oxide Ksurf.0=74.6 Ksurf.E=1.53
interst silicon Cstar.0=5.1e11 Cstar.E=0.0

# Bulk recombination
interst silicon Kr.0=1.4 Kr.E=3.99
vacancy silicon Kr.0=1.4 Kr.E=3.99

# read in initial boron epi profile
profile infile=initepi.dat bor
```

```
# initialize defects
method fermi
diffuse temp=750 time=0.001

# turn on "clusters"
dislocation rinit=10e-8 looploc=1750e-8 rho=1.35e10 fdrdt=0.35
dislocation loopgdt=0.001 maxsi=1.5e3 gamma=4.375e13 omega=2.0e-23
dislocation burger=3.14e-8 mu=4.975e23 nu=0.27864583333 ro=3.14e-8

# read in initial interstitial profile
profile infile=si.50.5e12 interst

method two.d init=1e-6

# 750C inert anneals
diffuse time=2 temp=750 argon
struct out=2min.str
select z=log10(bor)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16

diffuse time=8 temp=750 argon cont
struct out=10min.str
select z=log10(bor)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16

diffuse time=20 temp=750 argon cont
struct out=30min.str
select z=log10(bor)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16

diffuse time=90 temp=750 argon cont
struct out=120min.str
select z=log10(bor)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16

profile infile=initepi.dat ant
select z=log10(ant)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16
```

Compared to Example 18, the dislocation cards in this example specify one additional parameter, maxsi, the maximum ratio of the equilibrium interstitial concentration surrounding the cluster to the normal equilibrium interstitial concentration.

The main effect of adding interstitial cluster dissolution is a longer time dependence in the enhanced diffusion of the boron marker layer. The time dependence including cluster dissolution can be seen by inputting the following lines:

```
init inf=2min.str
profile infile=initepi.dat ant
select z=log10(ant)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16 line.type=9
```

```
select z=log10(bor)
plot.1d cl=f ax=f line.type=1
init inf=10min.str
select z=log10(bor)
plot.1d cl=f ax=f line.type=3
```

```
init inf=30min.str
select z=log10(bor)
plot.1d cl=f ax=f line.type=5
```

```
init inf=120min.str
select z=log10(bor)
plot.1d cl=f ax=f line.type=7
```

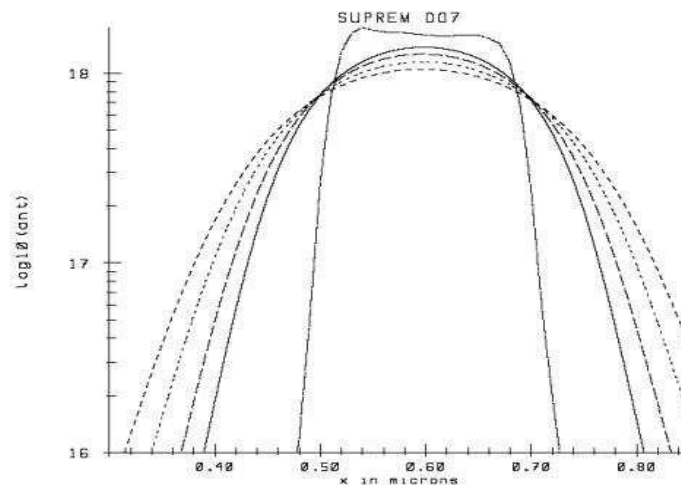


Figure 5. 27: Simulation results using the dislocation loop model to mimic cluster dissolution which prolongs the implant-damage enhanced diffusion.

Figure 5.28 can be compared to Figure 5.27 which contains the simulation results without the interstitial clusters (example19b.in).

```
select z=log10(bor)
plot.1d cl=f ax=f line.type=5
```

```
init inf=120min.str
select z=log10(bor)
plot.1d cl=f ax=f line.type=7
```

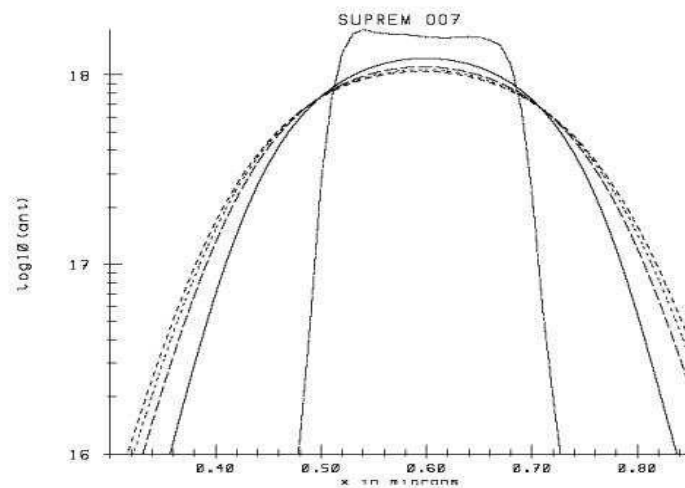


Figure 5. 28: Simulation results of implant-damage enhanced diffusion without the dislocation loop model.

5.12 Example 20: Nondilute Diffusion Using Full.Cpl

In this example, the new full.cpl model, which accounts for nondilute dopant/defect pair concentrations, is used. The input file for this simulation is in the "examples/Standford_Original/exam20" directory in the file example20.in.

```
option quiet
set echo
mode one.dim
```

```

# set boron fi and diffusion coefficient
boron silicon Fi=0.8
boron silicon Dix.0=0.037 Dix.E=3.46 Dip.0=0.72 Dip.E=3.46

# set up grid
line x loc=0.0    spacing= 0.002    tag=top
line x loc= 0.14  spacing= 0.002
line x loc= 0.16  spacing= 0.002
line x loc= 0.18  spacing= 0.002
line x loc= 0.28  spacing= 0.002
line x loc=0.3    spacing=0.02
line x loc=0.6    spacing=0.02
line x loc=1.0    spacing=0.02
line x loc= 10.0  spacing= 2.0
line x loc= 50    spacing= 5.0
line x loc= 200   spacing= 10.0    tag=bottom

# initialize structure
region silicon    xlo=top xhi=bottom
bound exposed xlo=top xhi=top
bound backside xlo=bottom xhi=bottom
init phosphorus  conc= 1.00e+14  orient = 100

# set method and error
method full.cpl
method inter      rel.err=.001
method vacan      rel.err=.001

# no traps
trap  silicon total=0 enable=f
interst silicon ktrap.0=1.0e-30 ktrap.E=0.0

# Use neutral defects
interst silicon neu.0=1.0 neg.0=0.00 dneg.0=0.0 pos.0=0.00 dpos.0=0.0
interst silicon neu.E=0.0 neg.E=0.00 dneg.E=0.0 pos.E=0.00 dpos.E=0.0
vacancy silicon neu.0=1.0 neg.0=0.00 dneg.0=0.0 pos.0=0.00 dpos.0=0.0
vacancy silicon neu.E=0.0 neg.E=0.00 dneg.E=0.0 pos.E=0.00 dpos.E=0.0

# Set interface parameters
interstitial silicon /gas  Krat.0= 0.      Krat.E= 0.
interstitial silicon /gas  Kpow.0= 0.      Kpow.E= 0.
interstitial silicon /oxide Krat.0= 0.      Krat.E= 0.
interstitial silicon /oxide Kpow.0= 0.      Kpow.E= 0.

```

```

vacancy silicon /gas      Krat.0= 0.      Krat.E= 0.
vacancy silicon /gas      Kpow.0= 0.      Kpow.E= 0.
vacancy silicon /oxide    Krat.0= 0.      Krat.E= 0.
vacancy silicon /oxide    Kpow.0= 0.      Kpow.E= 0.

```

```
# dopant/defect equilibrium reaction constants
```

```
vacancy silicon boron neu.0=8.0e-23 neu.E=-0.8 neg.0=0. dneg.0=0. pos.0=0. dpos.0=0.
```

```
interst silicon boron neu.0=2.15e-17 neu.E=0.0
```

```
#interst silicon boron neu.0=2.15e-20 neu.E=0.0
```

```
# interstitial parameters
```

```
interst silicon D.0=2.15e-11 D.E=0.0
```

```
interst silicon Cstar.0=3.4e12 Cstar.E=0.0
```

```
interst silicon /oxide Ksurf.0=3.28e-7 Ksurf.E=0.0
```

```
interst silicon /gas Ksurf.0=3.28e-7 Ksurf.E=0.0
```

```
# vacancy parameters
```

```
vacancy silicon D.0=1.0e-11 D.E=0.0
```

```
vacancy silicon Cstar.0=5.835e13 Cstar.E=0.0
```

```
vacancy silicon /oxide Ksurf.0=3.1e-9 Ksurf.E=0.0
```

```
vacancy silicon /gas Ksurf.0=3.1e-9 Ksurf.E=0.0
```

```
# bulk recombination
```

```
interst silicon Kr.0=1.4 Kr.E=3.99
```

```
vacancy silicon Kr.0=1.4 Kr.E=3.99
```

```
# read in initial boron epi profile
```

```
profile infile=initepi.dat bor
```

```
# initialize defects
```

```
method fermi
```

```
diffuse temp=750 time=0.001 argon
```

```
# read in interstitial profile
```

```
profile infile=si.50.6e12 interst
```

```
# deposit oxide cap
```

```
deposit oxide thick=.003
```

```
method full.cpl init.time=1e-6
```

```
# 750C, inert anneal
```

```
diffuse time=2 temp=750 argon
```

```
struct out=example20.str
```

```
select z=log10(bor)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16
```

```
profile infile=initepi.dat ant
select z=log10(antimony)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16
```

The equilibrium reaction constant between boron and neutral interstitials is specified on the "interstitial silicon boron" card as $2.15 \times 10^{-17} \text{ cm}^3$. The maximum enhancement in boron diffusivity using this model and the given parameters is $(1/K_{BI}C_I^*)$. Figure 5.29 shows that with a much smaller value for K_{BI} ($2.15 \times 10^{-20} \text{ cm}^3$), the amount of boron diffusion during 2 minutes of annealing at 750 C is much larger.

```
init inf=example20.str
profile infile=initepi.dat ant
select z=log10(antimony)
plot.1d x.mi=0.3 x.ma=0.85 y.mi=16 line.type=5
```

```
select z=log10(bor)
plot.1d cl=f ax=f line.type=1
```

```
init inf=example20b.str
select z=log10(bor)plot.1d cl=f ax=f line.type=3
```

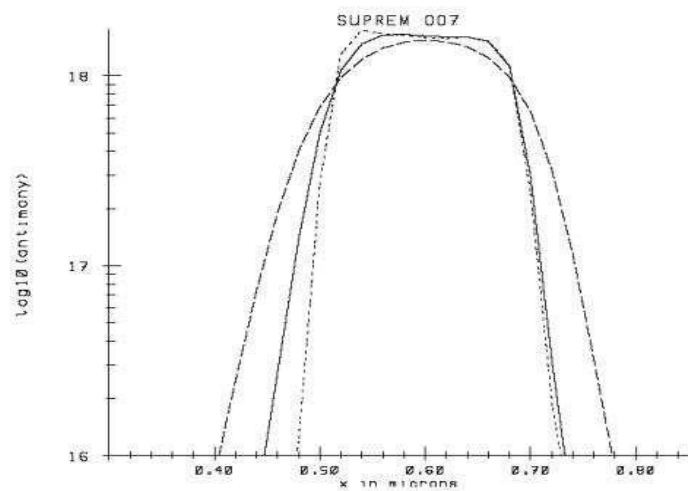


Figure 5. 29: Enhanced boron diffusion after silicon implantation with a "large" equilibrium

reaction constant between the boron and the interstitials compared to enhanced diffusion using a "small" reaction constant.

5.13 Example 21: Stanford BiCMOS 2D Field Oxidation Simulation

This example shows a SUPREM-IV.GS simulation of the Stanford Center for Integrated Systems' BiCMOS process. The input file is listed as follows.

```
#Title CIS BiCMOS Bipolar part in 2D

#set some stuff
set echo
option chat

#x dimension
line x loc=0.0 spacing=0.2 tag=left
line x loc=1.0 spacing=0.05
line x loc=2.0 spacing=0.2 tag=right

#y dimension
line y loc=0.0 tag=top
line y loc=1.0 tag=bottom

#silicon
region silicon xlo=left xhi=right ylo=top yhi=bottom

#exposed surface
bound exposed xlo=left xhi=right ylo=top yhi=top

#Comment Start with <100> Silicon, p doped to 20 ohm resistivity.
initialize boron conc=9.0e14 ori=100

oxide orient=100 wet lin.h.0=2.428e6 lin.l.0=2.793e4

#suprem iv default coefficients
boron silicon Dix.0=0.37 Dip.0=0.72 Dix.E=3.46 Dip.E=3.46
arsenic silicon Dix.0=8.0 Dim.0=12.8 Dix.E=4.05 Dim.E=4.05

#Comment Pad Oxidation.
method compress init=1.0e-3
```

```
#suprem4 barfs after the 1st anneal so take this one out
#diffuse time=3  temp=950  argon
diffuse time=42  temp=950  dryo2

deposit nitride thick=0.08 div=1
deposit photoresist thick=1

#Field implant mask at edge of collector

etch photoresist left  p1.x=1.0
etch nitride  left  p1.x=1.0

implant boron energy=100 dose=1e13

etch photoresist all
struct out=prefieldox.s

#Comment Field Ox

#-----Field oxidation 1000
# High Stress  $\{g\} \{NITRIDE\} = 1.59 \times 10^{+10} \exp(1.12/kT)$ 

oxide  Vc=425 Vr=12.5 Vd=65 stress.dep=t
material  oxide  visc.0 = 2.25e14 visc.E = 0.000 visc.x = 0.499 wet
mater  nitride visc.0=4.3e14 visc.E=0.0

meth  viscous oxide.rel=1e-2

diffuse time=18  temp=1000  argon
diffuse time=10  temp=1000  dryo2
struct out=init.str

#diffuse time=190 temp=1000  weto2
#break up this step into several substeps

diffuse time=2 temp=1000 weto2
struct out=2.str

diffuse time=7 temp=1000 weto2 cont
struct out=9.str

diffuse time=15 temp=1000 weto2 cont
struct out=24.str
```

```
diffuse time=20 temp=1000 weto2 cont
struct out=44.str
```

```
diffuse time=26 temp=1000 weto2 cont
struct out=70.str
```

```
diffuse time=30 temp=1000 weto2 cont
struct out=100.str
```

```
diffuse time=40 temp=1000 weto2 cont
struct out=140.str
```

```
diffuse time=50 temp=1000 weto2 cont
struct out=190.str
```

```
diffuse time=10 temp=1000 dryo2
structure outfile=fieldox.s
```

The initial and final mesh structures are shown in Figures 5.30 and 5.31. Please note the similarity between this example and example 7 where an etch step is used to form a recessed structure before wet oxidation.

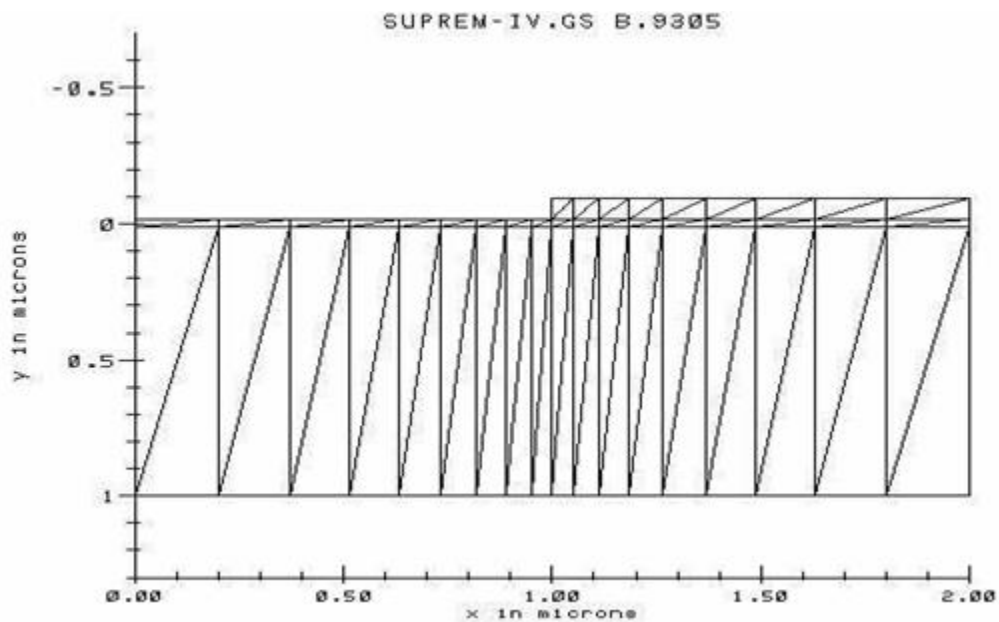


Figure 5. 30: Initial mesh structure for field oxidation.

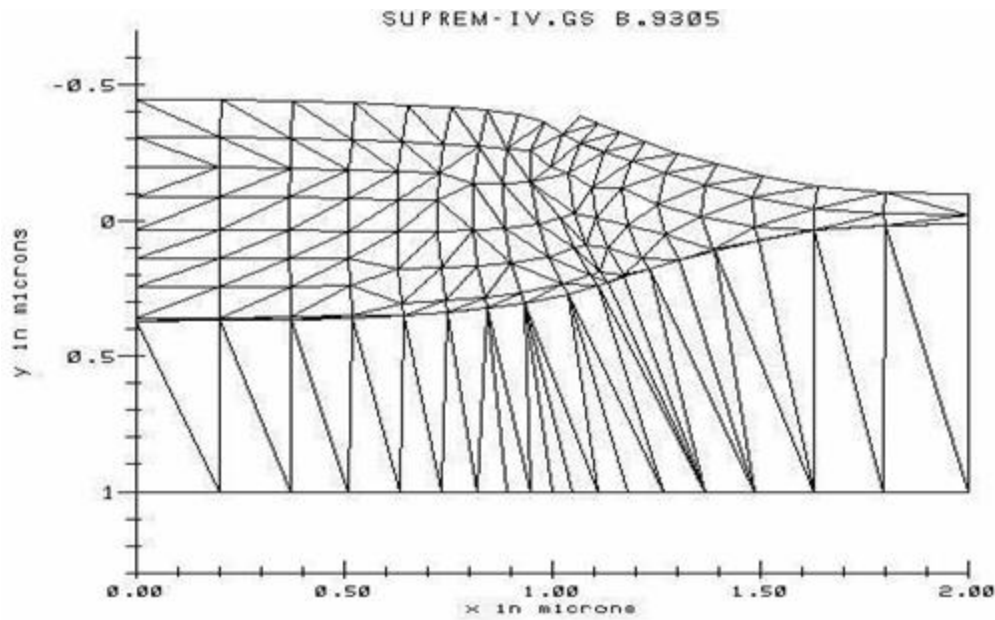


Figure 5.31: Final mesh structure for field oxidation.

5.14 SiGe induced uniaxial stress in silicon

This example illustrates the calculation of uniaxial stress induced by SiGe pocket. The example directory is SiGe_stress/. The input file is listed as follows:

option quiet

```
line x loc=0.0 tag=l spa=0.004
line x loc=0.06 spa=0.004
line x loc=0.1 spa=0.004
line x loc=0.2 spa=0.004
line x loc=0.3 tag=r spa=0.004
line y loc=0 tag=si spa=0.003
line y loc=0.08 spa=0.003
line y loc=0.1 spa=0.003
line y loc=0.19 tag=b spa=0.003
# 0.25 ---> 0.19
```

```
region silicon xlo=l xhi=r ylo=si yhi=b
bound expos xlo=l xhi=r ylo=si yhi=si
initial ori=100
```

```
mask x1.from=-0.06 x1.to=0.06 thick=1.5
etch silicon avoidmask depth=0.06
deposit imaterial thick=0.1 meshlayer=20

etch imaterial start x=-0.3 y=-3
etch imaterial continue x=0.0 y=0
etch imaterial continue x=0.3 y=0
etch imaterial done x=0.3 y=-3
etch photoresist all

# sigeflag is the material to apply instrinsic stress
# gecomp=composition of Ge
# hhoriz=length of the SiGe pocket for van der Walle model
# hvert=height of the SiGe pocket for van der Walle model
# thickness of Si is actually very large, beyond the diffusion mesh

material imaterial sigeflag=1 gecomp=0.17 hhoriz=0.24 hvert=0.06
material silicon hhoriz=0.06 hvert=10.

stress temp1=1000 temp2=1000
struct mirror left
struct outf=sigeim.str
```

Please note the use sigeflag which defines the material region where intrinsic stress is applied. The strain is determined by the van der Walle's theory described in the reference. The program needs to know the material dimensions in order to determine the strained lattice constant. We used a large thickness for silicon to reflect the fact that silicon is much more massive.

In this example, the device is grown on (001) surface but the stress direction (xx in our 2D notation) is along $\langle 110 \rangle$. The output is shown in Figure 5.32

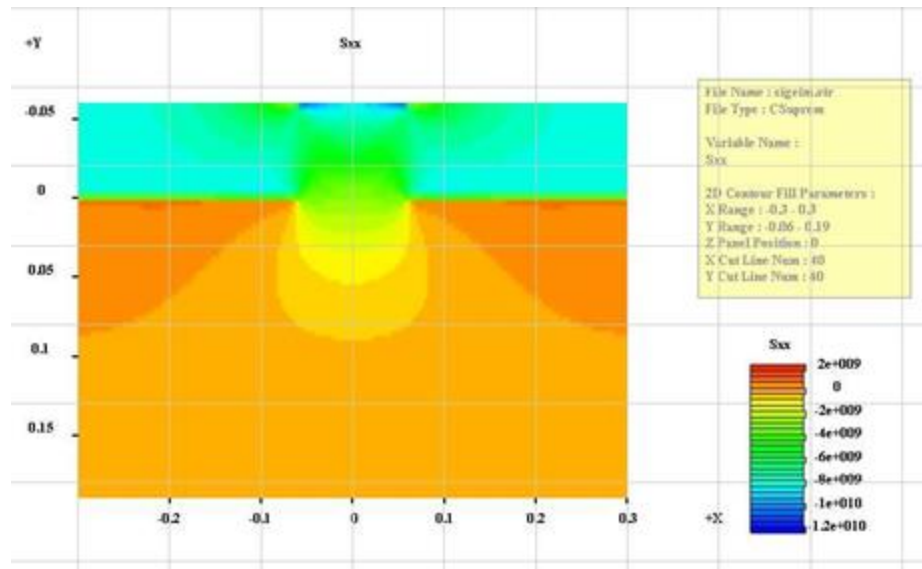


Figure 5. 32: Uniaxial stress distribution in Si/SiGe struture.

CHAPTER 6

Three Dimensional Examples

6.1 Introduction

This chapter contains examples in 3D. Since the basic idea of performing 3D is to stack up many 2D simulation cross sections, it is important that reasonable results are first obtain for at least one of the 2D cross sections. We make an effort to explain and demonstrate the 3D effects using simple structures.

The basic capability for Suprem4.gs is solution of dopant diffusion equations using various advanced point defect models. Another important transport model is segregation of dopants between material interfaces. Such segregation effect becomes more and more important as silicon devices are scaled down and interface effects are more pronounced. Any full 3D simulator should take into account segregation effect in all three directions. This example demonstrates the segregation effect in z-direction between Si and SiO₂ interface. To check the accuracy, the dopant distribution on y-z plane is compared with a reference structure where segregation happens on x-y plane (pure 2D effect).

This example has special importance for small MOSFET devices since SiO₂ shallow trench isolation (STI) in such devices has Si/SiO₂ interfaces surrounding the conduction channel in all three directions (in below, length and width directions). Careful study of this example would help to ensure the essential physical models are included properly for more complicated 3D

structures. The files corresponding to this example is placed under 3D_segregation directory.

6.1.1 Reference structure with segregation on x-y plane

This structure contains only one z-segment with uniform material in z-direction. Thus the transport model involved is purely 2D and the results are used as a reference check for the segregation effect in z-direction in the next subsection.

The main input file for the reference case (file name xy.in) is listed as follows:

```
option chat
mode three.dim
3d_mesh nsegm=1 infile=gs

init

implant boron dose=3.0e14 energy=80.0
struct outf=xy1.str

diff temp=1000 time=30 struct outf=xy2.str
```

The above file operates on a simple single z-segment structure, implant some boron and anneal for 30 minutes. The gs1.in file contains the xy-mesh definition as given below.

```
line x loc=0.0 tag=lft spacing=0.03
line x loc=0.3 tag=mid spacing=0.03
line x loc=0.6 tag=rht spacing=0.03

line y loc=0.0 tag=top spacing=0.1
line y loc=1 tag=int spacing=0.1
line y loc=2.0 tag=bot spacing=0.2

region silicon xlo=lft xhi=rht ylo=int yhi=bot
region silicon xlo=lft xhi=mid ylo=top yhi=int
region oxide xlo=mid xhi=rht ylo=top yhi=int

bound exposed xlo=lft xhi=rht ylo=top yhi=top
bound backside xlo=lft xhi=rht ylo=bot yhi=bot
```

Please note the definition of SiO₂ which forms Si/SiO₂ interface to demonstrate dopant segregation. To perform any 3D simulation, an input template file named zmesh.zst must be used. This is listed as follows.

```

begin_zst
$
$Set up 3d flow option
$-----
$
3d_solution_method 3d_flow=yes
$
$Set up Z-boundaries, number of planes and plane index of each segment
$-----
$
$segment 1
z_structure uniform_zseg_from=0.0 uniform_zseg_to=0.3 && zplanes=20 zseg_num=1
$
$-----
$Read the mesh files for the faces of each segment.
$-----
$
load_mesh mesh_inf=f1.msh zseg_num=1
$
$Please do not modify these settings
$-----
$
output sol_outf=tmp.out
$
export_3dgeo file=h_cvd.3dgeo
$
end_zst

```

The above zmesh.zst is used to control the size and mesh distribution in the z-direction. The reference boron profile after diffusion is shown in Figure 6.1 which clearly shows high concentration at material interface due to segregation effect.

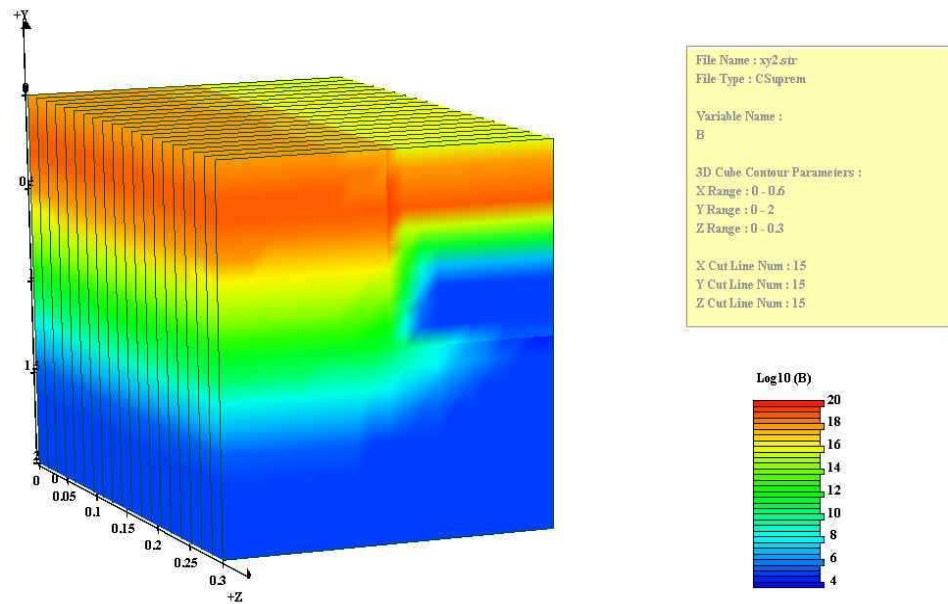


Figure 6. 1: Boron profile of reference structure. Please note the segregation effect at material interface.

6.1.2 Structure with segregation in z-direction

The main input file is `segr3d.in` list as follows

```
option chat
mode three.dim
```

```
3d_mesh nsegm=2 infile=gs zstfile=zmesh.zst
```

```
init
```

```
implant boron dose=3.0e14 energy=80.0
```

```
struct outf=segr3d_1.str
```

```
diff temp=1000 time=30
```

```
struct outf=segr3d_2.str
```

The above input file uses two files to define the xy-structure of two z-segments. The first is `gs1.in`:

```

line x loc=0.0 tag=lft      spacing=0.03
line x loc=0.3 tag=mid      spacing=0.03
line x loc=0.6 tag=rht      spacing=0.03

```

```

line y loc=0.0 tag=top      spacing=0.1
line y loc=1 tag=int        spacing=0.1
line y loc=2.0 tag=bot      spacing=0.2

```

```

region silicon xlo=lft xhi=rht ylo=int yhi=bot
region silicon xlo=lft xhi=mid ylo=top yhi=int
region oxide xlo=mid xhi=rht ylo=top yhi=int

```

```

bound exposed xlo=lft xhi=rht ylo=top yhi=top
bound backside xlo=lft xhi=rht ylo=bot yhi=bot

```

The second is gs2.in:

```

line x loc=0.0 tag=lft spacing=0.03
line x loc=0.3 tag=mid spacing=0.03
line x loc=0.6 tag=rht spacing=0.03

```

```

line y loc=0.0 tag=top spacing=0.1
line y loc=1 tag=int spacing=0.1
line y loc=2.0 tag=bot spacing=0.2

```

```

region silicon xlo=lft xhi=rht ylo=int yhi=bot
region oxide xlo=lft xhi=mid ylo=top yhi=int
region oxide xlo=mid xhi=rht ylo=top yhi=int

```

```

bound exposed xlo=lft xhi=rht ylo=top yhi=top
bound backside xlo=lft xhi=rht ylo=bot yhi=bot

```

Please note that the 2nd z-segment consists of two regions, one for Si and the other for SiO₂ which makes an interface plane between the first segment (consists of silicon only) and the second. The z-structure is controlled by the following zmesh.zst in the same directory:

```

begin_zst
$
$Set up 3d flow option
$-----
$
3d_solution_method 3d_flow=yes
$
$Set up Z-boundaries, number of planes and plane index of each segment

```



```

$-----
$
$segment 1
z_structure uniform_zseg_from=0.0 uniform_zseg_to=0.3 &&
zplanes=20 zseg_num=1
$
$Segment2 z_structure uniform_zseg_from=0.3 uniform_zseg_to=0.6 &&
zplanes=20 zseg_num=2

$-----
$Read the mesh files for the faces of each segment.
$-----

$
load_mesh mesh_inf=f1.msh zseg_num=1
load_mesh mesh_inf=f2.msh zseg_num=2
$
$Please do not modify these settings
$-----
$
output sol_outf=tmp.out
$
export_3dgeo file=h_cvd.3dgeo
$
end_zst

```

After the same 30 minute anneal, we obtain the doping profile as shown in Figure 6.2. Comparison of the results with those in the reference structure finds that except for axis rotation, the profiles are nearly identical. This indicates that that the 3D diffusion and segregation effects have been correctly implemented.

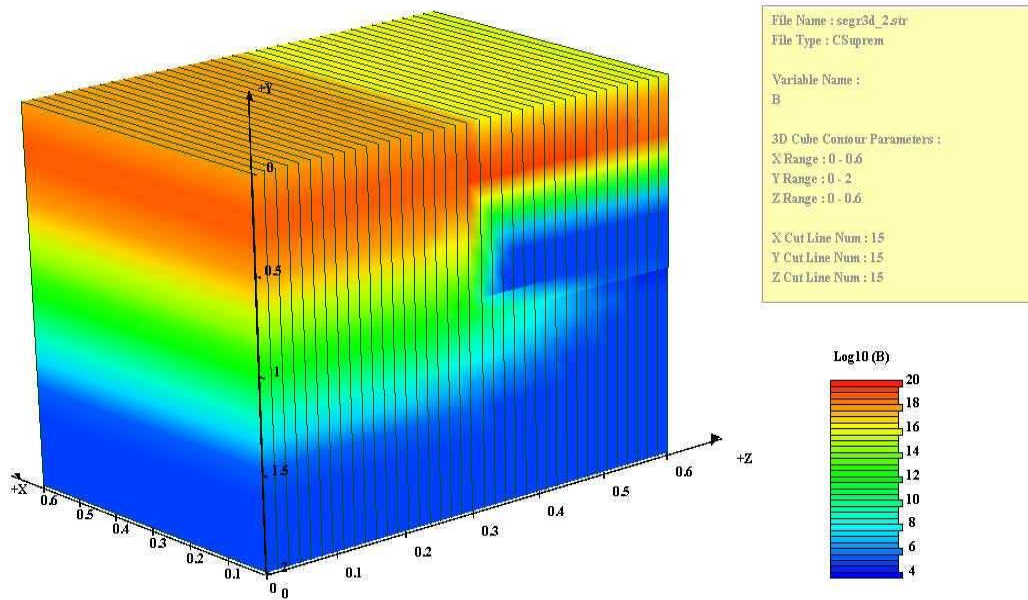


Figure 6. 2: Boron profile of structure with Si/SiO₂ interface plane between the first and second z-segments. Please note the segregation effect at material interface.

6.2 Example: Reference Structure for Full 3D MOSFET Simulation

6.2.1 Description

This example simulates the anneal of lightly doped drain in a 3d structure with one 2 segments. The input file *zmesh.zst* should be used to set up the mesh in z-direction and number of mesh points in each segment. The number of planes in each segment is 1.

The structure, Phosphorous and Arsenic distributions after diffusion are plotted below.

The GUI CrosslightView was used to visualize the structure and the doping profiles. This input file **ldd.in** for the simulation and its associated geometry files *geo1.in*, and *geo2.in* are in the \Csuprem\Examples\CSuprem_To_APSYS\11_MOSFET_3D\ref\ldd.in directory.

```
option quiet
mode three.dim
#
phos poly /gas Trn.0=0.0
bor poly /gas Trn.0=0.0
phos oxide /gas Trn.0=0.0
```

```
bor oxide /gas Trn.0=0.0
#change the phosphorus diffusivity like this
#phos silicon Dix.0=0.0 Dim.0=0.0 Dimm.0=0.0
#

3d_mesh nsegm=2 infile=geo zstfile=zmesh.zst
init boron conc=1.0e16

#deposit the gate oxide
deposit oxide thick=0.025

#channel implant
implant boron dose=1.0e12 energy=15.0

#deposit the gate poly
deposit poly thick=0.500 div=10 phos conc=1.0e19

#anneal the beast
diff time=10 temp=1000
#
#etch the poly away
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55 segm=1
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55 segm=2

#anneal this step
diffuse time=30.0 temp=950

struct outf=poly.str
#
#do the phosphorus implant
implant phos dose=1.0e13 energy=50.0

#deposit the oxide spacer
deposit oxide thick=0.400 spac=0.05

#etch the spacer back
etch dry oxide thick=0.420 segm=1
etch dry oxide thick=0.420 segm=2

struct outf=imp2.str
#after etch anneal
method vert fermi grid.ox=0.05
diffuse time=30 temp=950 dry
```

```
##implant the arsenic
implant ars dose=5.0e15 energy=80.0

#struct outf=imp4.str
#
##do the final anneal
diffuse time=20 temp=950

# contact etch
etch oxide p1.x=1.3 right segm=1
etch oxide p1.x=1.3 right segm=2

struct outf=ldd.str

struct mirror left

struct outf=ldd_mir.str

export outf=ldd.aps xpsize=0.0001 triangle.based=f
```

6.2.2 Numerical Results and discussion

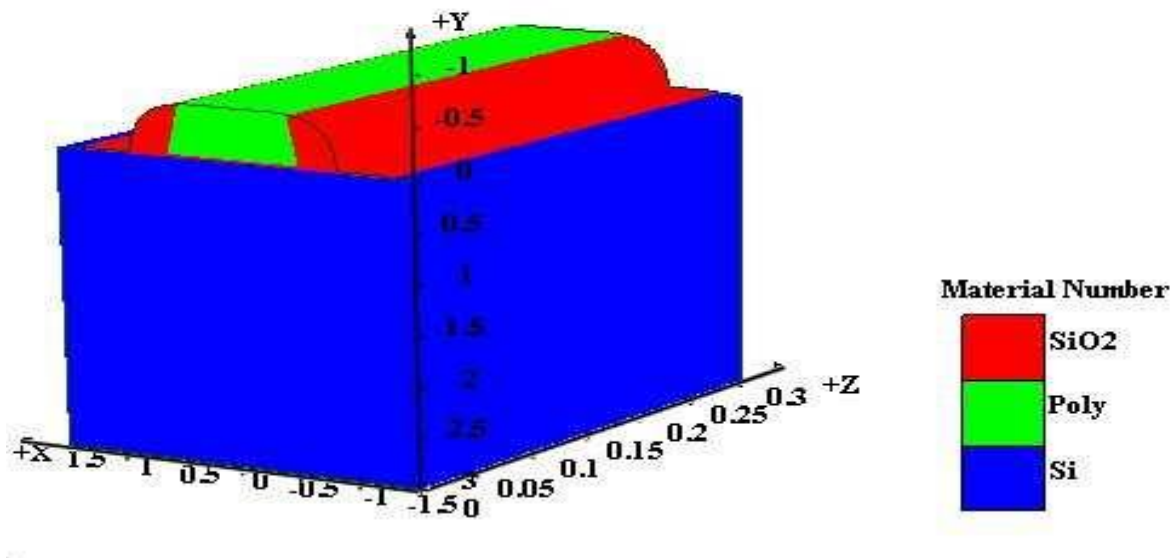


Figure 6. 3: 3D LDD structure

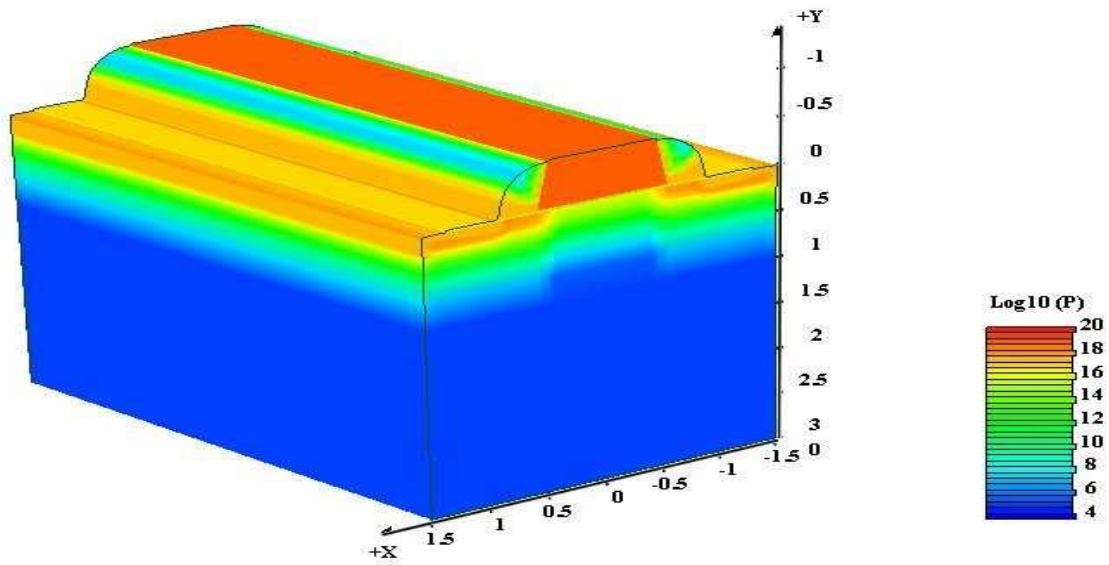


Figure 6. 4: 3D Phosphorous profile after diffusion

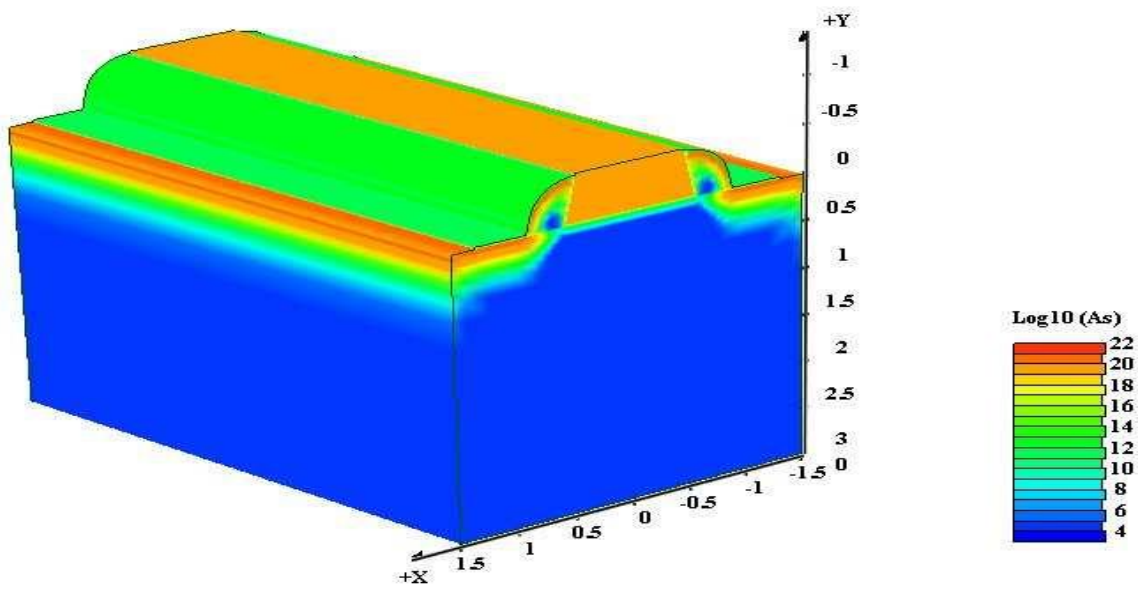


Figure 6. 5: 3D Arsenic profile after diffusion

6.3 Example: STI Structure for Full 3D MOSFET Simulation

6.3.1 Description

This example simulates the anneal of lightly doped drain in a 3d structure with STI. In this example, we use 4 segments. The input file *zmesh.zst* should be used to set up the mesh in z-direction and number of mesh points in each segment. The number of planes in each segment is 1. In Crosslight's simulators, planes within a segment is assumed to have the same mesh structure. Using unity for plane number within a segment allows easy treatment of oxidation if the dryo2/weto2 process flows are to produce different mesh points for different planes.

The STI structure, Phosphorous and Arsenic distributions after diffusion are plotted below.

The GUI CrosslightView was used to visualize the structure and the doping profiles. This input file **ldd.in** for the simulation and its associated geometry files *geo1.in*, *geo2.in*, *geo3.in*, *geo4.in* are in the \Csuprem\Examples\CSuprem_To_APSYS\11_MOSFET_3D\with_STI\ldd.in directory.

```
option quiet
mode three.dim
#
phos poly /gas Trn.0=0.0
bor poly /gas Trn.0=0.0
phos oxide /gas Trn.0=0.0
bor oxide /gas Trn.0=0.0
#change the phosphorus diffusivity like this
#phos silicon Dix.0=0.0 Dim.0=0.0 Dimm.0=0.0
#
3d_mesh nsegm=4 infile=geo zstfile=zmesh.zst
init boron conc=1.0e16

#deposit the gate oxide
deposit oxide thick=0.025

#channel implant
implant boron dose=1.0e12 energy=15.0

#deposit the gate poly
deposit poly thick=0.500 div=10 phos conc=1.0e19

#anneal the beast
diff time=10 temp=1000
#
```

```
#etch the poly away
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55 segm=1
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55 segm=2
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55 segm=3
etch poly right p1.x=0.55 p1.y=-0.020 p2.x=0.45 p2.y=-0.55 segm=4
```

```
#anneal this step
diffuse time=30.0 temp=950
```

```
struct outf=poly.str
#
#do the phosphorus implant
implant phos dose=1.0e13 energy=50.0
```

```
#deposit the oxide spacer
deposit oxide thick=0.400 spac=0.05
```

```
#etch the spacer back
etch dry oxide thick=0.420 segm=1
etch dry oxide thick=0.420 segm=2
etch dry oxide thick=0.420 segm=3
etch dry oxide thick=0.420 segm=4
```

```
struct outf=imp2.str
```

```
#after etch anneal
method vert fermi grid.ox=0.05
diffuse time=30 temp=950 dry
```

```
##implant the arsenic
implant ars dose=5.0e15 energy=80.0
```

```
#struct outf=imp4.str
#
##do the final anneal
diffuse time=20 temp=950
```

```
# contact etch
etch oxide p1.x=1.3 right segm=1
etch oxide p1.x=1.3 right segm=2
etch oxide p1.x=1.3 right segm=3
etch oxide p1.x=1.3 right segm=4
```

```
struct outf=ldd.str
```

```
struct mirror left
```

```
struct outf=ldd_mir.str
```

```
export outf=ldd.aps xpsize=0.0001 triangle.based=f
```

6.3.2 Numerical Results and discussion

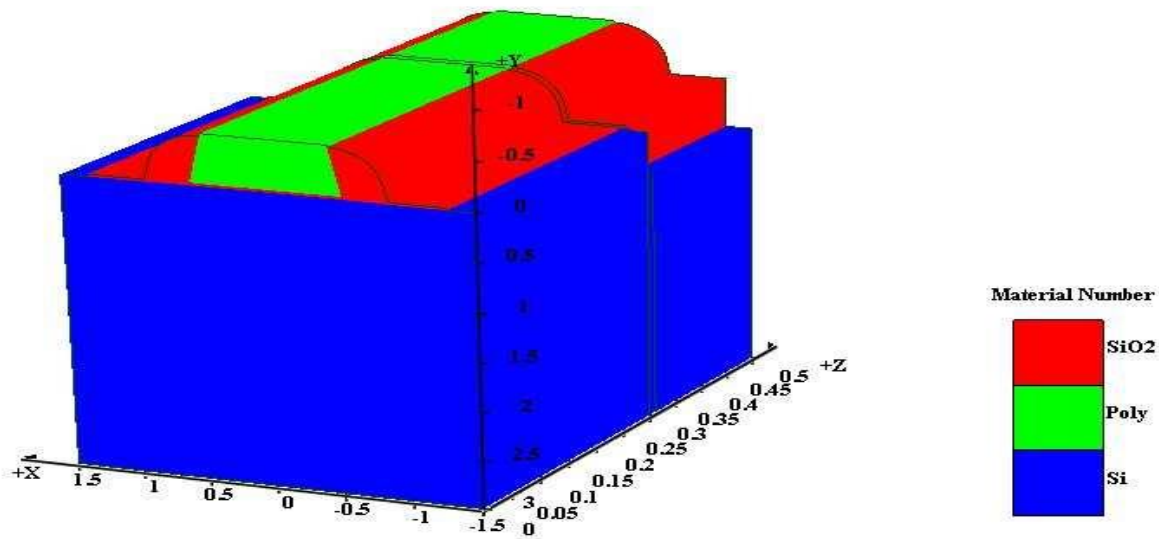


Figure 6. 6: 3D LDD structure with STI

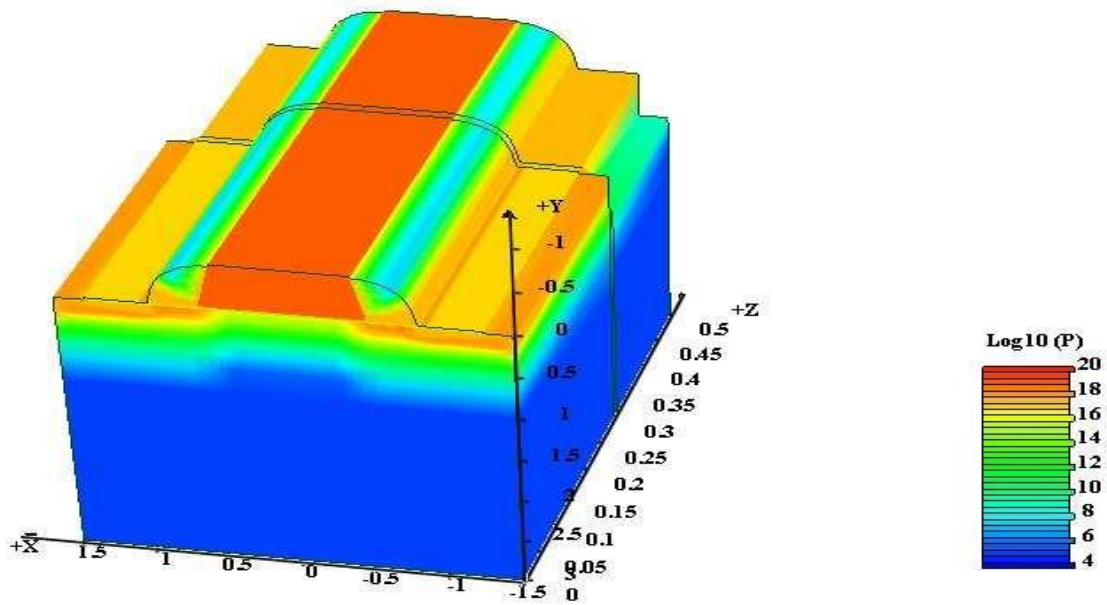


Figure 6. 7: 3D Phosphorous profile after diffusion in STI structure

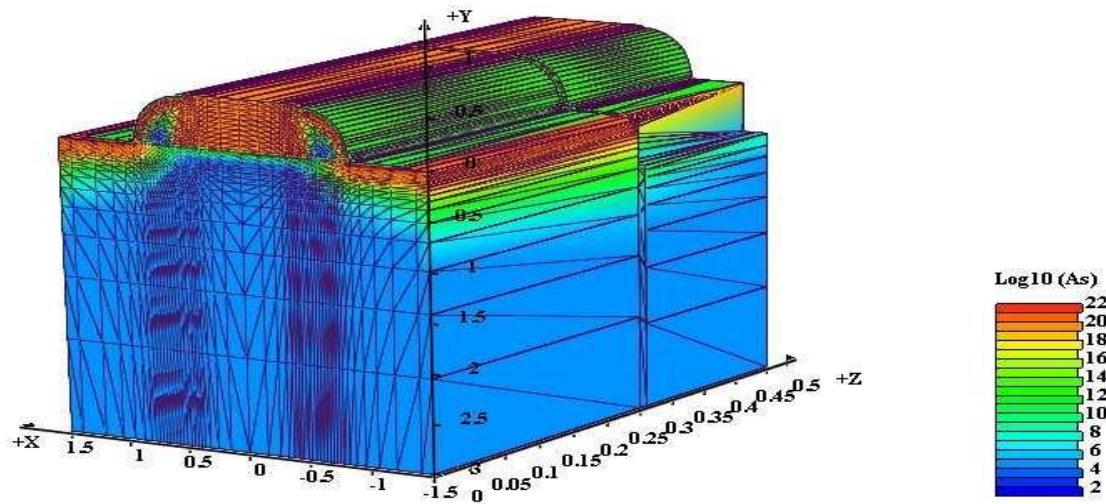


Figure 6. 8: 3D Arsenic profile after diffusion in STI structure

6.4 Example: Independent Gate FinFET

6.4.1 Introduction

The independent gate FinFET has the unique behavioral characteristics of a four-terminal double-gate FET and the processing ease and integration capabilities of the FinFET. Some circuit applications, such as multiple threshold CMOS, equivalent well-bias control, and analog circuits can take advantage from independent control of two isolated gate on the same fully depleted double-gate FET [62]. Using one gate to adjust threshold voltage can help designers to use intrinsic device bodies avoiding the random dopant fluctuation scaling limits [62].

6.4.2 Description

This example simulates the processing steps of an independent gate N-type FinFET structure [62]. The input file *zmesh.zst* should be used to set up the mesh in z-direction and number of mesh points in each segment. The z-coordinate of the segments is between 0 and 5 microns. To build these 3d structure, 5 segments have been used. The number of planes in each segment is 2. The 3d results associated with the final anneal:

- *Arsenic Active profile*
 - *Boron Active profile*
 - *Phosphorus Active profile*
- are plotted bellow.

The GUI CrosslightView was used to visualize the structure and the different doping profiles. This input file **main_example.txt** for the simulation and its associated geometry files are in FINFET1.

main_example.txt

```
option quiet
```

```
# Set Dimensionality
#mode two.dim
mode three.dim
#mode quasi3d
```

```
# Set Number of Segments and Their Geometry Files
3d_mesh nsegm=5  infile=geo
```

```
# Generate 3D Mesh
init
```

```
# Save strcuture
struct outfile=Substrate_3d.str

# Deposit Silicon 0.2um Thick
deposit silicon thick=0.2 divisions=3 boron conc=5e15
struct outfile=deposit1_3d.str

# Etch Silicon away from Segment 1 and 5
etch segm=1 silicon all
etch segm=5 silicon all
struct outfile=etch1_3d.str

# Make Silicon Fins 0.1um Thick: 1.35 - 1.25
etch segm=2 silicon right p1.x=1.35
etch segm=2 silicon left p1.x=1.25

etch segm=3 silicon right p1.x=1.35
etch segm=3 silicon left p1.x=1.25

etch segm=4 silicon right p1.x=1.35
etch segm=4 silicon left p1.x=1.25

struct outfile=etch2_3d.str

# Deposit Gate Oxide 0.2um Thick
deposit oxide thick=0.2 divisions=4
struct outfile=deposit2_3d.str

# Etch Segment 1:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=1 oxide start x=0 y=-0.68
etch segm=1 oxide continue x=0 y=0.
etch segm=1 oxide continue x=2.6 y=0.
etch segm=1 oxide done x=2.6 y=-0.68

# Etch Segment 5:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=5 oxide start x=0.0 y=-0.68
etch segm=5 oxide continue x=0.0 y=0.
etch segm=5 oxide continue x=2.6 y=0.
etch segm=5 oxide done x=2.6 y=-0.68
```

```
# Etch Segment 2:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=2 oxide start x=0 y=-0.68
etch segm=2 oxide continue x=0 y=0.
etch segm=2 oxide continue x=2.6 y=0.
etch segm=2 oxide done x=2.6 y=-0.68
```

```
# Etch Segment 4:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=4 oxide start x=0.0 y=-0.68
etch segm=4 oxide continue x=0.0 y=0.
etch segm=4 oxide continue x=2.6 y=0.
etch segm=4 oxide done x=2.6 y=-0.68
```

```
struct outfile=etch3_3d.str
```

```
# Etch Segment 3:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(1.1,0), P4(1.1,-0.68)
etch segm=3 oxide start x=0 y=-0.68
etch segm=3 oxide continue x=0 y=0.
etch segm=3 oxide continue x=1.1 y=0.
etch segm=3 oxide done x=1.1 y=-0.68
```

```
# Etch Segment 3:
# Etch the Polygone defined by P1(1.5,-0.68), P2(1.5,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=3 oxide start x=1.5 y=-0.68
etch segm=3 oxide continue x=1.5 y=0.
etch segm=3 oxide continue x=2.6 y=0.
etch segm=3 oxide done x=2.6 y=-0.68
```

```
struct outfile=etch4_3d.str
```

```
# Deposit N+ Doped Polysicon 0.26um Thick
deposit poly thick=0.26 div=3 phos conc=1e19
struct outfile=deposit3_3d.str
```

```
# Etch away Polysilicon from Segments 1,2,4,5
etch segm=1 poly all
etch segm=2 poly all
etch segm=4 poly all
etch segm=5 poly all
```

```
struct outfile=etch5_3d.str
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(2.4,-0.98), P2(2.4,0), P3(2.6,0), P4(2.6,-0.98)
```

```
etch segm=3 poly start x=2.4 y=-0.98
```

```
etch segm=3 poly continue x=2.4 y=0.
```

```
etch segm=3 poly continue x=2.6 y=0.
```

```
etch segm=3 poly done x=2.6 y=-0.98
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(0,-0.98), P2(0,0), P3(0.2,0), P4(0.2,-0.98)
```

```
etch segm=3 poly start x=0 y=-0.98
```

```
etch segm=3 poly continue x=0 y=0.
```

```
etch segm=3 poly continue x=0.2 y=0.
```

```
etch segm=3 poly done x=0.2 y=-0.98
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(0,-0.9), P2(0,-0.26), P3(2.6,-0.26), P4(2.6,-0.9)
```

```
etch segm=3 poly start x=0.0 y=-0.98
```

```
etch segm=3 poly continue x=0.0 y=-0.26
```

```
etch segm=3 poly continue x=2.6 y=-0.26
```

```
etch segm=3 poly done x=2.6 y=-0.98
```

```
# Save structure
```

```
struct outfile=etch6_3d.str
```

```
# Deposit Nitride Cap 0.22um Thick
```

```
deposit nitride thick=0.22 divisions=3
```

```
struct outfile=deposit4_3d.str
```

```
# Etch Nitride from Segments 1,2,4,5
```

```
etch segm=1 nitride all
```

```
etch segm=2 nitride all
```

```
etch segm=4 nitride all
```

```
etch segm=5 nitride all
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(2.4,-0.98), P2(2.4,0), P3(2.6,0), P4(2.6,-0.98)
```

```
etch segm=3 nitride start x=2.4 y=-0.98
```

```
etch segm=3 nitride continue x=2.4 y=0.
```

```
etch segm=3 nitride continue x=2.6 y=0.
```

```
etch segm=3 nitride done x=2.6 y=-0.98
```

```

# Anisotropic Etch of Segment 3:
# Etch the Polygon defined by P1(2.4,-0.98), P2(2.4,0), P3(2.6,0), P4(2.6,-0.98)
etch segm=3 nitride start x=0 y=-0.98
etch segm=3 nitride continue x=0 y=0.
etch segm=3 nitride continue x=0.2 y=0.
etch segm=3 nitride done x=0.2 y=-0.98

struct outfile=etch7_3d.str

# Arsenic Implant
implant arsenic    dose=8.0e14 energy=10.0 pearson
struct outfile=implant1_3d.str

# Diffusion step
diffuse time=10 temp=950
struct outfile=finfet.str

quit

```

6.4.3 Numerical Results

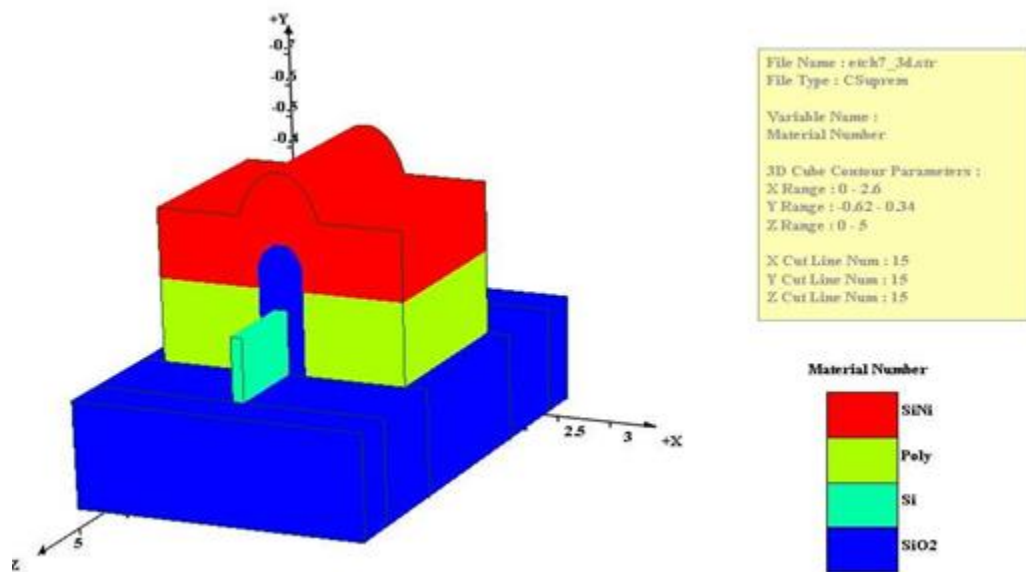


Figure 6. 9: Independent Gate FinFET Structure

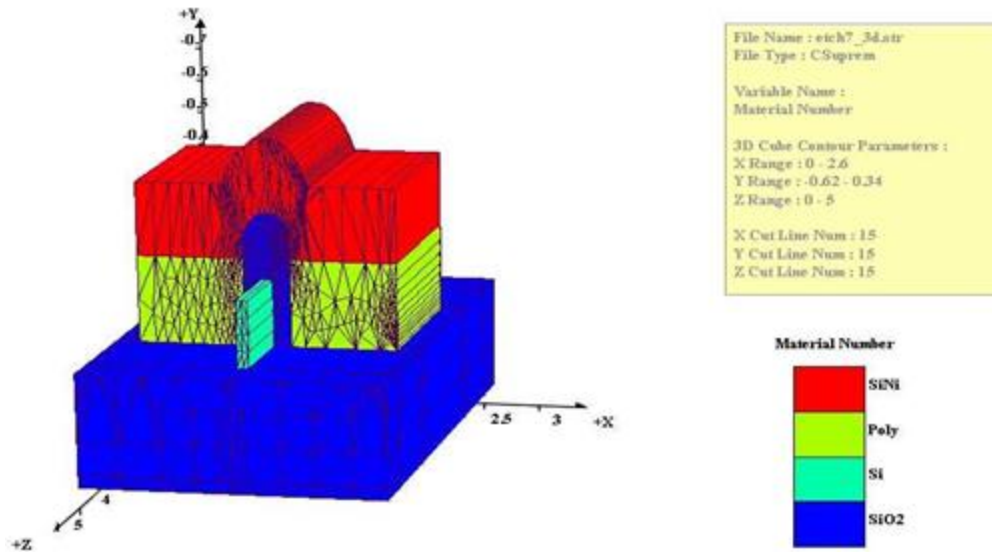


Figure 6. 10: 3d mesh

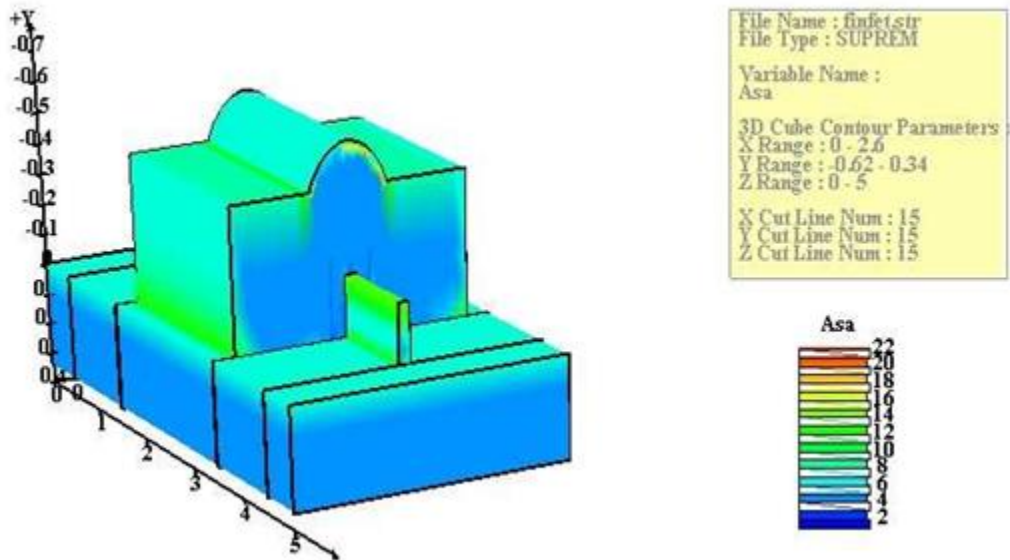


Figure 6. 11: Arsenic active after 10mn diffusion

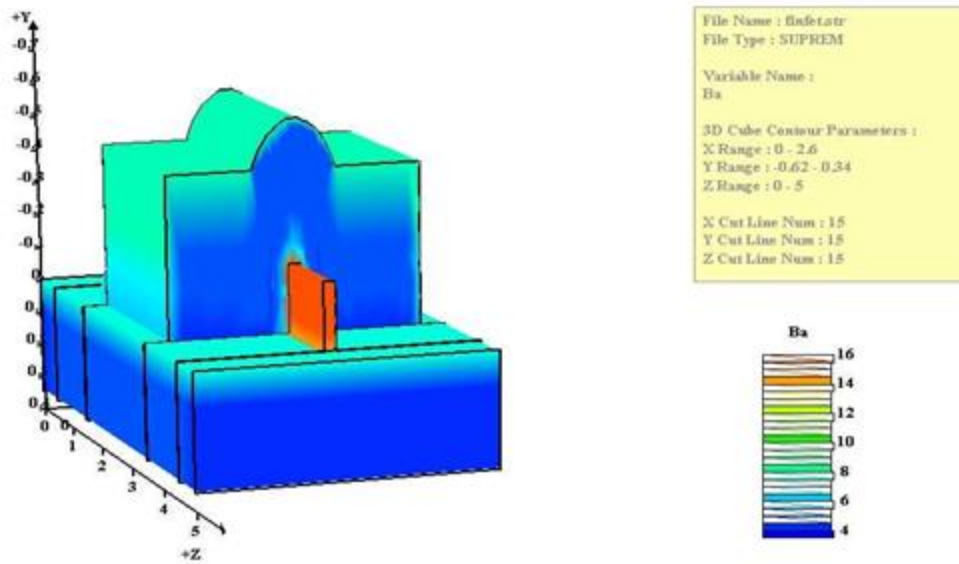


Figure 6. 12: Boron active after 10mn diffusion

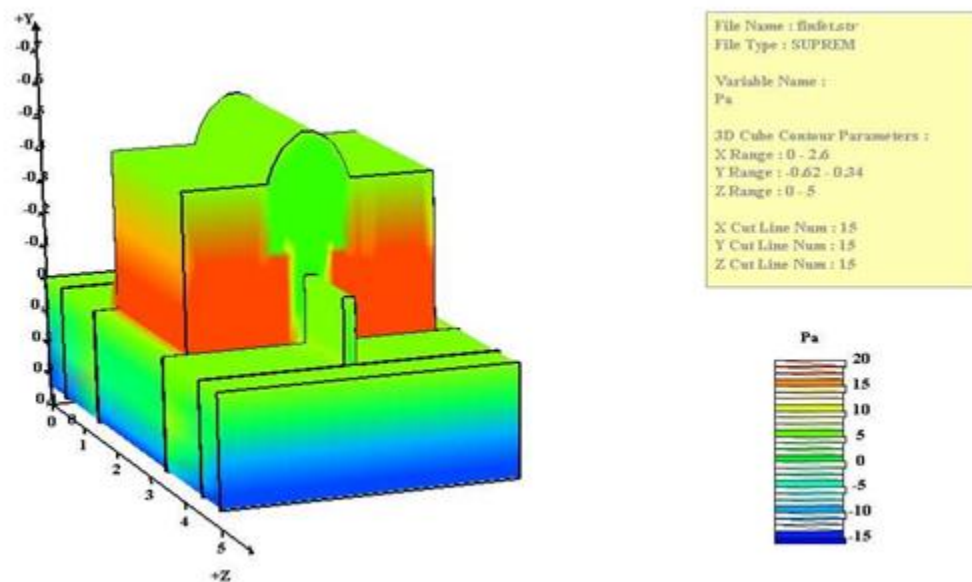


Figure 6. 13: Phosphorus active after 10mn diffusion

6.5 Example: Dependent Double Gate FinFET

6.5.1 Description

This example simulates the processing steps of a dependent gate FinFET structure. The input file *zmesh.zst* should be used to set up the mesh in z-direction and number of mesh points in each segment. The z-coordinate of the segments is between 0 and 5 microns. To build these 3d structure, 5 segments have been used. The number of planes in each segment is 2. The 3d results associated with the final anneal:

- *Arsenic Active profile*
 - *Boron Active profile*
 - *Phosphorus profile*
 - *Potential profile*
- are plotted bellow.

The GUI CrosslightView was used to visualize the structure and the different doping profiles. This input file **main_example.txt** for the simulation and its associated geometry files are in the \Csuprem\examples\CSuprem_Only\09_FINFET2_3D directory.

main_example.txt

option verbose

Set Dimensionality

#mode two.dim

mode three.dim

Set Number of Segments and Their Geometry Files

3d_mesh nsegm=5 infile=mygeo

Generate 3D Mesh

init

Save strcuture

struct outfile=Substrate_3d.str

Deposit Silicon 0.2um Thick

deposit silicon thick=0.2 divisions=3 boron conc=5e15

struct outfile=deposit1_3d.str

implant arsenic dose=8.0e14 energy=10.0 pearson

```
#struct outfile=implant1_3d.str
```

```
# Diffusion step
```

```
diffuse time=2 temp=950
```

```
type_etch type=1
```

```
# Etch Silicon away from Segment 1 and 5
```

```
etch segm=1 silicon all
```

```
etch segm=5 silicon all
```

```
#struct outfile=etch1_3d.str
```

```
# Make Silicon Fins 0.1um Thick: 1.35 - 1.25
```

```
etch segm=2 silicon right p1.x=1.35
```

```
etch segm=2 silicon left p1.x=1.25
```

```
etch segm=3 silicon right p1.x=1.35
```

```
etch segm=3 silicon left p1.x=1.25
```

```
etch segm=4 silicon right p1.x=1.35
```

```
etch segm=4 silicon left p1.x=1.25
```

```
#struct outfile=etch2_3d.str
```

```
# Deposit Gate Oxide 0.2um Thick
```

```
deposit oxide thick=0.2 divisions=4
```

```
#struct outfile=deposit2_3d.str
```

```
# Etch Segment 1:
```

```
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
```

```
etch segm=1 oxide start x=0 y=-0.68
```

```
etch segm=1 oxide continue x=0 y=0.
```

```
etch segm=1 oxide continue x=2.6 y=0.
```

```
etch segm=1 oxide done x=2.6 y=-0.68
```

```
# Etch Segment 5:
```

```
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
```

```
etch segm=5 oxide start x=0.0 y=-0.68
```

```
etch segm=5 oxide continue x=0.0 y=0.
```

```
etch segm=5 oxide continue x=2.6 y=0.
```

```
etch segm=5 oxide done x=2.6 y=-0.68
```

```
# Etch Segment 2:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=2 oxide start x=0 y=-0.68
etch segm=2 oxide continue x=0 y=0.
etch segm=2 oxide continue x=2.6 y=0.
etch segm=2 oxide done x=2.6 y=-0.68
```

```
# Etch Segment 4:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=4 oxide start x=0.0 y=-0.68
etch segm=4 oxide continue x=0.0 y=0.
etch segm=4 oxide continue x=2.6 y=0.
etch segm=4 oxide done x=2.6 y=-0.68
```

```
#struct outfile=etch3_3d.str
```

```
# Etch Segment 3:
# Etch the Polygone defined by P1(0,-0.68), P2(0,0), P3(1.1,0), P4(1.1,-0.68)
etch segm=3 oxide start x=0 y=-0.68
etch segm=3 oxide continue x=0 y=0.
etch segm=3 oxide continue x=1.1 y=0.
etch segm=3 oxide done x=1.1 y=-0.68
```

```
# Etch Segment 3:
# Etch the Polygone defined by P1(1.5,-0.68), P2(1.5,0), P3(2.6,0), P4(2.6,-0.68)
etch segm=3 oxide start x=1.5 y=-0.68
etch segm=3 oxide continue x=1.5 y=0.
etch segm=3 oxide continue x=2.6 y=0.
etch segm=3 oxide done x=2.6 y=-0.68
```

```
struct outfile=etch4_3d.str
```

```
# Deposit N+ Doped Polysicon 0.26um Thick
deposit poly thick=0.26 div=3 phos conc=1e19
#struct outfile=deposit3_3d.str
```

```
# Etch away Polysilicon from Segments 1,2,4,5
etch segm=1 poly all
etch segm=2 poly all
etch segm=4 poly all
etch segm=5 poly all
```

```
#struct outfile=etch5_3d.str
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(2.4,-0.98), P2(2.4,0), P3(2.6,0), P4(2.6,-0.98)
```

```
etch segm=3 poly start x=2.4 y=-0.98
```

```
etch segm=3 poly continue x=2.4 y=0.
```

```
etch segm=3 poly continue x=2.6 y=0.
```

```
etch segm=3 poly done x=2.6 y=-0.98
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(0,-0.98), P2(0,0), P3(0.2,0), P4(0.2,-0.98)
```

```
etch segm=3 poly start x=0 y=-0.98
```

```
etch segm=3 poly continue x=0 y=0.
```

```
etch segm=3 poly continue x=0.2 y=0.
```

```
etch segm=3 poly done x=0.2 y=-0.98
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(0,-0.9), P2(0,-0.26), P3(2.6,-0.26), P4(2.6,-0.9)
```

```
etch segm=3 poly start x=0.0 y=-0.98
```

```
etch segm=3 poly continue x=0.0 y=-0.26
```

```
etch segm=3 poly continue x=2.6 y=-0.26
```

```
etch segm=3 poly done x=2.6 y=-0.98
```

```
# Save structure
```

```
#struct outfile=etch6_3d.str
```

```
# Deposit Nitride Cap 0.22um Thick
```

```
deposit nitride thick=0.22 divisions=3
```

```
#struct outfile=deposit4_3d.str
```

```
# Etch Nitride from Segments 1,2,4,5
```

```
etch segm=1 nitride all
```

```
etch segm=2 nitride all
```

```
etch segm=4 nitride all
```

```
etch segm=5 nitride all
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(2.4,-0.98), P2(2.4,0), P3(2.6,0), P4(2.6,-0.98)
```

```
etch segm=3 nitride start x=2.4 y=-0.98
```

```
etch segm=3 nitride continue x=2.4 y=0.
```

```
etch segm=3 nitride continue x=2.6 y=0.
```

```
etch segm=3 nitride done x=2.6 y=-0.98
```

```
# Anisotropic Etch of Segment 3:
```

```
# Etch the Polygone defined by P1(2.4,-0.98), P2(2.4,0), P3(2.6,0), P4(2.6,-0.98)
etch segm=3 nitride start x=0 y=-0.98
etch segm=3 nitride continue x=0 y=0.
etch segm=3 nitride continue x=0.2 y=0.
etch segm=3 nitride done x=0.2 y=-0.98

#struct outfile=etch7_3d.str

# Arsenic Implant

implant arsenic dose=8.0e14 energy=10.0 pearson

#struct outfile=implant1_3d.str

# Diffusion step

diffuse time=10 temp=950

struct outfile=finfet.str

#export outf=device.str xpsize=0.001

quit
```

6.5.2 Numerical Results

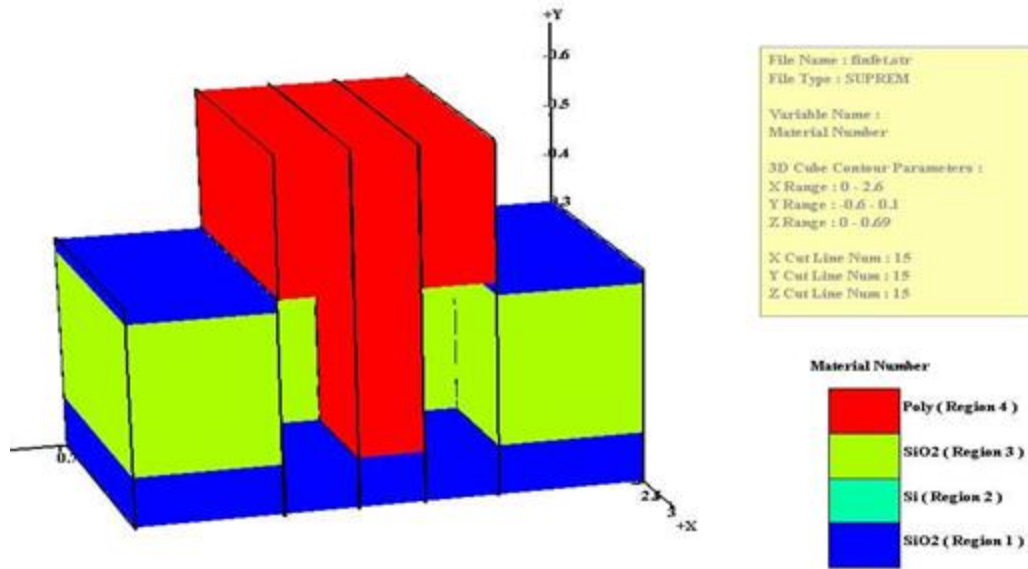


Figure 6. 14: Dependent Double Gate FinFET Structure

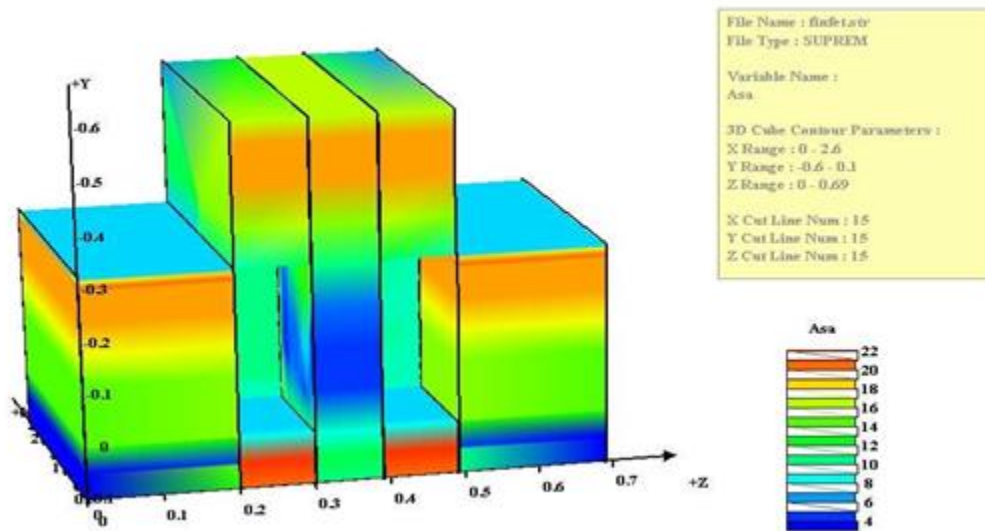


Figure 6. 15: Arsenic active after 20mn diffusion

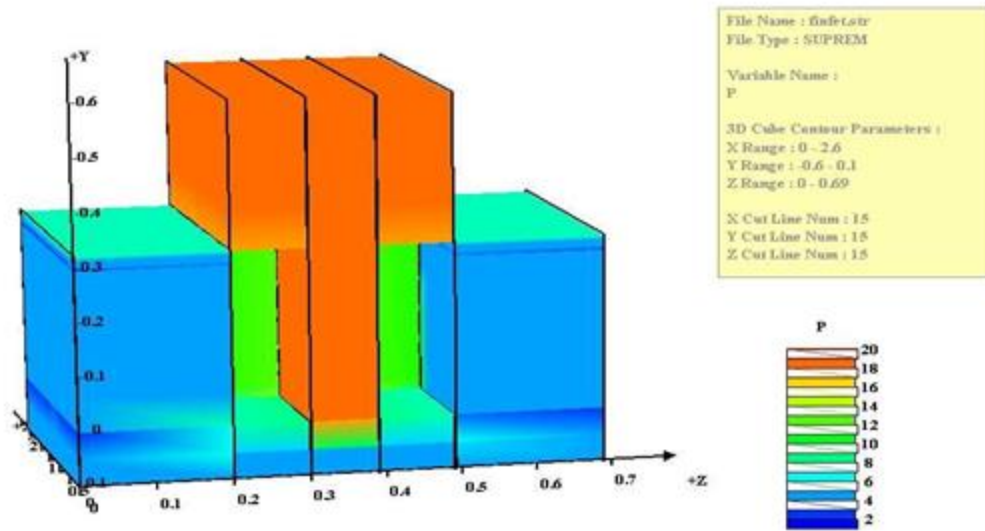


Figure 6. 16: Phosphorus after 20mn diffusion

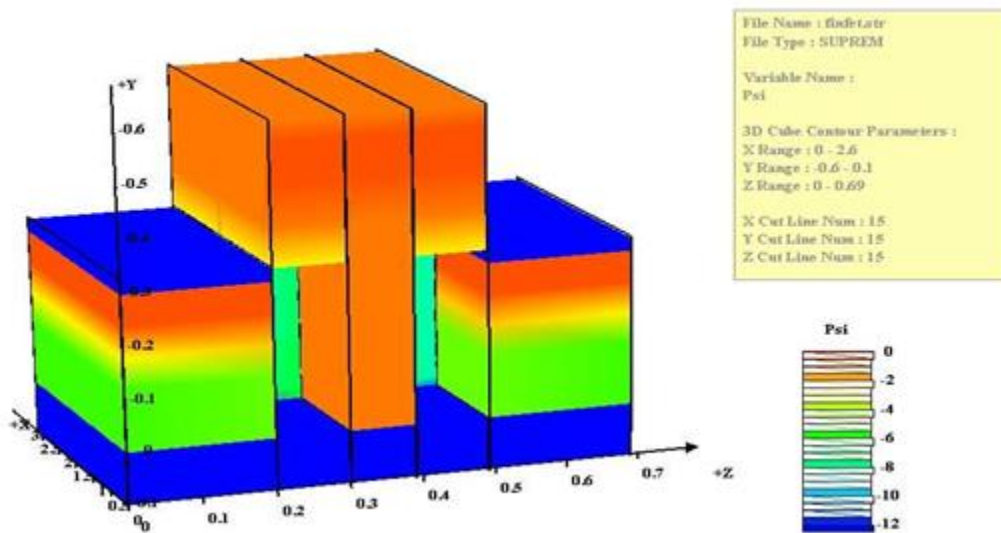


Figure 6. 17: Potential after 20mn diffusion

CHAPTER 7

CSuprem Statements

Commands in CSuprem is arranged in alphabetical order. Whenever possible, the syntax follows that from the original reference manual SUPREM.IV.GS from Stanford University.

7.1 3d.implant.method

7.1.1 Synopsis

3d.implant.method segm=<n> lateral.angle=<n> (unit=degrees)

7.1.2 Description

This command is used to scale the lateral doping profile in a quasi-3D simulation. In quasi-3D modeling of the implant process, lateral scattering of the dopants is only considered within the same plane. If the z-plane of 3D simulation is not perpendicular to mask edge or any blocking structure, the apparent lateral profile should be scaled by a $1/\cos(\theta)$, where θ is angle between z-plane of the 3D simulation and the plane perpendicular to the mask edge.

- **segm**

It is the segment number of the 3D structure.

- **lateral.angle**

It is the angle (in degree) between z-plane of the 3D structure and the plane perpendicular to the mask edge.

7.1.3 Examples

`3d.implant.method segm=3 lateral.angle=30`

This command should be used if the z-plane is not perpendicular to the implant mask edge but makes a 30 degree angle with the plane normal to the mask edge.

7.2 3d.method

7.2.1 Synopsis

```
3d.method
[zflow_oxidant=<logical>(default=false)]
[shear_stress=<n>(default=0)]
[zcoupling=<logical>(default=true)]
[3d.iter.check = <n> (default=30)]
[scale.z.segregation = <n> (default=1)]
[precondition = <n> (default=1)]
```

7.2.2 Description

This command is used to control the numerical solution for 3D simulation.

- **zflow_oxidant**

This would cause the oxidant to flow between 3D planes in an oxidation process. Turn off the oxidant flow in z-direction only affect the oxidant, not other impurities.

- **shear_stress**

Set the value of shear stress to be used by stress

- **zcoupling**

This parameter would enable the mesh points to couple between 3D mesh planes in diffusion

calculation. If this parameter is set to true then we are solving a full 3d diffusion. If it is set to false, then we are solving a quasi3d diffusion model. This means there is no coupling between the planes in 3d.

- **scale.z.segregation**

This parameter allows the user to control the segregation between the segments in 3d. Since the segments are shifted in 3d it is difficult to assure the segregation between them. So, this parameter is used to enforce the segregation in z-direction. Its default value is true.

- **precondition**

There are three ways to defragment a matrix, namely: diag, knot, full.fac. The default method will be diag.

7.2.3 Examples

```
3d.method    zflow_oxidant=true    scale.z.segregation=1    zcoupling=1    shear_stress=0
precondition=1
```

This command would enable a full 3D oxidation simulation with oxidant flowing in all three directions.

7.3 3d_mesh

7.3.1 Synopsis

```
3d_mesh nsegm=<n> infile=<string> zstfile=<string>
```

7.3.2 Description

This command is used to load commands contained in a number of input decks representing mesh lines for different planes.

- **nsegm**

It is the number of segments with each having a different mesh.

- **infile**

It is the mesh command file name base. The segment number is to be appended to such a file base to form the full file name. For example, infile=mymesh would mean loading input files of mymesh1.in, mymesh2.in,

- **zstfile**

Input the file that contains data in the z direction, if this command is not used, zmesh.zst will be applied.

7.3.3 Examples

```
3d_mesh nsegm=3 infile=mymesh zstfile=zmesh.zst
```

This command would load the mesh input files of mymesh1.in mymesh2.in and mymesh3.in.

7.4 activation.model

7.4.1 Synopsis

activation.model

(arsenic | phosphorus | boron | gallium | antimony | gold | beryllium | magnesium | selenium | isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric | vgeneric | indium | al.impurity)
[fraction=<n> (default=1.0)]

7.4.2 Description

This variable is used to set the activatoin fraction for certain impurity.

- **arsenic, phosphorus, boron, gallium, antimony, gold, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic, iigeneric, iiigeneric, ivgeneric, vgeneric, indium, al.impurity** Select an impurity type

- **fraction**

Set activatoin fraction

7.4.3 Examples

```
activation.model boron fraction=1.0
```

This command set boron's activation fraction to 1.0

7.5 al.impurity

7.5.1 Synopsis

al.impurity

(silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)
[Dix.0=<n>] [Dix.E=<n>] [Dip.0=<n>] [Dip.E=<n>]
[Fi = <n>]
[implanted] [grown.in]
[ss.clear] [ss.temp=<n>] [ss.conc=<n>]
[(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial)]
[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]
[(donor | acceptor)]

7.5.2 Description

This statement allows the user to specify values for coefficients of al.impurity diffusion and segregation. The diffusion equation for al.impurity is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{p}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{p}{n_i} \right) \right] \quad 7.1$$

where C_T and C_A are the total chemical and active concentrations of al.impurity, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} \right) \quad 7.2$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} \right) \quad 7.3$$

D_X , and D_P are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.4$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of DX , the al.impurity diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.28 cm²/sec in silicon, and Dix.E defaults to 3.46 eV in silicon [3]. DX is calculated using a standard Arrhenius relationship.

- **Dip.0,Dip.E**

These floating point parameters allow the specification of DP, the al.impurity diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to 0.23 cm²/sec in silicon, and Dip.E defaults to 3.46 eV in silicon [3]. DP is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether al.impurity diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.8[4].

- **implanted, grown.in**

Specifies whether the Dix , Dip , or Fi coefficients apply to implanted or grown-in al.impurity. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm ⁻³)
300	1.0000e+19
600.0	1.8000e+19

700.0	2.5000e+19
800.0	3.4499e+19
825.0	4.1291e+19
850.0	4.9027e+19
875.0	5.7777e+19
900.0	6.7615e+19
925.0	7.8610e+19
950.0	9.0832e+19
975.0	1.0435e+20
1000.	1.1922e+20
1025.	1.3552e+20
1050.	1.5331e+20
1075.	1.7263e+20
1100.	1.9356e+20
1125.	2.1613e+20
1150.	2.4041e+20
1175.	2.6643e+20
1200.	2.9423e+20
1225.	3.2387e+20
1250.	3.5536e+20
1275.	3.8876e+20

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default al.impurity is an acceptor.

7.5.3 Examples

`al.impurity silicon Dix.0=0.28 Dix.E=3.46`

This command changes the neutral defect diffusivity in silicon.

`al.impurity silicon /oxide Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7`

This command will change the segregation parameters at the silicon - silicon dioxide interface. The silicon concentration will be half the oxide concentration in equilibrium at 1100 C.

7.5.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/ Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. D. Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.
3. G.P. Barbuscia, G. Chin, R.W. Dutton, T. Alvarez and L. Arledge, "Modeling of Polysilicon Dopant Diffusion for Shallow Junction Bipolar Technology," International Electron Devices Meeting, San Francisco, p. 757, 1984.
4. P.A. Packan and J.D. Plummer, "Temperature and Time Dependence of B and P Diffusion in Si During Surface Oxidation," J. Appl. Phys., 68(8), 1990.

7.6 antimony

7.6.1 Synopsis

antimony

(silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)

[Dix.0= <n>] [Dix.E= <n>] [Dim.0= <n>] [Dim.E= <n>]

[Fi=<n>] [implanted] [grown.in]

[ss.clear] [ss.temp= <n>] [ss.conc= <n>]

[(/silicon | /oxide | /oxynit | /nitride | /gas /poly | /gaas | /imaterial | /iimaterial | /iiimaterial | /ivmaterial | /vmaterial)]

[Seg.0= <n>] [Seg.E= <n>] [Trn.0= <n>] [Trn.E= <n>]

[(donor | acceptor)]

7.6.2 Description

This statement allows the user to specify values for coefficients of antimony diffusion and segregation. The diffusion equation for antimony is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.5$$

where C_T and C_A are the total chemical and active concentrations of arsenic, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_M \frac{n}{n_i} \right) \quad 7.6$$

$$D_I = F_i \left(D_X + D_M \frac{n}{n_i} \right) \quad 7.7$$

D_X and D_M are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.8$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial | iimaterial | iimaterial | ivmaterial | vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the antimony diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.214 cm²/sec in silicon, and Dix.E defaults to 3.65 eV in silicon [3]. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the antimony diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 15.0 cm²/sec in silicon, and Dim.E defaults to 4.08 eV in silicon [3]. D_M is

calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether antimony diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.05 [4].

- **Ctn.0, Ctn.E**

These parameters specify the clustering coefficients for antimony. The parameters are respectively the pre-exponential coefficient and the activation energy. The total cluster coefficient obeys a standard Arrhenius relationship. The active concentration, C_A is calculated using:

$$C_T = C_A \left[1 + C_{tn} \left(\frac{n}{n_i} \right)^2 \frac{C_V}{C_V^*} \right] \quad 7.9$$

- **implanted, grown.in**

Specifies whether the Dix , Dim , or Fi coefficients apply to implanted or grown-in antimony. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm^{-3})
800	2.3e19
900	3.0e19
1000	4.0e19
1100	4.8e19
1200	6.2e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial | /iimaterial | /iiimaterial | /ivmaterial | /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M 12 is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default antimony is a donor.

7.6.3 Examples

`antimony silicon Dix.0=0.214 Dix.E=3.65`

This command changes the neutral defect diffusivity in silicon.

`antimony silicon /oxide Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the silicon - silicon dioxide interface. The silicon concentration will be 30.0 times the oxide concentration in equilibrium.

7.6.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. D. Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.
3. R.B. Fair, "Concentration Profiles of Diffused Dopants in Silicon," Impurity Doping Processes in Silicon, Yang ed., 1981, North-Holland, New York.
4. P. Fahey, G. Barbuscia, M. Moslehi and R.W. Dutton, "Kinetics of Thermal Nitridation Processes in the Study of Dopant Diffusion Mechanisms in Silicon," Appl. Phys. Lett., 46(8), p. 784, 1985.
5. F.A. Trumbore, "Solid Solubilities of Impurity Elements in Germanium and Silicon," Bell System Tech Journal, 1960.

7.7 arsenic

7.7.1 Synopsis

arsenic

(silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial | ivmaterial | vmaterial)

[Dix.0=<n>] [Dix.E=<n>] [Dim.0=<n>] [Dim.E=<n>]

[Fi = <n>]

[implanted] [grown.in]

[Ctn.0=<n>] [Ctn.E=<n>]

[ss.clear] [ss.temp=<n>] [ss.conc=<n>]

[(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial | /iimaterial | /ivmaterial | /vmaterial)]

[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]

[(donor | acceptor)]

7.7.2 Description

This statement allows the user to specify values for coefficients of arsenic diffusion and segregation. The diffusion equation for arsenic is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.10$$

where C_T and C_A are the total chemical and active concentrations of arsenic, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_M \frac{n}{n_i} \right) \quad 7.11$$

$$D_I = F_i \left(D_X + D_M \frac{n}{n_i} \right) \quad 7.12$$

D_X and D_M are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.13$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and

T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial | iimaterial | iimaterial | ivmaterial | vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the arsenic diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.214 cm²/sec in silicon, and Dix.E defaults to 3.65 eV in silicon [3]. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the arsenic diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 15.0 cm²/sec in silicon, and Dim.E defaults to 4.08 eV in silicon [3]. D_M is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether arsenic diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.05 [4].

- **implanted, grown.in**

Specifies whether the Dix, Dim, or Fi coefficients apply to implanted or grown-in arsenic. If neither is specified then a specified parameter applies to both.

- **Ctn.0, Ctn.E** These parameters specify the clustering coefficients for arsenic. The parameters are respectively the pre-exponential coefficient and the activation energy. The total cluster coefficient obeys a standard Arrhenius relationship. The active concentration, C_A is calculated using:

$$C_T = C_A \left[1 + C_{tn} \left(\frac{n}{n_i} \right)^2 \frac{C_V}{C_V^*} \right] \quad 7.14$$

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm^{-3})
800	2.3e19
900	3.0e19
1000	4.0e19
1100	4.8e19
1200	6.2e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial | /iimaterial | /iiimaterial | /ivmaterial | /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default arsenic is a donor.

7.7.3 Examples

arsenic silicon Dix.0=8.0 Dix.E=3.65

This command changes the neutral defect diffusivity in silicon.

arsenic silicon /oxide Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0

This command will change the segregation parameters at the silicon - silicon dioxide interface. The silicon concentration will be 30.0 times the oxide concentration in equilibrium.

7.7.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.

2. D. Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving

nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.

3. R.B. Fair, "Concentration Profiles of Diffused Dopants in Silicon," Impurity Doping Processes in Silicon, Yang ed., 1981, North-Holland, New York.

4. P. Fahey, G. Barbuscia, M. Moslehi and R.W. Dutton, "Kinetics of Thermal Nitridation Processes in the Study of Dopant Diffusion Mechanisms in Silicon," Appl. Phys. Lett., 46(8), p. 784, 1985.

5.F.A. Trumbore, "Solid Solubilities of Impurity Elements in Germanium and Silicon," Bell System Tech Journal, 1960.

7.8 beryllium

7.8.1 Synopsis

beryllium

(silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial | ivmaterial | vmaterial)

[Dix.0=<n>] [Dix.E=<n>]

[Dip.0=<n>] [Dip.E=<n>] [Dipp.0=<n>] [Dipp.E=<n>]

[Fi = <n>] [implanted] [grown.in]

[ss.clear] [ss.temp=<n>] [ss.conc=<n>]

[(/silicon | /oxide | /oxynit | /nitride | /gas | /poly | /gaas /imaterial | /iimaterial | /iimaterial | /ivmaterial | /vmaterial)]

[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]

[(donor | acceptor)]

7.8.2 Description

This statement allows the user to specify values for coefficients of beryllium diffusion and segregation. The diffusion equation for beryllium is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{p}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{p}{n_i} \right) \right] \quad 7.15$$

where C_T and C_A are the total chemical and active concentrations of beryllium, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.16$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.17$$

D_X , D_P and D_{PP} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 \frac{C_2}{M_{12}} \right) \quad 7.18$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of D_X , the beryllium diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.0 cm²/sec in gallium arsenide, and Dix.E defaults to 0.0 eV in gallium arsenide. D_X is calculated using a standard Arrhenius relationship.

- **Dip.0, Dip.E**

These floating point parameters allow the specification of D_P , the beryllium diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to $2.1 \times 10^{-6} \text{ cm}^2/\text{sec}$ for implanted beryllium in gallium arsenide and $2.1 \times 10^{-6} \text{ cm}^2/\text{sec}$ for grown-in beryllium in gallium arsenide, and Dip.E defaults to 1.74 eV for both in gallium arsenide [1, 2, 3]. D_P is calculated using a standard Arrhenius relationship.

- **Dipp.0, Dipp.E**

These floating point parameters allow the specification of D_{PP} , the beryllium diffusivity with doubly positive defects. Dipp.0 is the pre-exponential constant and Dipp.E is the activation energy. Dipp.0 defaults to 0.0 cm²/sec in gallium arsenide, and Dipp.E defaults to 0.0 eV in gallium arsenide. D_{PP} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether beryllium diffuses through interaction with interstitials or vacancies. The value of Fi

defaults to 1.

- **implanted, grown.in**

Specifies whether the D_{ix} , D_{ip} , D_{ipp} , or F_i coefficients apply to implanted or grown-in beryllium. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default beryllium is an acceptor.

7.8.3 Examples

beryllium gaas implanted Dip.0=2.2e-6 Dip.E=1.76

This command changes the positive defect diffusivity for implanted beryllium in gallium arsenide.

beryllium gaas /nitride Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7

This command will change the segregation parameters at the gallium arsenide "C silicon nitride interface. The gallium arsenide concentration will be half the nitride concentration in equilibrium at 1100jãC.

7.8.4 References

1. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1990.
2. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1991.
3. M.D. Deal and H.G. Robinson, Diffusion of Implanted Beryllium in P and N Type GaAs, Appl. Phys. Lett. 55, 1990.

7.9 boron

7.9.1 Synopsis

boron (silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial | ivmaterial | vmaterial) [Dix.0=<n>] [Dix.E=<n>] [Dip.0=<n>] [Dip.E=<n>] [Fi = <n>] [implanted] [grown.in] [ss.clear] [ss.temp=<n>] [ss.conc=<n>] [(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas /imaterial | /iimaterial | /iimaterial | /ivmaterial | /vmaterial)] [Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>] [(donor | acceptor)]

7.9.2 Description

This statement allows the user to specify values for coefficients of boron diffusion and segregation. The diffusion equation for boron is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V p}{C_V^* n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I p}{C_I^* n_i} \right) \right] \quad 7.19$$

where C_T and C_A are the total chemical and active concentrations of boron, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are

given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} \right) \quad 7.20$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} \right) \quad 7.21$$

D_X , and D_P are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.22$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of DX , the boron diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.28 cm²/sec in silicon, and Dix.E defaults to 3.46 eV in silicon [3]. DX is calculated using a standard Arrhenius relationship.

- **Dip.0,Dip.E**

These floating point parameters allow the specification of DP, the boron diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to 0.23 cm²/sec in silicon, and Dip.E defaults to 3.46 eV in silicon [3]. DP is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether boron diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.8[4].

- **implanted, grown.in**

Specifies whether the Dix , Dip , or Fi coefficients apply to implanted or grown-in boron. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm^{-3})
300	1.0000e+19
600.0	1.8000e+19
700.0	2.5000e+19
800.0	3.4499e+19
825.0	4.1291e+19
850.0	4.9027e+19
875.0	5.7777e+19
900.0	6.7615e+19
925.0	7.8610e+19
950.0	9.0832e+19
975.0	1.0435e+20
1000.	1.1922e+20
1025.	1.3552e+20
1050.	1.5331e+20
1075.	1.7263e+20
1100.	1.9356e+20
1125.	2.1613e+20
1150.	2.4041e+20
1175.	2.6643e+20
1200.	2.9423e+20
1225.	3.2387e+20
1250.	3.5536e+20
1275.	3.8876e+20

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default boron is an acceptor.

7.9.3 Examples

`boron silicon Dix.0=0.28 Dix.E=3.46`

This command changes the neutral defect diffusivity in silicon.

`boron silicon /oxide Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7`

This command will change the segregation parameters at the silicon "C silicon dioxide interface. The silicon concentration will be half the oxide concentration in equilibrium at 1100 C.

7.9.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/ Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. D.Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.
3. G.P. Barbuscia, G. Chin, R.W. Dutton, T. Alvarez and L. Arledge, "Modeling of Polysilicon Dopant Diffusion for Shallow Junction Bipolar Technology," International Electron Devices Meeting, San Francisco, p. 757, 1984.
4. P.A. Packan and J.D. Plummer, "Temperature and Time Dependence of B and P Diffusion in Si During Surface Oxidation," J. Appl. Phys., 68(8), 1990.
5. A. Armigliato, D. Nobili, P. Ostojic, M. Servidori and S. Solmi, "Solubility and Precipitation of Boron in Silicon and Supersaturation Resulting by Thermal Predeposition," Journal of Applied Physics.

7.10 Boundary

Specify a surface type.

7.10.1 Synopsis

```
boundary
( reflecting | exposed | backside )
xlo = <string> ylo = <string>
xhi = <string> yhi = <string>
code = <n> (default=-999)
```

7.10.2 Description

This statement is used to specify what conditions to apply at each surface in a rectangular mesh.

- **reflecting | exposed | backside**
- **code** In CSuprem, user-defined boundary tag can be used for both top and bottom edge of the device. These tags will set the boundary limit.

At present, three surface types are recognized. exposed surfaces correspond to the top of the wafer. Materials are only deposited on exposed surfaces (so if a deposit does nothing you know where to start looking). Impurity predeposition also happens at exposed surfaces, as does defect recombination and generation. backside surfaces roughly correspond to a nitride- or oxide-capped backside. Defect recombination and generation happen here. reflecting surfaces correspond to the sides of the device, and are also good for the backside if defects are not being simulated. The default for surfaces is reflecting .

- **xlo, ylo, xhi, yhi**

The bounds of the rectangle being specified. The <string> value should be one of the tags created in a preceding line statement

7.10.3 Examples

```
boundary exposed xlo=left xhi=right ylo=surf yhi=surf boundary backsid xlo=left xhi=right
ylo=back yhi=back
```

These lines specify that the top of the mesh is the exposed surface, and that the bottom is the backside.

7.10.4 Bugs

The top surface should probably default to exposed, and the bottom to backside. We took a poll, though, and decided it was safer that our users knew about the existence of boundary codes.

7.11 carbon

7.11.1 Synopsis

carbon

```
( silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial |
ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ]
[ Dip.0=<n> ] [ Dip.E=<n> ] [ Dipp.0=<n> ] [ Dipp.E=<n> ]
[ Fi = <n> ]
[ implanted ] [ grown.in ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ] [ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly
| /gaas /imaterial | /iimaterial | /iimaterial | /ivmaterial | /vmaterial ) ]
[ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]
```

7.11.2 Description

This statement allows the user to specify values for coefficients of carbon diffusion and segregation. The diffusion equation for carbon is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{p}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{p}{n_i} \right) \right] \quad 7.23$$

where C_T and C_A are the total chemical and active concentrations of carbon, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.24$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.25$$

D_X , D_P and D_{PP} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.26$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of D_X , the carbon diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to $1.0 \times 10^{-16} \text{ cm}^2/\text{sec}$ in gallium arsenide, and Dix.E defaults to 0.0 eV in gallium arsenide [1, 2]. D_X is calculated using a standard Arrhenius relationship.

- **Dip.0, Dip.E**

These floating point parameters allow the specification of D_P , the carbon diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to $0.0 \text{ cm}^2/\text{sec}$ in gallium arsenide and Dip.E defaults to 0.0 eV in gallium arsenide. D_P is calculated using a standard Arrhenius relationship.

- **Dipp.0, Dipp.E**

These floating point parameters allow the specification of D_{PP} , the carbon diffusivity with doubly positive defects. Dipp.0 is the pre-exponential constant and Dipp.E is the activation energy. Dipp.0 defaults to $0.0 \text{ cm}^2/\text{sec}$ in gallium arsenide, and Dipp.E defaults to 0.0 eV in gallium arsenide. D_{PP} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether carbon diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 1.

- **implanted, grown.in**

Specifies whether the Dix, Dip, Dipp, or Fi coefficients apply to implanted or grown-in carbon. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default carbon is an acceptor.

7.11.3 Examples

carbon gaas implanted Dip.0=2.2e-6 Dip.E=1.76

This command changes the positive defect diffusivity for implanted carbon in gallium arsenide.

carbon gaas /nitride Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7

This command will change the segregation parameters at the gallium arsenide "C silicon nitride interface. The gallium arsenide concentration will be half the nitride concentration in equilibrium at 1100°C.

7.11.4 References

1. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford

University Technical Report, 1990.

2. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1991.

7.12 cesium

7.12.1 Synopsis

cesium

(silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial |
ivmaterial | vmaterial)
[D.0=<n>] [D.E=<n>]
[implanted] [grown.in]
[mobile] [(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial
| /iimaterial | /ivmaterial | /vmaterial)]
[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]
[g.0=<n>] [g.E=<n>]

7.12.2 Description

This statement allows the user to specify values for coefficients of cesium diffusion and segregation.

Cesium is treated as a charge neutral impurity which diffuses according to the Fick's law:

$$\frac{\partial C_T}{\partial t} = -\nabla \cdot J_T \quad 7.27$$

$$J_T = -D\nabla C_T \quad 7.28$$

where C_T is the total chemical concentrations of cesium, and D is diffusivity. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) - g C_1 C_2 \quad 7.29$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively. M_{12} is related to ratio of solubility of the two materials and T_r is the surface velocity. The last term above represents detrapping of the neutral impurity.

- silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **D.0, D.E**

These floating point parameters allow the specification of D, the cesium diffusivity as a neutral defect. D.0 is the pre-exponential constant and D.E is the activation energy. By default, D.0 = $0.037 \text{ (cm}^2/\text{sec)}$ and D.E = 3.46 (eV).

- **implanted, grown.in**

Specifies whether the coefficients apply to implanted or grown-in cesium. If neither is specified then a specified parameter applies to both.

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship. By default, Trn.0 = 1.0 (cm/s), Trn.E = 0.0 (eV)

- **g.0, g.E**

It specifies the detrapping constant. By default, g.0 = $1.0 \text{ (cm}^4/\text{s)}$, g.E = 0.0 (eV).

- **mobile**

May be used to enable or disable the movement of the impurity.

7.12.3 Examples

cesium silicon implanted D.0=0.05

This command changes the diffusivity for implanted cesium in silicon.

7.13 change_material

7.13.1 Synopsis

```
change_material
[segm=<n> (default=-1)]
(silicon | oxide | oxynitride | nitride | poly | photoresist | aluminum | gaas | AlGaAs | InP |
AlInGaAs | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)
(/silicon | /oxide | /oxynitride | /nitride | /poly | /photoresist | /aluminum | /gaas | /AlGaAs |
/InP | /AlInGaAs | /imaterial | /iimaterial | /iiimaterial | /ivmaterial | /vmaterial)
(none | arsenic | antimony | boron | gallium | phosphorus | beryllium | magnesium | selenium
| isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric |
vgeneric | indium | ni.impurity | al.impurity)
[concentration=<n> (default=1e+10)]
[file=<string>]
```

7.13.2 Description

This command is used to substitute a certain material within the structure defined in the input file to a new material. The area to be substituted is contained in the input file.

- **segm**

During 3D simulation, this command specifies which segment to work on.

- **silicon, oxide, oxynitride, nitride, poly, photoresist, aluminum, gaas, AlGaAs, InP, AlInGaAs, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

Set the material type that needs to be substituted

- **/silicon, /oxide, /oxynitride, /nitride, /poly, /photoresist, /aluminum, /gaas, /AlGaAs, /InP, /AlInGaAs, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

Set the material type after substitution

- **none, arsenic, antimony, boron, gallium, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic, iigeneric, iiigeneric, ivgeneric, vgeneric, indium, ni.impurity, al.impurity**

Set impurity after the substitution. It can be left blank.

- **concentration**

If new impurity has been set, this variable sets the doping concentration for the new impurity.

- **file**

The file contains x and y coordinates of the area to be substituted. The format for each row is as follows: x1,y1 x2,y2 ... xn,yn

7.13.3 Examples

```
change_material silicon /oxide file=input.txt boron conc=1e+14
```

This command will change silicon to oxide within the area specified in the file input.txt and initialize the boron doping concentration to 1e+14 in the new oxide area.

7.14 connect_region

7.14.1 Synopsis

```
connect_region [ npoint=<n> xk=<n> yk=<n> (k=1,...,9) ]
```

7.14.2 Description

This command is used to control the oxide mesh connectivity between z-planes in a 3D weto2 process for application such as LOCOS oxidation. It works as follows. Oxidation involves diffusion of oxidant from gas phase through the oxide to the SiO₂/Si interface. In a 3D oxidation process oxidant must also diffuse from plane to plane in order to produce oxide expansion (under Si₃N₄ layer) which would otherwise be suppressed without interplane oxidant diffusion.

To allow the correct amount of oxidant diffusion, the oxide region on one plane must be fully coupled to the oxide region on the next plane. This may be a problem if the size and shape of the oxide regions are vastly different since by default, the mesh on one plane only connects to the mesh points nearest to it on the next plane. This command guides the program to disregard the default mesh connectivity and let the full oxide region couple to each other from plane to plane. You may imagine this command to let oxide region on one plane to fully "touch" that on the next plane.

It is not trivial to enable the coupling of each oxide region during active oxidation steps for the following reasons: 1) oxide regions can appear, merge with each other and may disappear totally; 2) there may be regions of oxide which are not at all involved in weto2 and for which it is better to use the default mesh connection for better numerical convergence. Therefore, it is better to define specific regions to allow the full region coupling for those regions only.

To specify a region to be connected for 3D field oxide growth, we provide a reference x-y coordinate close to the region. The program then searches on each plane for oxide nodes closest to that coordinate. Once the closest SiO₂ node is found, the program uses such information to compute the oxidant flow between planes.

- **npoint**

It is to inform the program how many reference points are used to search for active oxide regions. CSUPREM divides mesh into regions according to geometric features. Usually, one rectangle area is defined as one region. If you are unsure, you may use more points just to make sure no oxide regions are missed.

- **xk,yk (k=1,...,9)**

These are x-y coordinates in micron meters used by all planes to search for oxide regions to be connected during weto2 diffusion step.

7.14.3 Examples

```
connect_region npoint=3 x1=0.0 y1=-1.e-7 x2=4. y2=-1.e-7 x3=7.5 y3=-1.e-7
```

This command defines 3 reference points located in 3 different geometric regions.

7.15 contour

Plot contours in the selected variable on a two-dimensional plot.

7.15.1 Synopsis

```
contour
[ line.type= <n> ] [ value= <n> ] [ symb= <n> ]
[ print ] [label ]
```

7.15.2 Description

The contour statement draws a contour in the selected variable (see the select statement) at the value specified. value must be specified in the range of the computed variable. For instance, if plotting log boron, value should be in the range 10 to 20 and not e10 to e20 . This statement assumes a plot.2d has been specified and the screen has been set up for plotting a two dimensional picture. If this has not been done, the routine will probably produce garbage on the screen.

- **line.type**

This allows the line type to be specified for this contour. The default is 1. Different line types have different meanings to different graphics devices.

- **value**

This floating parameter expresses the value that the contour line should be plotted at. If boron had been selected, a value of 10^{16} would produce a line of constant boron concentration of 10^{16} cm^{-3} .

- **symb**

This integer parameter indicates that the contour line should be drawn with symbols on the contour line. The integer value indicates different symbols.

- **print**

This boolean parameter instructs the contour command to print the line segments coordinates rather than plot them. This is useful for exporting data to other programs and plot software.

- **label**

The command will attempt to place a text string on the contour in an ideal location to label the value.

7.15.3 Examples

`contour val=1e10`

This command will draw a line at a isoconcentration of $1e10$.

7.15.4 Bugs

This should probably check to make sure a plot.2d has been done. It is conceivable that this statement could produce floating point exceptions when a plot.2d has not been done. This is, to put it mildly, annoying. Plot software bugs will show up in this command.

7.16 define

7.16.1 Synopsis

`define`

7.16.2 Description

Define a variable so that it can be used later

7.16.3 Examples

```
define @Lsti 0.3 line x loc=@Lsti spac=0.1
```

The above command defines the variable Lsti and then called this variable using the command:line.

7.17 deposit

Deposit a layer.

7.17.1 Synopsis

deposit

(silicon | oxide | oxynitr | nitride | poly | photores | alumin | gaas | AlGaAs | Inp | AlInGaAs |
imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)

[thickness = <n>]

[divisions = <n>]

[none | arsenic | antimony | boron | gallium | phosphor | beryllium | magnesium | selenium |
isilicon | tin | germanium | zinc | carbon | generic | indium | ni.impurity | al.impurity |
iigeneric | iiigeneric | ivgeneric | vgeneric]

[conc= <n>][space= <n>] [file= <string>]

[meshlayer= <n>] [segm = <n> (default=-1)]

7.17.2 Description

This statement provides a primitive deposit capability. Material is deposited on the "exposed" surface of the structure, and its upper surface becomes the new exposed surface. There have been several enhancements in the handling of the offset line. It is now possible, for instance, to fill a trench completely.

- **silicon | oxide | oxynitr | nitride | poly | photores | alumin | gaas | AlGaAs | Inp | AlInGaAs | imaterial | iimaterial | iiimaterial | ivmaterial**

The material to be deposited.

- **thickness**

The thickness in microns.

- **divisions**

The number of vertical grid spacings in the layer, default 1.

- **none** | **arsenic** | **antimony** | **boron** | **phosphor** | **beryllium** | **magnesium** | **selenium** | **isilicon** | **tin** | **germanium** | **zinc** | **carbon** | **generic** | **iigeneric** | **iiigeneric** | **ivgeneric** | **vgeneric**

The type of doping in the deposited layer.

- **conc**

This floating point parameter specifies the level of the doping in the deposited layer.

- **space**

This floating point parameter represents the average spacing between points along the curved edge of the deposit. On a flat surface, deposited layer lets the mesh line to extend upwards from the mesh layer below and the horizontal mesh spacing inherits from below. Such extension is not possible if deposit is performed to fill a dent between two horizontal step structures. In such a case, a curved surface is formed between the two steps and this mesh spacing takes effect on the curved outer surface.

- **resistivity**

Parameter resistivity has been used in the following commands to replace the doping concentration:

```
region
deposit
fill_hole
```

This is based on known relation between doping and resistivity for some well known materials such as silicon and GaAs. For details, see example:

<http://inside.mines.edu/~sagarwal/phgn435/images/resistivity.jpg>

We plot the resistivity versus doping for n-type and p-type doping for silicon in the figure above. This has been implemented for phosphorous and boron in silicon.set doping concentrations.

- **file**

This string parameter specifies the name of a file that contains the coordinates associated with the deposit surface. This file could be generated using a physically based simulator. An auxiliary program called etch_geometry.exe is available to generate, among other profiles, the etching profile as a result of reactive ion etching (RIE).

- **meshlayer**

this causes the mesh generator to place meshlayer number of horizontal mesh lines (mesh-layer) for the deposited material. This command may be used to generate layered mesh structures.

- **segm**

In 3D simulation, when segm doesn't equal to -1, only specified segment will be deposited.

7.17.3 Examples

```
deposit oxide thick=0.02 meshlayer=4
```

This deposits silicon dioxide 200 angstroms thick on the surface of the silicon. At least 4 horizontal mesh lines are used for the SiO₂ area.

7.18 diffuse

Run a time temperature step on the wafer and calculate oxidation and diffusion of impurities.

7.18.1 Synopsis

```
diffuse time= <n> temperature= <n> final_temp=<n>
[ ( dryo2 | weto2 | nitrogen | ammonia | argon | antimony | arsenic | boron | gallium |
phosphorus | beryllium | magnesium | selenium | isilicon | tin | germanium | zinc | carbon |
generic | speedie | indium | ni.impurity | al.impurity | iigeneric | iiigeneric | ivgeneric |
vgeneric) ]
[ gas.conc= <n> ] [ pressure= <n> ] [stepwise_ramp=<n>]
[ gold.surf ] [ continue ]
[ movie= <string> ] [ dump= <n> ]
[ flow.control ] [ H2=<n> ] [ O2=<n> ] [ steam=<n> ] [ HCL=<n> ] [ other=<n> ]
```

7.18.2 Description

The diffuse statement allows the user to specify a diffusion or oxidation step. Any impurities present in the wafer are diffused. If the wafer is exposed to a gas, a predeposition or oxidation can be performed. This statement represents the core simulation capability of SUPREM-IV. Users can expect that this statement will take a lot of CPU time.

The parameters for oxidation and diffusivities can be found on the statements in the models section of the manual. Refer to either the oxide statement or any of the impurity statements to determine how to change coefficients of the models.

- **time**

This parameter represents the amount of time the wafer will spend in the furnace in minutes.

- **temp**

This parameter allows specification of the furnace temperature in degrees centigrade. This parameter is also used to represent the initial temperature of a rapid thermal annealing (RTA).

- **final_temp**

This parameter allows specification of the final furnace temperature in degrees centigrade. If defined, it represents the final temperature in a temperature ramp of an RTA. The ramp rate may be calculated using the following formula:

$$\text{ramp_rate} = (\text{final_temp} - \text{temp}) / \text{time}$$

Similarly, given the initial temperature and the ramp rate, one may figure out the final temperature using:

$$\text{final_temp} = \text{temp} + \text{tim} * \text{ramp_rate}$$

One should be careful about the units of the ramp rates since many use degree per second while the time here is in minutes. By default, this parameter is not used and diffuse means constant temperature annealing.

- **stepwise_ram**

It is a positive integer. If defined, the program performs a linear temperature ramp by dividing it into N number of constant temperature annealing. Where N is defined by this parameter.

- **antimony, arsenic, boron, phosphorus, beryllium, magnesium, selenium, silicon, tin, germanium, zinc, carbon, generic**

The above switched booleans of gas types allow the user to specify the type of impurity that may be in the gas stream. The value of gas.conc applies to these gas types. Only one gas type may be specified per diffusion step.

- **dryo2 weto2 nitrogen ammonia argon antimony arsenic boron gallium phosphorus beryllium magnesium selenium isilicon tin germanium zinc carbon generic speedie indium ni.impurity al.impurity iiigeneric iiiigeneric ivgeneric vgeneric**

The above switched booleans of gas types allow the user to specify the type of gas present in the furnace during the diffusion step. These gas types are not affected by the gas.conc parameter. Only one gas type may be specified per diffusion step. There is currently no difference between nitrogen, argon, and ammonia.

- **gas.conc**

This allows the user to specify the gas concentration of an impurity during prepositions, in atoms per cubic centimeter.

- **pressure**

This is the partial pressure of the active species, in atmospheres. It defaults to 1 for both wet and dry oxidation. In many process flow description, the gas flow of different gas species are

defined instead of partial pressure. In such as case, one needs to use the following formula to convert gas flow rate into oxidant partial pressure:

$$\text{partial_oxidant_pressure} = \frac{\text{furnace_total_pressure} \times \text{O2_or_H2O_flow_rate}}{(\text{O2_or_H2O_flow_rate} + \text{other_flow_rate})}$$

- **gold.surf**

This floating point parameter specifies the gettered gold concentration value as a sink. It is used for the gettering model described on the gold command.

- **continue**

This specifies a continuing diffusion. Do not reset the total time to 0, do not reset the analytic oxide thickness to the default value of "initial" on the oxide command, and do not initialize the defects.

- **movie**

This character string parameter should be made up of SUPREM-IV commands. They will be executed at the beginning of each time step. It is typically used to produce plots of a variable as a function of time during the diffusion process. Any legal SUPREM-IV command can be specified in the string. The variable time is set by the movie routine. Consequently, time will be expanded as a macro for use in plot labels or print outs.

- **dump**

This parameter instructs the simulator to output files on after every n'th time step. The files will be structure files and will be readable with the structure statement. The names will be of the form s.time, where time is the current total time of the simulation.

- **flow.control**

flow.control=true/false allows the user to set up the flow amount of different gas species that are present in the furnace during diffusion. Its default value is false. If this parameter is set to true then the user can set up the flow rate of the following gas/stream species:

H2: the flow rate of hydrogen

O2: the flow rate of oxygen

steam: the flow rate of the steam

HCL: the flow rate of HCL

other: this paramter is used to set up the flow rate of other user defined species other than the above.

The default value of these parameters is 0. The partial pressure of each specy is calculated automatically using its flow rate and the total furnace pressure. The units of the flow rates are arbitrary since only the ratio is used to compute the partial pressure based on the total pressure. The program converts the flow rates into steam (if applicable) and enable simultaneous dry and wet oxidations. Please see the following list of examples.

7.18.3 Examples

```
diffuse time=30 temp=1000 boron gas.conc=1.e20
```

This statement specifies that a 1000°C, 30 minute, boron predeposition is to be performed.

```
diffuse time=30 temp=1000 dryo2
```

This statement instructs the simulator to grow oxide for 30 minutes at 1000°C in a dry oxygen ambient.

```
diffuse time=30 temp=1000 movie="select z=log10(bor); plot.1d x.v=1.0;"
```

This command instructs SUPREM-IV to do a 30 minute, 1000°C diffusion. Before each time step, the current boron concentration will be plotted along a horizontal slice 1.0 μm into the wafer. (See select , plot.1d).

```
diffuse time=30 temp=1100 movie="select z=doping; printf Time is: time yfn(0.0, 0.0)
sil@oxi(0.0)"
```

This command instructs SUPREM-IV to do a 30 minute, 1000°C diffusion. Before each time step, the total net active doping concentration is chosen as the plot z variable. The printf statement will then print the string "Time is:", the current time of the diffusion in seconds, the y value along a vertical slice at x = 0.0 where the doping is equal to zero (i.e. the junction), and the location of the silicon oxide interface in the vertical slice at x = 0.0.

```
diffuse time=30 temp=1000 flow.control=true O2=50 H2=50 other=200
```

This command instructs Csuprem to do a 30 minute, 1000°C oxidation where the flow rates of oxygen and H2 are defined. The program converts the above into O2 and H2O (steam) with proper partial pressures for simultaneous dry and wet oxidations.

7.18.4 Bugs

There can still be occasional convergence problems. If these occur when defects are being computed, they can usually be fixed by performing a complete LU decomposition instead of a partial. Use the " method full " command to achieve this.

The models implemented in the diffusion and oxidation are involved in ongoing research. Disagreements between simulation and experiment is possible. This can be due to the differences in processing conditions between the lab where the original experiment was performed and your lab, or due to model shortcomings.

7.19 Dislocation

7.19.1 Synopsis

dislocation

```
[rinit=<n> (default=0.0)] [ro=<n> (default=3.14e-8)]
[looploc=<n> (default=0.0)] [rho=<n> (default=0.0)]
[burger=<n> (default=3.14e-8)] [fdrdt=<n> (default=0.0)]
[loopgdt=<n> (default=0.001)] [lfactor=<n> (default=0.0)]
[maxsi=<n> (default=5.0e22)] [gamma=<n> (default=4.375e13)]
[omega=<n> (default=2.0e-23)] [mu=<n> (default=4.975e23)]
[nu=<n> (default=0.27864583333)]
```

7.19.2 Description

This command sets parameters for dislocation, please refer to example 18 for how to use this command in detail.

- **rinit**

Initial radius of dislocation loops (cm), radius must be positive.

- **ro**

Core radius of dislocation (cm), core radius must be positive.

- **looploc**

Center of dislocation loops (cm).

- **rho**

Density of dislocation loops (atoms/cm²), density must be positive.

- **burger**

Magnitude of Burger's vector of dislocation loop (cm), magnitude must be positive.

- **fdrdt**

Fitting constant used in calculating dr/dt (unitless).

- **loopgdt**

Parameter used in calculating maximum time step (unitless).

- **lfactor**

$dl/dt = lfactor * dr/dt$ (unitless), $\rho = 1/(2l\hat{2})$.

- **maxsi**

Maximum ration of Cl^*/Cl^* (unitless).

- **gamma**

Internal energy of stacking fault ($eV/cm\hat{2}$).

- **omega**

Volume per silicon atom ($cm\hat{3}$).

- **mu**

Shear modulus of silicon (eV/cm).

- **nu**

Poisson's ratio for silicon (unitless).

7.19.3 Examples

```
dislocation rinit=235e-8 looploc=1750e-8 rho=4.5e9 fdrdt=1/1.75
dislocation loopgdt=0.001 gamma=4.375e13 omega=2.0e-23 burger=3.14e-8
dislocation mu=4.975e23 nu=0.27864583333 ro=3.14e-8
```

These commands will turn on dislocatoin loop model and set parameters for it.

7.20 double_mesh

7.20.1 Synopsis

```
double_mesh
[segm=<n>(default=-1)
(silicon | oxide | oxynitr | nitride | poly | photores | alumin | gaas | AlGaAs | InP | AlInGaAs |
imaterial | iimaterial | iimaterial | ivmaterial | vmaterial)
x1=<n>(default=-9999. um) x2=<n>(default=9999. um)
y1=<n>(default=-9999. um) y2=<n>(default=9999. um)
x.direction=<logical> (default=false)
y.direction=<logical> (default=false)
```

7.20.2 Description

This command is used to double the mesh density within one or more rectangle regions. It can be used multiple times to further densify a specific area.

- **material**

It is used to identify the material in which the double mesh operation will take place.

- **x1, x2, y1, y2**

These are used to define a rectangle region within which double mesh operation is to be performed.

- **segm**

If segm does not equal to -1, then only specified segment will have double mesh.

- **x.direction**

To determine whether or not the x direction needs to have double mesh.

- **y.direction**

To determine whether or not the y direction needs to have double mesh.

7.20.3 Example

```
double_mesh silicon x1=0. x2=5 y1=4 y=6
```

The above command will double the mesh density in the rectangle silicon region defined by x1, y1, x2, y2.

7.21 eliminate

7.21.1 Synopsis

```
eliminate [ segm=<n>(default=-1) x.direction | y.direction xlo = <n> xhi = <n> ylo = <n> yhi = <n>
ntimes = <n> ]
```

7.21.2 Description

This command is used to manually control the mesh line densities in the initial stage of a simulation. Given a mesh line direction and an area, using this command once results in elimination of half of mesh lines (in equal intervals). Using it more than once results in further reduction of mesh lines in the same manner. This is a useful command to reduce excess mesh lines in areas of less physical activities or less variation of physical variables such as in the

substrate regions of a device.

- **x.direction | y.direction**

Indicate whether mesh lines in the x or y direction are to be eliminated.

- **xlo, xhi, ylo, yhi**

These are the low and high coordinates of an area within which mesh elimination is to happen. Please note that this area should be within the region as defined by the **region** command.

- **ntimes**

It is the number of "half-mesh" action to be performed.

- **segm**

If segm does not equal to -1, then it will only be applied to the specified segment

7.21.3 Examples

`eliminate y.dir xlo=0. xhi=15 ylo=3. yhi=10. ntimes=2`

This command performs "half-mesh" action for mesh lines in the y-direction twice. If there were 16 lines, this command would reduce them to 4.

7.22 etch

7.22.1 Synopsis

```
etch [segm=<n>(default=-1) spacing=<n>(default=-999.0)
silicon | oxide | oxynitr | nitride | poly | photores | alumin | gaas | AlGaAs | InP | AlInGaAs |
imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial]
[ left | right | start | continue | done | dry | avoidmask | physical | all ]
[ x=<n> ] [ y=<n> ] [ thick=<n> ] [ depth=<n> ]
[ p1.x=<n> ] [ p1.y=<n> ] [ p2.x=<n> ] [ p2.y=<n> ]
[ file=<string> ] [ follow.surface=<logical>]
[ shift.x=<n> (default=0.0)] [ shift.y=<n> (default=0.0)]
[ ref.position=<string> ] [tag_etch_geometry=<string>]
[ horizontal_shift<n> (default=0.0)]
```

7.22.2 Description

This statement provides an etching capability. Material is etched from the structure, and the

underlying regions are marked as the $j^{\circ}\text{exposed}j_{\pm}$ surface. The etch shape can be described in several different ways. The user must specify the final shape of the region to be etched, there is no capability at present to simulate the etch region shape.

- **segm**

If segm does not equal to -1, then only specified segment will be etched

- **spacing**

If spacing is specified, this value will be used for the points after etching.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, alumin, AlGaAs, InP, AlInGaAs, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

The material to be etched. If a material is specified, only that material is etched even if other materials lie within the etch region. If no material is specified, all materials in the etch region are removed.

- **left, right**

These provide the user with a quick means of doing trapezoidal shaped etches. The etch region is to the left / right of the line specified by the coordinates given in p1.x, p1.y and p2.x, p2.y.

- **start, continue, done**

This allows the user to specify an arbitrarily complex region to be etched. Several lines can be combined to specify the several points that make up the region. See the examples.

- **dry**

This parameter indicates that the etch shape should be a replica of the current surface, but straight down $j^{\circ}\text{thickness}j_{\pm}$ microns.

- **all**

All of the specified material is removed.

- **x, y**

These parameters allow the user to specify point in the start / continue / done mode of etch region specification.

- **p1.x, p1.y, p2.x, p2.y**

These parameters allow the specification of a line for left / right etching.

- **thick**

This parameter allows specification of a thickness for the dry etch type.

- **file**

This string parameter specifies the name of a file that contains a string that contains the coordinates associated with the etched surface. This file could be generated using the

etch_geometry.exe program.

- **follow.surface**

This would cause imported etching profile (see **file above**) be adjusted according to (or follow) the original surface shape. This is useful for RIE etching of a surface which is not flat. The calculated RIE profile would be modified by the original surface. This is said to follow the original surface.

- **shift.x shift.y**

If the etch profile comes from the file, user can use this command to shift the entire profile in x and/or y direction by setting values to shift.x and/or shift.y

- **ref.position**

This command is used to set the location of the reference point. the reference point is defined by interface_label. The x y position obtained from the file combined with shift.x and shift.y and the position of the reference point will determine the new etch profile.

- **avoidmask**

This is to use avoidmask method to etch. Please refer to mask command

- **depth**

This parameter allows specification of a depth for the avoidmask type.

- **physical**

This is to use physical method to etch. It's necessary to specify the etch speed in different materials, (e.g. the etch speed of silicon will be specified by the speed parameter r.silicon)

- **horizontal_shift**

If dry etch is chosen, the command horizontal_shift will further change the profile after etch dryo2 command. User can alter the value for horizontal_shift to change the profile in x direction according to certain ratio.

- **tag_etch_geometry**

If the etch profile is created by etch_geometry.exe, the profile will be selected by tag_etch_geometry from etch_geometry command. For detailed information about etch_geometry, please refer to set_etch__geometry command.

7.22.3 Examples

etch nitride left p1.x = 0.5 p1.y=0.0 p2.x=0.5 p2.y=-1.0

This command etches all the nitride left of a vertical line at 0.5.

etch oxide start x=0.0 y=0.0

```
etch continue x=1.0 y=0.0  
etch continue x=1.0 y=1.0  
etch done x=0.0 y=1.0
```

This etches the oxide in the square defined at (0,0), (1,0), (1,1), and (1,0).

```
etch dry thick=0.1
```

This etch will find the exposed surface, lower it straight down 0.1 um and this line will be the new surface.

7.23 export

7.23.1 Synopsis

```
export  
xpsize=<n> outfile=<string>  
[ triangle.based=<logical> ]  
[ dbgapstr=<logical> ]  
[ mat.priority=<logical> ]  
[ repair.mesh=<n> (default=false)]
```

7.23.2 Description

This statement is to instruct the program to export the mesh and doping profile to a form readable by Crosslight's APSYS device simulation package.

- **xpsize**
Extra points will be added if a point is on the material boundary since each point has one single material property in Apsys. The material boundary will form a thin gap after extra points are added. The parameter specifies the space of the gap.

- **outfile**
It is the output file name to which the data are dumped.

- **triangle.based**
There are two ways to add extra points for Apsys. One is based on triangle and the other is by material skeleton. The default is false. True is mainly for back compatibility.

- **mat.priority**
It only works while triangle.based=false. It defines the material property of extra points. While

mat.priority=true extra points will be assigned to materials other than silicon.

- **dbgapstr**

It only works while triangle.based=false in 2D. While dbgapstr=true gap.str file will be generated for preview.

- **repair.mesh**

Csumpre will check whether it is necessary to repair mesh before export, so that it can make sure the mesh is correct

7.23.3 Examples

```
export outfile=mydevice.aps xpsize=0.0001
```

This statement exports to a file named mydevice.aps

7.24 extend

7.24.1 Synopsis

extend

[left=<logical>] [right=<logical>]

[length=<n>] [spacing=<n>]

7.24.2 Description

This command is used to extend the mesh to both left and right direction. During ion implant, if certain implant angle is required, the ion implantation will be partially shielded at the left and right edge of the device. This command is used to extend the mesh to inclose the device edges, so that better accuracy will be obtained. After implant, the extra mesh in the x direction will be shrunk to the original condition.

- **left**

Extend to the left

- **right**

Extend to the right

- **length**

The length that extend to left or right.

- **sapcing**

Set the spacing for the extended mesh. Refer to line command about how to use spacing

7.24.3 Examples

`extend right length=1.0 spacing=0.1`

This command will move the mesh on the right hand side of the device horizontally by 1um.

7.25 fill_hole

7.25.1 Synopsis

`fill_hole`

`[segm=<n> (default=-1)]`

`(silicon | oxide | oxynitride | nitride | poly | photoresist | aluminum | gaas | AlGaAs | InP | AlInGaAs | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)`

`(none | arsenic | antimony | boron | gallium | phosphorus | beryllium | magnesium | selenium | isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric | vgeneric | indium | ni.impurity | al.impurity)`

`[concentration=<n> (default=1e+10)]`

`[file=<string>]`

`[x.ref=<n>] [y.ref=<n>]`

7.25.2 Description

This command is used to fill up a hole with certain material

- **segm**

In 3D simulatoin, specify which segment to deal with.

- **silicon, oxide, oxynitride, nitride, poly, photoresist, aluminum, gaas, AlGaAs, InP, AlInGaAs, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

Set the material type for the hole filling

- **none, arsenic, antimony, boron, gallium, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic, iigeneric, iiigeneric, ivgeneric, vgeneric, indium, ni.impurity, al.impurity**

Set impurity in the filled area. You don't have to set it.

- **concentration**

if impurity is set, this variable will set the doping concentration for the new impurity.

- **x.ref**

The x coordinate for the hole, it should be within the hole, and it is used to find the hole.

- **y.ref**

The y coordinate for the hole, it should be within the hole, and it is used to find the hole.

- **resistivity**

Parameter resistivity has been used in the following commands to replace the doping concentration:

region
deposit
fill_hole

This is based on known relation between doping and resistivity for some well known materials such as silicon and GaAs. For details, see example:

<http://inside.mines.edu/~sagarwal/phgn435/images/resistivity.jpg>

We plot the resistivity versus doping for n-type and p-type doping for silicon in the figure above. This has been implemented for phosphorous and boron in silicon.set doping concentrations.

7.25.3 Examples

`fill_hole oxide x.ref=1.0 y.ref=1.0 boron conc=1e+14`

This command is used to fill the hole located at x=1.0,y=1.0 with oxide. The initial boron doping concentration within the oxide filled area is 1e+14

7.26 flip_y

7.26.1 Synopsis

`flip_y`

7.26.2 Description

This command is used to flip the whole device structure in y-direction and do a coordinate transform of $y \rightarrow -y$. This is useful if one wish to process on both the front and the back surface. This command has no related parameters.

7.26.3 Examples

flip_y

This command would flip the structure up-side-down so that process model can be performed on the backside.

7.27 Generic

7.27.1 Synopsis

```
generic ( silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial
| ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ]
[ Dim.0=<n> ] [ Dim.E=<n> ]
[ Dimm.0=<n> ] [ Dimm.E=<n> ]
[ Dimmm.0=<n> ] [ Dimmm.E=<n> ]
[ Dip.0=<n> ] [ Dip.E=<n> ]
[ Dipp.0=<n> ] [ Dipp.E=<n> ]
[ Dippp.0=<n> ] [ Dippp.E=<n> ]
[ Fi = <n> ]
[ implanted ] [ grown.in ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ] [ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly
| gaas | /imaterial | /iimaterial | /iimaterial | /ivmaterial | /vmaterial) ]
[ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]
```

7.27.2 Description

This statement allows the user to specify values for coefficients of a generic impurity's diffusion and segregation. The diffusion equation for the generic impurity is:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{p}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{p}{n_i} \right) \right] \quad 7.30$$

where C_T and C_A are the total chemical and active concentrations of generic, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 + D_{MMM} \left(\frac{n}{n_i} \right)^3 + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 + D_{PPP} \left(\frac{p}{n_i} \right)^3 \right] \quad 7.31$$

$$D_I = F_i \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 + D_{MMM} \left(\frac{n}{n_i} \right)^3 + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 + D_{PPP} \left(\frac{p}{n_i} \right)^3 \right] \quad 7.32$$

D_X , D_M , D_{MM} , D_{MMM} , D_P , D_{PP} , and D_{PPP} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12} T_r} \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.33$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the generic diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.0 cm²/sec in silicon, and Dix.E defaults to 0.0 eV. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the generic diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 0.0 cm²/sec in silicon, and Dim.E defaults to 0.0 eV. D_M is calculated using a standard Arrhenius relationship.

- **Dimm.0, Dimm.E**

These floating point parameters allow the specification of D_{MM} , the generic impurity's diffusivity with doubly positive defects. Dimm.0 is the preexponential constant and Dimm.E is the activation energy. Dimm.0 defaults to 0.0 cm²/sec and Dimm.E defaults to 0.0 eV. D_{MM} is

calculated using a standard Arrhenius relationship.

- **Dimmm.0, Dimmm.E**

These floating point parameters allow the specification of D_{MMM} , the generic impurity's diffusivity with triply positive defects. Dimmm.0 is the pre-exponential constant and Dimmm.E is the activation energy. Dimmm.0 defaults to 0.0 cm²/sec and Dimmm.E defaults to 0.0 eV. D_{MMM} is calculated using a standard Arrhenius relationship.

- **Dip.0, Dip.E**

These floating point parameters allow the specification of D_p , the generic impurity's diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to 0.0 cm²/sec and Dip.E defaults to 0.0 eV. D_p is calculated using a standard Arrhenius relationship.

- **Dipp.0, Dipp.E**

These floating point parameters allow the specification of D_{pp} , the generic impurity's diffusivity with doubly positive defects. Dipp.0 is the pre-exponential constant and Dipp.E is the activation energy. Dipp.0 defaults to 0.0 cm²/sec and Dipp.E defaults to 0.0 eV. D_{pp} is calculated using a standard Arrhenius relationship.

- **Dippp.0, Dippp.E**

These floating point parameters allow the specification of D_{ppp} , the generic impurity's diffusivity with triply positive defects. Dippp.0 is the pre-exponential constant and Dippp.E is the activation energy. Dippp.0 defaults to 0.0 cm²/sec and Dippp.E defaults to 0.0 eV. D_{ppp} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether generic diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.0.

- **implanted, grown.in**

Specifies whether the D_{ix} , D_{im} , D_{ip} , or F_i coefficients apply to implanted or grown-in generic. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm ⁻³)
700	1.e19

1250	1.e19
------	-------

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific.

7.27.3 Examples

generic gaas implanted Dip.0=2.2e-6 Dip.E=1.76

This command changes the positive defect diffusivity for an implanted generic impurity in gallium arsenide.

generic gaas /nitride Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7

This command will change the segregation parameters at the gallium arsenide "C silicon nitride interface. The gallium arsenide concentration will be half the nitride concentration in equilibrium at 1100 degree C.

7.28 germanium

7.28.1 Synopsis

germanium

(silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)

[Dix.0=<n>] [Dix.E=<n>] [Dim.0=<n>] [Dim.E=<n>]

[Dimm.0=<n>] [Dimm.E=<n>] [Fi = <n>]
 [implanted] [grown.in]
 [ss.clear] [ss.temp=<n>] [ss.conc=<n>]
 [(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas /imaterial | /iimaterial |
 /iiimaterial | /ivmaterial | /vmaterial)]
 [Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]
 [(donor | acceptor)]

7.28.2 Description

This statement allows the user to specify values for coefficients of germanium diffusion and segregation. The diffusion equation for germanium is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.34$$

where C_T and C_A are the total chemical and active concentrations of germanium, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.35$$

$$D_I = F_i \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.36$$

D_X , D_M and D_{MM} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.37$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the germanium diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0. cm²/sec in silicon, and Dix.E defaults to 0.0 eV in silicon. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0,Dim.E**

These floating point parameters allow the specification of the germanium diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 0.0 cm²/sec in silicon, and Dim.E defaults to 0.0 eV in silicon. D_M is calculated using a standard Arrhenius relationship.

- **Dimm.0,Dimm.E**

These floating point parameters allow the specification of the germanium diffusing with doubly negative defects. Dimm.0 is the pre-exponential constant and Dimm.E is the activation energy. Dimm.0 defaults to $1.1 \times 10^{-3} \text{ cm}^2/\text{sec}$ in gallium arsenide, and Dimm.E defaults to 2.9 eV in gallium arsenide [1,2]. D_{MM} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether germanium diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.0.

- **implanted, grown.in**

Specifies whether the Dix , Dim , Dimm, or Fi coefficients apply to implanted or grown-in germanium. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default germanium is a donor.

7.28.3 Examples

`germanium gaas implanted Dimm.0=2.1e-3 Dimm.E=3.1`

This command changes the doubly negative defect diffusivity for implanted germanium in silicon.

`germanium gaas /nitride Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the gallium arsenide "C silicon nitride interface. The gallium arsenide concentration will be 30.0 times the nitride concentration in equilibrium.

7.28.4 References

1. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1990.
2. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1991.
3. E.L. Allen, J.J. Murray, M.D. Deal, J.D. Plummer, K.S.Jones, and W.S. Robert, Diffusion of Ion-Implanted Group IV Dopants in GaAs, J. Electrochem. Soc., 138, p. 3440, 1991.

7.29 gold

7.29.1 Synopsis

gold

(silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial |
ivmaterial | vmaterial)

[K.O=<n>] [K.E=<n>]

[implanted] [grown.in]

[(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iimaterial | /ivmaterial | /vmaterial)]

[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]

7.29.2 Description

This statement allows the user to specify values for coefficients of gold diffusion and segregation.

Cesium is treated as a charge neutral impurity which diffuses according to the Fick's law:

$$\frac{\partial C_T}{\partial t} = -\nabla \cdot J_T \quad 7.38$$

$$J_T = -D\nabla C_T \quad 7.39$$

where C_T is the total chemical concentrations of gold, and D is diffusivity expressed as a function of concentration:

$$D = K/C_T^2 \quad 7.40$$

The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.41$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively. M_{12} is related to ratio of solubility of the two materials and T_r is the surface velocity.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **K.O, K.E**

These floating point parameters allow the specification of D , the gold diffusivity as a neutral defect. $K.O$ is the pre-exponential constant and $K.E$ is the activation energy. By default, $K.O = 0.037 \text{ (cm}^{-4}/\text{sec)}$ and $D.E = 3.46 \text{ (eV)}$.

- **implanted, grown.in**

Specifies whether the coefficients apply to implanted or grown-in gold. If neither is specified then a specified parameter applies to both.

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship. By default, Trn.0 = 1.0 (cm/s), Trn.E = 0.0 (eV)

- **mobile**

May be used to enable or disable the movement of the impurity.

7.29.3 Examples

gold silicon implanted K.0=1.e38

This command changes the diffusivity for implanted gold in silicon.

7.30 iigeneric

This command is the same as **generic** except it is for the second generic impurity. Please see the command for **generic** for more details.

7.31 iiigeneric

This command is the same as **generic** except it is for the third generic impurity. Please see the command for **generic** for more details.

7.32 implant

7.32.1 Synopsis

```

implant
(pearson gauss dualpearson (default=pearson))
( antimony | arsenic | boron | bf2 | cesium | phosphorus | beryllium | magnesium | selenium
| isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric |
vgeneric | indium | ni.impurity | al.impurity)
dose=<n> energy=<n>
[ rotation=<n> (default=0)] [ angle=<n> (default=0)]
[ damage ] [ max.damage=<n> ]
[ range=<n> (default=-1)] [ std.dev=<n> (default=-1)]
[ gamma=<n> (default=-1)] [ kurtosis=<n> (default=-1)]
[ lateral=<n> (default=-1)] [ fsims=<string> ] [tab=<string> ]
[ monte_carlo=<logical> ]

```

7.32.2 Description

This statement is used to simulate ion implantation. There are three different types of analytic models available. The first implements a simple Gaussian distribution. The second uses the moments data file to compute a Pearson-IV distribution. The third is dual pearson distribution used in low energy. These models are only as good as the data in the file (sup4gs.imp). However the sims data can be imported as well. The program will automatically choose a mothod by the data format in sup4gs.imp. The file data can be overridden by directly specifying the appropriate values on the implant command line.

- **antimony, arsenic, boron, bf2, cesium, phosphorus, beryllium , magnesium , selenium , isilicon , tin , germanium , zinc , carbon, generic, iigeneric, iiigeneric, ivgeneric, vgeneric, indium, ni.impurity, al.impurity**

These parameters specify the impurity to be implanted. Default data for the range statistics exists only for antimony, arsenic, boron, BF2, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, and carbon.

- **dose**

This parameter allows the user to specify the dose of the implant. The units are in atoms/cm2.

- **energy**

This parameter specifies the implant energy in KeV.

- **damage**

This parameter indicates that a computation of the damage due to the implant should be

performed. The data is from Hobler and Selberherr [1] and exist only for antimony, arsenic, boron, and phosphorus.

- **max.damage**

The maximum amount of damage to the crystal that makes sense to calculate. Clearly damage above the crystal density is absurd.

- **range, std.dev, gamma, kurtosis, lateral**

These parameters allow the user to override the table values. The full set of data has to be given. It is not possible, for instance, to override only the range parameter. The gamma and kurtosis variables only have meaning for the Pearson-IV distribution.

- **angle**

This parameter allows the user to specify the angle normal to the substrate that the impurity was implanted at. This is also called tilt angle.

- **rotation**

This is the angle of rotation in degrees relative to the xy plane.

- **tab**

It allows user to use their own implant table. This command is used to load the user-defined imp table. Use `tab=mytab.txt` to include the implant file. Within `mytab.txt`, define material number, dopant number, sims flag, the sims file or the Gaussian or Pearson moments, for all materials. Example: `implant tab=mytab.txt boron energy=40 dose=1e13`. Please refer to `sup4gs.imp` for more information about table format.

- **fsims**

This command allows user to apply SIMS data to implant. Use this command to input the corresponding SIMS data used by the implant.

- **monte_carlo** If this variable is turned on, the parameters above will be sent to Monte_carlo simulation. The output of Monte_carlo simulation will be used as implant SIMS data.

7.32.3 Examples

`implant phosph dose=1e14 energy=100`

This statement specifies that a 100KeV implant of phosphorus was done with a dose of $1 \times 10^{14} \text{cm}^{-2}$.

`implant phosph dose=1e14 energy=100 damage`

This statement specifies that a 100KeV implant of phosphorus was done with a dose of $1 \times 10^{14} \text{cm}^{-2}$. The damage from this implant is calculated and stored as point defects.

```
implant arsenic dose=1e14 energy=4
```

The program checks into the implant table and finds dual Pearson moments to be available for this low energy. Then, dual Pearson model would be used.

```
implant indium dose=1e14 energy=8
```

The program checks into the implant table and finds sims data be available for this energy. Then, sims data will be used directly.

7.33 include

7.33.1 Synopsis

```
include [ file=<string> ]
```

7.33.2 Description

This command is used to include an input file to be part of the current input file.

- **file**

It is the file containing input commands to be included into the current input file.

7.33.3 Examples

```
include file=layer8.msk
```

This command would cause all mask commands within layer8.msk to be included into the current line position of the input file.

7.34 indium

7.34.1 Synopsis

```
indium  
( silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
```

```

ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ] [ Dip.0=<n> ] [ Dip.E=<n> ]
[ Fi = <n> ]
[ implanted ] [ grown.in ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ]
[ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial ) ] [ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]

```

7.34.2 Description

This statement allows the user to specify values for coefficients of indium diffusion and segregation. The diffusion equation for indium is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{p}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{p}{n_i} \right) \right] \quad 7.42$$

where C_T and C_A are the total chemical and active concentrations of indium, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} \right) \quad 7.43$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} \right) \quad 7.44$$

D_X , and D_P are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.45$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of DX , the indium diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.28 cm²/sec in silicon, and Dix.E defaults to 3.46 eV in silicon [3]. DX is calculated using a standard Arrhenius relationship.

- **Dip.0,Dip.E**

These floating point parameters allow the specification of DP, the indium diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to 0.23 cm²/sec in silicon, and Dip.E defaults to 3.46 eV in silicon [3]. DP is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether indium diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.8[4].

- **implanted, grown.in**

Specifies whether the Dix , Dip , or Fi coefficients apply to implanted or grown-in indium. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm ⁻³)
300	1.0000e+19
600.0	1.8000e+19
700.0	2.5000e+19
800.0	3.4499e+19
825.0	4.1291e+19
850.0	4.9027e+19
875.0	5.7777e+19
900.0	6.7615e+19
925.0	7.8610e+19
950.0	9.0832e+19
975.0	1.0435e+20
1000.	1.1922e+20
1025.	1.3552e+20

1050.	1.5331e+20
1075.	1.7263e+20
1100.	1.9356e+20
1125.	2.1613e+20
1150.	2.4041e+20
1175.	2.6643e+20
1200.	2.9423e+20
1225.	3.2387e+20
1250.	3.5536e+20
1275.	3.8876e+20

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default indium is an acceptor.

7.34.3 Examples

indium silicon Dix.0=0.28 Dix.E=3.46

This command changes the neutral defect diffusivity in silicon.

indium silicon /oxide Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7

This command will change the segregation parameters at the silicon "C silicon dioxide interface. The silicon concentration will be half the oxide concentration in equilibrium at 1100 C.

7.34.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/ Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. D.Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.
3. G.P. Barbuscia, G. Chin, R.W. Dutton, T. Alvarez and L. Arledge, "Modeling of Polysilicon Dopant Diffusion for Shallow Junction Bipolar Technology," International Electron Devices Meeting, San Francisco, p. 757, 1984.
4. P.A. Packan and J.D. Plummer, "Temperature and Time Dependence of B and P Diffusion in Si During Surface Oxidation," J. Appl. Phys., 68(8), 1990

7.35 initialize

7.35.1 Synopsis

```
initialize
[ infile=<string> ]
[ ( antimony | arsenic | boron | phosphorus | gold | gallium | beryllium | magnesium |
selenium | silicon | tin | germanium | zinc | carbon | generic | iiigeneric | ivgeneric
| vgeneric) ]
[ conc=<n> ]
[ orientation=<n> ]
[ line.data ] [ scale ] [ flip.y ]
[ interval.r=<n> (default=1.5)] ]
```

7.35.2 Description

The initialize statement sets up the mesh from either a rectangular specification or from a previous structure file in 3D. The statement also initializes the background doping concentration.

- **infile**

The infile parameter selects the file name for reading in 3D. The file must contain a previously saved structure, see the structure command. If no filename is specified, the program will finish generating a rectangular mesh.

- **antimony, arsenic, beryllium, boron, carbon, generic, germanium, gold, magnesium,**

phosphorus, selenium, isilicon, tin, zinc, iigeneric, iiigeneric, ivgeneric, vgeneric

These parameters specify the type of impurity that forms the background doping.

- **conc**

This parameter allows the user to specify the background concentration in cm^{-3} .

- **orientation**

This parameter allows the user to specify the substrate orientation. Only 100, 110 and 111 are recognized.

- **line.data**

This boolean parameter indicates that the locations of mesh lines should be listed.

- **scale**

Parameter to scale the size of an incoming mesh. The default is 1.0.

- **flip.y**

This boolean parameter indicates if the mesh should be flipped around the x axis. It is sometimes useful for reading grids prepared with tri or other grid generation programs.

- **interval.r**

The interval ratio between two points, normally default value is used, since the initial mesh informtoin has been defined in the "line" command.

7.35.3 Examples

`init infile=foo`

This command reads in a previously saved structure in file foo.

`initialize conc=1e15 boron`

This command finishes a rectangular mesh and sets up with boron doping of $1 \times 10^{15} cm^{-2}$.

7.36 interface_label

7.36.1 Synopsis

`interface_label`

`segm=<n> label=<string> xref=<n>`

(silicon| oxide| oxynitride| nitride| poly| gas| gaas| imaterial| iimaterial| iimaterial|
ivmaterial| vmaterial) (/silicon| /oxide| /oxynitride| /nitride| /poly| /gas| /gaas| /imaterial|

/iimaterial| /iiimaterial| /ivmaterial| /vmaterial)

7.36.2 Description

This command is used to define and label an interface point so that later command may refer to such an interface position. Given an x-position, the program searches the interface between two specific materials and find the y-position. The point found is then given a label for later reference.

- **segm**

It is the segment number in a 3D simulation. Not to be used in 2D.

- **label**

It is a string label for the convenience of later reference.

- **xref**

It is the x-position at which the interface point is to be searched.

- **(silicon, oxide, ...**

It is one of the material of the interface.

- **(/silicon, /oxide, ...**

It is the other material of the interface.

7.36.3 Examples

```
interface_label label=mask_edge xref=0.5 silicon /oxide
```

This command finds an interface point between silicon and oxide at x=0.5 and labels it ``mask_edge."

7.37 interstitial

7.37.1 Synopsis

interstitial

(silicon | oxide | poly | oxynitr | nitride | gas | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)

[D.O=<n>] [D.E=<n>]

[Kr.O=<n>] [Kr.E=<n>]


```

[ Cstar.0=<n> ] [ Cstar.E=<n> ]
[ ktrap.0=<n> ] [ ktrap.E=<n> ]
[ neu.0=<n> ] [ neu.E=<n> ] [ neg.0=<n> ] [ neg.E=<n> ]
[ dneg.0=<n> ] [ dneg.E=<n> ] [ tneg.0=<n> ] [ tneg.E=<n> ]
[ pos.0=<n> ] [ pos.E=<n> ] [ dpos.0=<n> ] [ dpos.E=<n> ]
[ tpos.0=<n> ] [ tpos.E=<n> ]
[ ( /silicon | /oxide | /poly | /oxynitr | /nitride | /gas | /gaas | generic | iigeneric | iiigeneric |
ivgeneric | vgeneric ) ]
[ time.inj ] [ growth.inj ] [ recomb ] [ segregation ]
[ Ksurf.0=<n> ] [ Ksurf.E=<n> ]
[ Krat.0=<n> ] [ Krat.E=<n> ]
[ Kpow.0=<n> ] [ Kpow.E=<n> ]
[ vmole=<n> ] [ theta.0=<n> ] [ theta.E=<n> ]
[ Gpow.0=<n> ] [ Gpow.E=<n> ]
[ A.0=<n> ] [ A.E=<n> ] [ t0.0=<n> ] [ t0.E=<n> ]
[ Tpow.0=<n> ] [ Tpow.E=<n> ]
[ rec.str=<s> ] [ inj.str=<s> ]
[ Seg.E=<n> ] [ Seg.0=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]

```

7.37.2 Description

This statement allows the user to specify values for coefficients of interstitial continuity equation. The statement allows coefficients to be specified for each of the materials. Of course, the meaning of a lattice interstitial in an amorphous region is a philosophical topic. SUPREM-IV has default values only for silicon, gallium arsenide, and the interfaces with silicon. Polysilicon has not been characterized as extensively, and its parameters default to those for silicon. The interstitials obey a complex diffusion equation which can be written

$$\frac{\partial(C_I - C_{ET})}{\partial t} = \nabla(-J_I - \sum_{imp} J_{AI}) - R \quad 7.46$$

where C_I is the interstitial concentration, C_{ET} is the number of empty interstitial traps, J_I refers to the interstitial flux, J_{AI} refers to the flux of impurity A diffusing with interstitials, and R is all sources of bulk recombination of interstitials. In the two.dim model (see method) the fluxes from the dopants, J_{AI} , are ignored. Only in the full.cpl model are they computed and included in the interstitial equation. This model represents the diffusion of all interstitials, paired or unpaired, and can be derived from assuming thermal equilibrium between the species and that the simple pairing reactions are dominant [1].

The trap reaction was described by Griffin [2]. This model explains some of the wide variety of diffusion coefficients extracted from several different experimental conditions. The trap equation is:

$$\frac{\partial C_{ET}}{\partial t} = -K_T \left[C_{ET} C_I - \frac{e^*}{1-e^*} C_I^* (C_T - C_{ET}) \right] \quad 7.47$$

where C_T is the total trap concentration, K_T is the trap reaction coefficient, e^* is the equilibrium trap occupancy ratio. (see trap for a more complete discussion). Instead of the reaction, the time derivative is used in the interstitial equation because it has better properties in the numerical calculation. The pair fluxes are the interstitial portions of the flux as described in each of the models for the impurities. (see antimony, arsenic, boron, and phosphorus) The interstitial flux can be written [1]:

$$J_I = D_{IC} \nabla \frac{C_I}{C_I^*} \quad 7.48$$

It is important to note that the equilibrium concentration, C_I^* is a function of Fermi level [3]. This flux accounts correctly for the effect of an electric field on the charged portion of the defect concentration. The bulk recombination is simple interaction between interstitials and vacancies. This can be expressed:

$$R = K_R (C_{IC} V - C_I^* C_V : ifexpand(""): ifexpand(" \quad 7.49$$

where K_R is the bulk recombination coefficient, and C_V and C_V^* are the vacancy and vacancy equilibrium concentration, respectively. The defects obey a flux balance boundary condition, as described by Hu [4]:

$$J_I \hat{n} + K_I (C_I - C_I^*) = g \quad 7.50$$

where $\hat{n}$ is the surface normal, K_I is the surface recombination constant, and g is the generation, if any, at the surface. This expression has been used with success on several different surface types.

- **silicon, oxide, poly, oxynitr, nitride, gaas, gas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial,**

These parameters allow the specification of the material for which the parameters apply.

- **D.0, D.E**

These floating point parameters are used to specify the diffusion coefficient of the interstitials. The units are in cm^2/sec . The default values are $1 \times 10^6 \text{cm}^2/\text{sec}$ and 3.22 eV [5] for Si and $1 \times 10^{-12} \text{cm}^2/\text{sec}$ and 0.0 eV for GaAs.

- **Kr.0, Kr.E**

These floating point parameters allow the specification of the bulk recombination rate in cm^3/sec . The default is $1.4 \text{cm}^3/\text{sec}$ and 3.99 eV [6] for Si and $1 \times 10^{14} \text{cm}^3/\text{sec}$ and 0.0 eV for GaAs.

- **Cstar.0, Cstar.E**

These parameters allow the specification of the total equilibrium concentration of interstitials

in intrinsically doped conditions. The default values are $3.1 \times 10^{19} \text{cm}^{-3}$ and 1.58 eV [5] for Si and $1 \times 10^{14} \text{cm}^{-3}$ and 0.0 eV for GaAs.

- **ktrap.0, ktrap.E**

These values allow the specification of the trap reaction rate. At present, it is very difficult to extract exact values for these parameters. The default values assume that the trap reaction is limited by the interstitial concentration. A diffusion limited model has been assumed [6] and results in defaults of $1.1 \times 10^{-2} \text{cm}^3/\text{sec}$ and 3.22 eV for Si, the same values are used for GaAs.

- **neu.0, neu.E, neg.0, neg.E, dneg.0, dneg.E, tneg.0, tneg.E, pos.0, pos.E, dpos.0, dpos.E, tpos.0, tpos.E**

These values allow the user to specify the relative concentration of interstitials in the various charge states (neutral, negative, double negative, triple negative, positive, double positive, and triple positive) under intrinsic doping conditions. For example, in intrinsic conditions, the concentration of doubly negative interstitials is given by:

$$C_{I^*} = \frac{C_{stardneg}}{neu+neg+dneg+tneg+pos+dpos+tpos} \quad 7.51$$

where C_{Star} is the total equilibrium concentration in intrinsic doping conditions. In extrinsic conditions, the total equilibrium concentration is given by [3]:

$$C_{I^*} = C_{star} \frac{neu+neg\left(\frac{n}{n_i}\right)+dneg\left(\frac{n}{n_i}\right)^2+tneg\left(\frac{n}{n_i}\right)^3+pos\left(\frac{p}{n_i}\right)+dpos\left(\frac{p}{n_i}\right)^2+tpos\left(\frac{p}{n_i}\right)^3}{neu+neg+dneg+tneg+pos+dpos+tpos} \quad 7.52$$

There is very little data for the interstitial charge states; it is matter of ongoing research. The default values for Si are chosen based on an analysis of Giles [7]. See Table 4 in the section Adding SUPREM 3.5s GaAs Models and Parameters to SUPREM-IV for GaAs values.

- **/silicon, /oxide, /poly, /oxynitr, /nitride, /gaas, /gas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters allow the specification of the interface for which the boundary condition parameters apply. For example, parameters about the oxide silicon effect on interstitials should be specified by listing silicon / oxide. Only one of these can be specified at a time.

- **time.inj, growth.inj, recomb**

These parameters specify the type of reactions occurring at the specified interface. The time.inj parameter means that a time dependent injection model should be chosen. The growth.inj parameter ties the injection to the interface growth velocity. The recomb parameter indicates a finite surface recombination velocity.

- **Ksurf.0, Ksurf.E, Krat.0, Krat.E, Kpow.0, Kpow.E**

These parameters allow the specification of the surface recombination velocity. The formula used in computing the actual recombination velocity is:

$$K_I = K_{surf} \left[K_{rat} \left(\frac{V_{ox}}{B/A} \right)^{K_{pow}} + 1 \right] \quad 7.53$$

where V_{ox} is the interface velocity, and B/A is the Deal Grove oxide growth coefficient. (see oxide). The full model applies only to oxide, since it is the only interface that can grow in SUPREM-IV. This formulation allows an inert interface to have different values than a growing interface. For inert interfaces, (gas, oxynitride, nitride, poly) KSurf is the only important parameter.

- **vmole, theta.0, theta.E, Gpow.0, Gpow.E**

These parameters allow the specification of generation that is dependent on the growth rate of the interface. The formula is:

$$g = \theta V_{mole} V_{ox} \left(\frac{V_{ox}}{B/A} \right)^{G_{pow}} \quad 7.54$$

Theta, θ , is the fraction of silicon atoms consumed during growth that are reinjected as interstitials. V_{mole} is the lattice density of the consumed material. Once again, this model only makes sense for oxide since it is currently the only material which can have a growth velocity.

- **A.0, A.E, t0.0, t0.E, Tpow.0, Tpow.E**

These parameters allow an injection model with a fairly flexible time dependency. The equation for the injection is:

$$g = A(t + t_0)^{T_{pow}} \quad 7.55$$

where t is the time in the diffusion in seconds. This can be used to represent the injection of interstitials from an oxynitride layer.

- **rec.str, inj.str**

These two string parameters are useful for experimenting with new models for injection or recombination at interfaces. Three macros are defined for use, t the time in seconds and x and y the coordinates. If these are specified, they are used in place of any other model. For example,

`interst silicon /oxide inj.str = (10.0e4 * exp( t / 10.0 ))`

describes an injection at the silicon oxide interface that decays exponentially in time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. The segregation constant follows an Arrhenius relationship. This is for use in help supporting future model development, and is currently not in use.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. The transfer coefficient follows an Arrhenius relationship. This is for use in help supporting future model development, and is currently not in use.

7.37.3 Examples

```
inter silicon Di.0=5.0e-7 D.E=0.0 Cstar.0=1.0e13 Cstar.E=0.0
```

This statement specifies the silicon diffusion and equilibrium values for interstitials.

```
inter sil /oxi growth vmole=5.0e22 theta.0=0.01 theta.E=0.0
```

This parameter specifies the oxide "C silicon interface injection is to be computed using the oxide growth velocity and with 1injected as interstitials.

```
inter silicon /nitride Ksurf.0=3.5e-3 Ksurf.E=0.0 Krat.0=0.0
```

This specifies that the surface recombination velocity at the nitride silicon interface is $3.5 \times 10^{-3} \text{ cm/sec}$.

```
inter sil neu.0=1.0 neg.0=1.0 pos.0=0.0 dneg.0=0.0 dpos.0=0.0 inter sil neu.E=0.0 neg.E=0.0 pos.E=0.0 dneg.E=0.0 dpos.E=0.0
```

This specifies that there are equal numbers of negative and neutral charged interstitials in intrinsic doping.

7.37.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/ Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. P.B. Griffin and J.D. Plummer, "Process Physics Determining 2-D Impurity Profiles in VLSI Devices," International Electron Devices Meeting, Los Angeles, p. 522, 1986.
3. W. Shockley and J.T. Last, "Statistics of the Charge Distribution for a Localized Flaw in a Semiconductor," Phys. Rev., 107(2), p. 392, 1957.
4. S. M. Hu, "On Interstitial and Vacancy Concentrations in Presence of Injection," J. Appl. Phys., 57, p. 1069, 1985.
5. C. Boit, F. Lau and R. Sittig, "Gold Diffusion in Silicon by Rapid Optical Annealing," Appl. Phys. A, 50, p. 197, 1990.

6. M.E. Law, "Parameters for Point Defect Diffusion and Recombination," IEEE Trans. on CAD, Accepted for Publication, Sept., 1991.
7. M.D. Giles, "Defect Coupled Diffusion at High Concentrations," IEEE Trans. on CAD, 8(4), p. 460, 1989.
8. G.A. Baraff and M. Schluter, "Electronic Structure, Total Energies, and Abundances of the Elementary Point Defects in GaAs," Phys. Rev. Lett., 55, 1327 (1985).
9. R.W. Jansen, D.S. Wolde-Kidane, and O.F. Sankey, "Energetics and Deep Levels of Interstitial Defects in the Compound Semiconductors GaAs, AlAs, ZnSe, and ZnTe," J. Appl. Phys., 64, 2415 (1988).

7.38 isilicon

7.38.1 Synopsis

isilicon

(silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)
[Dix.0=<n>] [Dix.E=<n>] [Dim.0=<n>] [Dim.E=<n>]
[Dimm.0=<n>] [Dimm.E=<n>] [Fi = <n>]
[implanted] [grown.in]
[ss.clear] [ss.temp=<n>] [ss.conc=<n>]
[(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial)]
[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]
[(donor | acceptor)]

7.38.2 Description

This statement allows the user to specify values for coefficients of isilicon diffusion and segregation. The diffusion equation for isilicon is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^* n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^* n_i} \right) \right] \quad 7.56$$

where C_T and C_A are the total chemical and active concentrations of isilicon, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium

concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.57$$

$$D_I = F_i \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.58$$

D_X , D_M and D_{MM} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.59$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the silicon diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. For implanted silicon in gallium arsenide Dix.0 defaults to $1.8 \times 10^{-9} \text{ cm}^2/\text{sec}$ and Dix.E defaults to 1.6 eV. For grown-in silicon in gallium arsenide Dix.0 defaults to 0.0 cm^2/sec and Dix.E defaults to 0.0 eV [1,2,3,4]. DX is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the isilicon diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 0.0 cm^2/sec in silicon, and Dim.E defaults to 0.0 eV in silicon. D_M is calculated using a standard Arrhenius relationship.

- **Dimm.0, Dimm.E**

These floating point parameters allow the specification of the silicon diffusing with doubly negative defects. Dimm.0 is the pre-exponential constant and Dimm.E is the activation energy. For implanted silicon in gallium arsenide Dimm.0 defaults to 0.0 cm^2/sec and Dimm.E defaults to 0.0 eV. For grown-in silicon in gallium arsenide Dimm.0 defaults to 33.0 cm^2/sec and Dimm.E defaults to 3.9 eV [1,2,3]. D_{MM} is calculated using a standard Arrhenius

relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether isilicon diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.0.

- **implanted, grown.in**

Specifies whether the Dix , Dim , Dimm, or Fi coefficients apply to implanted or grown-in isilicon. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default isilicon is a donor.

7.38.3 Examples

isilicon gaas implanted Dimm.0=2.1e-3 Dimm.E=3.1

This command changes the doubly negative defect diffusivity for implanted silicon in gallium arsenide.

`isilicon gaas /nitride Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the gallium arsenide "C silicon nitride interface. The gallium arsenide concentration will be 30.0 times the nitride concentration in equilibrium.

7.38.4 References

1. J. D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1990.
2. J. D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1991.
3. E.L. Allen, J.J. Murray, M.D. Deal, J.D. Plummer, K.S. Jones, and W.S. Robert, Diffusion of Ion-Implanted Group IV Dopants in GaAs, J. Electrochem. Soc., 138, p. 3440, 1991.
4. J.J. Murray, M.D. Deal, E.A. Allen, D.D. Stevenson, and S. Nozaki, Modeling Silicon Diffusion in GaAs Using Well Defined Silicon Doped Molecular Beam Epitaxy Structures, J. Electrochem. Soc., 139, p. 2037, 1992.

7.39 ivgeneric

This command is the same as **generic** except it is for the fourth generic impurity. Please see the command for **generic** for more details.

7.40 line

7.40.1 Synopsis

`line ( x | y | z ) location = <n> [spacing = <n>] [tag = <string>]`

7.40.2 Description

This statement is used to specify the position and spacing of mesh lines. All line statements should come before region and boundary statements, which should in turn be followed by an initialize statement. Only flat structures can be specified with line / region / boundary statements. To create the grid for a more complicated structure, use the interactive grid generator skel.

- **x, y, z**

A mesh line is either horizontal, vertical, or into the screen. The x coordinate increases from left to right. The y coordinate increases into the substrate, that is, from top to bottom. People who learned analytic geometry with the y axis pointing up seem to find this confusing. The z coordinate is supported, somewhat. Rectangular grid generation in three dimensions is the only three dimensional part of the program. It is anticipated that the next release will be fully one, two, and three dimensional.

- **location**

The location along the chosen axis, in microns.

- **spacing**

The local grid spacing, in microns. SUPREM-IV will add mesh lines to the ones given according to the following recipe. Each user line has a spacing, either specified by the user or inferred from the nearest neighbor. These spacings are then smoothed out so no adjacent intervals will have a ratio greater than 1.5. New grid lines are then introduced so that the line spacing varies geometrically from the user spacing at one end of each interval to that at the other. The example below might alleviate the confusion you must be experiencing.

- **tag**

Lines can be tagged with a label for later reference by boundary and region statements. The label is any word you like.

7.40.3 Examples

```
line x loc=0 spa=1 tag=left line x loc=1 spa=0.1 line x loc=2 spa=1 tag=right line y loc=0 spa=0.02
tag=surf line y loc=3 spa=0.5 tag=back
```

There are three user-specified x lines and two user y lines. Taking the x lines for illustration, there is finer spacing in the center than at the edges. After processing, SUPREM-IV produces a mesh with x lines at 0.0, 0.42, 0.69, 0.88, 1.0, 1.12, 1.31, 1.58, and 2.0 μm . Around the center, the spacing is 0.12, approximately what was requested. At the edge, the spacing is 0.42 μm , because that was as coarse as it could get without having an interval ratio greater than 1.5. If the interval ratio was allowed to be 9, say, then we would have got one interval of 0.9 μm and one interval of 0.1 μm on each side. In this example, specifying a spacing of 1 μm at the edges was redundant, because that's what the spacing of the user lines was anyway.

7.41 local_heating

7.41.1 Synopsis

```
local_heating  
(uniform | left_right_piece_model)  
[xdiv=<n>] [left.ratio=<n>] [right.ratio=<n>]  
[infile=<string>] [min.temperature=<n>]
```

7.41.2 Description

In CSuprem, during anneal, the temperature in the whole device under simulation is the same at certain moment, the purpose of the command of local_heating is to change the temperature within the device and make the temperature unevenly distributed throughout the device at certain moment.

- **uniform | left_right_piece_model**

This is a model for local heating. Uniform means the temperature is uniformly distributed. Left_right_piece_model is the model for unevenly distributed temperature. Suppose the user uses uneven distribution model during certain diffusion step only, while for the next diffusion step, the temperature is uniformly distributed within the device, the model variable uniform is then used to switch back to uniform model.

- **xdiv**

Set value for x, apply different temperature for left and right handside of x.

- **left.ratio**

There is a temperature value on every mesh point, left.ratio is used to multiply a value defined by left.ratio to the points located to the left of x to get the final temperature on the left handside of x.

- **right.ratio**

There is a temperature value on every mesh point, right.ratio is used to multiply a value defined by right.ratio to the points located to the right of x to get the final temperature on the right handside of x.

- **infile**

Input temperature data file. This data file is obtained from APSYS.

- **min.temperature**

If the temperature of certain point is less than min.temperature, then the value defined by min.temperature will be used instead. This is to make sure the convergency during diffusion.

7.41.3 Examples

`local_heating infile=lt.dat min.temperature=500`

This command will be used to import the temperature data lt.dat. If the temperature of certain point is less than 500K, this command will reset it to 500k.

7.42 loose_mesh

7.42.1 Synopsis

`loose_mesh [x1=<n> (default=-9999.0)] [x2=<n> (default=9999.0)] [y1=<n> (default=-9999.0)] [y2=<n> (default=9999.0)]`

7.42.2 Description

This command will loose the mesh in specified area

- **x1**

Set the minimum x value for the specified area

- **x2**

Set the maximum x value for the specified area

- **y1**

Set the minimum y value for the specified area

- **y2**

Set the maximum y value for the specified area

7.42.3 Examples

`loose_mesh x1=1.0 x2=2.0 y1=1.0 y2=2.0`

Loose the triangles in the area defined by x=1.0,2.0 y=1.0,2.0

7.43 magnesium

7.43.1 Synopsis

magnesium

(silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial |
ivmaterial | vmaterial)

[Dix.0=<n>] [Dix.E=<n>]

[Dip.0=<n>] [Dip.E=<n>] [Dipp.0=<n>] [Dipp.E=<n>]

[Fi = <n>]

[implanted] [grown.in]

[ss.clear] [ss.temp=<n>] [ss.conc=<n>]

[(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iimaterial | /ivmaterial | /vmaterial)]

[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]

[(donor | acceptor)]

7.43.2 Description

This statement allows the user to specify values for coefficients of magnesium diffusion and segregation. The diffusion equation for magnesium is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{p}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{p}{n_i} \right) \right] \quad 7.60$$

where C_T and C_A are the total chemical and active concentrations of magnesium, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.61$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.62$$

D_X , D_P and D_{PP} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 \frac{C_2}{M_{12}} \right) \quad 7.63$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of DX , the magnesium diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.0 cm²/sec in gallium arsenide, and Dix.E defaults to 0.0 eV in gallium arsenide. DX is calculated using a standard Arrhenius relationship.

- **Dip.0,Dip.E**

These floating point parameters allow the specification of DP, the beryllium diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to $6.1 \times 10^{-2} \text{ cm}^2/\text{sec}$ for implanted magnesium in gallium arsenide and $6.8 \times 10^{-3} \text{ cm}^2/\text{sec}$ for grown-in magnesium in gallium arsenide, and Dip.E defaults to 2.8 eV for both in gallium arsenide [1,2,3]. D_p is calculated using a standard Arrhenius relationship.

- **Dipp.0,Dipp.E**

These floating point parameters allow the specification of DPP, the magnesium diffusivity with doubly positive defects. Dipp.0 is the pre-exponential constant and Dipp.E is the activation energy. Dipp.0 defaults to 0.0 cm²/sec in gallium arsenide, and Dipp.E defaults to 0.0 eV in gallium arsenide. D_{PP} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether magnesium diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 1.

- **implanted, grown.in**

Specifies whether the Dix , Dip , Dipp, or Fi coefficients apply to implanted or grown-in magnesium. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default magnesium is an acceptor.

7.43.3 Examples

magnesium gaas implanted Dip.0=2.2e-6 Dip.E=1.76

This command changes the positive defect diffusivity for implanted magnesium in gallium arsenide.

magnesium gaas /nitride Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7

This command will change the segregation parameters at the gallium arsenide / silicon nitride interface. The gallium arsenide concentration will be half the nitride concentration in equilibrium at 1100C.

7.43.4 References

1. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1990.

2. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1991.
3. H. G. Robinson, M.D. Deal, and D.A. Stevenson, Hole Dependent Diffusion of Implanted Mg in GaAs, Appl. Phys. Lett. 58, p. 2801, 1990.
4. H.G. Robinson, Diffusion of P-Type Dopants in GaAs, Stanford University Ph.D. Thesis, 1992.

7.44 mask

7.44.1 Synopsis

```
mask
segm=<n> (default=-1)
thick=<n>
xi.from=<n> xi.to=<n> xi.len=<n> xi.left.theta=<n> xi.right.theta=<n> (i=1,...,9)
meshlayer=<n>
```

7.44.2 Description

This statement should be regarded as a macro that expands into a set of deposit and etch commands for the convenience of setting up device. The program stores the mask geometry information so that subsequent etch statements may use the same mask coordinates.

This command causes the program to deposit a layer of photoresist and etch away segments outside of the mask lines.

- **segm**

In 3D simulation, this command is used to select which segment to use the mask.

- **thick**

It is the thickness of the photoresist layer to be deposited.

- **xi.from (i=1..9)**

It is the starting coordinate of the ith mask segment.

- **xi.to (i=1..9)**

It is the ending coordinate of the ith mask segment.

- **xi.len (i=1..9)**

It is the length of the ith mask segment. This parameter conflicts with xi.to and only one of these should be used, not both.

- **xi.left.theta (i=1..9)**

It is the counterclockwise angle measured from the downward direction at position xi.from. This angle may be used by subsequent etch statements to make a downward cut at such an angle of the material to be etched.

- **xi.right.theta (i=1..9)**

It is the counterclockwise angle measured from the downward direction at position xi.to. This angle may be used by subsequent etch statements to make a downward cut at such an angle of the material to be etched.

- **meshlayer**

It causes the mesh generator to place meshlayer number of horizontal mesh lines (mesh-layer) for the deposited photoresist.

7.44.3 Examples

```
mask thick=0.2 x1.from=1 x1.to=2 x2.from=4 x2.to=6
```

This line would deposit 0.2 micron thick photoresist and etch away segments left of 1 um, between 2 and 4 um and right of 6 um.

7.45 material

7.45.1 Synopsis

material

(silicon | oxide | poly | oxynitr | nitride | photores | aluminum | gaas | imaterial | iimaterial |
iiimaterial | ivmaterial | vmaterial)

[(wet | dry)]

[Ni.0=<n>] [Ni.E=<n>] [Ni.Pow=<n>]

[eps=<n>] [visc.0=<n>] [visc.E=<n>] [visc.x=<n>]

[Young.m=<n>] [Poiss.r=<n>]

[lcte=<string>]

[intrin.sig=<n>]

[act.a=<n>] [act.b=<n>] [(n.type | p.type)]

7.45.2 Description

This statement allows the user to specify values for coefficients of the intrinsic concentration and relative permittivity for all the materials.

- **silicon , oxide , oxynitride , nitride , poly , photores , aluminu, gaas, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

These parameters indicate the material that the remainder of the parameters apply to.

- **wet | dry**

These parameters indicate whether the parameters apply to in wet or dry oxides. When oxide is specified, it also necessary to specify the type of oxide.

- **Ni.0, Ni.E, Ni.Pow**

These parameters specify the dependencies of the intrinsic electron concentration as a function of temperature.

$$n_i = N_{i,0} e^{\left(-\frac{N_{i,E}}{kT}\right)} T^{N_{i,pow}} \quad 7.64$$

Ni.0 is the pre-exponential term, Ni.E is the activation energy, and Ni.pow is the power dependence of temperature. The default in silicon is $Ni.0 = 3.9 \times 10^{16} cm^3$, $Ni.E = 0.605$ eV, and $Ni.Pow = 1.5$.

- **eps**

This parameter allows the specification of the relative permittivity of the different materials.

- **visc.0, visc.E, visc.x**

These are the parameters specifying viscosity. visc.0 is the pre-exponential coefficient, visc.E is the activation energy. visc.x is the incompressibility factor. Normally this is left at the value of 0.499 (0.5 is infinitely incompressible), but if it is desired to allow more compressibility, it can be lowered.

- **Young.m, Poiss.r**

Young.m is the material's Young's modulus, in dynes/cm². Poiss.r is the material's Poisson ratio.

- **lcte**

This is an expression giving the linear coefficient of thermal expansion as a function of temperature, called T in the expression. It is given as a fraction, not as a percentage.

- **intrin.sig**

This parameter specifies the initial uniform stress state of a material such as a thin film of nitride deposited on the substrate. It can be specified as a function of temperature by using an expression and the variable T (see examples).

- **act.a, act.b**

GaAs activation modeling includes a compensation mechanism. This is modeled in SUPREM-IV.GS using the following expression:

$$(N_D - N_A)_{effective} = \frac{A(N_D - N_A)}{1 + (N_D - N_A)/B} \quad 7.65$$

Where A and B in the above expression are set by the act.a and act.b parameters. act.a and act.b may be expressions in terms of T, the temperature in degrees C. The values of A and B are dependent upon the local dominant impurity type so that one of the p.type or n.type parameters must be specified when specifying act.a or act.b.

- **n.type, p.type**

When setting the GaAs activation coefficients using the act.a and act.b parameters, one of the n.type or p.type parameters must be specified to indicate the type of region that they are to apply to.

7.45.3 Examples

`material silicon eps = 11.9`

This statement specifies the silicon relative permittivity.

`mate nitride lcte=(3e-6+2*1e-10*T) intrin.sig=1.4e10 You=3e12`

This statement gives the thermal expansion coefficient of nitride as a function of absolute temperature T. Thus at 0K the coefficient is 0.0003%/K. The initial stress in the nitride film is $1.4 \times 10^{10} \text{ dynes/cm}^2$ and the Young's modulus is $3 \times 10^{12} \text{ dynes/cm}^2$.

7.46 material_define

7.46.1 Synopsis

```
material_define
(=<string>)
[material_label=<string>]
[macro_name=<string>]
[grade_var=<n>]
[grade_from=<n>]
[grade_to=<n>]
```

```
[grade_dir=<x or y>]  
[var_simbol1=<string>]  
[grade_var=<n>]  
[active_macro=<string>]  
[avar1=<n>]  
[avar_symbol1=<string>]
```

7.46.2 Description

This command is used to define a new material, and the components grading within the material.

- **material_label**

This is a label of the specified mater and will be cited by other commands such as region, deposit, etc.

- **macro_name**

The macro name will be used for Apsys load_macro.

- **grade_from**

This is to specify the grade starting value.

- **grade_to**

This is to specify the grade ending value.

- **grade_dir**

This is used to determine the grade direction of the material, either vertical direction or horizontal direction.

- **var_symbol1**

This is to specify the grade variable symbol number 1.

- **grade_var**

This is to specify the grade variable number (e.g 1,2,...) for the variable symbol.

- **active_macro**

This is used to determine the quantum well and barrier materials (e.g. InGaN/InGaN).

- **avar1**

This is used to specify the value of the first active grade variable.

- **avar_symbol1**

This is to specify the symbol of the first active grade variable.

7.46.3 Examples

```
mater_define material_label=In0.08Ga0.92N macro_name=ingan var1=0.08 var_symbol1=x
```

This command will specify a new material $\text{In}_{0.08}\text{Ga}_{0.92}\text{N}$.

7.47 memselectivetch

7.47.1 Synopsis

```
memselectivetch [ segm=<n> (default=-1) ] (silicon | oxide | oxynitride | nitride | poly |  
photoresist | aluminum | gaas | imaterial | iimaterial | iiimaterial | ivmaterial)
```

7.47.2 Description

This command is used to entirely etch away certain material from the device.

- **segm**

In 3D simulation, specify the segment to work on.

- **silicon, oxide, oxynitride, nitride, poly, photoresist, aluminum, gaas, imaterial, iimaterial, iiimaterial, ivmaterial**

Specify the material to be etched away.

7.47.3 Examples

```
memselectivetch segm=1 silicon
```

This command will etch away all the silicon in segment 1.

7.48 merge

7.48.1 Synopsis

```
merge  
[inf.1=<string>] [flipy.1=<logical> (default=false)]
```

```
[deltax.1=<n> (default=0.0)]  
[deltay.1=<n> (default=0.0)]  
[inf.2=<string>] [flipy.2=<logical> (default=false)]  
[deltax.2=<n> (default=0.0)] [deltay.2=<n> (default=0.0)]
```

7.48.2 Description

This command read the data of two files and then combine them into one device. It is mainly used for merging.

- **inf.1**

Input the first file

- **flipy.1**

Y direction flipping is necessary or not for the first data file

- **deltax.1**

x direction movement for the first data file

- **deltay.1**

y direction movement for the first data file

- **inf.2**

Input the second file

- **flipy.2**

Y direction flipping is necessary or not for the second data file

- **deltax.2**

x direction movement for the second data file

- **deltay.2**

y direction movement for the second data file

7.48.3 Examples

```
merge inf.1=input1.str inf.2=input2.str deltax.2=1.0
```

Two files: input1.str and input2.str will be imported by this command and the data in input2.str will be shifted by 1.0 in the y direction. Finally the two structures will be merged to form a new device.

7.49 method

7.49.1 Synopsis

method

```
[ ( vacancies | interstitial | arsenic | phosphorus | antimony | boron | gallium | oxidant |
velocity | traps | gold | psi | cesium | beryllium | magnesium | selenium | isilicon | tin |
germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric | vgeneric | indium |
ni.impurity | al.impurity ) ]
[ rel.error=<n> ] [ abs.error=<n> ]
[ init.time=<n> ] [ ( trbdf | formula ) ]
[ min.fill ] [ min.freq=<n> ]
[ ( gauss | cg ) ] [ back=<n> ] [ blk.itlim=<n> ]
[ ( time | error | newton ) ]
[ ( diag | full.fac ) ]
[ ( fermi | two.dim | steady | full.cpl ) ]
[ ( erfc | erfg | erf1 | erf2 | vertical | compress | viscous ) ]
[ grid.oxide=<n> (default=0.1) ] [ skip.sil ]
[ oxide.gdt=<n> (default=0.25) ] [ redo.oxide=<n> (default=10) ]
[ oxide.early=<n> (default=0.5) ] [ oxide.late=<n> (default=0.9) ]
[ oxide.rel=<n> (default=1.0e-6) ]
[ gloop.emin=<n> ] [ gloop.emax=<n> ] [ gloop.imax=<n> (default=170) ]
[ boolean zflow_oxidant ]
[ mf.solver.level=<n> (default=0) ]
[ outfile=<string> ]
[ relax.rhs.ratio=<n> (default=9.99e-1) ]
[ max.iter.num=<n> (default=100000) ]
[ min.time.step=<n> (default=0) ]
[ tolerance=<n> (default=1.e-6) ]
[ rhs.tolerance=<n> (default=10.0) ]
[ no.triangle.flip=<logical> (default=false) ]
[ 3d.iter.check=<n> (default=30) ]
[ ox.obfix=<n> (default=2) ]
[ grid.grain=<n> (default=0.1) ]
[ grain.gdt=<n> (default=0.25) ]
[ norm.style=<n> (default=0) ]
[ blk.itlim=<n> (default=10) ]
```

7.49.2 Description

Select numerical methods and models for diffusion and oxidation. This statement sets the flags for selection of algorithm for various mathematical solutions. The statement also allows the user to select the complexity desired for the diffusion and oxidation models. Appropriate defaults for the numerical parameters are included in the model start up file so users should only have to specify the type of model desired for the diffusion and oxidation. The numerical methods used in SUPREM-IV for solution of the diffusion equations are described in more detail in Law and Dutton [1].

- **vacancies, interstitial, arsenic, phosphorus, antimony, boron, gallium, oxidant, velocity, traps, gold, psi, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic, iigeneric, iigeneric, ivgeneric, vgeneric, indium, ni.impurity, al.impurity**

These options allow the user to specify a single impurity. The error bound parameters are specific for each impurity. If error bounds are listed, an impurity must also be selected.

- **rel.err**

This parameter indicates the precision with which the impurity block must be solved. In general, the error will be less than half of the indicated error since the program is conservative in its error estimates. The defaults are 0.01 for all impurities except the potential which is solved to 0.001.

- **abs.err**

This parameter indicates the size of the absolute value of the error. For the dopants, the absolute error defaults to 109. For defects, the absolute error defaults to 105. For the potential, the error defaults to 10⁻⁶.

- **init.time**

This parameter specifies the size of the initial time step. It defaults to 0.1 seconds. A routine to estimate the initial time step is being worked on.

- **trbdf, formula**

These parameters specify the method of time integration to be used. The trbdf option indicates a combination trapezoidal rule/backward difference should be used. The error is estimated using Milne's device. The formula method allows the user to specify the time step directly as a function of time (t), previous time step (dt) and grid time (gdt). This option is primarily for testing. The trbdf method is the default. The time step methods have been taken from literature, see Yeager and Dutton [2] and Bank et al. [3].

- **min.fill, min.freq**

This option allows the user to specify a minimum fill. It defaults to true. This is a highly recommended option since it can reduce the matrix sizes by a factor of two or more. The size of the matrix is proportional to speed. min.freq is a parameter which controls the frequency of min fill reordering. It is only partially implemented and will have no effect on the calculation.

- **gauss, cg**

These parameters allow the user to specify the type of iteration performed on the linear system as a whole. `gauss` specifies that this iteration should be Gauss-Seidel, `cg` specifies that a conjugate residual should be used. The default is `cg`. The `gauss` option is considerably slower, it should be deleted in the future. The CG algorithm is described by Elman [4].

- **back**

This integer parameter specifies the number of back vectors that can be used in the `cg` outer iteration. The default is 3, and the maximum that can be used is 5. The higher the number, the better the convergence and the more memory used.

- **blk.itlim**

The maximum number of block iterations that can be taken. The block iteration will finish at this point independent of convergence.

- **time, error, newton**

These parameters specify the frequency with which the matrix should be re-factored. The `time` parameter specifies that the matrix should be re-factored twice per time step. This option takes advantage of the similarity in the matrix across a time integration. The `error` option indicates the matrix should be re-factored whenever the error in that block is decreasing. The `newton` option will force the re-factorization at every newton step. The default is `time`.

- **diag, knot, full.fac**

These parameters specify the amount of fill to be included in the factorization of the matrix. `full.fac` indicates that the entire amount of fill is to be computed. The `diag` option indicates that the only the diagonal blocks should be factored in the matrix. The `diag` option is the default, although under certain conditions (one dimensional) `full.fac` may perform better. `knot` is a factor.

- **fermi, two.dim, steady, full.cpl**

These parameters specify the type of defect model to be used. The `fermi` option indicates that the defects are assumed to be a function of the fermi level only. The `two.dim` option indicates that a full time dependent transient simulation should be performed. The `steady` option indicates that the defects can be assumed to be in steady state. The `full.cpl` option indicates that the full coupling between defects and dopants should be included. The default is `fermi`.

- **erfc, erfg, erf1, erf2, vertical, compress, viscous**

These are oxide model parameters. The `erfc` option indicates that a simple error function approximation to a bird's beak shape should be used. The `erf1` and `erf2` models are analytic approximations to the bird's beak from the literature (see the oxide statement). The `erfg` model chooses whichever of `erf1` or `erf2` is most appropriate. The `vertical` model indicates that the oxidant should be solved for and that the growth is entirely vertical. The `compress` model treats the oxide as a compressible liquid. The `viscous` model treats the oxide as an incompressible viscous liquid. Real oxide is believed to be incompressible, but the compressible model is a lot faster. It's an accuracy vs. speed trade-off.

- **grid.oxide**

This parameter is the desired thickness of grid layers to be added to the growing oxide. The value is specified in microns. It represents the desired thickness of grid layers to be added to the growing oxide, in microns. It has a side effect on time steps (see oxide.gdt).

- **skip.sil**

This boolean parameter indicates whether or not to compute stress in the silicon. The default is false. The calculation can only be performed when the viscous oxide model is used. The silicon substrate is treated as an elastic solid subject to the tractions generated by the oxide flow. It should not be turned on indiscriminately, since the silicon grid is generally much larger than the oxide grid, and the stress calculation correspondingly more expensive.

- **oxide.gdt**

The time step may be limited by oxidation as well as diffusion; the oxide limit is specified as a fraction (oxide.gdt) of the time required to oxidize the thickness of one grid layer (grid.oxide).

- **redo.oxide**

To save time, the oxide flow field need not be computed every time the diffusion equation for impurities is solved. The parameter redo.oxide specifies the percentage of the time required to oxidize the thickness of one grid layer which should elapse before resolving the flow field. Usually redo.oxide is much less than oxide.gdt, which is an upper bound on how long the solution should wait. It is mainly intended to exclude solving oxidation at each and every one of the first few millisecond time steps when defects are being tracked.

- **oxide.early, oxide.late, oxide.rel**

These parameters are not normally modified by users. They relate to internal numerical mechanisms, and are described only for completeness. A node whose spacing decreases proportionally by more than oxide.late is marked for removal. If there are any nodes being removed, then in addition all nodes greater than oxide.early are removed. The default values are oxide.early = 0.5 and oxide.late = 0.9. For earlier node removal (less obtuse triangles), try oxide.late = 0.3 and oxide.early = 0.1. It makes no sense, but does no harm, to have oxide.early greater than oxide.late. The oxide.rel parameter is the solution tolerance in the nonlinear viscous model. The default is 10⁻⁶ (that is, a 10⁻⁴ percentage error in the velocities). It can be increased for a faster solution, but a value greater than about 10⁻³ will probably give meaningless results.

- **gloop.emin, gloop.emax, gloop.imin**

This group of parameters controls loop detection during grid manipulation. The loop detection checks for intrusions and extrusions in the boundary. The intrusion-fixing algorithm is triggered by angles greater than gloop.imax. The default value is gloop.imax = 170°. A larger value means that more extreme intrusions can develop and increases the possibility of a tangled grid. A smaller value leads to earlier intrusion-fixing; too small a value will lead to inaccuracy due to premature intervention. Similar concerns apply to the other parameters. The values are a

compromise between safety and accuracy. The extrusion-fixing algorithm is always triggered by angles greater than `gloop.emax`. Again, the default is 170° . In addition, it be triggered by lesser extrusions, anything greater than `gloop.emin`, if the extrusion is a single-triangle glitch in the boundary. The default value is `gloop.emin` = 120° . None of these parameters should be less than 90° , because then the rectangular edges of the simulation space would be smoothed out!

- **zflow_oxidant**

which takes the default of false. It is used to enable or disable the interplane oxidant flow for the wet-oxidation process in 3D simulation. In some cases, flow of oxidant in z-direction may cause convergence problem and one may wish to disable the interplane oxidant flow for less accurate 3D results. This parameter only affects 3D oxide growth and does not affect diffusion of dopants. For more information on interplane oxidant diffusion, please see command `connect_region`.

- **mf.solver.level**

This command is to define whether `mf.solver.level` method is used to solve the matrix. This is a newly added method

- **outfile**

If errors occur in the program, the error information will be printed to certain file.

- **relax.rhs.ratio**

This command is used to control the convergency issue during diffusion. When solving a matrix like $AX=B$, for item B, after certain iteration, the ratio of error residue to that of the previous iteration result is greater than the value of `relax.rhs.ratio` will yield the satisfactory result.

- **max.iter.num**

When solving matrix $AX=B$ during diffusion, if the number of iteration is greater than the value defined by `max.iter.num`, the iteration is considered not converged.

- **min.time.step**

During diffusion, if the time step is smaller than the value of `min.time.step`, it means there is little movement of time. An error message will appear for not converging.

- **tolerance**

During diffusion, if the ratio of error residue of right handside items of the matrix to that of the last iteration is smaller than the value in the pre-set tolerance value, it means the result is satisfactory.

- **rhs.tolerance**

During diffusion, if the error residue of the right handside items is less than the value set by `rhs.tolerance`, then a satisfactory result is obtained. Otherwise is not satisfactory.

- **no.triangle.flip**

This command determines whether it is necessary to check triangle flip. The flip check includes checking whether the triangle obeys counterclockwise order. False means all triangles need to be checked with flip

- **3d.iter.check**

In 3D simulation, if the number of iteration is greater than 3d.iter.check, an error message will appear.

- **blk.itlim**

During cg iteration, the maximum number of iteration for cg. Value needs to be increased in case of convergency problem

7.49.3 Examples

```
method arsenic rel.err=0.01 abs.err=1.0e9
```

This command indicates that the arsenic equation should be solved with a relative error of 1.

```
method min.fill cg back=3 init.ti=0.1 trbdf fermi vert skip.sil=f
```

This command specifies that minimum fill reordering should be done and that the entire system should be solved using a conjugate residual technique with three back vectors. The initial time step should be 0.1 seconds and time should be integrated using the trbdf model. The fermi model should be assumed for the defects and the vertical model for the oxide growth. The stresses and potential and the silicon should not be computed.

7.50 mode

7.50.1 Synopsis

```
mode  
[ one.dim | two.dim | three.dim | quasi3d | cylindrical]
```

7.50.2 Description

The mode statement determines the dimensionality of the simulator. The internal data structure is changed slightly to support true one dimensional mode operation when one dimension is selected. Two dimensions is the default, and is the original operation mode.

- **one.dim , two.dim , three.dim , quasi3d**

These parameters set the simulator dimensionality.

- **cylindrical**

This command is to determined whether cylindrical coordinate will be used, cylindrical coordinate will generate different diffuse result.

7.50.3 Examples

`mode three.dim`

This will instruct the simulator to perform a full 3D simulation.

7.51 ni.impurity

7.51.1 Synopsis

```
ni.impurity
( silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ] [ Dim.0=<n> ] [ Dim.E=<n> ]
[ Fi = <n> ]
[ implanted ] [ grown.in ]
[ Ctn.0=<n> ] [ Ctn.E=<n> ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ]
[ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial) ]
[ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]
```

7.51.2 Description

This statement allows the user to specify values for coefficients of nitrogen diffusion and segregation. The diffusion equation for ni.impurity is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.66$$

where C_T and C_A are the total chemical and active concentrations of ni.impurity, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities

are given by:

$$D_V = (1 - F_i) \left(D_X + D_M \frac{n}{n_i} \right) \quad 7.67$$

$$D_I = F_i \left(D_X + D_M \frac{n}{n_i} \right) \quad 7.68$$

D_X and D_M are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.69$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial | iimaterial | iimaterial | ivmaterial | vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the ni.impurity diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.214 cm²/sec in silicon, and Dix.E defaults to 3.65 eV in silicon [3]. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the ni.impurity diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 15.0 cm²/sec in silicon, and Dim.E defaults to 4.08 eV in silicon [3]. D_M is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether ni.impurity diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.05 [4].

- **implanted, grown.in**

Specifies whether the Dix, Dim, or Fi coefficients apply to implanted or grown-in ni.impurity. If neither is specified then a specified parameter applies to both.

- **Ctn.0, Ctn.E**

These parameters specify the clustering coefficients for ni.impurity. The parameters are respectively the pre-exponential coefficient and the activation energy. The total cluster coefficient obeys a standard Arrhenius relationship. The active concentration, C_A is calculated using:

$$C_T = C_A \left[1 + C_{tn} \left(\frac{n}{n_i} \right)^2 \frac{C_V}{C_V^*} \right] \quad 7.70$$

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm^{-3})
800	2.3e19
900	3.0e19
1000	4.0e19
1100	4.8e19
1200	6.2e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial | /iimaterial | /iiimaterial | /ivmaterial | /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default ni.impurity is a donor.

7.51.3 Examples

ni.impurity silicon Dix.0=8.0 Dix.E=3.65

This command changes the neutral defect diffusivity in silicon.

`ni.impurity silicon /oxide Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the silicon - silicon dioxide interface. The silicon concentration will be 30.0 times the oxide concentration in equilibrium.

7.51.4 References

1. D. Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.
2. R.B. Fair, "Concentration Profiles of Diffused Dopants in Silicon," Impurity Doping Processes in Silicon, Yang ed., 1981, North-Holland, New York.
3. P. Fahey, G. Barbuscia, M. Moslehi and R.W. Dutton, "Kinetics of Thermal Nitridation Processes in the Study of Dopant Diffusion Mechanisms in Silicon," Appl. Phys. Lett., 46(8), p. 784, 1985.
4. F.A. Trumbore, "Solid Solubilities of Impurity Elements in Germanium and Silicon," Bell System Tech Journal, 1960.

7.52 option

7.52.1 Synopsis

option

[quiet | normal | chat | barf]

[fewer.mesh | vertical.mesh | original.mesh | rectangle.based]

[loose_mesh_size =<n> (default=-999.0)] [etch.type=<n> (default=3)]

[poly.clip=<logical> (default=false)]

[optim.angle=<logical> (default=false)]

[stanford_implant_table=<logical> (default=true)]

7.52.2 Description

In the previous version of Csuprem, this command was used only to specify the verbosity of the simulator. These parameters are also used to set up the mesh generation algorithms for deposition etching.

- **quiet, normal, chat, barf**

These parameters determine the amount of information that is output to the user about errors, cpu times, behavior of the algorithms, etc. The default is quiet. The chat and barf modes are mainly useful for debugging and would not normally be chosen by any user in their right mind.

- **fewer.mesh, vertical.mesh, original.mesh , rectangle.based**

These parameters are used to set up the mesh generation algorithms for deposition. The option *original.mesh=true* is based on the original suprem.4gs mesh algorithm. The option *vertical.mesh=true* is based on a new mesh algorithm of Crosslight. The option *fewer.mesh=true* is based on a modified version of the original suprem.4gs meshing algorithm. The user is recommended to use the options *original.mesh=true* or *vertical.mesh=true*. The option *fewer.mesh* is based on the original mesh scheme with method to truncate the mesh generation if it detects the triangle size would be less than 80 percent of the deposition thickness.

Rectangle.based is a new algorithm, rectangular mesh grids are built on the surface during deposition, and then these grids are etched away. The remaining mesh grids will be added to the device. This method is the default method, if it somehow failed, then priority will be given to vertical.mesh.

- **loose_mesh_size**

The parameter is used to enable the *fewer.mesh=true* method with a size of triangle for truncation relative to thickness. *loose_mesh_size* means during deposition, if newly created mesh length is greater than the value set by *loose_mesh_size*, it needs denser mesh grids.

- **stanford_implant_table**

Implant table is needed during implant. If *stanford_implant_table* is set to be true, it means the original stanford implant table will be applied. If *stanford_implant_table* is set to false, then crosslight modified implant table will be used.

- **etch.type**

There are three new etching models that can now be set. The parameter *etch.type=1,2,3* is used to set up the type of the mesh generator algorithm. The case *etch.type=1* will use the original mesh generator algorithm *grid* of suprem.4gs with improved safe guards against several exceptions such as isolated regions after dry etch. The case *etch.type=2* will use a modified version of the mesh generator *grid* with efforts to keep the original mesh intact. The case *etch.type=3* which is the default will use a new mesh generator provided by Crosslight. This last type seems to be more stable, reliable and allows the mesh spacing control after etching. It is then advised to be used for complicated structures. Switching between different etch types may help the user to resolve program crashes due to complicated device geometries.

- **poly.clip , optim.angle**

It is used with *etch.type=3* to choose between two methods developed by Crosslight to get the

skeleton of the subtraction of two regions. Its default value is false. The user may not need to worry about setting up this parameter except for difficult cases.

7.52.3 Examples

`option vertical.mesh=true etch.type=3`

The command choose vertical.mesh deposition algorithm and 3rd etching model for the following processes.

7.53 oxide

7.53.1 Synopsis

```
oxide orientation= ( 111 | 110 | 100 )
( dry | wet )
[ lin.l.0=<n> ] [ lin.l.e=<n> ] [ lin.h.0=<n> ] [ lin.h.e=<n> ]
[ l.break=<n> ] [ l.pde=<n> ]
[ par.l.0=<n> ] [ par.l.e=<n> ] [ par.h.0=<n> ] [ par.h.e=<n> ]
[ p.break=<n> ] [ p.pdep=<n> ]
[ ori.dep ] [ ori.fac=<n> ]
[ thinox.0=<n> ] [ thinox.e=<n> ] [ thinox.l=<n> ]
[ hcl.pc=<n> ] [ hclT=<string> ] [ hclP=<string> ]
[ hcl.par=<string> ] [ hcl.lin=<string> ]
[ baf.dep ] [ baf.ebk=<n> ] [ baf.pe=<n> ] [ baf.ppe=<n> ]
[ baf.ne=<n> ] [ baf.nne=<n> ] [ baf.k0=<n> ] [ baf.ke=<n> ]
[ stress.dep ] [ Vc=<n> ] [ Vr=<n> ] [ Vd=<n> ] [ Vt=<n> ]
[ Dlim=<n> ]
[ gamma=<n> ] [ alpha=<n> ]
[ henry.coeff=<n> | theta=<n> ]
[ ( silicon | oxide | nitride | poly | gaas | gas | imaterial | iimaterial | iimaterial | ivmaterial |
vmaterial) ]
[ ( /silicon | /oxide | /nitride | /poly | /gaas | /gas | /imaterial | /iimaterial | /iimaterial |
/ivmaterial | /vmaterial) ]
[ diff.0=<n> ] [ diff.e=<n> ] [ seg.0=<n> ] [ seg.E=<n> ]
[ trn.0=<n> ] [ trn.E=<n> ]
[ initial=<n> ] [ spread=<n> | mask.edge=<n> ]
[ erf.q=<n> ] [ erf.delta=<n> ] [ erf.lbb=<n> ] [ erf.h=<n> ]
[ nit.thick=<n> ]
```

7.53.2 Description

All parameters relating to oxidation, and a few more besides, are specified here. Five oxide models are supported in this release: 1) an error-function fit to bird's beak shapes; 2) a parameterized error-function model from the literature; 3) a model where oxidant diffuses and the oxide boundaries move vertically at a rate determined by the local oxidant concentration; 4) a compressible viscous flow model, similar to 3 but in addition the new oxide formed at each time step forces the overlying oxide (and nitride) to flow; 5) an incompressible viscous flow model. The error function is by far the fastest. It should be used for uniform oxidation, and is the most reliable for semi-recessed oxidation. It requires fitting data for the lateral spread of the bird's beak. The parameterized model is by Guillemot [1]. The bird's beak length and nitride lifting were measured as a function of process conditions. The diffusion model has no fitting parameters, but is only accurate when the growth is reasonably vertical – less than say 30° interface angle. The compressible model is more accurate, but requires more computer time. It is still cheaper than the incompressible model, which uses many more grid points. The incompressible model is required whenever accurate stresses are desired. Most coefficients need to know whether wet or dry oxidation is intended, and some need to know what the substrate orientation is.

- **orientation**

The substrate orientation to which the coefficients specified apply. Required for orientation factor (see below) and thin oxide coefficients.

- **dry | wet**

The type of oxidation to which the specified coefficients apply. Required for everything except for the one-dimensional coefficients and the volume ratio.

- **lin.l.0, lin.l.e, lin.h.0, lin.h.e, l.break, l.pdep**

The linear rate coefficients (B/A). A doubly activated Arrhenius model is assumed. l.break is the temperature breakpoint between the lower and higher ranges, in degrees Celsius. lin.l.0 is the prefactor in $\mu\text{m}/\text{min}$, and lin.l.e is the activation energy in eV for the low temperature range. lin.h.0 and lin.h.e are the corresponding high temperature numbers. l.pdep is the exponent of the pressure dependence. The value given is taken to apply to $\langle 111 \rangle$ orientation and later adjusted by ori.fac according to the substrate orientation present.

- **par.l.0, par.l.e, par.h.0, par.h.e, p.break, p.pdep**

The parabolic rate coefficients (B). Like the linear rate coefficients, but different.

- **ori.dep, ori.fac**

The numerical coefficient ori.fac is the ratio of B/A on the specified orientation to that on the $\langle 111 \rangle$ orientation. The boolean parameter ori.dep (defaults true) specifies whether the local orientation at each point on the surface should be used to calculate B/A. If it is false, the substrate orientation is used at all points.

- **thinox.0, thinox.e, thinox.l**

Coefficients for the thin oxide model proposed by Massoud [2]. thinox.0 is the prefactor in $\mu\text{m}/\text{min}$, thinox.e is the activation energy in eV, and thinox.l is the characteristic length in μm .

- **hcl.pc, hclT, hclP, hcl.par, hcl.lin**

The numerical parameter hcl.pc is the percentage of HCl in the gas stream. It defaults to 0. The HCl dependence of the linear and parabolic coefficients is obtained from a look-up table specified in the model file. The rows of the table are indexed by HCl percentage. The row entries can be specified with the parameter hclP, which is an array of numerical values, surrounded by double quotes and separated by spaces or commas. The columns are indexed by temperature. The column entries can be specified with the parameter hclT, which is an array of numerical values, surrounded by double quotes and separated by spaces or commas. The dependence of B/A can be specified with the parameter hcl.lin, which is an array of numerical values, surrounded by double quotes and separated by spaces or commas. The number of entries in hcl.lin must be the product of the number of entries in hclP and hclT. The dependence of B can be specified with the parameter hcl.par, which is an array of numerical values, surrounded by double quotes and separated by spaces or commas. The number of entries in hcl.par must be the product of the number of entries in hclP and hclT.

- **baf.dep, baf.ebk, baf.pe, baf.ppe, baf.ne, baf.nne, baf.k0, baf.ke**

These parameters relate to the doping dependence of the oxidation rate. The doping dependence is turned on if baf.dep is true. The linear rate coefficient, B/A, is assumed to depend on the Fermi level as follows [3].

$$B/A = (B/A)_0 [1 + K([V]/[V]_0 - 1)] \quad 7.71$$

[V] is the total concentration of vacancies and K is an activated coefficient with prefactor baf.k0 and activation energy baf.ke. The remaining parameters above serve to compute [V]. The total concentration of vacancies [V] is the sum of the concentrations in each charge state (see vacancies).

- **stress.dep, Vc, Vr, Vd, Vt, Dlim**

These parameters control the stress dependence of oxidation, which is only calculated under the viscous model. The parameter stress.dep turns on the dependence. The parameter Vc is the activation volume of viscosity. The parameter Vr is the activation volume of the reaction rate with respect to normal stress. The parameter Vt is the activation volume of the reaction rate with respect to tangential stress. The parameter Vd is the activation volume of oxidant diffusion with respect to pressure. The parameter Dlim is the maximum increase of diffusion permitted under tensile stress.

- **gamma**

The parameter gamma is the oxidant surface energy in ergs/cm^2 , which controls reflow.

- **alpha**

The volume expansion ratio between material 1 and material 2. In a nutshell, alpha oxide/silicon is 2.2, and alpha x /y is 1.0 for everything else (until someone tells us otherwise).

- **silicon, oxide, nitride, poly, gaas, gas, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial** What material 1 is.

- **/silicon, /oxide, /nitride, /poly, /gaas, /gas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

What material 2 is.

- **henry.coeff,theta**

Henry's coefficient is the solubility of oxidant in material 1 at one atmosphere, in cm^{-3} . Theta is the number of oxygen atoms incorporated in a cubic centimeter of oxide. Note: Don't change these unless you really know what you are doing. Change the Deal-Grove coefficients instead.

- **diff.0, diff.e, seg.0, seg.E, trn.0, trn.E**

The diffusion coefficients of oxidant in material 1, and the boundary coefficients ($j^{\circ}\text{transport}\pm$ and $j^{\circ}\text{segregation}\pm$) from material 1 to material 2. See the note above. For the record, diff.0 is the diffusivity prefactor in cm^2/sec , diff.e is the energy in eV. The transport coefficient represents the gasphase mass-transfer coefficient in terms of concentrations in the solid at the oxide-gas interface, the chemical surface-reaction rate constant at the oxide-silicon surface, and a regular diffusive transport coefficient at other interfaces. The segregation coefficient is 1 at the oxide-gas interface, infinity at the oxide-silicon interface, and represents a regular segregation coefficient at other interfaces. initial The thickness of the existing oxide at the start of oxidation, default 2 nm. If the wafer is bare, an oxide layer of this thickness is deposited before oxidation begins.

- **spread, mask.edge**

These coefficients are only used in the error-function approximation to a bird's beak shape. Spread is the relative lateral to vertical extension, defaults to 1. It's a fitting parameter to make erfc birds look realistic. Mask.edge is the position of the mask edge in microns, and defaults to negative infinity. Oxide grows to the right of the mask edge. erf.q, erf.delta, erf.lbb, erf.h, nit.thick This group of parameters all apply to the $j^{\circ}\text{erfg}\pm$ model. See Guillemot et al. for their interpretation.

- **erf.q, erf.delta**

The delta and q parameters for the $j^{\circ}\text{erfg}\pm$ model. These probably do not need to be changed, but they're available if you want them.

- **erf.lbb**

The erf.lbb parameter is the length of the bird's beak. It can be specified as an expression in Eox (the field oxide thickness (μm)), eox (the pad oxide thickness (μm)), Tox (the oxidation temperature (Kelvin)), and en (the nitride thickness (μm)). The published expression can be

found in the models file. Specifying $i^{\circ} \text{erf.llbb} = E_{ox} \pm$ for instance would give a lateral spread equal to the field thickness, similar to the Hee-Gook Lee model with a spread of 1.

- **erf.h**

The erf.h parameter is the ratio of the nitride lifting to the field oxide thickness. (It corresponds to the Guillemot H parameter except that it is normalized to the field oxide thickness). Again it is specified as an expression of E_{ox} , e_{ox} , T_{ox} , e_n .

- **nit.thick**

The nitride thickness to substitute for the parameter e_n .

7.53.3 Examples

Please see example input files.

7.54 phosphorus

7.54.1 Synopsis

phosphorus

```
( silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ] [ Dim.0=<n> ] [ Dim.E=<n> ]
[ Dimm.0=<n> ] [ Dimm.E=<n> ] [ Fi = <n> ]
[ implanted ] [ grown.in ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ]
[ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial) ]
[ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]
```

7.54.2 Description

This statement allows the user to specify values for coefficients of phosphorus diffusion and segregation. The diffusion equation for phosphorus is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.72$$

where C_T and C_A are the total chemical and active concentrations of phosphorus, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium

concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = D_{VX} + D_{VM} \frac{n}{n_i} + D_{VMM} \left(\frac{n}{n_i} \right)^2 \quad 7.73$$

$$D_I = D_{IX} + D_{IM} \frac{n}{n_i} + D_{IMM} \left(\frac{n}{n_i} \right)^2 \quad 7.74$$

The D -coefficients are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.75$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the phosphorus diffusing with interstitial neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to $3.85 \text{ cm}^2/\text{sec}$ in silicon, and Dix.E defaults to 3.66 eV in silicon [3]. DX is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the phosphorus diffusing with singly negative interstitial defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to $4.44 \text{ cm}^2/\text{sec}$ in silicon, and Dim.E defaults to 4.00 eV in silicon [3]. D_M is calculated using a standard Arrhenius relationship.

- **Dimm.0, Dimm.E**

These floating point parameters allow the specification of the phosphorus diffusing with doubly negative interstitial defects. Dimm.0 is the pre-exponential constant and Dimm.E is the activation energy. Dimm.0 defaults to $44.2 \text{ cm}^2/\text{sec}$ in silicon, and Dim.E defaults to 4.37 eV in silicon [3]. D_{MM} is calculated using a standard Arrhenius relationship.

- **Dvx.0, Dvx.E**

These floating point parameters allow the specification of the phosphorus diffusing with neutral vacancy defects. Dvx.0 is the pre-exponential constant and Dvx.E is the activation energy. Both

of these are 0.0, since phosphorus is an interstitial diffuser [4].

- **Dvm.0, Dvm.E**

These floating point parameters allow the specification of the phosphorus diffusing with singly negative vacancy defects. Dvm.0 is the pre-exponential constant and Dvm.E is the activation energy. Both of these are 0.0, since phosphorus is an interstitial diffuser [4].

- **Dvmm.0, Dvmm.E**

These floating point parameters allow the specification of the phosphorus diffusing with doubly negative vacancy defects. Dvmm.0 is the pre-exponential constant and Dvmm.E is the activation energy. Both of these are 0.0, since phosphorus is an interstitial diffuser [4].

- **implanted, grown.in**

Specifies whether the diffusivity coefficients apply to implanted or grown-in phosphorus. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are [5]:

ss.temp (C)	ss.conc (cm^{-3})
800	2.79e20
900	3.16e20
1000	3.4e20
1100	3.8e20

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor

in a semiconductor material. These parameters are not presently material specific. By default phosphorus is a donor.

7.54.3 Examples

`phosphorus silicon Dix.0=3.85 Dix.E=3.65`

This command changes the neutral defect diffusivity in silicon.

`phosphorus silicon /oxide Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the silicon / silicon dioxide interface. The silicon concentration will be 30.0 times the oxide concentration in equilibrium.

7.54.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/ Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. D. Mathiot and J.C. Pfister, "Dopant Diffusion in Silicon: A consistent view involving nonequilibrium defects," J. Appl. Phys., 55(10), p. 3518, 1984.
3. R.B. Fair, "Concentration Profiles of Diffused Dopants in Silicon," Impurity Doping Processes in Silicon, Yang ed., 1981, North-Holland, New York.
4. P. Fahey, G. Barbuscia, M. Moslehi and R.W. Dutton, "Kinetics of Thermal Nitridation Processes in the Study of Dopant Diffusion Mechanisms in Silicon," Appl. Phys. Lett., 46(8), p. 784, 1985.
5. F.A. Trumbore, "Solid Solubilities of Impurity Elements in Germanium and Silicon," Bell System Tech Journal, 1960.

7.55 plot.1d

7.55.1 Synopsis

`plot.1d`

`[ (x.value=<n> | y.value=<n>) ]`

`[ exposed | backside | reflect | silicon | oxide | nitride | poly | oxynitride | aluminum | photoresist | gaas | gas ]`

```
[ /exposed | /backside | /reflect | /silicon | /oxide | /nitride | /poly | /oxynitride | /aluminum |
/photoresist | /gaas | /gas ]
[ boundary ] [ axis ] [ clear ]
[ symb=<n> ]
[ x.max=<n> ] [ x.min=<n> ] [ y.max=<n> ] [ y.min=<n> ]
[ arclength ]
```

7.55.2 Description

The plot.1d statement allows the user to plot cross sections vertically or horizontally through the device. The statement has options to provide for initialization of the graphics device and plotting of axes. The statement can optionally draw vertical lines whenever a material boundary is crossed. The vertical axis is the variable that was selected (see the select statement). The plot can have limits given to it so that only a portion of the entire device will be shown, or more than one variable can be conveniently plotted.

- **x.value, y.value**

These parameters specify that the cross section is to be done at a constant value as specified by the parameter. This parameter value is in microns. Only one of x.value or y.value can be specified for a given device. Specifying x.value will produce a vertical slice through the device and y.value will specify a horizontal slice. An easy way to remember is that the cross section is taken at the constant value specified. In one dimensional mode these parameters do not need to be specified.

- **silicon | oxide | nitride | - /silicon | /oxide | /nitride**

In addition to constant x or y cross sections, a one-dimensional plot can be generated along an interface. The interface lies between material 1, named without an initial $j^\circ/j_\pm$, and material 2, named with a leading $j^\circ/j_\pm$. For example, $j^\circ\text{plot.1d oxide /silicon}j_\pm$ shows the values in the oxide at the oxide/silicon interface. Thus $j^\circ\text{plot.1d oxide /silicon}j_\pm$ will usually show something different from $j^\circ\text{plot.1d silicon /oxide}j_\pm$. The backside, reflecting or exposed surfaces of a material can be specified with the appropriate parameter. The ordinate on the axis is the x coordinate of the points along the interface. If the optional parameter arclength is set, the ordinate is instead the arc length along the boundary from the leftmost point on the curve. The leftmost point itself has ordinate equal to its x coordinate in the mesh.

- **boundary**

This parameter specifies that any material boundaries that are crossed should be drawn in as vertical lines on the plot. It defaults false.

- **axis**

This parameter specifies that the axes should be drawn. If this parameter is set false and clear is false, the plot will be drawn on a top of the previous axes. It defaults true.

- **clear**

The clear parameter specifies whether the graphics screen should be cleared before the graph is drawn. If true, the screen is cleared. It defaults true.

- **clear**

The clear parameter specifies whether the graphics screen should be cleared before the graph is drawn. If true, the screen is cleared. It defaults true.

- **symb**

This parameter allows the user to specify a symbol type to be drawn on the cross sectional line. Each point is drawn with the specified symbol. symb = 0 means no symbol should be drawn. It defaults to 0. symb = 1 and higher will add one of the predefined symbols to the plot.

- **x.max, x.min**

These parameters allow the user to specify the range of the x axis. The units are in microns and the default is the length of the device in the appropriate direction of the cross section.

- **y.max, y.min**

These parameters allow the user to specify a range for the plot variable. The values have to be in units of the selected variable. If log of the boron concentration was selected, the values for this should be 16 and 10 to see the concentrations between 10^{16} and 10^{10} .

- **arclength**

If this parameter is set, the ordinate is the arc length along the boundary from the leftmost point on the curve. The leftmost point itself has ordinate equal to its x coordinate in the mesh. This parameter only has meaning when used with a plot along an interface.

7.55.3 Examples

`plot.1d x.v=1.0 symb=1 axis clear`

This command will clear the screen, draw a set of axes, and draw the data cross section at $x = 1.0 \mu\text{m}$. Each point will be drawn with symbol 1.

`plot.1d x.v=2.0 axis=f clear=f`

This command draws a cross section at $x = 2.0 \mu\text{m}$ on the previous set of axes.

`plot.1d sil /oxi`

Plots the selected variable along the silicon interface facing oxide.

7.56 plot.2d

7.56.1 Synopsis

```
plot.2d  
[ x.max=<n> ] [ x.min=<n> ] [ y.min=<n> ] [ y.max=<n> ]  
[ clear ] [ fill ] [ boundary ] [ grid ] [ axis ]  
[ vornoi ] [ diamonds ]  
[ stress ] [ vmax=<n> ] [ vleng=<n> ]  
[ flow ]
```

7.56.2 Description

The plot.2d statement allows the user to prepare a two dimensional xy plot. This routine does not plot any solution values, however, it can prepare the screen for isoconcentration lines to be plotted. The statement can draw the xy space of the device, label it, draw material boundaries, and draw stress and flow vectors. If in one dimensional mode, it will return and print an error message.

- **x.min, x.max**

These parameters allow the user to specify a subset of the x axis to be plotted. The parameters will default to the current device limits. The units are in microns.

- **y.min, y.max**

These parameters allow the user to specify a subset of the y axis to be plotted. The parameters will default to the current device limits. The units are in microns.

- **clear**

The clear parameter specifies whether the graphics screen should be cleared before the graph is drawn. If true, the screen is cleared. It defaults to true.

- **fill**

The fill parameter specifies that the device should be drawn with the proper aspect ratio. If fill is false, the device will be drawn with the proper aspect ratio. When true, the device will be expanded to fill the screen. It defaults to false.

- **boundary**

This parameter specifies that the device outline and material interfaces should be drawn. They will be drawn with the color specified by the line.bound parameter. It defaults to off.

- **grid**

The grid parameter specifies that the numerical grid the problem was solved on should be drawn. The color will be with the color will be with the line.grid parameter. It defaults to off.

- **axis**

The axis parameter controls the drawing and labeling of axes around the plot. If axis is true, the graph will be drawn with labeled axes. This parameter defaults true.

- **line.grid**

The line.grid parameter controls the line type or color that the axes and grid will be drawn in. The parameter defaults to 1.

- **line.bound**

This parameter specifies the line color to be used for the material boundaries. If boundary is not specified, this parameter is unimportant.

- **vornoi**

This parameter specifies that the vornoi tessellation to be drawn.

- **diamonds**

This option will cause the plot to draw small diamonds out at each point location. Actually crosses are now plotted but the parameter name has not yet been changed, just to confuse the opposition.

- **stress**

This parameter plots the principal stresses throughout the structure. Vectors are drawn along the two principal axes of the stress tensor at each point.

- **vleng**

The length of the vectors is scaled so that the maximum stress has length vleng, specified in microns.

- **line.com, line.ten**

The sign of the stress is indicated by the color of the vectors drawn. line.com is the color for compressive stress, line.ten is the color for tensile stress.

- **vmax**

If vmax is specified (in dynes/cm²), that stress will have length vleng, and higher stresses will be omitted.

- **flow**

Plots little arrows representing the velocity at each point. The vleng and vmax parameters help out here too.

7.56.3 Examples

plot.2d grid

This command will draw the triangular grid and axes.

plot.2d bound x.min=2 x.max=5 clear=false

This command will draw the material interfaces and x axis between 2.0 and 5.0 μm and will not clear the screen first.

plot.2d bound line.bound=2 diamonds y.max=5 axis=false

This command will draw the material interfaces with a maximum value of y equal to 5.0 μm . The boundaries will be drawn with line type 2, no axes, and will draw the grid points.

7.57 print.1d

7.57.1 Synopsis

```
print.1d
[ ( x.value=<n> | y.value=<n> ) ]
[ silicon | oxide | nitride ]
[ /exposed | /backside | /reflect | /silicon | /oxide | /nitride ]
[ arclength ] [ layers ] [ x.min=<n> ] [ x.max=<n> ]
[ format=<string> ]
```

7.57.2 Description

The print.1d command allows the user to print the values along cross sections through the device. It is also possible to integrate along a specified line. The value printed is the value that has been selected, see select.

- **x.value, y.value**

These parameters specify that the cross section is to be done at a constant value as specified by the value of the parameter. x.value specifies a vertical cross section of the device, and y.value a horizontal slice. The parameter value is in microns. Only one of x.value or y.value can be

specified for a given device. In one dimensional mode, it is not necessary to specify a position.

- **silicon | oxide | nitride**

In addition to constant x or y cross sections, a one-dimensional print can be specified along one side of an interface. The interface lies between material 1, named without an initial / and material 2, named with a leading /. For example, `print.1d oxide /silicon`. Thus `print.1d oxide /silicon` will usually show something different from `print.1d silicon /oxide`. The backside, reflecting or exposed surfaces of a material can be specified with the appropriate parameter.

- **arclength**

This parameter only has meaning for printing along the interface. The ordinate printed is the arc length along the boundary from the leftmost point on the curve, in microns, if `arclength` is chosen. Otherwise the x value of the interface location is printed. The leftmost point has ordinate equal to its x coordinate in the mesh.

- **layers**

This option instructs that the selected plot variable should be integrated in each material that is crossed. The integrated value and material width is reported. Zero crossings of the variable are treated the same as material interfaces. This option imitates the SUPREM-III command, `i°print layersj±`, and is probably most useful when doping is the selected variable.

- **x.min, x.max**

These parameters allow the user to specify the limits of the print region. Only values between these two will be displayed.

- **format**

This allows the user to change the print format for the variable. It uses standard C format expressions (e.g. 8.2f), minus the unfamiliar with the C programming language, it might be best to not experiment.

7.57.3 Examples

```
print.1d x.val=1.0 x.max=3.0
```

This command will print the selected value at x equal to 1um between the top of the mesh and the 3.0um point.

```
print.1d y.val=0 layers
```

This will print the integrated value selected in each material layer crossed by a horizontal slice at depth of 0.0.

```
print.1d sil /oxi
```

This will print the selected variable along the silicon side of the silicon oxide interface.

7.58 profile

7.58.1 Synopsis

```
profile
infile=<string>
( antimony | arsenic | boron | phosphorus | gallium | interstitial | vacancy | beryllium | carbon
| germanium | selenium | isilicon | tin | magnesium | zinc | indium | ni.impurity | al.impurity |
generic | iigeneric | iiigeneric | ivgeneric | vgeneric) [ offset=<n> ]
```

7.58.2 Description

The profile allows the user to specify a file of one dimensional doping data. This command allows the user to read data from another program, e.g. SUPREM-III, or measured data. The new doping is added to any current doping that may exist. The file should have two columns of numbers, depth and concentration. The file should have the depth in microns, and the concentration in cm⁻³. Any lines which do contain two values are ignored as comments. A grid must be present for this command.

- **infile**

The infile parameter selects the file name for data.

- **antimony, arsenic, boron, phosphorus, interstitial, vacancy, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, indium, ni.impurity, al.impurity, generic, iigeneric, iiigeneric, ivgeneric | vgeneric**

These parameters specify the type of impurity that forms the doping.

- **offset**

The amount of displacement added to the data.

7.58.3 Examples

```
profile infile=foo phos
```

This command reads in a doping file named foo, containing a phosphorus doping profile. The file foo, for example, could look like:

#pulse doped buried phosphorus profile:

```
0.2    1.0e5
0.21   1.0e19
0.30   1.0e19
0.31   1.0e5
```

Any area outside the specified doping layer is unchanged.

7.59 quit

7.59.1 Synopsis

quit

7.59.2 Description

This command is used to interrupt the example at present location

7.59.3 Examples

quit

7.60 rectangle_deposit_mesh

7.60.1 Synopsis

```
rectangle_deposit_mesh
(x.direction | y.direction | z.direction)
[location=<n> (default=0.0)] [spacing=<n> (default=-999.0)]
[interval.r=<n> (default=1.5)]
```

7.60.2 Description

This command is used to set user defined mesh used for deposit. refer to rectangle_deposit_method for details. rectangle_deposit_method needs to be used to turn on the user defined mesh function.

- **x.direction, y.direction, z.direction**

set x.direction for x direction, y.direction as y direction and z.direction as z direction.

- **location**

set the position of the coordinates

- **spacing** set the spacing for the point coordinates. Refer to line command for details about how to use spacing.

- **interval.r** The interval ratio for the point coordinates. Refer to line command for details about how to use interval.r

7.60.3 Examples

```
rectangle_deposit_mesh x location=0 spacing=0.05
rectangle_deposit_mesh x location=6 spacing=0.05
rectangle_deposit_mesh y location=3 spacing=0.05
rectangle_deposit_mesh y location=0 spacing=0.05
```

The commands above defined a rectangular shape whose x location is 0,6 and y location is 3,0

7.61 rectangle_deposit_method

7.61.1 Synopsis

```
rectangle_deposit_method
[x.auto=<n> (default=true)] [y.auto=<n> (default=true)]
[min_delta_x=<n> (default=0.0)]
```

7.61.2 Description

rectangle.based is a new function for deposit. (Use option rectangle.based to turn on this function). rectangle.based will initialize a normal rectangular mesh, and then etch away this rectangular shape, the residue of the mesh after etching will be used for deposit.

The initial normal mesh play an important role for the final deposit layer. Before deposit, surface line needs to be found and arranged according to x coordinate, and then extends upwards vertically. Horizontal lines will also be created in the y direction. The lines cross over and form initial rectangular mesh. Sometimes the mesh points on the surface lines are too dense, which caused smaller distance between points in x direction, This command is used to control the mesh points in the x direction so that a more reasonable initial rectangular mesh

will be formed.

This command should be used in conjunction with `rectangle_deposit_mesh`, `rectangle_deposit_method` will turn on the function that lets users define the mesh, while the `rectangle_deposit_mesh` command needs to be used to define the mesh in x and y direction.

- **x.auto**

Whether or not to use auto control in the x direction.

- **y.auto**

Whether or not to use auto control in the y direction.

- **min_delta_x**

If the space between two adjacent points is less than `min_delta_x` in the x direction, then it will be set as `min_delta_x`. In order to apply this function, one must set `x.auto=false`.

7.61.3 Examples

```
rectangle_deposit_method x.auto=false min_delta_x=0.01
```

This command assigns 0.01 to the initial mesh if the space between two points in the x direction is less than 0.01.

7.62 region

7.62.1 Synopsis

region

(silicon | oxide | nitride | poly | gas | oxynitr | photores | aluminum | gaas | AlGaAs | InP | AlInGaAs | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)

xlo = <string> ylo = <string> xhi = <string> yhi = <string>

[conc=<n>]

(arsenic | phosphorus | boron | gallium | antimony | gold | beryllium | magnesium | selenium | isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric | vgeneric | indium | ni.impurity | al.impurity)

7.62.2 Description

This statement is used to specify the material of rectangles in a rectangular mesh. Region statements should follow line statements. Every element must be given some material, so at

least one region statement is required for each rectangular mesh.

- **silicon, oxide, nitride, poly, gas, oxynitr, photores, aluminum, gaas**

The material being specified.

- **xlo, ylo, xhi, yhi**

The bounds of the rectangle being specified. The <string> value should be one of the tags created in a preceding line statement.

- **conc**

Set doping concentration

- **resistivity**

Parameter resistivity has been used in the following commands to replace the doping concentration:

```
region
deposit
fill_hole
```

This is based on known relation between doping and resistivity for some well known materials such as silicon and GaAs. For details, see example:

<http://inside.mines.edu/~sagarwal/phgn435/images/resistivity.jpg>

We plot the resistivity versus doping for n-type and p-type doping for silicon in the figure above. This has been implemented for phosphorous and boron in silicon.set doping concentrations.

- **arsenic | phosphorus | boron | gallium | antimony | gold | beryllium | magnesium | selenium | isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric | vgeneric | indium | ni.impurity | al.impurity**

Set dopant type

7.62.3 Examples

```
region silicon xlo=left xhi=right ylo=surf yhi=back
```

This line would make the entire mesh silicon (from the example on the line page).

7.63 regrid

7.63.1 Synopsis

```

regrid
( silicon | oxide | nitride | poly | gas | oxynitr | photores | aluminum | gaas | AlGaAs | InP |
AlInGaAs | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial (default=silicon))
[repair.mesh.allsteps=<logical>(default=true)]
[log10.change=<n>(default=10.0)]
[oxide.repair=<logical>(default=true)]
[etch.repair = <n> (default=true)]
[deposit.repair = <n> (default=true)]
[optimize.mesh = <n> (default=false)]
[segm = <n> (default=-1)]
[delta.doping = <n> (default=1e15)]
[ratio = <n> (default=10)]
[max.increase = <n> (default=2)]
[max.space = <n> (default=-50)]
[doping=<logical>]
(refine | reduce | auto)
[max\_mesh\_number=<n> (default=100.0)]
[min\_dopant\_change=<n> (default=10.0)]

```

7.63.2 Description

This command is used to control the mesh quality after etching, deposition, and oxidation. It does apply some meshing algorithms to repair the original mesh of suprem4gs. It also uses the doping profile to optimize the shape of some elements. The mesh repair parameters are all turned on by default. But the user may choose to turn them off if it's desired to speed up the processing steps.

- **log10.change:**

If this parameter is turned on then the mesh is densified at pn junctions according to doping concentration changes. This is the primary method of regrid.

- **repair.mesh.allsteps:**

If this parameter is turned on then the mesh repair is called after each step.

- **oxide.repair:**

This parameter will enable the mesh repair utility after oxidation step if it is turned on.

- **etch.repair:**

This parameter allows the mesh repair after etching step.

- **deposit.repair:**

This parameter allows the mesh repair after deposition step.

- **optimize.mesh:**

This parameter when turned on will provide the mesh adaptation using the doping profile.

- **delta.doping:**

This parameter is used to set up the doping value that will be used to optimize the mesh.

- **max.space**

This parameter can be used to set up the maximum spacing.

- **segm:**

This parameter is used to set up the segment index in 3d of the segment that will be repaired.

- **silicon | oxide | nitride | poly | gas | oxynitr | photores | aluminum | gaas | AlGaAs | InP | AllnGaAs | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial:**

This parameter is used to set up the material name that will be affected by the repair mesh tool. Silicon is the default.

- **doping**

This command determines whether it is necessary to deal with the doping of the mesh points in the specified area. If no action is necessary, the three parameters: "(refine | reduce | auto), max_mesh_number,min_dopant_change" are not going to be effective.

- **refine | reduce | auto**

The three modes to deal with the mesh points within specified area. refine: densify the mesh, reduce: relax the mesh, auto: automatic mode.

- **max_mesh_number**

The maximum mesh number after regrid in the specific area.

- **min_dopant_change**

In the area where doping needs to be dealt with, if the ratio of doping at certain point to its adjacent point is greater than the value specified by min_dopant_change, then the point will be kept and no action will be taken.

- **ratio**

Increase the mesh points when variations of doping is greater than that specified in ratio.

- **max.increase**

The upper limit of the ratio of the mesh points before densification and after densification.

- **delta.doping**

When doping value is less than delta.doping, no densification needed.

7.63.3 Examples

```
regrid log10.change=5 refine
```

This command is used to densify the mesh according to the variation of the doping.

7.64 restart

7.64.1 Synopsis

```
restart file=<string>
```

7.64.2 Description

This command is used to force the program flow to restart from a previously saved structure within an input deck. The file can be saved using the command **structure**.

- **file**

It is a previously saved structure file.

7.64.3 Examples

```
restart file=step23.str
```

This command would cause the program to jump directly to "struct outf=step23.str" and start the process below it. Please note that for 2D simulation, this command should be placed at the beginning of the input file after symbolic definitions model parameter override.

For 3D or quasi3d simulation, since the program must load the plane z-dimensions, the **restart** command must be placed right after the **3d_mesh** statement.

7.65 select

7.65.1 Synopsis

```
select
```

7.65.2 Description

Select the plot variable for the post-processing routines. The select statement is used to specify the variable for display in the contour statement, plot.1d statement, print.1d statement, and plot.2d statement. In short, all of the post-processing commands. No data will be available for any of these statements unless this statement is specified first. Only one variable can be selected at any one time and each select statement overrides any previous statements.

- **z**

This parameter accepts a vector expression of several different vector variables for the z parameter. The operators *, /, +, -, all work in the expected way. The vector variables are listed below:

al.impurity	aluminum concentration
antimony	antimony concentration
arsenic	arsenic concentration
beryllium	beryllium concentration
bf2	bf2 concentration
boron	boron concentration
carbon	carbon concentration
cesium	cesium concentration
phosphorus	phosphorus concentration
gallium	gallium concentration
magnesium	magnesium concentration
selenium	selenium concentration
isilicon	isilicon concentration
tin	tin concentration
germanium	germanium concentration
zinc	zi concentration
generic	generic concentration
iigeneric	iigeneric concentration
iiigeneric	iiigeneric concentration
ivgeneric	ivgeneric concentration
vgeneric	vgeneric concentration
indium	indium concentration
ni.impurity	nitrogen concentration
ci.star	equilibrium interstitial concentration
cv.star	equilibrium vacancies concentration
doping	net active concentration
electrons	electron concentration
germanium	germanium

generic	generic impurity concentration
gold	gold concentration
interstitial	interstitial concentration
isilicon	silicon impurity concentration
magnesium	magnesium concentration
ni	intrinsic electron concentration
oxygen	oxygen concentration
phosphorus	phosphorus concentration
psi	potential
selenium	selenium concentration
Sxx, Sxy, Syy	components of stress in rectangular coordinates
tin	tin concentration
trap	unfilled interstitial trap concentration
vacancy	vacancy concentration
x	x coordinates
x.v	x velocity
y	y coordinates
y.v	y velocity
zinc	zinc concentration

Many of these are self explanatory. Potential is computed using charge neutrality. The electron concentration is computed from the potential using Boltzmann statistics. There also several function that are available for the user. These are:

abs	absolute value
active	active portion of the specified dopant
erf	error function
erfc	complimentary error function
exp	exponential
gradx	differentiates the argument with respect to x
grady	differentiates the argument with respect to y
log	logarithm log10 logarithm base 10 scale scales the value given by the maximum value
sqrt	square root

- **label**

This is the string that appears on the y axis in a 1D plot. The default is the select string.

- **title**

This is the string printed in large letters across the top of the plot. Default is the version number of the program.

- **temp**

The temperature at which expressions are evaluated. It defaults to the last temperature of diffusion.

7.65.3 Examples

`select z=log10(arsen)`

Choose as the plot variable the base 10 logarithm of the arsenic concentration.

7.66 selenium

7.66.1 Synopsis

selenium

(silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)

[Dix.0=<n>] [Dix.E=<n>] [Dim.0=<n>] [Dim.E=<n>]

[Dimm.0=<n>] [Dimm.E=<n>] [Fi = <n>]

[implanted] [grown.in]

[ss.clear] [ss.temp=<n>] [ss.conc=<n>]

[(/silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial)]

[Seg.0=<n>] [Seg.E=<n>] [Trn.0=<n>] [Trn.E=<n>]

[(donor | acceptor)]

7.66.2 Description

This statement allows the user to specify values for coefficients of selenium diffusion and segregation. The diffusion equation for selenium is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.76$$

where C_T and C_A are the total chemical and active concentrations of selenium, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i

refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.77$$

$$D_I = F_i \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.78$$

D_X , D_M and D_{MM} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.79$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the selenium diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0. cm²/sec in silicon, and Dix.E defaults to 0.0 eV in silicon. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the selenium diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 0.0 cm²/sec in silicon, and Dim.E defaults to 0.0 eV in silicon. D_M is calculated using a standard Arrhenius relationship.

- **Dimm.0, Dimm.E**

These floating point parameters allow the specification of the selenium diffusing with doubly negative defects. Dimm.0 is the pre-exponential constant and Dimm.E is the activation energy. For implanted selenium in gallium arsenide Dimm.0 defaults to 0.0 cm²/sec and Dimm.E defaults to 0.0 eV. For grown-in selenium in gallium arsenide Dimm.0 defaults to 1.2 cm²/sec and Dimm.E defaults to 3.6 eV [1,2,3]. DMM is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether selenium diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.0.

- **implanted, grown.in**

Specifies whether the Dix , Dim , Dimm, or Fi coefficients apply to implanted or grown-in selenium. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default selenium is a donor.

7.66.3 Examples

selenium gaas implanted Dimm.0=2.1e-3 Dimm.E=3.1

This command changes the doubly negative defect diffusivity for implanted selenium in silicon.

`selenium gaas /nitride Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the gallium arsenide C silicon nitride interface. The gallium arsenide concentration will be 30.0 times the nitride concentration in equilibrium.

7.66.4 References

1. J.D. Plummer et al., ``Process Simulators for Si VLSI and High Speed GaAs Devices," Stanford University Technical Report, 1990.
2. J.D. Plummer et al., ``Process Simulators for Si VLSI and High Speed GaAs Devices," Stanford University Technical Report, 1991.
3. M.D. Deal, S.E. Hansen, and T.W. Sigmon, ``SUPREM 3.5 ``C Process Modeling of GaAs Integrated Circuit Technology," IEEE Trans. CAD, 9, p. 939, 1989.

7.67 set_etch_geometry

7.67.1 Synopsis

```
set_etch_geometry
[profile_type=<n> (default=1)] [x_length=<n>] [vertical_rate=<n>]
[vertical_time=<n>] [left_angle=<n>] [right_angle=<n>] [straight_spac=<n>]
[isotropic_rate=<n>] [isotropic_time=<n>] [arc_spac=<n>] [arc_height=<n>]
[arc_width=<n>] [arc_upper_type=<n>] [arc_mesh_spac=<n>] [tag=<string>]
```

7.67.2 Description

This command is mainly used for setting parameters for etch_geometry.exe. During etching, the geometries defined by these parameters can be called upon. Refer to 3.7.5 Auxiliary etch program for more detail about how to use the command etch_geometry.exe. Specially attention should be paid to the variable "tag". This variable is used to identify the profiles created by etch_geometry.exe. If the profiles needed to be implemented during etching, etch_tag_etch_geometry=label needs to be used.

7.67.3 Examples

```
set_etch_geometry profile_type=* x_length=* ... tag=etch_label1
etch ... tag_etch_geometry=etch_label1
```

The example above first defined an etch profile by using `set_etch_geometry`, note that the profile is labeled as `etch_label1`. If the profile needs to be applied during etching, `ta_etch_geometry = etch_label1` needs to be used to find the previously defined `etch_label1`.

7.68 smooth_interface

7.68.1 Synopsis

```
smooth_interface
[factorial=<n> (default=6)]
[x1=<n> (default=-9999.0)] [x2=<n> (default=9999.0)]
[top_interface=<logical> (default=true)]
[bottom_interface=<logical> (default=true)]
[fix_end_points=<logical>]
(silicon | oxide | oxynitride | nitride | poly | gaas | imaterial | iimaterial | iimaterial |
ivmaterial | vmaterial)
(/silicon | /oxide | /oxynitride | /nitride | /poly | /gaas | /imaterial | /iimaterial | /iimaterial |
/ivmaterial | /vmaterial)
```

7.68.2 Description

This command uses polynomial interpolating function to smooth out the interface of two materials

- **factorial**

Set the factorial of the polynomial interpolating function.

- **x1**

Set the start point for x coordinate.

- **x2**

Set the end point for x coordinate.

- **top_interface**

Smooth the top interface.

- **bottom_interface**

Smooth the bottom interface.

- **fix_end_points**

Whether to smooth the end point or not. In the polynomial interpolating function, whether to smooth the end points is a boundary condition, the end points will affect the profile shape after smoothing.

- **silicon, oxide, oxynitride, nitride, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

Set the interface material 1.

(/silicon | /oxide | /oxynitride | /nitride | /poly | /gaas | /imaterial | /iimaterial | /iimaterial | /ivmaterial | /vmaterial) Set the interface material 2

7.68.3 Examples

`smooth_interface silicon /oxide x1=0.0 x2=1.0 fix_end_points=true top_interface=true`

This command is to smooth the top interface of silicon and oxide from x=0 to x=1. It also smooth the end points of the interface.

7.69 stress

7.69.1 Synopsis

`stress [ temp1=<n> temp2=<n> ] [ nel=<n> ]`

7.69.2 Description

Calculate elastic stresses.

- **temp1, temp2**

These are the initial and final temperatures for calculating thermal mismatch stresses.

- **nel**

This is the number of nodes per triangle to use. Currently only 6 or 7 are allowed. 6 nodes is faster than 7 and usually gives adequate results, so 6 is the default.

7.69.3 Examples

material nitride intrin.sig=1.4e10 stress

This calculates the stresses in the substrate and film arising from a nitride layer which has an intrinsic stress of 1.4×10^{10} when deposited uniformly.

7.70 structure

7.70.1 Synopsis

```
structure
[ (infile=<string> | outfile=<string>) ]
[ show ] [ backside.y=<n> ]
[ mirror ] [ left ] [ right ] [ top ] [ bottom ]
[ imagetool=<string> ]
[ x.min=<n> | x.max=<n> | y.min=<n> | y.max=<n> | z.min=<n> | z.max=<n> | pixelx=<n> |
pixely=<n> | nxfac=<n> | nyfac=<n> | mode=<n> ]
[ simpl=<string> ] [ header=<string> ]
[ segm=<n> (default=-1)]
[ prseg2d=<logical> (default=false)]
[ flip.y=<logical> (default=false) ]
```

7.70.2 Description

Read/write the mesh and solution information. The structure statement allows the user to read and write the entire mesh and solution set. The data saved is from the current set of solution and impurity values.

- **infile, outfile**

These parameters specify the name of the file to be read or written. An existing file will be overwritten.

- **show**

The numbering of electrodes is displayed by plotting the structure and labeling each electrode node with its electrode number.

- **backside.y**

Used in conjunction with the pisces parameter, the substrate (assumed to be silicon) will be etched from the location specified by the argument to backside.y to the bottom of the wafer. A backside contact extending across the substrate in the x-direction will be placed at this y-value.

- **mirror left, mirror right**

Mirrors the grid around its left or right boundary. Useful for turning half-a- MOSFET simulations into PISCES grids. The default is to reflect around the right axis.

- **imagetool**

This string parameter specifies the name of the binary raster file that can be used by NCSA programs such as XImage. The entire structure will be plotted unless clipped by the window parameters x.min, x.max, y.min, and y.max. In addition the size of the raster image is controlled by the parameters pixelx and pixely. Z values are derived using the select command. All z values will be plotted unless clipped with z.min and z.max. For details on the parameters associated with imagetool files, see below.

- **x.min**

This float parameter is minimum x-value in microns used in generating the raster image. x.min defaults to the left edge of the entire structure.

- **x.max**

This float parameter is maximum x-value in microns used in generating the raster image. x.max defaults to the right edge of the entire structure.

- **y.min**

This float parameter is minimum y-value in microns used in generating the raster image. y.min defaults to the top edge of the entire structure.

- **y.max**

This float parameter is maximum y-value in microns used in generating the raster image. y.max defaults to the bottom edge of the entire structure.

- **z.min**

This float parameter is the minimum z-value used for generating the raster image. The z array is created using the select command. If the actual z-value is less than z.min, the value will be given the same color as z.min.

- **z.max**

This float parameter is the maximum z-value used for generating the raster image. The z array is created using the select command. If the actual z-value is greater than z.max, the value will be given the same color as z.max.

- **pixelx**

This integer parameter is the number of pixels in the x-direction for the raster image. The default value is 400 pixels.

- **pixely**

This integer parameter is the number of pixels in the y-direction for the raster image. The default value is 200 pixels.

- **nxfac**

This integer parameter is the amount of linear interpolation in the x-direction to be done on the rectangular grid that will be used to create the raster image. The default value of 1 gives the most resolution at the cost of higher CPU usage. A higher value will give a slightly coarser picture but will take less CPU time.

- **nyfac**

This integer parameter is the amount of linear interpolation in the y-direction to be done on the rectangular grid that will be used to create the raster image. The default value of 1 gives the most resolution at the cost of higher CPU usage. A higher value will give a slightly coarser picture but will take less CPU time.

- **mode**

This integer parameter is a special flag (0 or 1) useful for plotting doping concentrations. It assumes that the follow select statement was issued: `select z=sign(bor-phos-ars)*log10(abs(bor-phos-ars))` The colors are then mapped between -14 and 14. This compresses the color spectrum to allow more colors in the region of interest.

- **clear**

This integer parameter clears a global counter which counts frame numbers in association with the movie parameter of the diffuse command.

- **count**

This integer parameter increments the global counter. When used with the movie parameter of the diffuse command, a sequence of image files can be generated whose last 3 characters are numbers. Files in this type of sequence can be easily animated using NCSA XImage.

- **simpl**

This string parameter generates a file in the SIMPL-2 format. This is necessary to incorporate SUPREM-IV information into the SIMPL-IPX system. Another parameter, header, must also be specified.

- **header**

This string parameter is the name of the file used to describe the cross section to be described. This is the first few lines from a SIMPL-2 file. Usually these files are automatically generated when using SUPREM-IV within the SIMPL-IPX environment.

- **segm**

In 3D simulation, if certain segment data needs individual output, then this variable needs to be specified.

- **prseg2d**

This parameter specifies whether only certain segment will be the output. In 3D simulation, if only certain segment needs to be the output, the parameter segm needs to be specified. preseg2d also needs to be set to true.

- **flip.y**

if you need to change the y coordinate to -y in the mesh, this parameter needs to be specified.

7.70.3 Examples

`structure infile=mystructure.str`

This command reads in a previously saved structure in file mystructure.str.

7.71 tin

7.71.1 Synopsis

tin

```
( silicon | oxide | oxynit | nitride | gas | poly | gaas | imaterial | iimaterial | iimaterial |
ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ] [ Dim.0=<n> ] [ Dim.E=<n> ]
[ Dimm.0=<n> ] [ Dimm.E=<n> ] [ Fi = <n> ]
[ implanted ] [ grown.in ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ]
[ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial) ]
[ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]
```

7.71.2 Description

This statement allows the user to specify values for coefficients of tin diffusion and segregation. The diffusion equation for tin is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V}{C_V^*} \frac{n}{n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I}{C_I^*} \frac{n}{n_i} \right) \right] \quad 7.80$$

where C_T and C_A are the total chemical and active concentrations of tin, C_V and C_I are the

vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and n and n_i refer to the electron concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.81$$

$$D_I = F_i \left[D_X + D_M \frac{n}{n_i} + D_{MM} \left(\frac{n}{n_i} \right)^2 \right] \quad 7.82$$

D_X , D_M and D_{MM} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12}} T_r \left(C_1 - \frac{C_2}{M_{12}} \right) \quad 7.83$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of the tin diffusing with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0. cm²/sec in silicon, and Dix.E defaults to 0.0 eV in silicon. D_X is calculated using a standard Arrhenius relationship.

- **Dim.0, Dim.E**

These floating point parameters allow the specification of the tin diffusing with singly negative defects. Dim.0 is the pre-exponential constant and Dim.E is the activation energy. Dim.0 defaults to 0.0 cm²/sec in silicon, and Dim.E defaults to 0.0 eV in silicon. D_M is calculated using a standard Arrhenius relationship.

- **Dimm.0, Dimm.E**

These floating point parameters allow the specification of the tin diffusing with doubly negative defects. Dimm.0 is the pre-exponential constant and Dimm.E is the activation energy. Dimm.0 defaults to 96. × cm²/sec in gallium arsenide, and Dimm.E defaults to 3.9 eV in gallium arsenide [1,2,3]. D_{MM} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether tin diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 0.0.

- **implanted, grown.in**

Specifies whether the Dix , Dim , Dimm, or Fi coefficients apply to implanted or grown-in tin. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default tin is a donor.

7.71.3 Examples

tin gaas implanted Dimm.0=2.1e-3 Dimm.E=3.1

This command changes the doubly negative defect diffusivity for implanted tin in silicon.

`tin gaas /nitride Seg.0=30.0 Trn.0=1.66e-7 Seg.E=0.0`

This command will change the segregation parameters at the gallium arsenide "C silicon nitride interface. The gallium arsenide concentration will be 30.0 times the nitride concentration in equilibrium.

7.71.4 References

1. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1990.
2. J.D. Plummer et al., Process Simulators for Si VLSI and High Speed GaAs Devices, Stanford University Technical Report, 1991.
3. E.L. Allen, J.J. Murray, M.D. Deal, J.D. Plummer, K.S.Jones, and W.S. Robert, Diffusion of Ion-Implanted Group IV Dopants in GaAs, J. Electrochem. Soc., 138, p. 3440, 1991.

7.72 trap

7.72.1 Synopsis

```
trap
( silicon | oxide | poly | oxynitr | nitride | gas | aluminum | photores | gaas | imaterial |
iimaterial | iimaterial | ivmaterial | vmaterial)
[ enable ]
[ total=<n> ]
[ frac.0=<n> ] [ frac.E=<n> ]
```

7.72.2 Description

Set coefficients of interstitial traps. This statement allows the user to specify values for coefficients of the interstitial traps. The statement allows coefficients to be specified for each of the materials. SUPREM-IV has default values only for silicon. Polysilicon has not been characterized as extensively, and its parameters default to those for silicon. The trap reaction was described by Griffin[1]. This model explains some of the wide variety of diffusion coefficients extracted from several different experimental conditions. The trap equation is:

$$\frac{\partial C_{ET}}{\partial t} = -K_T \left[C_{ET} C_I - \frac{e^*}{1-e^*} C_I^* (C_T - C_{ET}) \right] \quad 7.84$$

where C_T is the total trap concentration, K_T is the trap reaction coefficient, e^* is the equilibrium trap occupancy ratio. Instead of the reaction, the time derivative is used in the interstitial equation because it has better properties in the numerical calculation. The equation is derived from a simple interaction of an interstitial and a trap. To achieve balance in equilibrium, e^* is defined to be C_{ET}/C_T , the equilibrium number of empty traps divided by the total traps.

- **silicon, oxide, poly, oxynitr, nitride, gas, aluminum, photores, gaas, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

These parameters allow the specification of the material for which the parameters apply.

- **enable**

This indicates that material specified has a finite number of traps.

- **total**

This floating point parameter is used to specify the total number of traps, in cm^{-3} . The default for silicon is $2 \times 10^{18} \text{cm}^{-3}$. This value is appropriate for Czochralski silicon material. `frac.0, frac.E` These floating point parameters allow the specification of the equilibrium empty trap ratio, e^* . The default for these parameters is $2.39 \times 10^5 \text{cm}^{-3}$ and 1.57 eV for the pre-exponential and activation energy for silicon.

7.72.3 Examples

`trap silicon total=2.0e18 frac.0=0.5 frac.E=0.0 enable`

This statement turns on interstitial traps and sets the total to 2×10^{18} and the fraction to a half.

7.72.4 References

1. P.B. Griffin and J.D. Plummer, Process Physics Determining 2-D Impurity Profiles in VLSI Devices, International Electron Devices Meeting, Los Angeles, p. 522, 1986.

7.73 truncate

7.73.1 Synopsis

`truncate`

`[ segm=<n> left_to_x = <n> right_to_x = <n> below_y = <n> ]`

7.73.2 Description

This command is used to truncate the mesh of the device below, left or right to a certain point of the simulated structure.

- **left_to_x**

This would cause a whole structure left to this point to be truncated.

- **right_to_x**

This would cause a whole structure right to this point to be truncated.

- **below_y**

This would cause a whole structure below point to be truncated.

7.73.3 Examples

`truncate below_y=10`

This command would cause all mesh below y=10 micron to be removed from the simulation structure.

7.74 uniform_doping

7.74.1 Synopsis

```
uniform_doping
conc=<n> ylo=<n> yhi=<n> [ std.dev.ylo=<n> std.dev.yhi=<n> ]
( antimony | arsenic | boron | bf2 | cesium | phosphorus | beryllium | magnesium | selenium
| isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric |
vgeneric)
```

7.74.2 Description

This command is used to define a uniform doping region within a structure. The uniform doping can fall off with a Gaussian tail on each side of the uniform profile. It is commonly used to define an epi-layer.

- **conc**

This is the doping concentration in $1/cm^3$.

- **antimony, arsenic, boron, bf2, cesium, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic, iigeneric, iiigeneric, ivgeneric, vgeneric**

Select an impurity type.

- **ylo, yhi**

These are used to define the upper and lower ranges of the uniform profile.

- **std.dev.ylo, std.dev.yhi**

These are the Gaussian standard deviations for the top and bottom Gaussian tails, respectively.

- **impurity=logical**

This is the impurity being uniformly doped

7.74.3 Examples

`uniform_doping ylo=5 yhi=6`

This would define a doped epi-layer from 5 to 6 microns.

`uniform_doping ylo=5 yhi=6 std.dev.ylo=0.5`

This would add a Gaussian tail to the top of the epi-layer.

7.75 vacancy

7.75.1 Synopsis

`vacancy`

(silicon | oxide | poly | oxynitr | nitride | gaas | gas | aluminum | photoresist | imaterial | iimaterial | iiimaterial | ivmaterial | vmaterial)

(/silicon | /oxide | /poly | /oxynitr | /nitride | /gaas | /gas | /aluminum | /photoresist | /imaterial | /iimaterial | /iiimaterial | /ivmaterial | /vmaterial)

[D.0=<n>] [D.E=<n>]

[Kr.0=<n>] [Kr.E=<n>]

[Cstar.0=<n>] [Cstar.E=<n>]

[ktrap.0=<n>] [ktrap.E=<n>]

[neu.0=<n>] [neu.E=<n>]

[neg.0=<n>] [neg.E=<n>] [dneg.0=<n>] [dneg.E=<n>]

```

[ tneg.0=<n> ] [ tneg.E=<n> ]
[ pos.0=<n> ] [ pos.E=<n> ] [ dpos.0=<n> ] [ dpos.E=<n> ]
[ tpos.0=<n> ] [ tpos.E=<n> ]
(boron | gallium | antimony | arsenic | phosphorus | beryllium | magnesium | selenium |
isilicon | tin | germanium | zinc | carbon | generic | iigeneric | iiigeneric | ivgeneric | vgeneric
| indium | ni.impurity | al.impurity)
[ time.inj ] [ growth.inj ] [ recomb ]
[ Ksurf.0=<n> ] [ Ksurf.E=<n> ]
[ Krat.0=<n> ] [ Krat.E=<n> ]
[ Kpow.0=<n> ] [ Kpow.E=<n> ]
[ vmole=<n> ] [ theta.0=<n> ] [ theta.E=<n> ]
[ Gpow.0=<n> ] [ Gpow.E=<n> ]
[ A.0=<n> ] [ A.E=<n> ] [ t0.0=<n> ] [ t0.E=<n> ]
[ Tpow.0=<n> ] [ Tpow.E=<n> ]
[ rec.str=<s> ] [ inj.str=<s> ]

```

7.75.2 Description

This statement allows the user to specify values for coefficients of vacancy continuity equation. The statement allows coefficients to be specified for each of the materials. Of course, the meaning of a lattice vacancy in an amorphous region is a philosophical topic. SUPREM-IV has default values only for silicon, gallium arsenide, and the interfaces with silicon. Polysilicon has not been characterized as extensively, and its parameters default to those for silicon.

The vacancies obey a complex diffusion equation which can be written

$$\frac{\partial(C_V - C_{ET})}{\partial t} = \nabla(-J_V - \sum_{imp} J_{AV}) - R \quad 7.85$$

where C_V is the vacancy concentration, C_{ET} is the number of empty vacancy traps, J_V refers to the vacancy flux, J_{AV} refers to the flux of impurity A diffusing with vacancies, and R is all sources of bulk recombination of vacancies. In the two.dim model (see method) the fluxes from the dopants, J_{AV} , are ignored. Only in the full.cpl model are they computed and included in the vacancy equation. This model represents the diffusion of all vacancies, paired or unpaired, and can be derived from assuming thermal equilibrium between the species and that the simple pairing reactions are dominant [1].

The pair fluxes are the vacancy portions of the flux as described in each of the models for the impurities. (see antimony, arsenic, boron, and phosphorus) The vacancy flux can be written [1]:

$$-J_V = D_V C_V \nabla \frac{C_V}{C_V^*} \quad 7.86$$

It is important to note that the equilibrium concentration, C_V^* is a function of Fermi level [3]. This flux accounts correctly for the effect of an electric field on the charged portion of the defect concentration. The bulk recombination is simple interaction between vacancies and vacancies. This can be expressed:

$$R = K_R(C_I C_V - C_I^* C_V^*) \quad 7.87$$

where K_R is the bulk recombination coefficient, and C_V and C_V^* are the vacancy and vacancy equilibrium concentration, respectively.

The defects obey a flux balance boundary condition, as described by Hu [4]:

$$J_V \tilde{n} + K_V(C_V - C_V^*) = g \quad 7.88$$

where $\tilde{n}$ is the surface normal, K_V is the surface recombination constant, and g is the generation, if any, at the surface. This expression has been used with success on several different surface types.

- **silicon, oxide, poly, oxynitr, nitride, gaas, gas, aluminum, photoresist, imaterial, iimaterial, iiimaterial, ivmaterial, vmaterial**

These parameters allow the specification of the material for which the parameters apply.

- **/silicon, /oxide, /poly, /oxynitr, /nitride, /gaas, /gas, /aluminum, /photoresist, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters allow the specification of the interface for which the boundary condition parameters apply. For example, parameters about the oxide silicon effect on vacancies should be specified by listing silicon / oxide. Only one of these can be specified at a time.

- **boron, gallium, antimony, arsenic, phosphorus, beryllium, magnesium, selenium, isilicon, tin, germanium, zinc, carbon, generic, iigeneric, iiigeneric, ivgeneric, vgeneric, indium, ni.impurity, al.impurity**

This parameter sets an impurity variable.

- **D.0, D.E**

These floating point parameters are used to specify the diffusion coefficient of the vacancies. The units are in cm^2/sec . The default values are $6.34 \times 10^3 \text{cm}^2/\text{sec}$ and 3.29 eV [5] for silicon, and $1.0 \times 10^{-12} \text{cm}^2/\text{sec}$ and 0.0 eV for GaAs.

- **Kr.0, Kr.E**

These floating point parameters allow the specification of the bulk recombination rate in cm^3/sec . The default is 1.4 cm^3/sec and 3.99 eV [6] for silicon, and $1.0 \times 10^{-18} \text{cm}^3/\text{sec}$ and 0.0 eV for GaAs.

- **Cstar.0, Cstar.E**

These parameters allow the specification of the total equilibrium concentration of vacancies in

intrinsically doped conditions. The default values are $4.77 \times 10^{18} \text{ cm}^{-3}$ and 0.713 eV [5] for silicon, and $4.77 \times 10^{14} \text{ cm}^{-3}$ and 0.0 eV for GaAs.

- **neu.0, neu.E, neg.0, neg.E, dneg.0, dneg.E, tneg.0, tneg.E, pos.0, pos.E, dpos.0, dpos.E, tpos.0, tpos.E**

These values allow the user to specify the relative concentration of interstitials in the various charge states (neutral, negative, double negative, triple negative, positive, double positive, and triple positive) under intrinsic doping conditions. For example, in intrinsic conditions, the concentration of doubly negative vacancies is given by:

$$C_I^* = \frac{C_{star}.dneg}{neu+neg+dneg+tneg+pos+dpos+tpos} \quad 7.89$$

where C_{star} is the total equilibrium concentration in intrinsic doping conditions. In extrinsic conditions, the total equilibrium concentration is given by [3]:

$$C_I^* = C_{star} \frac{neu+neg\left(\frac{n}{n_i}\right)+dneg\left(\frac{n}{n_i}\right)^2+tneg\left(\frac{n}{n_i}\right)^3+pos\left(\frac{p}{n_i}\right)+dpos\left(\frac{p}{n_i}\right)^2+tpos\left(\frac{p}{n_i}\right)^3}{neu+neg+dneg+tneg+pos+dpos+tpos} \quad 7.90$$

There is very little data for the vacancy charge states; it is matter of ongoing research. The default values for Si are chosen based on an analysis of Giles [7]. See Table 4 in the section i°Adding SUPREM 3.5j's GaAs Models and Parameters to SUPREM-IVj± for GaAs values.

- **time.inj, growth.inj, recomb**

These parameters specify the type of reactions occurring at the specified interface. The time.inj parameter means that a time dependent injection model should be chosen. The growth.inj parameter ties the injection to the interface growth velocity. The recomb parameter indicates a finite surface recombination velocity.

- **Ksurf.0, Ksurf.E, Krat.0, Krat.E, Kpow.0, Kpow.E**

These parameters allow the specification of the surface recombination velocity. The formula used in computing the actual recombination velocity is:

$$K_V = K_{surf} \left[K_{rat} \left(\frac{V_{ox}}{B/A} \right)^{K_{pow}} + 1 \right] \quad 7.91$$

where V_{ox} is the interface velocity, and B/A is the Deal Grove oxide growth coefficient. (see oxide). The full model applies only to oxide, since it is the only interface that can grow in SUPREM-IV. This formulation allows an inert interface to have different values than a growing interface. For inert interfaces, (gas, oxynitride, nitride, poly) KSurf is the only important parameter.

- **vmole, theta.0, theta.E, Gpow.0, Gpow.E**

These parameters allow the specification of generation that is dependent on the growth rate of the interface. The formula is:

$$g = \theta V_{mole} V_{ox} \left(\frac{V_{ox}}{B/A} \right)^{G_{pow}} \quad 7.92$$

Theta, θ , is the fraction of silicon atoms consumed during growth that are reinjected as vacancies. V_{mole} is the lattice density of the consumed material. Once again, this model only makes sense for oxide since it is currently the only material which can have a growth velocity.

- **A.0, A.E, t0.0, t0.E, Tpow.0, Tpow.E**

These parameters allow an injection model with a fairly flexible time dependency. The equation for the injection is:

$$g = A(t + t_0)^{T_{pow}} \quad 7.93$$

where t is the time in the diffusion in seconds. This can be used to represent the injection of vacancies from an oxynitride layer.

- **rec.str, inj.str**

These two string parameters are useful for experimenting with new models for injection or recombination at interfaces. Three macros are defined for use, t the time in seconds and x and y the coordinates. If these are specified, they are used in place of any other model. For example,

`vacancy silicon /oxide inj.str = (10.0e4 * exp( t / 10.0 ))`

describes an injection at the silicon oxide interface that decays exponentially in time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. The segregation constant follows an Arrhenius relationship. This is for use in help supporting future model development, and is currently not in use.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. The transfer coefficient follows an Arrhenius relationship. This is for use in help supporting future model development, and is currently not in use.

7.75.3 Examples

`vacan silicon Di.0=5.0e-7 D.E=0.0 Cstar.0=1.0e13 Cstar.E=0.0`

This statement specifies the silicon diffusion and equilibrium values for vacancies.

`vacan sil /oxi growth vmole=5.0e22 theta.0=0.01 theta.E=0.0`

This parameter specifies the oxide "C silicon interface injection is to be computed using the oxide growth velocity and with 1 injected as vacancies.

`vacan silicon /nitride Ksurf.0=3.5e-3 Ksurf.E=0.0 Krat.0=0.0`

This specifies that the surface recombination velocity at the nitride silicon interface is 3.5×10^{-3} cm/sec.

`inter sil neu.0=1.0 neg.0=1.0 pos.0=0.0 dneg.0=0.0 dpos.0=0.0 inter sil neu.E=0.0 neg.E=0.0 pos.E=0.0 dneg.E=0.0 dpos.0=0.0`

This specifies that there are equal numbers of negative and neutral charged vacancies in intrinsic doping.

7.75.4 References

1. M.E. Law and J.R. Pfister, "Low Temperature Annealing of Arsenic/ Phosphorus Junctions," IEEE Trans. on Elec. Dev., 38(2), p. 278, 1991.
2. P.B. Griffin and J.D. Plummer, "Process Physics Determining 2-D Impurity Profiles in VLSI Devices," International Electron Devices Meeting, Los Angeles, p. 522, 1986.
3. W. Shockley and J.T. Last, "Statistics of the Charge Distribution for a Localized Flaw in a Semiconductor," Phys. Rev., 107(2), p. 392, 1957.
4. S.M. Hu, "On Interstitial and Vacancy Concentrations in Presence of Injection," J. Appl. Phys., 57, p. 1069, 1985.
5. T.Y. Tan and U. Goele, "Point Defects, Diffusion Processes, and Swirl Defect Formation in Silicon," J. Appl. Phys., 37(1), p. 1, 1985.
6. M.E. Law, "Parameters for Point Defect Diffusion and Recombination," IEEE Trans. on CAD, Accepted for Publication, Sept., 1991.
7. G.D. Watkins, "EPR Studies of the Lattice Vacancy and Low Temperature Damage Processes in Silicon," Lattice Defects in Semiconductors 1974, Huntley ed., 1975, Inst. Phys. Conf. Ser. 23, London.
8. G.A. Baraff and M. Schluter, "Electronic Structure, Total Energies, and Abundances of the Elementary Point Defects in GaAs," Phys. Rev. Lett., 55, 1327 (1985).

7.76 vgeneric

This command is the same as **generic** except it is for the fifth generic impurity. Please see the command for **generic** for more details.

7.77 zinc

7.77.1 Synopsis

```
zinc
( silicon | oxide | oxynitr | nitride | gas | poly | gaas | imaterial | iimaterial | iiimaterial |
ivmaterial | vmaterial)
[ Dix.0=<n> ] [ Dix.E=<n> ]
[ Dip.0=<n> ] [ Dip.E=<n> ] [ Dipp.0=<n> ] [ Dipp.E=<n> ]
[ Fi = <n> ]
[ implanted ] [ grown.in ]
[ ss.clear ] [ ss.temp=<n> ] [ ss.conc=<n> ]
[ ( /silicon | /oxide | /oxynitr | /nitride | /gas | /poly | /gaas | /imaterial | /iimaterial |
/iiimaterial | /ivmaterial | /vmaterial) ]
[ Seg.0=<n> ] [ Seg.E=<n> ] [ Trn.0=<n> ] [ Trn.E=<n> ]
[ ( donor | acceptor ) ]
```

7.77.2 Description

This statement allows the user to specify values for coefficients of zinc diffusion and segregation. The diffusion equation for zinc is [1,2]:

$$\frac{\partial C_T}{\partial t} = \nabla \left[D_V C_A \frac{C_V}{C_V^*} \nabla \ln \left(C_A \frac{C_V p}{C_V^* n_i} \right) + D_I C_A \frac{C_I}{C_I^*} \nabla \ln \left(C_A \frac{C_I p}{C_I^* n_i} \right) \right] \quad 7.94$$

where C_T and C_A are the total chemical and active concentrations of zinc, C_V and C_I are the vacancy and interstitial concentrations, the superscript * refers to the equilibrium concentration, D_V and D_I are the diffusivities with vacancies and interstitials, and p and n_i refer to the hole concentration and the intrinsic concentration respectively. The diffusivities are given by:

$$D_V = (1 - F_i) \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.95$$

$$D_I = F_i \left(D_X + D_P \frac{p}{n_i} + D_{PP} \left(\frac{p}{n_i} \right)^2 \right) \quad 7.96$$

D_X , D_P and D_{PP} are described in greater detail below. The segregation at material interfaces is computed using the following expression:

$$flux = \sqrt{M_{12} T_r} \left(C_1 \frac{C_2}{M_{12}} \right) \quad 7.97$$

where C_1 and C_2 are the concentrations in material 1 and 2 respectively, and the M_{12} and T_r terms are computed using expressions shown below with the parameters of the models.

- **silicon, oxide, oxynitr, nitride, gas, poly, gaas, imaterial, iimaterial, iimaterial, ivmaterial, vmaterial**

These allow the specification of parameters for that material. Only one of these can be specified per statement. The parameters specified in that statement will apply in the material listed. These parameters specify which material is material 1 for the segregation terms.

- **Dix.0, Dix.E**

These floating point parameters allow the specification of D_X , the zinc diffusivity with neutral defects. Dix.0 is the pre-exponential constant and Dix.E is the activation energy. Dix.0 defaults to 0.0 cm²/sec in gallium arsenide, and Dix.E defaults to 0.0 eV in gallium arsenide. D_X is calculated using a standard Arrhenius relationship.

- **Dip.0,Dip.E**

These floating point parameters allow the specification of D_P , the beryllium diffusivity with singly positive defects. Dip.0 is the pre-exponential constant and Dip.E is the activation energy. Dip.0 defaults to 0.0 cm²/sec in gallium arsenide, and Dip.E defaults to 0.0 eV in gallium arsenide. D_P is calculated using a standard Arrhenius relationship.

- **Dipp.0,Dipp.E**

These floating point parameters allow the specification of D_{PP} , the zinc diffusivity with doubly positive defects. Dipp.0 is the pre-exponential constant and Dipp.E is the activation energy. Dipp.0 defaults to $1.2 \times 10^2 \text{ cm}^2/\text{sec}$ for implanted zinc in gallium arsenide and $15.6 \text{ cm}^2/\text{sec}$ for grown-in zinc in gallium arsenide, and Dipp.E defaults to 3.9 eV for both in gallium arsenide[1,2]. D_{PP} is calculated using a standard Arrhenius relationship.

- **Fi**

This parameter allows the specification of the fractional interstitialcy. This value indicates whether zinc diffuses through interaction with interstitials or vacancies. The value of Fi defaults to 1.

- **implanted, grown.in**

Specifies whether the Dix, Dip, Dipp, or Fi coefficients apply to implanted or grown-in zinc. If neither is specified then a specified parameter applies to both.

- **ss.clear**

This parameter clears the currently stored solid solubility data.

- **ss.temp, ss.conc**

These parameters add a single temperature solid solubility concentration point to those already stored. The default values are:

ss.temp (C)	ss.conc (cm^{-3})
700	1.e19
1250	1.e19

- **/silicon, /oxide, /oxynit, /nitride, /gas, /poly, /gaas, /imaterial, /iimaterial, /iiimaterial, /ivmaterial, /vmaterial**

These parameters specify material 2. Only one of the these parameters can be specified at one time.

- **Seg.0, Seg.E**

These parameters allow the computation of the equilibrium segregation concentrations. M_{12} is calculated using a standard Arrhenius relationship.

- **Trn.0, Trn.E**

These parameters allow the specification of the transport velocity across the interface given. The units are in cm/sec. Tr is calculated using a standard Arrhenius relationship.

- **donor, acceptor**

These parameters determine whether the impurity is to be treated as a donor or as an acceptor in a semiconductor material. These parameters are not presently material specific. By default zinc is an acceptor.

7.77.3 Examples

zinc gaas implanted Dip.0=2.2e-6 Dip.E=1.76

This command changes the positive defect diffusivity for implanted zinc in gallium arsenide.

zinc gaas /nitride Seg.0=1126.0 Seg.E=0.91 Trn.0=1.66e-7

This command will change the segregation parameters at the gallium arsenide silicon nitride interface. The gallium arsenide concentration will be half the nitride concentration in equilibrium at 1100 °C.

Appendix A

Impurity and Material Indices

In this appendix, we list all the impurity, variable and material indices (also referred to as the impurity numbers and material numbers). These indices are needed if one needs to modify or enhance the modelrc material data base or the impurity table.

Table 1 is a list of the supported materials. For generic materials (from imaterial to vmaterial) all physical models applicable for poly silicon are implemented (with diffusion, implantation, deposit, etch, etc).

Table of material indices.

material	symbol	description
0	gas	gas phase
1	oxide	SiO ₂
2	nitride	Si ₃ N ₄
3	silicon	single crystal silicon
4	poly	poly crystal silicon
5	oxynitride	oxynitride
6	aluminum	aluminum
7	photoresist	photoresist
8	gaas	GaAs

9	imaterial	generic material number 1
10	iimaterial	generic material number 2
11	iiimaterial	generic material number 3
12	ivmaterial	generic material number 4
13	vmaterial	generic material number 5

In the Suprem4.gs code, all nodal variables are referred to as ``impurities" which has a general meaning of solved nodal variables. Impurities and variables are sometime interchangeable and we shall refer them as ``impurity-variables" and these are listed in Table 1.

Table of Csuprem nodal variables, part 1.

symbol	index	description
V	0	Vacancies
I	1	Interstitials
As	2	Arsenic
P	3	Phosphorus
Sb	4	antimony
B	5	Boron
N	6	electron concentration
H	7	hole concentration
XVEL	8	X velocity
YVEL	9	Y velocity
O2	10	Dry O2
H2O	11	Wet O2
T	12	Interstitial Traps
Au	13	Gold
Psi	14	Potential
Sxx	15	Component of stress(xx)
Syy	16	Component of stress(yy)
Sxy	17	Component of stress(xy)
Cs	18	Cesium for oxide charges
DELA	19	change in interface area - not solution variable
Asa	20	Arsenic active concentration
Pa	21	Phosphorus active concentration
Sba	22	antimony active concentration
Ba	23	Boron active concentration
GRN	24	Polysilicon grain size
Ga	25	p-type tracer to help model miyake

Table of Csuprem nodal variables, part 2.

symbol	index	description
iBe	31	Beryllium impurity
iBea	32	Beryllium active concentration
iMg	33	Magnesium impurity
iMga	34	Magnesium active concentration
iSe	35	Selenium impurity
iSea	36	Selenium active concentration
iSi	37	Silicon impurity
iSia	38	Silicon active concentration
iSn	39	Tin impurity
iSna	40	Tin active concentration
iGe	41	Germanium impurity
iGea	42	Germanium active concentration
iZn	43	Zinc impurity
iZna	44	Zinc active concentration
iC	45	Carbon impurity
iCa	46	Carbon active concentration
iG	47	Generic impurity
iGa	48	Generic active concentration
iGII	49	II Generic impurity
iGall	50	II Generic active concentration
iGIII	51	III Generic impurity
iGalll	52	III Generic active concentration
iGIV	53	IV Generic impurity
iGaIV	54	IV Generic active concentration
iGV	55	V Generic impurity
iGaV	56	V Generic active concentration
BF2	57	Implanted as BF2 but diffuses as Boron

Table 1 lists all the impurity of nodal variables acceptable by the program. For example, when using the **select** statement to select a variable for plotting, one of the input variables can be used. Note that this table refers to the CSuprem symbols which is defined in Tables 1 and 1. When referring to the active component of an impurity, the flag ``active" should be used in the **select** statement.

Table of input nodal variables.

input	symbol	input	symbol
arsenic	As	boron	B
gallium	Ga	antimony	Sb
phosphorus	P	interstitial	I
vacancy	V	time	TIM

x	X	y	Y
oxygen	OXY	trap	T
gold	Au	Sxx	Sxx
Syy	Syy	Sxy	Sxy
x.veloc	XVEL	v.x	XVEL
xv	XVEL	vx	XVEL
y.veloc	YVEL	v.y	YVEL
yv	YVEL	vy	YVEL
psi	Psi	doping	DOP
holes	H	electrons	N
ci.star	CIS	cv.star	CVS
cesium	Cs	grain	GRN
qfn	QFN	qfp	QFP
Ec	ECON	Ev	EVAL
beryllium	iBe	magnesium	iMg
selenium	iSe	isilicon	iSi
tin	iSn	germanium	iGe
zinc	iZn	carbon	iC
generic	iG	iigeneric	iGII
iiigeneric	iGIII	ivgeneric	iGIV
vgeneric	iGV		

Table 1 list all the impurities that are implemented into the ion implantation model. Please note that the ion used in implantation may be different than the diffused impurity associated with a species. When changing the impurity table, the ion index should be used while the program diffuses using the diffusion impurity. The diffused impurity index and ion index can be looked up in Tables 1 and 1. For example, when modeling BF₂ implantation, the ion index is 57 while the diffused impurity index is 5.

Table of implant variables. Ion index and diffused impurity index can be looked up in the previous table of nodal variables in this appendix.

input	diffusion variable/index	ion variable/index
silicon	I	I
arsenic	As	As
phosphorus	P	P
antimony	Sb	Sb
boron	B	B
bf2	B	BF2
cesium	Cs	Cs
beryllium	iBe	iBe
magnesium	iMg	iMg
selenium	iSe	iSe

isilicon	iSi	iSi
tin	iSn	iSn
germanium	iGe	iGe
zinc	iZn	iZn
carbon	iC	iC
generic	iG	iG
iigeneric	GII	GII
iiigeneric	GIII	GIII
ivgeneric	GIV	GIV
vgeneric	GV	GV

References

- [1] S. M. Sze, *Physics of semiconductor devices*, 2nd ed. John Wiley & Sons, 1981.
- [2] Suprem-IV.GS, *Tow dimensional process simulation for silicon and galium arsenide*. Edited by S.E. Hansen and M.D. Deal, 1993. Integrated Circuits laboratory, Stanford University, Stanford, California 94305.
- [3] Deal, B.E. Grove, A.S., *General relationship for the thermal oxidation of silicon*. J.App.Phys. 36,pp:3770-3778, 1965.
- [4] Selberherr S., *Analysis and simulation of semiconductor devices*. Spring-Verlag, Wien, New York, 1984.
- [5] H.Z. Masssoud, J.D. Plummer, and E.A. Irene, *Thermal oxidation of silicon in dry oxygen growth rate enhancement in the thin regime*. J. Electrochem, Soc., 132, pp:2685, 1985.
- [6] N. Guillemot, G. Pananakakis and P. chenevier, *A new analytical model of the Bird's Beak*. IEEE Transactions on Electron Devices, ED-34, 1987.
- [7] C.P. Ho, and J.D. Plummer, *Si/SiO₂ interface oxidation kinetics: A physical model for the influence of high substrate doping levels, Part I and II*. J. Electrochem, Soc, 126, pp: 1523, 1979.

- [8] H. Puchner and S. Selberherr, *Dynamic grain-growth and static clustering effects on dopants diffusion in poly-silicon*. In: NUPAD V, pp:109-112, 1994.
- [9] E. Leitner, *Diffusionsprozesse in dreidimensionalen strukturen*. Dissertation, Technischen Universität Wien, 1997.
- [10] R.B. Fair, *Concentration profiles diffused dopants in silicon*. Impurity doping process in silicon. Yang ed. 1981, North-Holland, New-York.
- [11] M.E. Law and R.W. Dutton, *Verification of analytic point defect models using Suprem-IV*. IEEE Trans. on CAD, 7(2), pp:181, 1988.
- [12] Yeager, S. , Dutton, W., *An approach to solving multi-particle diffusion exhibiting non-linear stiff coupling*. IEEE Trans. on Elec. Dev., 32(10), pp:1964-1976, 1985.
- [13] R.E. Bank, W.M. Coughran, W. Fichtner, E.H. Grosses, D.J. Rose, and R.K. Smith, *Transient simulation of silicon devices and circuits*. IEEE Trans. on Elec. Devices, ED-32, pp: 1992, 1985.
- [14] H.C. Elman, *Preconditioned conjugate gradient methods for nonsymmetric systems of linear equations*. Yale University Research Report, 1981.
- [15] D.B. Kao, J.P. McVittie, W.D. Nix and K.C. Saraswat, *Two-dimensional silicon oxidation experiments and theory*. IEDM Tech. Digest, 1985.
- [16] Christel, L.A., Gibbons, J.F. Mylroie, S., *An application of the Boltzmann equation to ion range and damage distribution in mylti-layered targets*. J. Appl. Phys. 51, No.12, pp: 6176-6182, 1981.
- [17] Titov, A. I., Christodoulides, C. E., Carter G., Nobes, M.J., *The depth distribution of disorder produced by room temperature 40keV N^+ ion irradiation of silicon*. Radiation effects 41, pp:107-111, 1979.
- [18] Troxell, J.R., *Ion-Implantation associated defect production in silicon*. Solid-State Electron. 26, No.6, pp:539-548, 1983.
- [19] Stone, J. L., Plunkett, J. C., *Ion implantation processes in silicon*. In: Impurity Doping Processes in Silicon, PP: 56-146. Amsterdam: North-Holland, 1981.
- [20] Gemmel, T.S., *Channeling and related effects in the motion of charged particles through crystals*. Rev. Mod. Phys. 46, No. 1, pp:129-227, 1974.
- [21] Mazzone, A.M., *Formation and growth of a damaged layer during implantation: A three dimensional Monte Carlo Simulation*. Proc. NASECODE III Conf., pp:179-184. Dublin: Boole Press, 1983.

- [22] Pichler, P. Jungling, W., Selberherr, S., Guerrero E., and Potzl, H., *Simulation of critical IC-Fabrication steps*. IEEE Trans. Computer-Aided Design, Vol. 4, pp:384-397, 1985.
- [23] Law, M.E., Dutton, R.W., *Verification of analytic point defect models using Suprem-IV*. IEEE Trans. CAD, vol. 7, No. 2, pp:181-190,1988.
- [24] Baccus, B., Collard, D., Dubois, E., Morel, D., *IMPACT4--A general two-di,mensional multilayer process simulator*. In: Baccarani and Rudan [25], pp:255-266.
- [25] Ed. by Baccarani, B., and Rudan, M., *Simulation of semiconductor devices and processes*. Bologna, vol. 3, Technoprint, 1988.
- [26] Mulvaney, B.J., Richardson, W.B., Crandle, T.L., *PEPPER-- A process simulator for VLSI*. IEEE Trans. on CAD of Integrated Circuits and systems, vol.8, no. 4, pp:336-349
- [27] Helmut, P., *Advanced process modeling for VLSI technology*. Ph.D. Thesis, Technical University of Wien, June 1996.
- [28] Ryssel, H., Prike, G., Habberger, K., Hoffman, K., Muller, K., *Range parameters of Boron implanted into silicon*. Appl. Phys. vol. 24, pp:39-43, 1981.
- [29] Tasch, A.F., Shin, H., Park, C., Alvis, J., Novak, S., *An improved approach to accurately model shallow B and BF2 implants in silicon*. J. Electrochem. Soc., vol.136, pp:810-814, 1989.
- [30] Hobler, G., Selberherr, S., *Two-dimensional modelling of ion implantation induced point defects*. IEEE Trans. on CAD, 7(2), pp: 174, 1988.
- [31] Bohmayer, W., Schrom, G., Selberherr, S., *Investigation of channeling in field oxide corners by three-dimensional Monte Carlo simulation of ion implantation*. In Proceedings: 4th Int. Conf. on Solid-State and Integrated-Circuits Technology, Beijing, China, pp:304-306, 1995
- [32] Radi, M., *Three-dimensional simulation of thermal oxidation*. Ph.D. Thesis, Technical University of Wien, June 1998.
- [33] Puchner, H., Neary, P., Aronwitz, S., Selberherr, S., *A transient activation model for phosphorus after sub-amorphizing channeling implants*. In: Baccarani and Rudan [34], pp:157-160
- [34] Ed. by Baccarani, G., Rudan, M., Proc. 26th Eurpean Solid State Device Research Conference, Gif-sur-Yvettte Cedex, France, 1996.
- [35] Hofker, W.K., *Concentration profiles of boron implanted in amorphous and polycrystalline silicon*. Philips Res. Rep., vol. supp. 8, pp:41-57, 1975.

- [36] Gibbons, J.F., Mylroie, S., *Estimation of impurity profiles in ion implanted amorphous targets using joined half-Gaussian distribution.*
- [37] Park, C., Klein, K.M., Tasch, A.F., *Efficient modeling parameter extraction for dual Pearson approach to simulation of implanted impurity profiles in silicon.* Sol. State Electron., vol. 33, no. 6, pp:645-650, 1990.
- [38] Yang, S.H., Morris, S., Parab, K., Tasch, A.F., *An accurate Monte Carlo model for the simulation of Arsenic and Boron implants into silicon.* UT-MARLOWE Version 2.0. The University of Texas, Austin 78712, USA, 1995.
- [39] Odanaka, S., Umimoto, H., Wakabayashi, M., Esaki, H. *SMART-P: Rigorous three-dimensional process simulator on supercomputer.* IEEE Transactions on CAD of Integrated Circuits and Systems, vol. 7, No. 6, pp:675-683, 1988.
- [40] Ushio, S., Nishi, K., Kuroda, S., Kai, K., Ueda, J., *A fast three-dimensional process simulator OPUS/3D with access to two-dimensional simulation results.* IEEE Trans. CAD, vol. 9, No. 7, pp:745-751, 1990.
- [41] Leitner, E., Bohmayer, W., Fleishchmann, P., Strasser, E., Selberherr, S., *3D TCAD at TU Vienna.* In: Lorenz [42], pp:136-161, 1995.
- [42] Ed. by J. Lorenz. *3-Dimensional process simulation, 1995, Spring.*
- [43] Zheng, J., McVittie, P., *Modeling of side wall passivation and ion saturation effects on etching profiles.* In: Proceedings: Int. Workshop on numerical modeling of processes and devices for Integrated Circuits NUPAD V, pp:37-40, 1994.
- [44] Vinay, S.R., *On the numerical modeling of thermal oxidation in silicon.* Ph.D. Thesis, Depart. of Civil Engineering, Stanford University, July 1997.
- [45] Stephan, C., *Multidimensional viscoelastic model modelling of silicon oxidation and titanium silicidation.* Ph.D. Thesis, University of Florida, 1996.
- [46] Mastumoto, H., Fukuma, M., *Numerical modeling of nonuniform Si thermal oxidation.* IEEE Trans. Electron Devices, Vol. 32, No. 2, pp:132-140, 1985.
- [47] Poncet, A., *Finite-Element simulation of local oxidation of silicon.* IEEE Trans. CAD, Vol. 4, No. 1, pp:41-53, 1985.
- [48] Chin, D., Oh, S.Y., Hu, S.M., Dutton, R.W., Moll, J.L., *Two-dimensional oxidation.* IEEE Trans. Electron Devices, Vol. 30, No. 7, pp:744-749, 1983.
- [49] Peng, J.P., Chidambarao, D., Srinivasan, G.R., *Viscoelastic modeling of thermal oxidation.*

177th Meeting Electrochem. Soc., pp:425-426, 1990.

[50] Tsami, C., Tsoukalas, D., *Modeling of silicon interstitial surface recombination velocity at non-oxidizing interfaces*. In: Lorenz [42], pp:452-455.

[51] Tiller, W.A., *On the kinetics of thermal oxidation of silicon I. A theoretical perspective*. J. Electrochem. Soc., Vol. 127, No. 3, pp:619-624, 1980.

[52] Law, M.E., *Grid adaptation near moving boundaries in two dimensions for IC process simulation*. IEEE Trans. CAD, Vol. 14, No. 10, pp:1223-1230, 1995.

[53] C.G. Van de Walle and R.M. Martin, "Theoretical calculations of heterojunction discontinuities in the Si/Ge system," Phys. Rev. B, vol. 34, No. 8, p. 5621, 1986.

[54] Takato, H., Sunouchi, K., Okabe, N., Nitayama, A., Hieda, K., Horiguchi, F., Masuoka, F., *Impact of surrounding gate transistor (SGT) for Ultra-High-Density LS's*. IEEE Trans. Electron Devices, Vol. 38, No. 3, pp:573-578, 1991.

[55] Fahey, P.M., Griffin, P.B., Plumer, J.D., *Point defects and dopant diffusion in silicon*. Review of Modern Physics, Vol. 61, No. 2, pp:289-384, 1989.

[56] Taylor, W., Gosele, U., Tan, T.Y., *Present understanding of point defect parameters and diffusion in silicon: An overview*. In proceedings: Process physics and modeling in semiconductor technology, 1993, the Electrochemical Society, pp:3-19

[57] Griffin, P.B., Plummer, J.D., *Process physics determining 2D-impurity profiles in VLSI devices*. International Electron Devices Meeting, Los Angeles, pp:522, 1986

[58] Hu, S.M., *On interstitial and vacancy concentrations in presence of injection*. J. Appl. Phys., 57, pp:1069, 1985.

[59] Baccus, B., Vnadenboss, E., *Boron diffusion at high concentrations during predeposition processes*. In Proceedings: Process Physics and Modelling in Semiconductor Technology, 1993, The Electrochemical Society, pp: 75-87.

[60] Guerrero, E., Potzl, H., Tielert, R., Grasserbauer, M., Stingder, G., *Generalized model for the clustering of As dopants in Si*. J. Electrochem. Soc., Vol. 129, No. 8, pp:1826-1831, 1982.

[61] Tsai, M., Morehead, F., Baglin, J., *Shallow junction by high dose As implants in Si: Experiments and modelling*. J. Appl. Phys., Vol. 51, No. 6, pp:3230-3235, 1980.

[62] David M., Jon S., Keven T., *Improved Independent Gate N-Type FinFET Fabrication and Characterization*. IEEE Electron Device Letters, Vol.24, No.9, Sept. 2003.